Machine Learning Handbook — Vol 1

foundations and core algorithms — ML basics, maths, Python stack, data and features, regression, evaluation, trees and ensembles, kNN/SVM/Naive Bayes, unsupervised learning, neural networks (Ch 1–10)

10 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 1 — What Machine Learning Is**
   - 1.1 The AI family tree
   - 1.2 Learning from data vs writing rules
   - 1.3 How a model learns: training, inference and feedback loops
   - 1.4 Types of learning by supervision
   - 1.5 Classification vs regression
   - 1.6 Key terms
   - 1.7 Batch vs online learning
   - 1.8 Instance-based vs model-based learning
   - 1.9 The end-to-end ML workflow
   - 1.10 The main challenges of machine learning
   - 1.11 A first end-to-end project with scikit-learn
   - Chapter TL;DR
2. **Chapter 2 — The Math You Need for Machine Learning**
   - 2.1 Vectors
   - 2.2 Matrices and matrix multiplication
   - 2.3 Eigenvalues, eigenvectors and SVD (intuition)
   - 2.4 Derivatives, partial derivatives and gradients
   - 2.5 Gradient descent
   - 2.6 Random variables and distributions
   - 2.7 Conditional probability and Bayes' theorem
   - 2.8 Descriptive statistics and standardisation
   - 2.9 Correlation vs causation
   - 2.10 Sampling and the central limit theorem
   - 2.11 Hypothesis tests, p-values and confidence intervals
   - 2.12 Information theory: entropy, cross-entropy and KL divergence
   - Chapter TL;DR
3. **Chapter 3 — The Python ML Stack**
   - 3.1 Setting up
   - 3.2 NumPy: fast arrays
   - 3.3 pandas: tables with labels
   - 3.4 matplotlib: seeing the data
   - 3.5 scikit-learn: the estimator API
   - 3.6 Jupyter workflow
   - 3.7 Deep-learning frameworks: when scikit-learn is not enough
   - Chapter TL;DR
4. **Chapter 4 — Data, EDA and Feature Engineering**
   - 4.1 Where training data comes from
   - 4.2 Exploratory data analysis (EDA)
   - 4.3 Data cleaning
   - 4.4 Encoding categorical variables
   - 4.5 Feature scaling
   - 4.6 Feature creation
   - 4.7 Train, validation and test splits
   - 4.8 Data leakage
   - 4.9 Class imbalance
   - 4.10 End-to-end: `Pipeline` + `ColumnTransformer`
   - Chapter TL;DR
5. **Chapter 5 — Linear and Logistic Regression**
   - 5.1 The supervised-learning workflow
   - 5.2 Linear regression: model and loss
   - 5.3 Fitting it: the normal equation vs gradient descent
   - 5.4 Gradient descent in practice
   - 5.5 Assumptions of linear regression
   - 5.6 Polynomial regression
   - 5.7 Regularization: Ridge, Lasso, ElasticNet
   - 5.8 Logistic regression
   - 5.9 Multiclass classification: one-vs-rest and softmax
   - 5.10 Interpreting coefficients
   - Chapter TL;DR
6. **Chapter 6 — Evaluation and Tuning**
   - 6.1 Baselines, and offline vs business metrics
   - 6.2 Regression metrics
   - 6.3 Classification metrics
   - 6.4 Ranking metrics (an introduction)
   - 6.5 Calibration
   - 6.6 Cross-validation
   - 6.7 The bias–variance trade-off
   - 6.8 Learning curves and validation curves
   - 6.9 Hyperparameter tuning
   - 6.10 Early stopping
   - 6.11 Putting it together: from ML metric to business decision
   - Chapter TL;DR
7. **Chapter 7 — Decision Trees and Ensembles**
   - 7.1 Decision trees
   - 7.2 Bias–variance of trees
   - 7.3 Bagging and Random Forests
   - 7.4 Boosting
   - 7.5 Voting and stacking
   - 7.6 When trees beat neural networks
   - 7.7 Hyperparameter cheat sheet
   - Chapter TL;DR
8. **Chapter 8 — k-Nearest Neighbours, Support Vector Machines, Naive Bayes**
   - 8.1 k-Nearest Neighbours (k-NN)
   - 8.2 Support Vector Machines (SVM)
   - 8.3 Naive Bayes
   - 8.4 Choosing a classic supervised algorithm
   - Chapter TL;DR
9. **Chapter 9 — Unsupervised Learning**
   - 9.1 k-means clustering
   - 9.2 Hierarchical (agglomerative) clustering
   - 9.3 DBSCAN — density-based clustering
   - 9.4 Gaussian mixture models (GMM) and EM
   - 9.5 Dimensionality reduction: PCA
   - 9.6 t-SNE and UMAP — non-linear visualisation
   - 9.7 Anomaly detection
   - 9.8 Association rules (Apriori)
   - 9.9 Evaluating unsupervised models
   - Chapter TL;DR
10. **Chapter 10 — Neural Networks and Deep Learning**
    - 10.1 From the perceptron to the multi-layer perceptron
    - 10.2 Activation functions
    - 10.3 The forward pass and loss functions
    - 10.4 Backpropagation

- 10.5 Optimizers and learning-rate schedules
- 10.6 Initialization and vanishing/exploding gradients
- 10.7 Regularization: fighting overfitting
- 10.8 Practical training habits
- 10.9 PyTorch basics
- Chapter TL;DR

Chapter 1 — What Machine Learning Is

Machine learning (ML) is getting a computer to improve at a task by studying examples instead of following hand-written rules. This chapter builds the map for the rest of the book and ends with a runnable **scikit-learn** project (the most widely used free Python library for classic ML: ready-made models, data-splitting tools and metrics behind one consistent interface).

What you'll learn

- Where ML sits inside AI (deep learning, generative and agentic AI)
- When learning from data beats writing rules, and when it does not
- The five learning setups; classification vs regression; batch vs online; instance- vs model-based
- Training, inference and feedback loops; the end-to-end workflow
- Core vocabulary: features, labels, parameters, hyperparameters, loss, train/validation/test
- How projects fail: bad data, overfitting, underfitting, leakage, non-representative data

All code is in `code/`; every script runs in seconds: `python code/<file>.py`.

1.1 The AI family tree

What it is

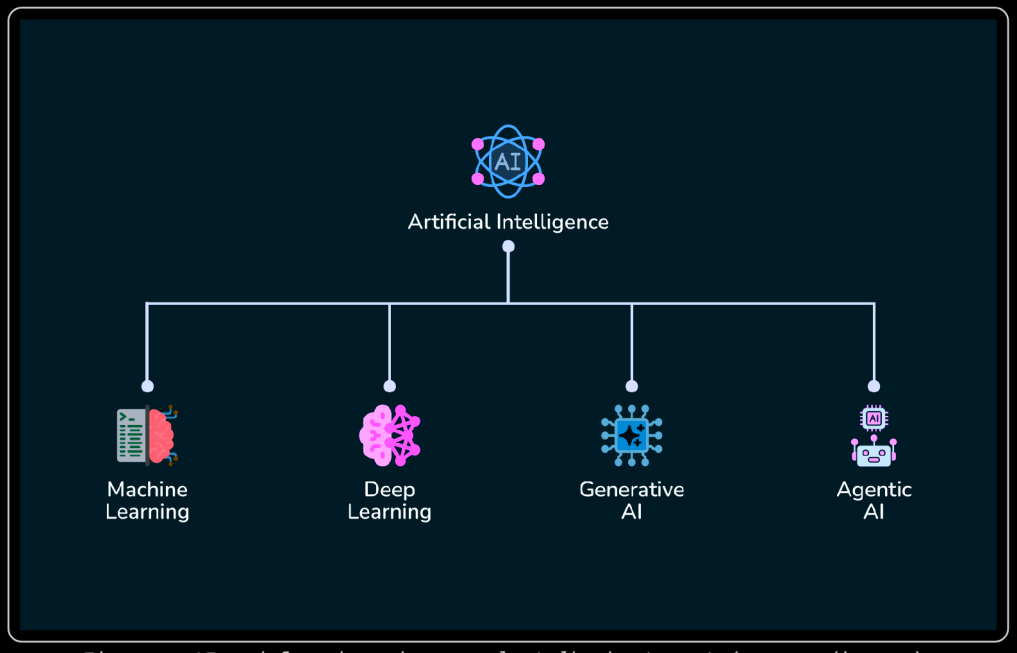
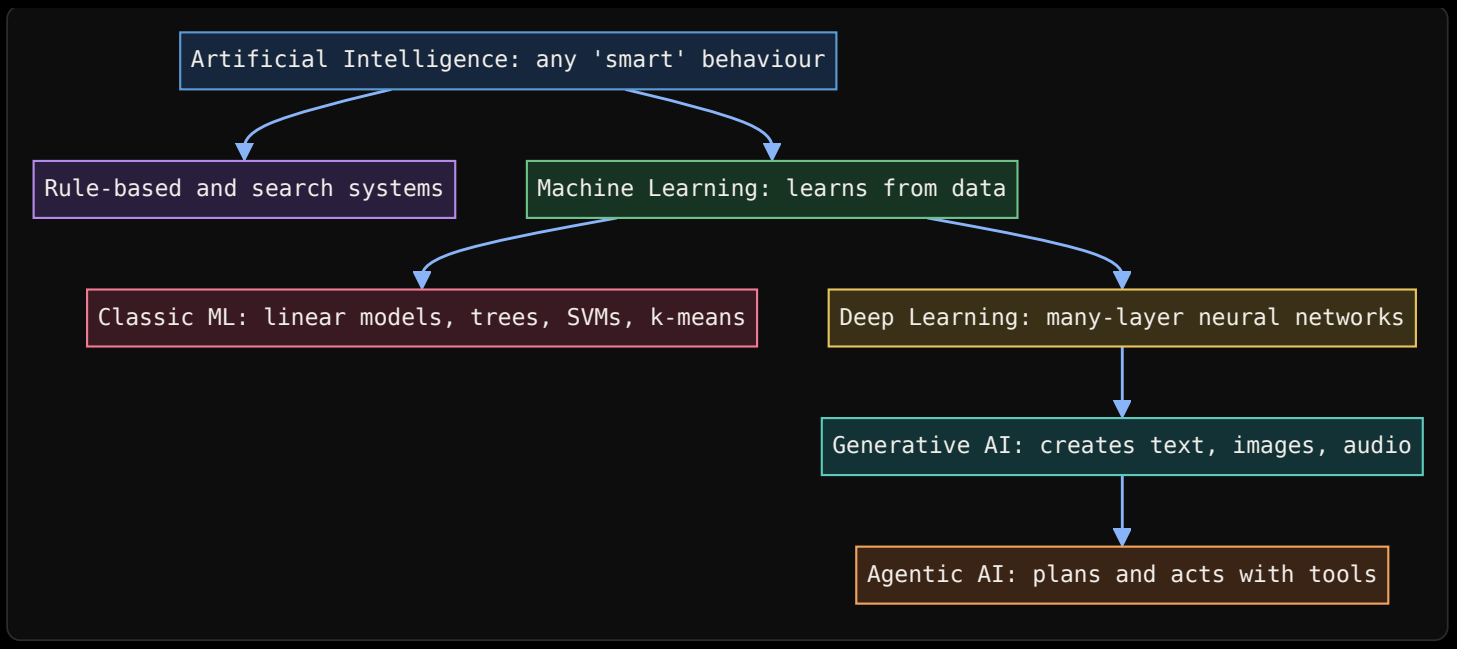


Figure — AI and four branches people talk about most (course diagram).

Term	Plain definition	Relationship	Typical example
Artificial intelligence (AI)	Any technique that makes a machine reason, perceive, plan or decide.	The whole field; includes hand-written rules and search, not only learning.	Chess engine searching moves; rules-based tax calculator; route planner
Machine learning (ML)	AI that *learns its behaviour from data* instead of having every rule spelled out.	Subset of AI.	Spam filter, fraud detector, "customers also bought"
Deep learning (DL)	ML using *neural networks* (many small units that each compute a weighted sum and pass it on) with many layers, each transforming the previous layer's output.	Subset of ML.	Face unlock, speech recognition, machine translation
Generative AI (GenAI)	Models that *produce new content* (text, images, audio, video, code) resembling their training data.	Mostly built with DL today.	Chat assistants, image generators (DALL·E, Midjourney)
Agentic AI	Systems that *plan and take multi-step actions* toward a goal, using tools and adapting.	Usually a generative model in a plan → act → observe loop.	Coding agent that edits files and runs tests; warehouse robot choosing routes



Every DL model is an ML model and every ML model is an AI system, not the reverse. GenAI and agentic AI are not "inside" DL by definition, but nearly every practical system today is built on deep networks, so the diagram shows the common case.

Why it matters

Knowing the boundaries lets you pick the right tool: a tabular dataset (rows and columns, like a spreadsheet) of 10,000 loan applications calls for classic ML, not a large language model (**LLM**: a very large neural network trained on huge amounts of text to predict and generate language); generating product descriptions calls for GenAI; a VAT calculation needs neither.

How each branch works, briefly

1. **ML** — show many labelled examples (photos marked "cat"/"dog"); an algorithm finds the patterns that separate them and labels unseen photos. The rules are discovered, not typed.
2. **DL** — the model is a stack of layers: in an image network early layers detect edges, middle layers textures and parts, late layers whole objects. Because layers learn their own features, DL shines on raw unstructured data (pixels, audio, text). **AlexNet** (a deep image network) winning the 2012 **ImageNet** contest (over a million labelled photos) is the usual marker of DL overtaking older vision methods.
3. **GenAI** — learns the structure of its material well enough to *sample* new material. A text model such as GPT predicts the next **token** (a word or piece of a word) repeatedly; image generators turn random noise into an image matching a prompt.
4. **Agentic AI** — a model gets a goal and tools (web search, a calendar **API** — a defined way for one program to ask another to do something — a code runner). It picks a step, calls a tool, reads the result and repeats until done: autonomy over *multiple* steps in a changing environment.

Trade-offs

	Classic ML	Deep learning	Generative AI	Agentic AI
Data needed	Hundreds to millions of rows	Usually large (or a pre-trained model)	Enormous for pre-training (the first, general run); small for adapting	Builds on a pre-trained model
Best input	Tables	Images, audio, text	Text, images, audio	Tasks with tools and steps
Interpretability (seeing why it decided)	Often good	Low	Low	Low, and actions compound
Compute	Low (CPU)	High (GPU: a graphics chip doing many calculations in parallel)	Very high to train; high to run	High (many model calls)
Main risk	Bad data	Needs lots of data and compute	Confident wrong output ("hallucination"); copyright, authenticity	Mistakes chain across steps; safety of actions

When not to use it

- Rules known, stable and few (tax brackets, unit conversion, access control) → write the rules.
- A mistake is unacceptable and you cannot measure the error rate → no learned model without human review.
- A classic model or a database query answers exactly → no GenAI.

Common mistakes

- Saying "AI" for a specific technique; say "a gradient-boosted classifier" or "a fine-tuned language model" (a *classifier* predicts a category; *fine-tuning* trains an already-trained model a little more on your smaller dataset).
- Assuming newer = better: on tabular business data, tree ensembles (many combined decision trees, i.e. flowcharts of yes/no questions) such as random forests and gradient boosting (Chapter 7) often beat deep networks.
- Believing a model "understands"; it maps inputs to outputs using statistical patterns.

Real-world examples

- **Classic ML:** card-fraud scoring, Gmail spam filtering, credit scoring, retail demand forecasting; Netflix/YouTube recommendations from your and similar users' history.
- **DL:** Apple's Face ID, Google Translate, speech recognition, abnormality detection in medical scans.
- **GenAI:** chat assistants drafting text, DALL·E and Midjourney, music and video tools.
- **Agentic:** warehouse robots planning routes, assistants that book appointments, coding agents (automated trading acts autonomously too, though much of it is rule-based).

In the interview

> **30-second answer:** "AI is the broad goal of making machines behave intelligently. Machine learning is the part of AI where behaviour is learned from data instead of hand-coded. Deep learning is ML with many-layer neural networks, which learn their own features from raw data like pixels and text. Generative AI uses such models to create new content, and agentic AI wraps them in a loop so they can plan and take actions with tools."

- *Is a rules engine AI?* — Yes, classical AI includes rule-based expert systems; it is just not ML.
- *Why did DL take off around 2012?* — Large labelled datasets (ImageNet), cheap parallel compute (GPUs), and training tricks: ReLU activations (keep positives, zero negatives), dropout (randomly switching off units during training so the net cannot rely on any one), better initialisation (smarter starting weights).
- *Classic ML over DL when?* — Small/medium tabular data, need for interpretability, tight latency (time for one prediction; see System Design Fundamentals) or budget.

Self-check

1. Is every ML model a DL model? → No; linear regression and decision trees are ML but not DL.
2. What makes a system "agentic"? → It pursues a goal over multiple steps, choosing actions and tools and adapting to what it observes.
3. What does a generative model output that a classifier does not? → New content rather than a label or number.

TL;DR

AI ⊃ ML ⊃ DL. GenAI creates content, agentic AI takes multi-step actions; both usually run on DL. Pick the simplest branch that solves the problem.

1.2 Learning from data vs writing rules

What it is

Traditional programming: *input + rules → output*. ML: *inputs + outputs → rules (the model)*, then *new input + model → output*.

Tom Mitchell (1997): a program *learns* from experience **E** with respect to task **T** and performance measure **P** if its performance on T, measured by P, improves with E. Spam filter: T = flag spam, E = emails labelled spam/not-spam, P = fraction classified correctly.

The problem it solves

- **Too complex:** nobody can write "is there a cat in this photo?" in pixel values.
- **Constantly changing:** spammers adapt; a learned filter is retrained on new examples.
- **Too many cases:** speech across accents, microphones, noise.

- **Discovery:** revealing patterns people missed (e.g. which feature combinations predict churn), sometimes called *data mining*.

How it works

Collect examples (with answers if available) → turn each into numbers (features) → choose a model family (e.g. "a weighted sum of word counts") → a training algorithm adjusts the model's internal numbers until its answers match the examples → use it on new inputs.

Code: a hand-written rule vs a learned model

`code/rules_vs_learning.py` trains a Naive Bayes spam filter (a simple probabilistic classifier, Chapter 8) on ten labelled messages and compares it with the rule "spam if it contains *free* or *prize*".

```
def rule_based(text: str) -> int:
    return int("free" in text or "prize" in text)

vec = CountVectorizer()                  # text -> word-count features
X = vec.fit_transform(texts)
model = MultinomialNB().fit(X, labels)   # learns how often each word appears in spam vs ham
p = model.predict_proba(vec.transform([t]))[0, 1]
```

Output:

```
message                          rule  learned  P(spam)
limited offer click to claim        0        1    0.95
free coffee in the kitchen          1        1    0.61
slides for the review meeting       0        0    0.06
```

The rule misses spam without its two words and cannot learn. The model catches the first message from words seen in spam ("limited", "offer", "click", "claim") but gets "free coffee" wrong: with ten examples, "free" looks spammy. **A model is only as good as its data**; with thousands of examples the office use of "free" would be learned.

Trade-offs

	Hand-written rules	Machine learning
Needs	Expert knowledge	Representative, labelled data
Transparency	Fully readable	Often opaque
Adapting to change	Rewrite code	Retrain
Fuzzy patterns (images, speech)	Poorly	Well
Guarantees	Exact	Statistical: right *most* of the time

When not to use it

Logic is known and must be exact (payroll, permissions), there is no data, or no error rate is tolerable.

Common mistakes

- Starting with ML before trying a simple rule or heuristic as a **baseline** (a simple reference solution any real model must beat).
- Forgetting that a learned model inherits every bias and gap in its examples.

Self-check

1. Mitchell's T, E, P for a house-price model? → T: predict price; E: past sales with prices; P: e.g. average error in dollars.
2. What did the spam example show about small data? → The model over-trusted one word ("free").

TL;DR

Rules come from people in programming and from data in ML. Use ML when rules are too complex or change too often, and only with good examples.

1.3 How a model learns: training, inference and feedback loops

What it is

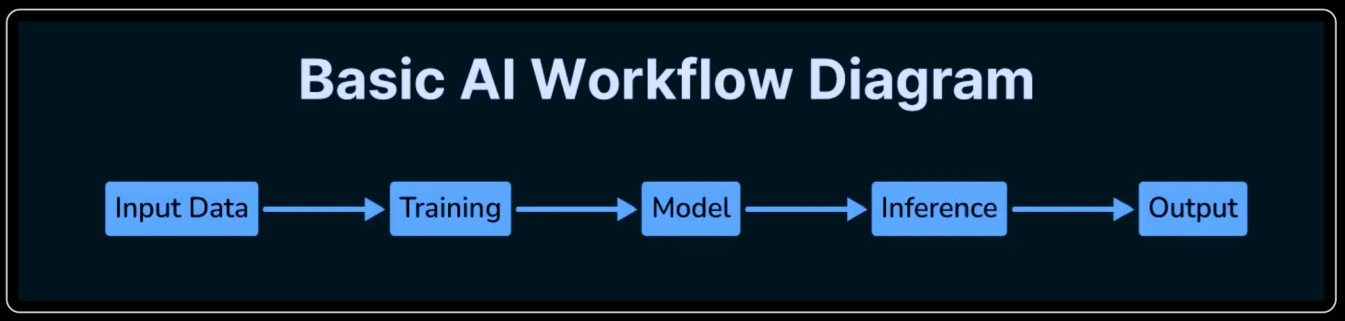
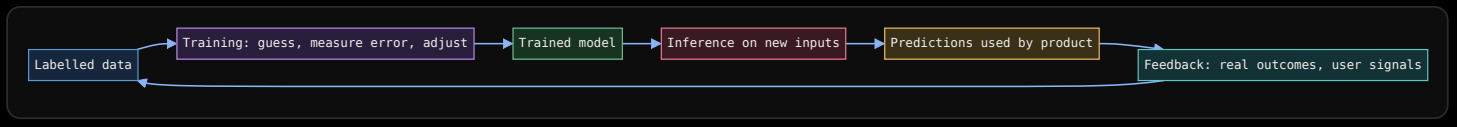


Figure — Data feeds training, which produces a model used for inference (course diagram).

1. **Training** — the model guesses on examples with known answers, measures how wrong it was, and nudges its internal numbers. After many rounds it predicts well on examples it has *not* seen.
2. **Inference** — the trained model is applied to new data (e.g. a voice assistant transcribing your command). No learning happens.
3. **Feedback loop** — in production, compare predictions with what actually happened, collect outcomes as fresh training data, retrain. A thumbs-down on a show becomes a new label for a recommender.



How it works (the core loop)

1. Initialise the **parameters** (the adjustable numbers, e.g. w and b in ŷ = w·x + b).
2. Predict for a batch (small group) of training examples.
3. Compute the **loss**: one number measuring how wrong the predictions are (e.g. mean squared error, the average squared difference between prediction and truth; worked numbers in §1.6).
4. Move the parameters in the direction that lowers the loss — **gradient descent**: repeatedly take a small step in the direction that reduces the loss fastest (Chapter 2).
5. Repeat until the loss stops improving.

Data is king

- **Garbage in, garbage out:** mislabelled, outdated or one-sided data yields a model with the same flaws, often hard to spot.
- **Diversity matters:** the *Gender Shades* study (Buolamwini and Gebru, 2018) found commercial face-analysis systems erred far more on darker-skinned women than lighter-skinned men, traced largely to unbalanced training and **benchmark** data (standard datasets used to compare models).
- **High stakes:** a diagnostic model trained on one population may misdiagnose another.

The goal is **generalisation**: performing well on new, real-world data, not just the training examples. Good, diverse data is the biggest lever.

Self-check

1. Does a model learn during inference? → Not by default; learning resumes on retraining (or with online learning, §1.7).
2. What is generalisation? → Performing well on data not seen in training.

TL;DR

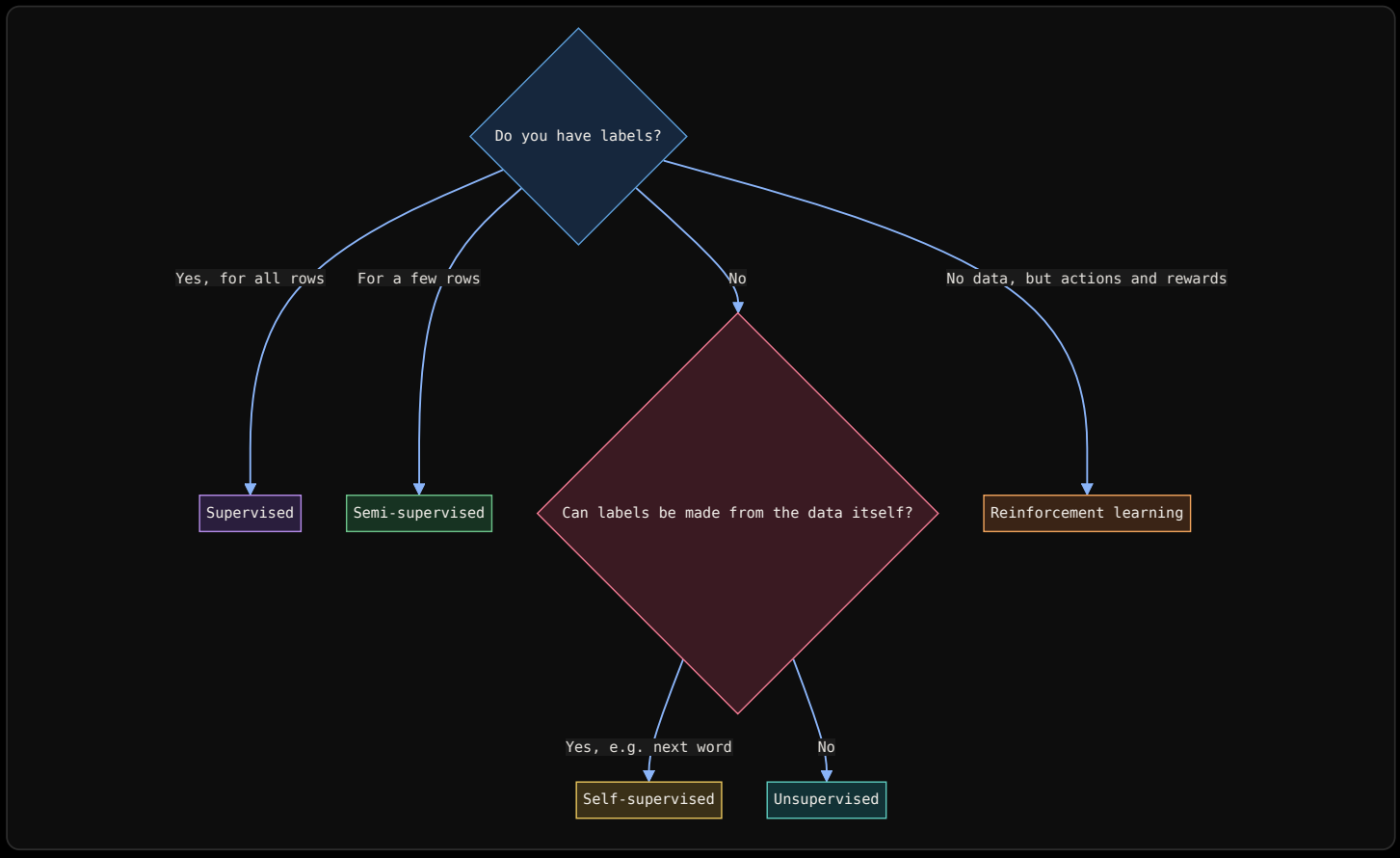
Training adjusts parameters to reduce loss on known examples; inference applies the model; feedback loops turn real outcomes into new training data. Data quality and diversity decide generalisation.

1.4 Types of learning by supervision

What it is

Grouped by *how much and what kind of answer* the system gets during training.

Type	Training data	Goal	Typical algorithms	Example
Supervised	Inputs **with** labels	Predict labels for new inputs	Linear/logistic regression, trees, random forests, gradient boosting, SVMs, neural nets	Classify new emails from emails marked spam/not spam
Unsupervised	Inputs **without** labels	Find structure: groups, low-dimensional summaries, anomalies	k-means, DBSCAN, PCA, isolation forest	Group customers; flag odd transactions
Semi-supervised	Few labelled + many unlabelled rows	Beat the labelled part alone	Self-training, label propagation	Photo app clusters faces, you name one per person, it names the rest
Self-supervised	Unlabelled data, labels *generated automatically*	Learn **representations** (internal numeric summaries of the input) by predicting a hidden part of the data	Next-word (GPT), masked-word (BERT), contrastive learning	LLMs pre-trained on next-token prediction
Reinforcement (RL)	No fixed dataset; an *agent* acts in an *environment* and gets *rewards*	Learn a *policy* maximising total reward	Q-learning, policy gradients, PPO	DeepMind's AlphaGo; robots learning to walk; RLHF for chatbots



Key terms (each covered later): *SVM* draws the widest boundary between classes; *k-means/DBSCAN* group similar points; *PCA* squeezes many features into a few summary ones; *isolation forest* flags easily separated points as anomalies. *Self-training/label propagation* spread a few labels to unlabelled rows; *BERT* fills in hidden words; *contrastive learning* teaches that two views of the same thing should look alike. In RL the *agent* is the learner, the *environment* its world, the *policy* its action rule; *Q-learning*, *policy gradients*, *PPO* are training methods; *RLHF* (RL from human feedback) uses human ratings as reward.

Why it matters

Labels are usually the most expensive part of a project (doctors labelling scans). The supervision type sets labelling cost, applicable algorithms and how you can evaluate.

How it works (one demo each)

`code/learning_types.py`:

```
# supervised: iris species (classification) and diabetes progression (regression)
clf = LogisticRegression(max_iter=1000).fit(Xtr, ytr)
reg = LinearRegression().fit(Xtr, ytr)
# unsupervised: k-means never sees the true groups
labels = KMeans(n_clusters=3, n_init=10, random_state=SEED).fit_predict(X)
# semi-supervised: hide 95% of labels (-1 = unlabelled), then self-train
self_train = SelfTrainingClassifier(LogisticRegression(max_iter=2000), threshold=0.9).fit(Xtr, y_partial)
# self-supervised: the "label" is the next value of the series
X = np.stack([series[i:i + 3] for i in range(len(series) - 3)]); y = series[3:]
# reinforcement: epsilon-greedy 3-armed bandit learns which arm pays best
arm = rng.integers(3) if rng.random() < 0.1 else int(np.argmax(estimates))
```

Output:

```
[classification] iris accuracy = 1.000
[regression]     diabetes MAE = 45.1 (target std = 77.0)
[unsupervised]   k-means agreement with hidden groups (ARI) = 0.767
[semi-supervised] 69 labels only: 0.867 | + self-training on unlabelled rows: 0.882
[self-supervised] next-value MAE = 0.068
[reinforcement]  estimated payouts [0.19, 0.46, 0.8], pulls [81, 72, 1847], avg reward 0.766
```

- *Supervised:* regression error (**MAE**, mean absolute error = average mistake size) of 45 is well below the target's **standard deviation** (std, how far a typical value sits from the average) of 77.
- *Unsupervised:* k-means recovered most of the hidden grouping without seeing it (**ARI**, adjusted Rand index: 1 = perfect agreement, 0 = random).
- *Semi-supervised:* with 69 labels, self-labelling confident rows and retraining added ~1.5 points; the gain is not guaranteed.
- *Self-supervised:* the future of the series served as the target; no human label.
- *Reinforcement:* a *multi-armed bandit* (slot machines with unknown payout rates). The *epsilon-greedy* agent usually pulls the best-looking arm but with probability epsilon (10%) a random one; it found arm 3 pays ~80% and spent most pulls there. The random pulls are the *exploration* cost.

Trade-offs

	Labelling cost	Evaluation	Main risk
Supervised	High	Easy: compare to labels	Label errors, bias
Unsupervised	None	Hard: no ground truth	Meaningless patterns
Semi-supervised	Low	Needs a labelled test set	Wrong pseudo-labels (self-assigned labels) reinforce themselves
Self-supervised	None, but huge data and compute	Via downstream tasks (the real tasks it is later adapted to)	Cost; web-scale biases
Reinforcement	Need reward and environment	Total reward	Reward hacking; slow, unstable training

Common mistakes

- Treating clusters as true categories without checking.
- An RL reward that can be "gamed" (a loophole earns reward without doing the task).
- Calling LLM pre-training "unsupervised": it is self-supervised — the target (next token) exists, generated from the data.

In the interview

30-second answer: "Supervised learning has labelled examples and predicts labels. Unsupervised has no labels and finds structure like clusters. Semi-supervised mixes a few labels with lots of unlabelled data. Self-supervised creates labels from the data itself, like predicting the next word, which is how large language models are pre-trained. Reinforcement learning learns actions from rewards by trial and error."

- *Is anomaly detection supervised?* — Usually unsupervised or semi-supervised (train mostly on normal data), because labelled anomalies are rare.
- *Exploration–exploitation trade-off?* — Trying actions to learn more (explore) vs using the best-known action (exploit).

- *How is ChatGPT-style training split?* — Self-supervised pre-training, supervised fine-tuning on example conversations, then RL from human (or AI) feedback.

Self-check

1. Fraud labels exist for 1% of transactions. Setup? → Semi-supervised (or supervised on the labelled part plus anomaly detection).
2. Why is unsupervised learning hard to evaluate? → No correct answer to compare against.

TL;DR

Labels → supervised; none → unsupervised; a few → semi-supervised; made from the data → self-supervised; rewards from actions → reinforcement.

1.5 Classification vs regression

What it is

The two main supervised tasks, split by label type:

- **Classification** predicts a **category**: spam/ham ("ham" = legitimate email), digit 0–9. Two classes = *binary*; more = *multiclass*; several labels per example (photo tagged "beach" and "dog") = *multilabel*.
- **Regression** predicts a **continuous number**: price, temperature, days until churn.

	Classification	Regression
Output	Class (often with a probability)	Real number
Example losses	Log loss (cross-entropy), hinge loss	MSE, MAE
Example metrics	Accuracy, precision, recall, F1, ROC-AUC	MAE, RMSE, R^2
Example algorithms	Logistic regression, tree classifier, k-NN classifier	Linear regression, tree regressor, k-NN regressor

Key terms: a *loss* is what training minimises; a *metric* is what you report. *Log loss* punishes confidently wrong probabilities; *hinge loss* (SVMs) punishes points on the wrong side of, or too close to, the boundary. *Accuracy* = share correct; *precision* = of flagged positives, share truly positive; *recall* = of true positives, share caught; *F1* balances the two; *ROC-AUC* = how well positives are ranked above negatives (1 perfect, 0.5 coin flip). *RMSE* = √MSE (target's units); R^2 = share of target variation explained (1 perfect, 0 = predicting the average). Full treatment in Chapter 6.

Logistic regression is a classifier: it predicts a class probability, then applies a *threshold* (cut-off, usually 0.5) above which the answer is "yes".

Common mistakes

- Encoding categories as numbers (red = 1, green = 2) and doing regression, inventing an order.
- Binning a regression target ("high/low" price) too early, discarding information.
- Judging accuracy on imbalanced data: if 99% of transactions are legitimate, "always legitimate" is 99% accurate and useless (Chapter 6).

Self-check

1. Star rating 1–5? → Either: regression respects order, classification treats ratings as categories; try both.

TL;DR

Category → classification; number → regression. The label type decides loss, metrics and algorithms.

1.6 Key terms

Term	Meaning	Example (house prices)
Example / instance / sample / row	One item in the dataset	One house sale
Feature (attribute, input, predictor, X)	Measurable property used as input	Size m², rooms, postcode
Label (target, output, y)	Answer to predict	Sale price
Feature vector	One example's features as numbers	[120, 3, 1995]
Model	Function with adjustable parts mapping features to a prediction	price = $w_1 \cdot$size + $w_2 \cdot$rooms + b
Parameters	Numbers *learned* in training	w_1, w_2, b
Hyperparameters	Settings *you choose before* training that control learning	Polynomial degree, regularisation strength C (how hard big weights are penalised), learning rate (training step size), number of trees
Loss (cost, objective)	Number measuring wrongness; training minimises it	Mean squared error
Training set	Data parameters are learned from	60–80%
Validation set	Data to compare models and tune hyperparameters	10–20%
Test set	Used once at the end to estimate real-world performance	10–20%, untouched before
Cross-validation	Rotating the validation role through k chunks ("folds") of training data and averaging	5-fold CV
Prediction (ŷ, "y-hat")	Model output	$412,000
Generalisation	Doing well on new data	Accurate prices next month
Inference	Using a trained model to predict	Pricing a new listing

Parameters vs hyperparameters

Training finds parameters (w, b in ŷ = w·x + b); you set hyperparameters (e.g. polynomial **degree**) and tune them on the *validation* set or with cross-validation, never on the test set.

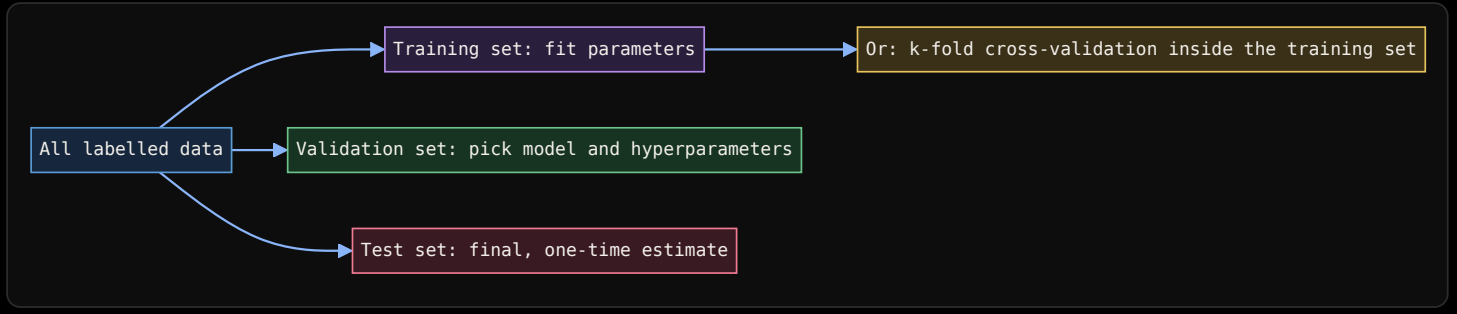
Loss, with worked numbers

True prices 300, 250, 400 ($1000s); predictions 310, 240, 380. Errors (true − predicted): −10, +10, +20.

- **MSE** = (100 + 100 + 400) / 3 = **200** ($1000², hard to read)
- **RMSE** = √200 ≈ **14.1** → "typically off by about $14k"
- **MAE** = (10 + 10 + 20) / 3 ≈ **13.3**

Squaring makes MSE punish the big error (20) much more.

Training, validation and test sets



Tuning while watching test performance slowly fits the test set and makes the reported score optimistic; the validation set absorbs tuning so the test set stays honest.

`code/train_val_test.py` splits 60/20/20, picks C by validation log loss, reports test accuracy once, then simulates *drift* (live data changing away from the training data) in production:

```
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, random_state=SEED)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=SEED)
for c in [0.001, 0.01, 1.0]:
    m = LogisticRegression(C=c).fit(X_train, y_train)
    loss = log_loss(y_val, m.predict_proba(X_val))
```

Output:

```
sizes train/val/test: 1800 600 600
C=0.001  validation log-loss = 0.529
C=0.01   validation log-loss = 0.287
C=1.0    validation log-loss = 0.063
chosen C=1.0; test accuracy = 0.998
after drift, live accuracy = 0.746  -> alert, retrain
after retraining on fresh labelled data, accuracy = 0.986
```

(0.25 of the remaining 80% = 20% of the total.) After drift accuracy fell from 99.8% to 74.6%; retraining on fresh data recovered it — why monitoring matters.

Self-check

1. Learning rate: parameter or hyperparameter? → Hyperparameter.
2. Why not tune on the test set? → Its score becomes biased upward.

TL;DR

Features in, label out. Training learns parameters by minimising a loss; hyperparameters are chosen with validation data or CV; the test set is used once.

1.7 Batch vs online learning

What it is

- **Batch (offline):** train on the full dataset, deploy the frozen model, retrain from scratch periodically (nightly, weekly).
- **Online (incremental):** feed mini-batches or single examples; the model updates as data arrives and old data can be discarded.
- **Out-of-core learning:** online learning used to train on data too large for memory by streaming chunks from disk.

Why it matters

Batch is simple and reproducible but slow to react and costly on huge data. Online adapts fast (stock prices, news trends, sensors) on limited memory but can drift badly if bad data arrives.

How it works

Initialise → for each arriving mini-batch, compute the loss and take one gradient-descent step → discard (or archive) the batch. The **learning rate** sets how strongly each batch moves the model: high adapts fast but forgets old patterns and chases **noise** (random variation with no real pattern); low is stable but slow.

`code/batch_vs_online.py` trains `SGDClassifier` (a linear classifier trained by **stochastic gradient descent**: updates after each small random batch instead of the whole dataset) both ways:

```
batch = SGDClassifier(average=True, random_state=SEED).fit(X[:16_000], y[:16_000])

online = SGDClassifier(average=True, random_state=SEED)
for start in range(0, 16_000, 2000):
    online.partial_fit(X[start:start + 2000], y[start:start + 2000], classes=[0, 1])
```

Output:

```
batch model (16,000 rows at once): test acc = 0.842
online after  2,000 rows: test acc = 0.833
online after  8,000 rows: test acc = 0.838
online after 16,000 rows: test acc = 0.839
```

The online model nearly matches batch while holding only 2,000 rows in memory. (`average=True` averages weights over updates; without it accuracy jumped between 0.66 and 0.83 here — online learners are sensitive to learning-rate

settings.) scikit-learn **estimators** (any model/transformer object with `fit`) supporting incremental learning expose `partial_fit`: `SGDClassifier`, `SGDRegressor`, `MultinomialNB`, `MiniBatchKMeans`.

Trade-offs

	Batch	Online
Adapts to change	At next retrain	Continuously
Memory / compute	Whole dataset per retrain	One chunk at a time
Reproducibility	High	Lower: depends on data order
Risk	Stale model	Bad data corrupts the live model quickly
Monitoring	Standard	Close watch on inputs and performance, plus a rollback plan

When not to use online learning

Data arrives slowly, changes are rare (weekly retrain suffices), or you cannot monitor inputs — poisoned (deliberately corrupted) or buggy data flows straight into the live model. Microsoft's Tay chatbot (2016) was taken offline within a day after users fed it offensive content it then repeated.

Self-check

1. Which scikit-learn method signals online support? → `partial_fit`.
2. Out-of-core learning? → Streaming data larger than memory in chunks through an online algorithm.

TL;DR

Batch: train on everything, redeploy periodically. Online: update continuously from small chunks; faster to adapt, needs careful monitoring.

1.8 Instance-based vs model-based learning

What it is

- **Instance-based:** memorise examples; predict from the most **similar** stored ones. E.g. **k-nearest neighbours (k-NN)** averages the labels of the k closest points.
- **Model-based:** fit a **compact model** (formula with parameters) and discard the examples. E.g. linear regression $\hat{y} = w \cdot x + b$.

How it works (worked example)

Data y ≈ 3x + 2 + noise, x in 0–10 (`code/batch_vs_online.py`):

```
lin = LinearRegression().fit(X, y)             # stores 2 numbers
knn = KNeighborsRegressor(n_neighbors=5).fit(X, y)  # stores all 5000 rows
x_new = np.array([[4.0], [15.0]])              # 15 lies outside 0..10
```

Output:

```
linear model: w=3.00, b=2.04 -> predicts [14. 47.]
k-NN (k=5): stores 5000 rows -> predicts [14.4 32. ]
prediction time for 10,000 queries: linear 0.22 ms vs k-NN 3.1 ms (exact times vary by machine)
```

At x = 4 both agree (true 14). At x = 15 the linear model **extrapolates** (predicts beyond the training range) correctly (47); k-NN can only average neighbours near x = 10 (≈ 32). The linear model is 2 numbers and much faster; k-NN searches all data at prediction time.

Trade-offs

	Instance-based (k-NN)	Model-based
Training	Almost instant	Takes time
Prediction	Slower; all data in memory	Fast, small memory
Flexibility	Any shape if data is dense	Limited by model form
Extrapolation	Poor	Possible if form is right
Needs	Good similarity measure; scaled features	Good model family

Self-check

1. Which approach makes new examples usable instantly without retraining? → Instance-based: add them to the store.

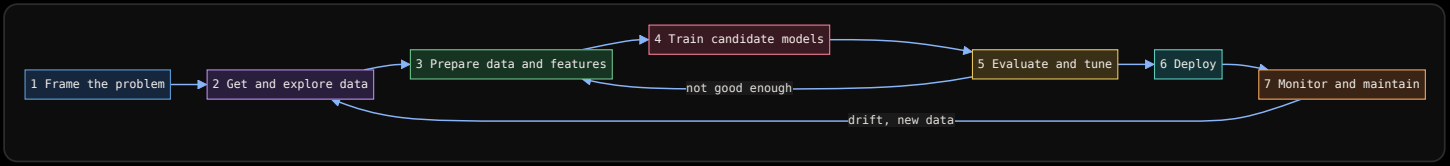
TL;DR

Instance-based learners compare to stored cases; model-based learners fit parameters and discard data. Most algorithms in this book are model-based.

1.9 The end-to-end ML workflow

What it is

A cycle in which modelling is usually the smallest part.



How it works

1. **Frame.** What decision does the prediction drive and what is it worth? Supervised? Classification/regression? Batch/online? Which metric reflects business value (fraud recall, dollars of price error)? What does the current solution achieve (the baseline)? Is ML needed at all?
2. **Get and explore data.** Check it is legal and ethical to use; look at distributions, missing values, **outliers** (values far from the rest) and **correlations** (how strongly two columns move together). **Set aside the test set now**, before deep exploration.
3. **Prepare data and features.** Clean, handle missing values, encode categories as numbers, scale features to comparable ranges, create informative features ("price per m²"). Put it all in a **pipeline** (a fixed chain of steps) so training and production apply it identically.
4. **Train candidates.** Start simple (linear), then try a few families (trees, ensembles); compare with cross-validation.
5. **Evaluate and tune.** Tune on validation data, analyse errors, check fairness across groups, then test *once*.
6. **Deploy** the whole pipeline as a batch job (scheduled scoring of many rows) or a web API (answers prediction requests over the network); roll out gradually — *shadow mode* (new model runs alongside, answers logged not used) or *A/B test* (a random share of users gets the new model). See System Design Fundamentals.
7. **Monitor and maintain.** Track input and prediction distributions and real performance as labels arrive. **Data drift**: inputs change; **concept drift**: the input→label relationship changes. Retrain, version models and data, keep a rollback path.

Real-world examples

- **Spam filtering:** every "Report spam" click is a new label.
- **Google Flu Trends** substantially overestimated the 2012–13 flu peak as search behaviour changed — drift without enough monitoring and recalibration.
- **Zillow Offers** (home buying on price predictions) shut down in 2021, citing difficulty forecasting prices in a fast-moving market.

Common mistakes

- Jumping to modelling without a metric or baseline.

- Preprocessing in a notebook and re-implementing it differently in production (**training–serving skew**: the model sees inputs unlike those it learned from).
- No monitoring: the model silently decays.

In the interview

> **30-second answer:** "I start by framing the problem: the decision, the metric, and a baseline. Then I collect and explore data, holding out a test set early. I build a preprocessing-plus-model pipeline, compare a few model families with cross-validation, tune on validation data and test once. Then I deploy the pipeline as a unit, and monitor drift and performance, retraining when needed."

- *When to retrain?* — Monitored performance drops as labels arrive, inputs shift noticeably from training, or on a schedule.

Self-check

1. Why hold out the test set before exploring? → To avoid tailoring choices to it (data snooping).
2. Why ship preprocessing and model together? → Production then applies exactly the training transformations.

TL;DR

Frame → data → features → model → evaluate → deploy → monitor, and loop. Baseline first, test once, pipeline everything, monitor always.

1.10 The main challenges of machine learning

Failures come from **bad data** or **a bad model**. `code/challenges.py` demonstrates each.

1. Not enough or poor-quality data

Too few examples, wrong labels, noise, missing values, outliers. **Demo** — flipping training labels on the breast-cancer dataset:

```
 0% wrong training labels -> test accuracy 0.958
20% wrong training labels -> test accuracy 0.951
40% wrong training labels -> test accuracy 0.636
```

Moderate random label noise partly cancels out but quickly becomes fatal; systematic errors (one annotator always mislabels a class) are worse because they do not cancel. **Fixes:** audit labels, more data, fix outliers, handle missing values deliberately.

2. Non-representative training data (sampling bias)

Training data unlike deployment data: convenience sampling, self-selection (only happy customers answer surveys), historical bias. **Demo** — trained only on houses under 120 m²:

```
trained on houses < 120 m^2: mean error small houses 14,086 | big houses 43,250
```

The relationship bends outside the seen range; error triples. **Real world:** the face-analysis gaps in §1.3; the 1936 *Literary Digest* poll predicted Landon over Roosevelt from ~2.4 million ballots — mailing lists leaned on telephone directories and car registrations, and only about a quarter replied, so both list and responses were skewed. **Fixes:** sample to match the deployment population, check per-subgroup performance, collect data where the model will run.

3. Irrelevant features

They add noise and invite **spurious patterns** (coincidences in the training data). **Feature engineering** — selecting, combining and creating features from domain knowledge — is often where most gains come from.

4. Overfitting

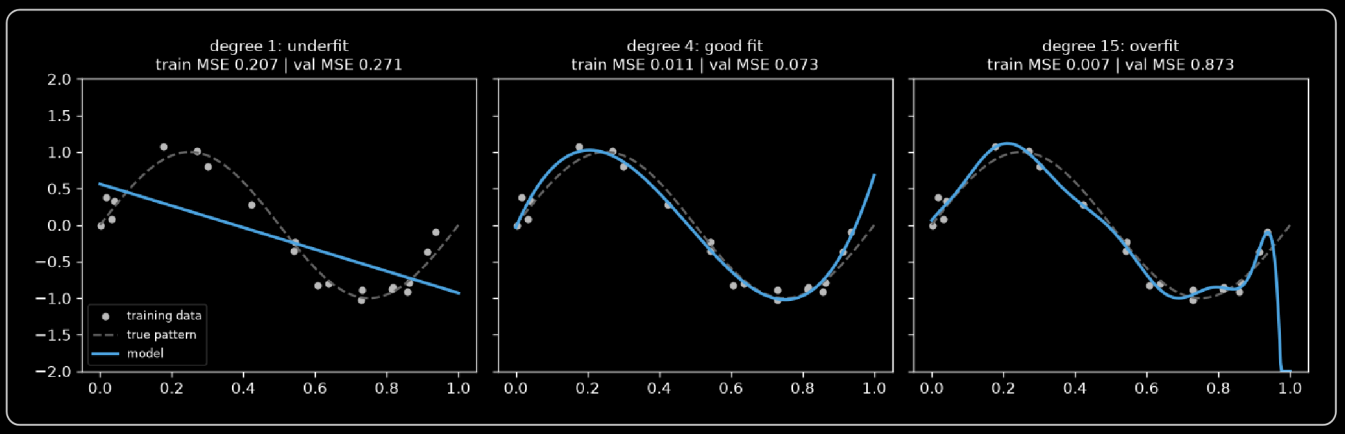
The model learns training data *too* well, noise included, and fails on new data (memorising past exams). **Signal:** low training error, much higher validation error. **Causes:** model too complex for the data, too many features, too little or noisy data. **Fixes:** simpler model, fewer features, **regularisation** (penalty on large parameters), more data, less noise, **early stopping** (halt when validation error stops improving), cross-validation to choose complexity.

5. Underfitting

Model too simple for the pattern. **Signal:** training *and* validation error both high. **Fixes:** more powerful model, better features, less regularisation, train longer.

Demo — polynomials of degree 1, 4, 15 on 20 noisy sine points:

```
degree  train MSE  val MSE
     1      0.207    0.271
     4      0.011    0.073
    15      0.007    0.873
```



Degree 15 has the *lowest* training error but 12× degree 4's validation error: it threads every noisy point and swings wildly between them. This is the **bias–variance trade-off**: simple models have high *bias* (systematic error), complex ones high *variance* (sensitivity to the particular sample) — Chapter 6.

Train error	Validation error	Diagnosis	First fixes
High	High	Underfitting	More capacity or better features
Low	High	Overfitting	Regularise, simplify, more data
Low	Low, close	Good fit	Test once, ship

6. Data leakage

Information unavailable at prediction time sneaks into training or evaluation, inflating scores.

- **Target leakage:** a feature that is a consequence of the label ("was prescribed antibiotics" for "has infection"; "account closed date" for churn).
- **Train–test contamination:** preprocessing (scaling, feature selection, **imputation** = filling missing values) fit on the *whole* dataset before splitting; random splits of **time-series** data (measurements in time order); the same patient in train and test.

Demo — 100 rows of random noise with random labels (honest answer 50%):

```python
# WRONG: select features using all rows, then cross-validate
X_leaky = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(max_iter=1000), X_leaky, y, cv=5).mean()
# RIGHT: selection inside the pipeline, refit on each training fold
pipe = make_pipeline(SelectKBest(f_classif, k=20), LogisticRegression(max_iter=1000))
right = cross_val_score(pipe, X, y, cv=5).mean()
```

```
leaky CV accuracy on pure noise: 0.85  |  honest CV accuracy: 0.48
```

85% accuracy on noise: selecting the 20 features best matching the labels *using the test folds* baked the answers in. **Real world:** Zech et al. (2018) found chest X-ray pneumonia models could recognise *which hospital* an image came from, which correlated with disease rates, so part of the skill did not transfer. **Fixes:** split first; put every data-dependent step in a `Pipeline`; split time series by time; group splits by patient/user; ask of every feature "would I know this at prediction time?"

When a result is too good to be true

Suspect leakage when a simple model scores near-perfectly, one feature dominates importance, or production falls far below test.

In the interview

30-second answer: "The main risks are data problems — too little, noisy, non-representative or leaky data — and model problems — overfitting, where the model memorises noise and generalises poorly, and underfitting, where it's

> too simple. I diagnose by comparing training and validation error, guard against leakage with pipelines and proper splits, and check performance across subgroups."

- *Detect overfitting?* — Low training error with much higher validation error; **learning curves** (train and validation error vs training-set size).
- *More data or a better model?* — Overfitting: more data helps. Underfitting: more data of the same kind barely helps; improve model or features.

Self-check

1. Why is fitting a scaler on the full dataset leakage? → Its mean and std include test-set information.
2. A churn model's top feature is "cancellation reason". Problem? → Target leakage: it exists only after churn.
3. How can a huge sample be biased? → Drawn from an unrepresentative population (Literary Digest).

TL;DR

Bad data (too little, noisy, non-representative, leaky) and bad models (overfit, underfit) cause most failures. Compare train vs validation error, use pipelines, split correctly, check each subgroup.

1.11 A first end-to-end project with scikit-learn

What it is

The whole workflow in `code/first_end_to_end.py` on the bundled **Breast Cancer Wisconsin** dataset: 569 tumour samples, 30 numeric cell-image features (radius, texture, smoothness, ...), label malignant (0) / benign (1).

Framing: supervised binary classification, batch. Missing a malignant tumour is worse than a false alarm, so we also care about **recall on the malignant class**.

How it works

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target

# 1. hold out a stratified test set first
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=SEED)

# 2. baseline: always predict the most common class
base = DummyClassifier(strategy="most_frequent").fit(X_train, y_train)

# 3. pipeline = scaling + logistic regression; cross-validate on training data
pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
cv = cross_val_score(pipe, X_train, y_train, cv=5)

# 4. tune the hyperparameter C with cross-validation
grid = GridSearchCV(pipe, {"logisticregression__C": [0.01, 0.1, 1, 10]}, cv=5)
grid.fit(X_train, y_train)

# 5. evaluate once on the test set
y_pred = grid.predict(X_test)

# 6. save the whole pipeline and reload it as a serving process would
joblib.dump(grid.best_estimator_, path)
served = joblib.load(path)
```

- `stratify=y` keeps the class ratio equal in train and test.
- `DummyClassifier` is the score to beat.
- `StandardScaler` rescales features to mean 0, std 1 (logistic regression trains better); inside the pipeline it is re-fit per training fold — no leakage.
- `GridSearchCV` tries each C with 5-fold CV and refits the best on all training data; `step__param` (`logisticregression__C`) addresses a parameter inside a pipeline step.
- `joblib` saves scaler + model as one file; load such files only from trusted sources, since unpickling (turning saved bytes back into Python objects) can run code.

Output:

```
shape: (569, 30) | class counts: [212, 357]
train: 455 test: 114
baseline accuracy (test): 0.632
5-fold CV accuracy: 0.980 +/- 0.013
best C: 0.1 | best CV accuracy: 0.980
test accuracy: 0.974
confusion matrix [rows=true, cols=pred]:
 [[40  2]
 [ 1 71]]
              precision    recall  f1-score   support

   malignant      0.976     0.952     0.964        42
      benign      0.973     0.986     0.979        72

    accuracy                          0.974       114
   macro avg      0.974     0.969     0.972       114
weighted avg      0.974     0.974     0.974       114

prediction for a new sample: malignant | P(benign) = 0.0
learned parameters: coef_ shape (1, 30) intercept [0.504]
```

- Baseline 63.2% (always "benign") → model 97.4%.
- CV (98.0%) and test (97.4%) are close: no sign of overfitting or leakage.
- The **confusion matrix** (counts of each true × predicted class pair) shows **2 malignant tumours predicted benign** (false negatives), the costly error; malignant recall = 40/42 = 0.952. Lowering the decision threshold would trade false alarms for fewer misses (Chapter 6).
- `coef_` = 30 learned weights, `intercept` = b; C = 0.1 is the chosen hyperparameter.
- *Support* = test examples per class; *macro avg* weights classes equally, *weighted avg* by support. `P(benign) = 0.0` is rounded.

Common mistakes

- Reporting the tuned model's CV score as final (slightly optimistic; the test set is the honest number).
- Forgetting `stratify` on small or imbalanced data.
- Saving the model without the scaler, then feeding unscaled data in production.

Self-check

1. What is a useless baseline for? → It sets the minimum bar; a model barely beating it is not worth deploying.

TL;DR

Split (stratified) → baseline → pipeline → cross-validate → tune → test once → save the whole pipeline. That skeleton carries through every chapter.

Chapter TL;DR

- **AI ⊃ ML ⊃ DL.** GenAI creates content; agentic AI plans and acts. Choose the simplest branch that works.
- **ML learns rules from examples**; use it when rules are too complex or change too often and you have good data.
- **Training** minimises a **loss**; **inference** applies the model; **feedback loops** feed outcomes back.
- **Supervision:** supervised, unsupervised, semi-supervised, self-supervised, reinforcement.
- **Classification** → categories; **regression** → numbers.
- **Batch** retrains periodically; **online** updates with `partial_fit`. **Instance-based** compares to stored examples; **model-based** fits parameters.
- **Workflow:** frame → data → features → model → evaluate → deploy → monitor, looped.
- **Parameters** are learned; **hyperparameters** chosen on validation data; **test set** used once.
- **Failures:** bad or non-representative data, overfitting (train ≪ validation error), underfitting (both high), leakage (too-good scores). Pipelines and correct splits prevent most.

Chapter 2 — The Math You Need for Machine Learning

Four areas of math appear in every model, library call and interview: **linear algebra** (how data and models are stored and combined), **calculus** (how models learn), **probability** (how models express uncertainty) and **statistics** (how we judge results). A small fifth, **information theory**, explains the most common classification **loss** (the formula scoring how wrong predictions are). Each idea comes with a plain definition, a hand calculation and runnable **NumPy** code (the core Python library for fast calculations on arrays of numbers).

All code is in `code/`. Notation: lowercase (x, w) = vectors, uppercase (X, A) = matrices, ŷ = prediction, Σ = "sum of", · = dot product.

2.1 Vectors

What it is

A **vector** is an ordered list of numbers, such as [2, 5, −3]. In ML **one data point is one vector**: a house with 120 m², 3 bedrooms, 2 bathrooms is x = [120, 3, 2]. Each position is a **feature**; the count of positions is the **dimension**. Geometrically it is an arrow from the origin to a point: a 784-pixel image is a point in 784-D space.

How it works

Operation	Rule	Example, a = [1, 2, 3], b = [4, 5, 6]
Addition	Position by position	a + b = [5, 7, 9]
Scalar (single-number) multiplication	Multiply every entry	2a = [2, 4, 6]
Dot product a · b	Multiply position by position, sum	1·4 + 2·5 + 3·6 = **32**
L2 norm $\|a\|_2$ (length)	√(sum of squares)	√14 ≈ 3.74
L1 norm $\|a\|_1$	Sum of absolute values	6
Max norm $\|a\|_\infty$	Largest absolute value	3
Unit vector	a / $\|a\|_2$	[0.27, 0.53, 0.80]
Cosine similarity	(a · b) / (‖a‖ ‖b‖)	32 / (3.74 · 8.77) ≈ 0.975

The dot product is the most important operation in ML. A *linear model* (prediction = weighted sum of features) predicts $\hat{y} = w \cdot x + b = w_1x_1 + w_2x_2 + w_3x_3 + b$. With w = [2, 10, 5] and x = [120, 3, 2]: w · x = 240 + 30 + 10 = **280**.

Geometrically a · b = ‖a‖ ‖b‖ cos θ (θ = angle between the arrows): large when vectors point the same way, zero when perpendicular, negative when opposite. **Cosine similarity** keeps only the angle; search and recommender systems use it to compare **embeddings** (learned vectors representing meaning).

Norms measure size; the distance between points is $\|a - b\|_2$. L1 and L2 norms of the *weights* are **regularisation** penalties (extra loss terms discouraging large weights, to reduce overfitting): Lasso regression uses L1, Ridge uses L2.

Code

`code/linear_algebra.py`:

```
house = np.array([120.0, 3.0, 2.0])
w = np.array([2.0, 10.0, 5.0])
print("dot w.x =", w @ house)                       # @ is the dot / matrix product
a, b = np.array([1.0, 0.0]), np.array([1.0, 1.0])
cos = a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
v = np.array([3.0, -4.0])
print(np.linalg.norm(v), np.linalg.norm(v, 1), np.linalg.norm(v, np.inf))
```

Output:

```
x = [120.   3.   2.] | w = [ 2. 10.  5.]
x + w = [122.  13.   7.] | 2 * x = [240.   6.   4.]
dot w.x = 280.0 (= 2*120 + 10*3 + 5*2)
cosine similarity of [1,0] and [1,1] = 0.707 (angle 45 deg)
L2 norm of [3,-4] = 5.0 | L1 norm = 7.0 | max norm = 4.0
unit vector = [ 0.6 -0.8]
```

Common mistakes

- `a * b` in NumPy is element-wise (returns a vector); `a @ b` / `np.dot` is the dot product.
- Comparing distances when features have wildly different scales (m^2 in hundreds, bedrooms in units): the large one dominates. Standardise first (§2.8).

Self-check

1. Why cosine over dot product for documents? → It ignores length (document size) and compares direction (content).

TL;DR

A data point is a vector. The dot product is how a linear model combines features; norms measure size and distance; cosine similarity measures direction.

2.2 Matrices and matrix multiplication

What it is

A **matrix** is an m × n grid of numbers. **A dataset is a matrix** (rows = examples, columns = features); so are weights (a layer mapping 3 inputs to 2 outputs has a 2 × 3 matrix).

How matrix multiplication works

A (m × n) times B (n × p): inner sizes must match; the result is m × p; entry (i, j) = **row i of A · column j of B**.

```
A = | 1 2 |     B = | 5 6 |
    | 3 4 |         | 7 8 |

(A B)[0,0] = 1·5 + 2·7 = 19      (A B)[0,1] = 1·6 + 2·8 = 22
(A B)[1,0] = 3·5 + 4·7 = 43      (A B)[1,1] = 3·6 + 4·8 = 50
```

Order matters: **AB ≠ BA** in general (here BA = [[23, 34], [31, 46]]).

Batch prediction is one matrix product: ŷ = Xw + b, each row of X dotted with w. Libraries run this as optimised CPU/GPU code (a GPU is a graphics chip doing thousands of simple calculations in parallel), so ML code avoids Python loops. A neural-network layer computes h = activation(Wx + b) (an *activation function* is a simple non-linear function, e.g. "replace negatives with 0"; without it stacked layers collapse into one linear map). Deep networks are long chains of matrix multiplications, which is why GPUs power deep learning.

Transpose, identity, inverse, determinant

Concept	Definition	Example / use
Transpose A^T	Swap rows and columns (n × m)	$[[1, 2], [3, 4]]^T = [[1, 3], [2, 4]]$; in gradients and covariance (how features vary together)
Identity I	1s on diagonal, 0s elsewhere; AI = A	The "1" of matrices
Inverse A^{-1}	$A A^{-1} = I$ (square matrices with det ≠ 0 only)	"Undoes" A; solves linear equations
Determinant det(A)	How A scales area/volume; 0 = squashes space flat, no inverse	2 × 2: ad − bc = 1·4 − 2·3 = **−2**

2 × 2 inverse: A^{-1} = (1/det) · [[d, −b], [−c, a]] = (1/−2) · [[4, −2], [−3, 1]] = [[−2, 1], [1.5, −0.5]].

Solving equations. $x_1 + 2x_2 = 5$, $3x_1 + 4x_2 = 11$ is Ax = c, so $x = A^{-1}c$ = [1, 2]. In code use `np.linalg.solve(A, c)`, not the inverse: faster and numerically more accurate. The **closed-form** (direct formula, no iterative search) linear-regression solution, the **normal equation** $w = (X^TX)^{-1}X^Ty$, is solved the same way; scikit-learn's `LinearRegression` uses a **least-squares** solver (finds weights minimising the sum of squared errors) based on matrix decompositions (rewriting a matrix as a product of simpler ones), not an explicit inverse.

Code

```
A = np.array([[1, 2], [3, 4]]); B = np.array([[5, 6], [7, 8]])
A @ B            # matrix product
A * B            # element-wise product (different!)
A.T              # transpose
np.linalg.det(A); np.linalg.inv(A)
np.linalg.solve(A, np.array([5.0, 11.0]))
X @ w + b        # predictions for all houses at once
```

Output:

```
X shape: (4, 3) | predictions X @ w + b = [330. 235. 505. 270.]
A @ B =
 [[19 22]
 [43 50]]
B @ A =
 [[23 34]
 [31 46]]
(A @ B != B @ A: order matters)
A * B (element-wise) =
 [[ 5 12]
 [21 32]]
A.T =
 [[1 3]
 [2 4]]
det(A) = -2.0
inv(A) =
 [[-2.   1. ]
 [ 1.5 -0.5]]
A @ inv(A) =
 [[1. 0.]
 [0. 1.]]
solve A x = [5, 11] -> [1. 2.]
layer output W @ x = [2.2 0.3]
```

Common mistakes

- **Shape errors.** Write shapes first: (4 × 3)(3,) → (4,). Mismatched inner sizes mean a missing transpose or the wrong matrix.
- Inverting nearly singular matrices (det ≈ 0), e.g. two features almost copies of each other (*multicollinearity*): results blow up. Regularisation or `solve` / `lstsq` helps.

In the interview

30-second answer: "A dataset is a matrix with examples as rows and features as columns, and a linear model's predictions for the whole dataset are one matrix-vector product, Xw + b. Neural network layers are matrix multiplications followed by nonlinearities (activation functions), which is why GPUs, which are very fast at matrix math, dominate deep learning."

- *Is matrix multiplication commutative?* — No; AB ≠ BA in general, and shapes may not even allow BA.
- *When does a matrix have no inverse?* — Not square, or det = 0 (columns linearly dependent: one is a weighted sum of others).

Self-check

1. det([[2, 1], [4, 2]])? → 0: no inverse (rows are multiples).

TL;DR

A dataset is a matrix; predictions are Xw + b. Multiplication = rows dotted with columns, order matters. Transpose swaps axes; the inverse exists only when det ≠ 0.

2.3 Eigenvalues, eigenvectors and SVD (intuition)

What it is

A matrix usually rotates and stretches a vector. An **eigenvector** of square A is a direction A only stretches (or flips): A v = λ v; λ is the **eigenvalue**.

Example: A = [[2, 1], [1, 2]], v = [1, 1]: A v = [3, 3] = 3v → eigenvalue 3. The other eigenvector is [1, −1] with eigenvalue 1.

Singular value decomposition (SVD) works for any matrix: $M = U \Sigma V^T$, "rotate (V^T), stretch along axes (Σ), rotate (U)". The **singular values** on Σ's diagonal, largest first, say how much each direction matters.

Why it matters

- **PCA (principal component analysis)**, the standard **dimensionality reduction** method (many features → a few that keep most information), takes eigenvectors of the data's **covariance matrix** (how every pair of features varies together) — equivalently the SVD of the centred data (column means subtracted). Top eigenvectors are the directions of most variation; eigenvalues say how much variance (§2.6) each captures.

- **Compression/denoising:** keeping the top k singular values gives the best **rank-k approximation** (closest matrix built from only k independent directions) — Eckart–Young theorem.
- **Recommenders:** matrix factorisation, made famous by the Netflix Prize (2006–2009), approximates the user × movie ratings matrix as a product of two thin matrices, an idea closely related to SVD.
- **PageRank**, Google's original ranking, is the leading eigenvector of a matrix built from the web's link graph.

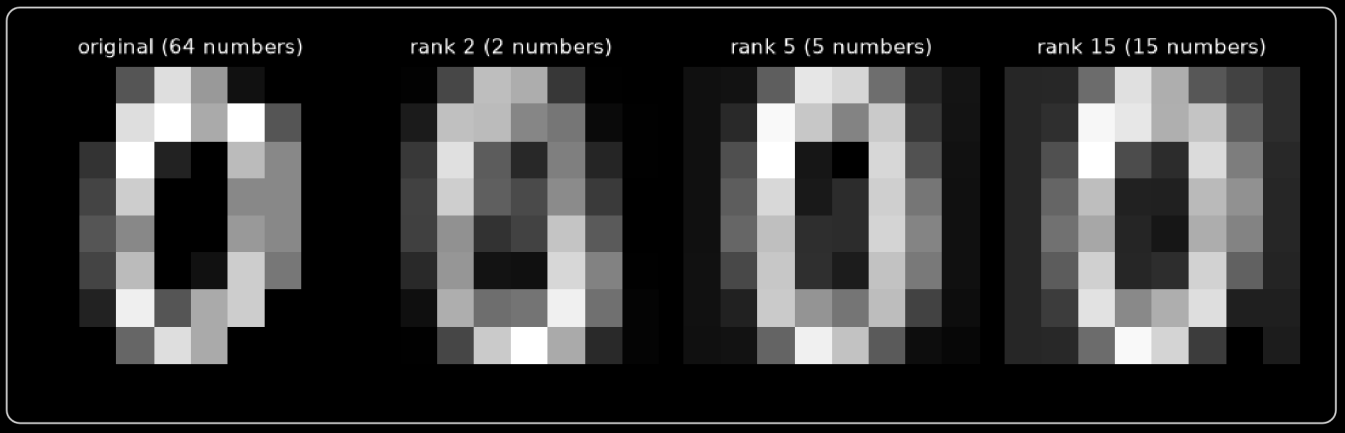
Code

```
vals, vecs = np.linalg.eigh(np.array([[2.0, 1.0], [1.0, 2.0]]))  # eigh: symmetric matrices
data = rng.multivariate_normal([0, 0], [[3, 2], [2, 2]], size=2000)
vals, vecs = np.linalg.eigh(np.cov(data, rowvar=False))          # PCA in two lines
U, s, Vt = np.linalg.svd(pixels - pixels.mean(0), full_matrices=False)
approx = (x @ Vt[:k].T) @ Vt[:k] + pixels.mean(0)                 # keep top-k directions
```

Output:

```
eigenvalues: [1. 3.]
eigenvectors (columns):
 [[-0.707  0.707]
 [ 0.707  0.707]]
A @ v = [2.121 2.121] = 3 * v = [2.121 2.121]
covariance eigenvalues: [0.433 4.584] -> share of variance on main axis: 0.914
singular values: [3.464 3.162]
rebuild U @ diag(s) @ Vt =
 [[ 3.  1.  1.]
 [-1.  3.  1.]]
top  5 of 64 singular values keep 54.5% of the variance
top 10 of 64 singular values keep 73.8% of the variance
top 20 of 64 singular values keep 89.4% of the variance
```

Eigenvectors come normalised ([1, 1] → [0.707, 0.707]). In the 2-D cloud one direction holds 91% of the variance; on 8 × 8 digit images (64 pixels), 20 directions keep 89%:



Common mistakes

- PCA without centring (and usually standardising).
- `np.linalg.eig` on symmetric matrices like covariance; `eigh` is faster, more stable and returns real, sorted eigenvalues.

TL;DR

Eigenvectors are directions a matrix only stretches; eigenvalues are the amounts. SVD factors any matrix into rotate–stretch–rotate. They power PCA, compression and recommenders.

2.4 Derivatives, partial derivatives and gradients

What it is

- A **derivative** f'(x) is the slope at a point. f(x) = x² → f'(x) = 2x; at x = 3, nudging x by 0.001 raises f by ≈ 0.006.
- A **partial derivative** $\partial f/\partial x$ is the slope along one input, others held fixed.
- The **gradient** ∇f is the vector of all partials; it points to steepest *increase*, its negative downhill.

Why it matters

The **loss** is a function of the parameters; its gradient tells each parameter which way and how much to move to reduce error. That is "learning" for most models, from linear regression to GPT-scale networks.

How it works: rules you actually need

Rule	Formula	Example
Constant	$(c)' = 0$	$(7)' = 0$
Power	$(x^n)' = n\,x^{n-1}$	$(x^3)' = 3x^2$
Sum	$(f + g)' = f' + g'$	$(x^2 + 3x)' = 2x + 3$
Constant multiple	$(c\,f)' = c\,f'$	$(5x^2)' = 10x$
Exponential / log	$(e^x)' = e^x$, $(\ln x)' = 1/x$	
Chain rule	$(f(g(x)))' = f'(g(x)) \cdot g'(x)$	$((3x + 1)^2)' = 2(3x + 1) \cdot 3$
Sigmoid (S-curve squashing any number into 0–1)	$\sigma(z) = 1/(1 + e^{-z})$, $\sigma'(z) = \sigma(z)(1 - \sigma(z))$	$\sigma(0) = 0.5$, $\sigma'(0) = 0.25$

Partials, worked. $f(x, y) = x^2y + 3y$ at $(2, 5)$: $\partial f/\partial x = 2xy =$ **20**; $\partial f/\partial y = x^2 + 3 =$ **7**; $\nabla f = [20, 7]$.

The chain rule (the engine of backpropagation)

For nested functions, multiply local slopes along the chain. A one-weight **neuron** (weighted input, then activation) with squared error: $z = w \cdot x \to \hat{y} = \sigma(z) \to L = (\hat{y} - y)^2$, with $x = 2$, $y = 1$, $w = 0.5$:

1. Forward: $z = 1.0$, $\hat{y} = \sigma(1) = 0.7311$, $L = 0.0723$
2. Local slopes: $dL/d\hat{y} = 2(\hat{y} - y) = -0.5379$; $d\hat{y}/dz = \hat{y}(1 - \hat{y}) = 0.1966$; $dz/dw = x = 2$
3. Chain: $dL/dw = -0.5379 \times 0.1966 \times 2 =$ **−0.2115**

Negative → increase w to reduce the loss ($\hat{y}$ is below the target). **Backpropagation** is this, applied efficiently to millions of weights from the output layer backwards, reusing shared intermediate products. PyTorch and similar libraries do it automatically (*autograd*, automatic differentiation: record each operation, derive gradients for you).

Code: check every derivative numerically

`code/calculus.py` compares hand formulas with the **central difference** $f'(x) \approx (f(x + h) - f(x - h)) / 2h$, a standard "gradient check":

```
def numerical_derivative(f, x, h=1e-5):
    return (f(x + h) - f(x - h)) / (2 * h)

# gradient of MSE for linear regression, vectorised: (2/n) X^T (Xw - y)
grad = 2 / len(yv) * X.T @ (X @ wv - yv)
```

Output:

```
d/dx x^2 at x=3: numeric 6.0000, exact 6
gradient of x^2*y + 3y at (2,5): numeric [20.  7.] exact [20. 7.]
d/dx (3x+1)^2 at x=2: numeric 42.0000, exact 42
z=1.0, y_hat=0.7311, dL/dyhat=-0.5379, dyhat/dz=0.1966, dz/dw=2.0
dL/dw via chain rule -0.21151 | numeric -0.21151
MSE gradient: formula [-4.         -9.33333333] | numeric [-4.     -9.3333]
```

The MSE gradient (MSE = mean squared error), seen everywhere: for $L(w) = (1/n)\,\Sigma\,(x_i \cdot w - y_i)^2$, $\nabla L = (2/n)\,X^T(Xw - y)$ — each example's error times its features, averaged. Here X has a column of 1s (so $w[0]$ is the **bias**, the constant b shifting every prediction), $y = [2, 4, 6]$, $w = [0, 1]$: errors $[-1, -2, -3]$ give $[2/3 \cdot (-6), 2/3 \cdot (-14)] = [-4, -9.33]$.

Common mistakes

- Dropping the inner function's factor in the chain rule (the "· 3" in $((3x + 1)^2)'$).
- The gradient points **uphill**; we step along **minus** the gradient.

Self-check

1. Why does $\sigma'(z)$ get tiny for large $|z|$? → σ flattens near 0 and 1, so $\sigma(1 - \sigma) \to 0$: the "vanishing gradient" problem in deep nets.

TL;DR

Derivative = slope; partial = slope along one input; gradient = vector of partials pointing uphill. The chain rule multiplies slopes through nested functions — that is backpropagation.

2.5 Gradient descent

What it is

Gradient descent minimises a function: start somewhere, compute the gradient, step the opposite way, repeat (walking down a foggy hill by the steepest slope underfoot). Update: $w \leftarrow w - \eta \cdot \nabla L(w)$, where η (eta) is the **learning rate**, a **hyperparameter** (set before training, not learned) controlling step size.

Worked by hand

Minimise $f(w) = (w - 3)^2$ (minimum at 3); $f'(w) = 2(w - 3)$; start $w = 0$, $\eta = 0.1$.

Step	w	f(w)	gradient 2(w − 3)	new w = w − 0.1 · gradient
0	0.0000	9.0000	−6.0000	0 + 0.6 = 0.6000
1	0.6000	5.7600	−4.8000	0.6 + 0.48 = 1.0800
2	1.0800	3.6864	−3.8400	1.4640
3	1.4640	2.3593	−3.0720	1.7712
4	1.7712	1.5099	−2.4576	2.0170

Each step closes 20% of the remaining distance ($w - 3$ is multiplied by $1 - 2\eta = 0.8$); steps shrink as the slope flattens.

The learning rate



- **Too small (0.02):** after 12 steps w is only 1.16.
- **Good (0.3):** reaches 3.00 in 12 steps.
- **Too large (1.05):** each step overshoots further; w diverges (−6.42 and growing). Here any $\eta > 1$ diverges because $|1 - 2\eta| > 1$.

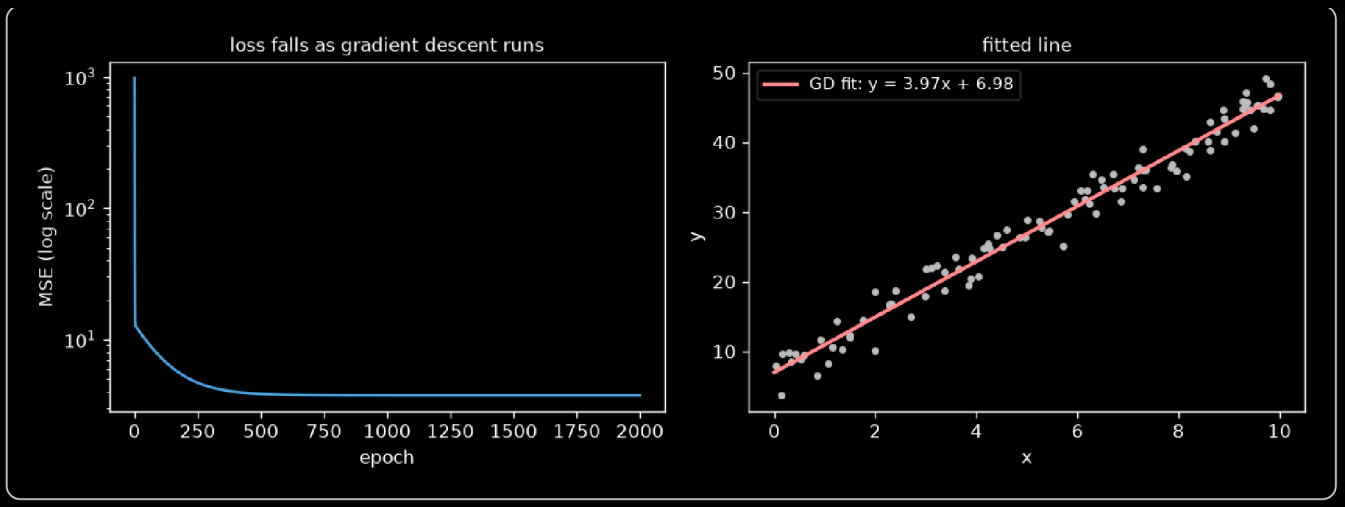
Gradient descent for linear regression

`code/gradient_descent.py` fits $y = w \cdot x + b$ to 100 noisy points from $y = 4x + 7$:

```
w, b, lr = 0.0, 0.0, 0.01
for epoch in range(2000):
    err = (w * x + b) - y
    dw = 2 * np.mean(err * x)      # dMSE/dw
    db = 2 * np.mean(err)          # dMSE/db
    w -= lr * dw
    b -= lr * db
```

Output:

```
epoch    0: w=0.000 b=0.000 MSE=976.742
epoch    1: w=3.885 b=0.575 MSE=51.369
epoch   10: w=4.820 b=0.966 MSE=12.240
epoch  100: w=4.533 b=3.007 MSE=7.464
epoch 1999: w=3.974 b=6.984 MSE=3.760
scikit-learn closed form: w=3.974 b=6.985
```



It matches scikit-learn's exact solver. w overshoots early while b creeps up: the **loss surface** (loss as a landscape over parameter values) is a long narrow valley because x (0–10) and the constant 1 have different scales. **Standardising features** rounds the valley and speeds convergence.

Variants

Variant	Gradient on	Pros	Cons
Batch GD	All n examples per step	Smooth, exact gradient	Slow per step on big data
Stochastic GD (SGD)	One random example	Very cheap steps; noise can escape shallow **local minima** (dips lowest only nearby)	Noisy; needs a decaying learning rate
Mini-batch GD	Small batch (e.g. 32–512)	Best of both; uses GPU parallelism	Extra hyperparameter (batch size)

One pass over the training set is an **epoch**. Deep-learning **optimisers** (update rules) such as Momentum, RMSProp and Adam are mini-batch GD with adaptive step sizes.

Convex vs non-convex. A convex (bowl-shaped) loss — MSE for linear regression, log loss for logistic regression — has no separate local minima, so GD with a suitable η reaches the global minimum. Neural-network losses are non-convex with many local minima and **saddle points** (flat spots curving up in some directions, down in others); mini-batch methods still find good solutions in practice.

When not to use it

A closed-form solver exists and data fits in memory (small linear regression). GD shines with large data, complex models or no closed form.

Common mistakes

- Unscaled features → slow, zig-zagging convergence.
- η too large → loss grows or becomes NaN ("Not a Number", from invalid maths like ∞ − ∞). First fix: divide η by 10.
- Not shuffling between SGD epochs; stopping too early (plot the loss curve).

In the interview

30-second answer: "Gradient descent minimises a loss by repeatedly moving parameters a small step opposite the gradient: $w \leftarrow w - \eta \nabla L$. The learning rate η controls step size; too small is slow, too large diverges. In practice we use mini-batch stochastic gradient descent, often with an adaptive optimiser like Adam, and we scale features so it converges quickly."

- *Why mini-batches?* — Cheaper than full-batch, less noisy than single examples, maps well to GPUs.
- *Always the global minimum?* — Guaranteed only for convex problems with a suitable η.

Self-check

1. One step on $f(w) = w^2$ from $w = 4$, $\eta = 0.25$? → gradient 8, $w = 2$.

TL;DR

$w \leftarrow w - \eta \nabla L$. Pick η carefully, scale features, use mini-batches, watch the loss curve.

2.6 Random variables and distributions

What it is

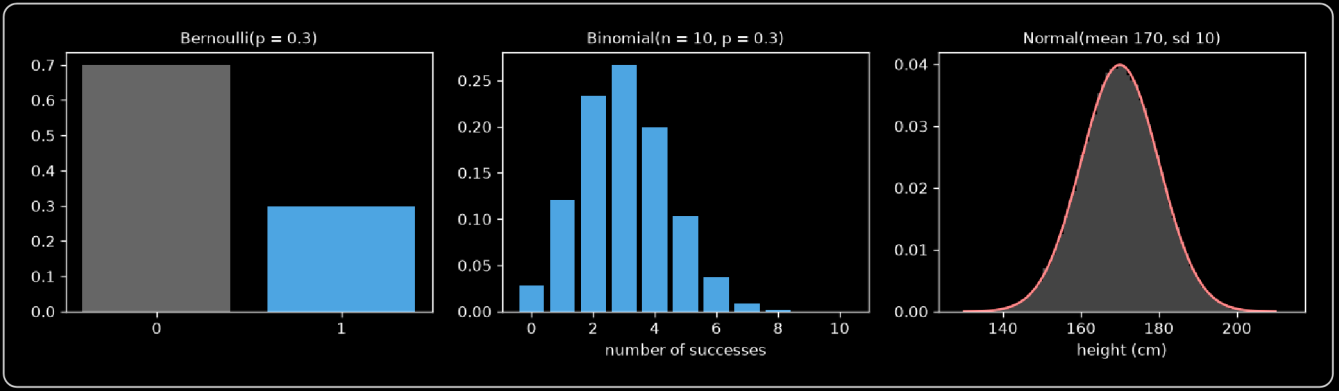
Probability is a number from 0 (impossible) to 1 (certain), e.g. P(spam) = 0.2. A **random variable** takes its value from a random process (die roll, spam or not, tomorrow's temperature): **discrete** (countable values) or **continuous** (any value in a range). A **probability distribution** says how likely each value is: a *probability mass function (PMF)* for discrete, a *probability density function (PDF)* for continuous (probabilities are areas under it).

The three distributions you'll meet most

Distribution	Models	Parameters	Mean	Variance	ML use
Bernoulli	One yes/no trial	p = P(1)	p	p(1 − p)	Binary label; logistic regression outputs p
Binomial	Successes in n independent trials	n, p	np	np(1 − p)	Clicks out of n ad impressions; A/B tests (comparing two versions on random user groups)
Normal (Gaussian)	Values clustered symmetrically around a mean	μ (mean), σ (std)	μ	σ^2	Measurement noise, heights, linear-regression errors; weight initialisation

Binomial: P(exactly 3 of 10, p = 0.3) = C(10, 3) · 0.3^3 · 0.7^7 = 120 · 0.027 · 0.0824 ≈ **0.2668** (C(10, 3) = 120 ways to pick the 3 successes).

68–95–99.7 rule: ~68% of normal values lie within 1σ, 95% within 2σ, 99.7% within 3σ. Heights ~ Normal(170, 10): P(height > 190) = P(z > 2) ≈ **2.3%**.



Expectation and variance

- **Expectation (mean)** E[X] = Σ x · P(x), the long-run average. Fair die: **3.5**.
- **Variance** Var[X] = E[(X − E[X])²], the average squared distance from the mean. Die: 17.5/6 ≈ **2.917**.
- **Standard deviation** σ = √Var, in X's units (die ≈ 1.71).

Rules: E[aX + b] = aE[X] + b; Var[aX + b] = a^2Var[X]; independent X, Y: Var[X + Y] = Var[X] + Var[Y].

Code

`code/probability.py` simulates each distribution against theory:

```
flips = rng.random(100_000) < 0.3                # Bernoulli(0.3)
draws = rng.binomial(10, 0.3, 100_000)           # Binomial(10, 0.3)
heights = rng.normal(170, 10, 100_000)           # Normal(170, 10)
stats.binom.pmf(3, 10, 0.3); 1 - stats.norm.cdf(190, 170, 10)
```

Output:

```
Bernoulli(0.3): sample mean 0.299 (theory 0.3), variance 0.210 (theory p(1-p) = 0.21)
Binomial(10, 0.3): P(X=3) = C(10,3) 0.3^3 0.7^7 = 0.2668 | scipy 0.2668
  mean 2.994 (theory np = 3.0), variance 2.093 (theory np(1-p) = 2.1)
Normal(170, 10): within 1 sd 0.683, within 2 sd 0.954 (theory 0.683, 0.954)
  P(height > 190) = 0.0228  (z = (190-170)/10 = 2)
die: E[X] = 3.500, Var[X] = 2.917 | simulated mean 3.510, var 2.925
```

Common mistakes

- Reading a PDF value as a probability: for continuous variables P(X = exactly 170) = 0; only ranges have probability.

- Assuming normality. Incomes, session lengths and prices are usually **right-skewed** (mostly small, long tail of large values); a log transform often helps.

Self-check

1. Mean and variance of Binomial(100, 0.1)? → 10 and 9.
2. Var[3X] if Var[X] = 2? → 18.

TL;DR

Bernoulli = one yes/no, binomial = count of yeses, normal = symmetric bell. Expectation = long-run average; variance = average squared spread.

2.7 Conditional probability and Bayes' theorem

What it is

Conditional probability P(A | B) = P(A and B) / P(B): the probability of A given B happened. Of 1,000 emails, 200 are spam; 120 spam and 40 normal emails contain "free": P(spam | free) = 120 / 160 = **0.75**, versus the base rate 0.2. A and B are **independent** if P(A | B) = P(A).

Bayes' theorem: P(A | B) = P(B | A) · P(A) / P(B), i.e. **posterior = likelihood × prior / evidence**:

- **Prior** P(A): belief before the data.
- **Likelihood** P(B | A): probability of the data if A is true.
- **Evidence** P(B) = P(B | A)P(A) + P(B | not A)P(not A).
- **Posterior** P(A | B): updated belief.

Worked example: a medical test

Disease prevalence 1%; the test catches 90% of sick people (**sensitivity** P(+ | sick) = 0.90) and flags 5% of healthy ones (false-positive rate 0.05, **specificity** 0.95). You test positive:

1. Prior P(sick) = 0.01
2. Evidence P(+) = 0.90 · 0.01 + 0.05 · 0.99 = **0.0585**
3. Posterior P(sick | +) = 0.009 / 0.0585 = **0.154**

Only ~15%. Of 10,000 people, 100 are sick (90 test positive) and 9,900 healthy (495 test positive): only 90 of 585 positives are sick. This **base-rate fallacy** matters for every rare-event classifier (fraud, rare disease, intrusion detection). Bayes chains: after a second independent positive, the old posterior is the new prior: 0.9 · 0.154 / (0.9 · 0.154 + 0.05 · 0.846) ≈ **0.766**.

```
evidence = sens * prior + fpr * (1 - prior)
posterior = sens * prior / evidence
```

Output (including a 1,000,000-person simulation):

```
P(free) = 0.16, P(spam | free) = 120/160 = 0.75
P(positive) = 0.9*0.01 + 0.05*0.99 = 0.0585
P(disease | positive) = 0.009 / 0.0585 = 0.154
simulation: P(disease | positive) = 0.153
after a second independent positive test: 0.766
```

Real-world examples

- **Naive Bayes spam filters** apply Bayes assuming words are independent given the class; Paul Graham's 2002 essay "A Plan for Spam" popularised them.
- **Calibration:** a fraud score of 0.9 should mean 90% of such cases are fraud; Bayes explains why scores trained on balanced samples must be adjusted when real fraud is rare.

Common mistakes

- Confusing P(A | B) with P(B | A): P(+ | sick) = 0.9 is not P(sick | +) = 0.15.
- Ignoring the prior (base rate).

In the interview

> **30-second answer:** "Bayes' theorem updates a prior belief with evidence: P(A|B) = P(B|A)P(A)/P(B). A classic example is a test with 90% sensitivity and 5% false positives for a disease with 1% prevalence: a positive result gives only about a 15% chance of disease, because the healthy population is so much larger. It underlies Naive

Bayes and explains why precision (the share of flagged cases that really are positive) is low for rare-event classifiers."

- *What's "naive" in Naive Bayes?* — Features assumed conditionally independent given the class.
- *Relation to precision?* — P(sick | positive) is exactly the test's precision.

TL;DR

P(A | B) = P(A and B) / P(B). Posterior ∝ likelihood × prior. Rare events + imperfect tests = many false alarms.

2.8 Descriptive statistics and standardisation

What it is

Statistics summarises data and quantifies how far conclusions can be trusted.

Measure	Definition	Robust to outliers?
Mean	Sum ÷ count	No
Median	Middle sorted value	Yes
Mode	Most frequent value	Yes
Variance	Average squared distance from the mean	No
Standard deviation	√variance, original units	No
Percentiles / IQR	Value below which a % of data falls; IQR = 75th − 25th percentile	Yes

Worked example. Salaries ($1000s): 40, 42, 45, 47, 50, 52, 55, 300 (one CEO). Mean = **78.9**, median = **48.5**: one outlier drags the mean far above a typical salary. **Population vs sample std:** dividing by n (NumPy default `ddof=0`) describes this exact group; n − 1 (`ddof=1`, the pandas default) gives an unbiased variance estimate for a sample of a larger population; negligible difference for large n.

Standardisation (z-scores): z = (x − μ)/σ gives mean 0, std 1; z = "standard deviations from the mean". For x = [2, 4, 4, 4, 5, 5, 7, 9]: μ = 5, σ = 2, so 9 → **2.0**, 2 → **−1.5**. It stops features dominating through units, speeds up gradient descent, and is essential for distance-based methods (k-NN, k-means, SVMs) and regularised models. scikit-learn's `StandardScaler` learns μ and σ from training data only.

Output of `code/statistics_basics.py`:

```
mean 78.9, median 48.5
population std (ddof=0) 83.7, sample std (ddof=1) 89.5
without the outlier: mean 47.3, median 47.0
x mean 5.0, std 2.0 -> standardised z = [-1.5 -0.5 -0.5 -0.5  0.   0.   1.   2. ]
z mean 0.0, z std 1.0
```

Self-check

1. Why fit the scaler on training data only? → Test data in μ and σ leaks test information.

TL;DR

Mean and std are pulled by outliers; median and IQR are robust. Standardise to mean 0, std 1 before distance- or gradient-based models.

2.9 Correlation vs causation

What it is

Correlation (Pearson's r, −1 to +1) measures how strongly two variables move together *in a straight line*: +1 perfect increasing line, −1 perfect decreasing, 0 no *linear* relationship. **Causation** means changing one actually changes the other. Correlation without causation comes from:

1. **Confounder:** a third variable drives both (hot weather raises ice-cream sales and swimming, hence drownings).
2. **Reverse causation:** the arrow runs the other way.
3. **Coincidence:** with enough variables some correlate by chance.
4. **Selection bias:** the data collection creates the pattern.

```
temp = rng.uniform(10, 35, 365)
ice_cream = 20 * temp + noise;  drownings = 0.3 * temp + noise
np.corrcoef(ice_cream, drownings)          # strong correlation
# remove temperature's effect from both, then correlate what is left
```

Output:

```
corr(ice cream, drownings) = 0.77
corr after controlling for temperature = -0.06
corr(x, x^2) = -0.00  (perfect dependence, zero LINEAR correlation)
```

Controlling for temperature removes the relationship. Conversely $y = x^2$ is fully determined by x yet $r = 0$ because the relationship is not a line.

Why it matters for ML

Models find correlations, not causes: fine for *prediction* (umbrella sales predict rain), dangerous for *decisions* ("sell umbrellas to cause rain"). Causal effects need **randomised experiments (A/B tests)** or causal-inference methods.

Self-check

1. More hospitals, more deaths. Causal? → Likely confounded by population size.
2. $r = 0$ means independent? → Not necessarily; there may be a non-linear relationship.

TL;DR

Correlation measures linear co-movement and can come from confounders, reverse causation or chance. Use experiments to establish cause.

2.10 Sampling and the central limit theorem

What it is

A **population** is everything you care about; a **sample** is what you measure. A sample **statistic** (e.g. sample mean) estimates a population **parameter**. Samples must be **random** and **representative**; a biased sample gives biased estimates however large (Chapter 1's *Literary Digest* poll).

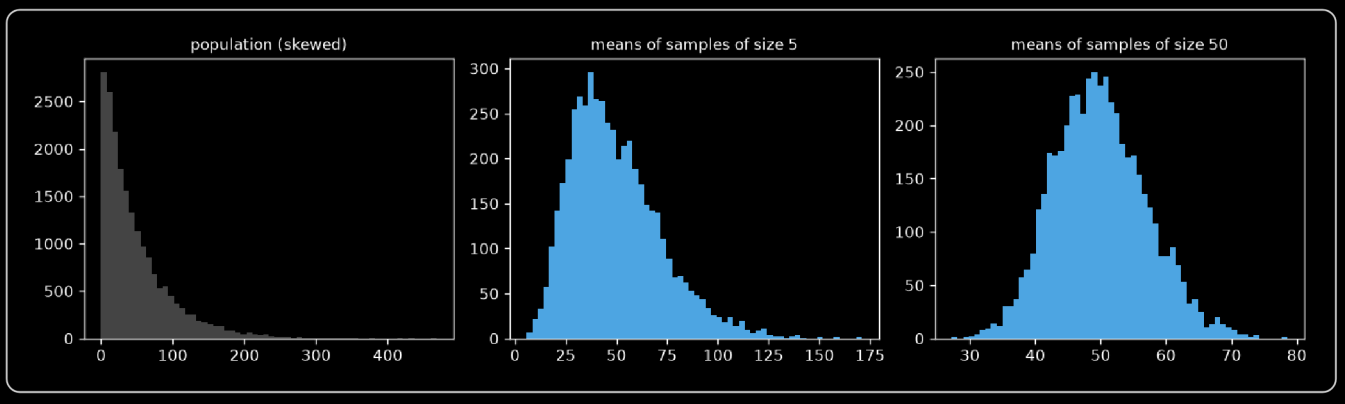
Central limit theorem (CLT): means of many random samples of size n are approximately normal around the true mean, even for a skewed population, if n is reasonably large. Their spread, the **standard error**, is **$\sigma/\sqrt{n}$**: quadrupling n halves the uncertainty.

Code

```
population = rng.exponential(scale=50, size=1_000_000)       # skewed
means = rng.choice(population, size=(5000, n)).mean(axis=1)  # 5000 samples of size n
```

Output:

```
population mean 49.99, std 50.07
n= 5: mean of sample means 49.81, std of sample means 22.48 (theory sigma/sqrt(n) = 22.39)
n=50: mean of sample means 49.89, std of sample means 7.11 (theory sigma/sqrt(n) = 7.08)
```



At n = 50 the means form a bell despite the skew — why normal-based CIs and t-tests (§2.11) work for averages of real data, and why larger test sets give more reliable accuracy.

TL;DR

Sample means cluster normally around the truth with spread σ/√n. Random, representative samples matter more than big ones.

2.11 Hypothesis tests, p-values and confidence intervals

What it is

A **hypothesis test** asks whether an effect could plausibly be noise:

1. State the **null hypothesis H_0** (no effect) and **alternative H_1**.
2. Choose **significance level α** beforehand, usually 0.05.
3. Compute a **test statistic** (e.g. t = difference in means / its standard error).
4. **p-value**: *if H_0 were true*, the probability of a result at least this extreme.
5. p < α → **reject H_0** ("statistically significant"); otherwise **fail to reject** (not proof of H_0).

Error	Meaning	Probability
Type I (false positive)	Reject a true H_0	α
Type II (false negative)	Miss a real effect	β; **power** = 1 − β

A **confidence interval (CI)** = estimate ± critical value × standard error (the *critical value* comes from a reference distribution, ≈ 1.96 for 95% with large samples). A 95% CI comes from a method that captures the true value in 95% of repeated experiments.

Worked example: an A/B test

Page load times, 60 users each: old 202.7 ms, new 187.3 ms (−15.4 ms). Welch's t-test (no equal-variance assumption):

```
t, p = stats.ttest_ind(new, old, equal_var=False)
m, se = new.mean(), new.std(ddof=1) / np.sqrt(len(new))
tcrit = stats.t.ppf(0.975, df=len(new) - 1)
```

Output:

```
old mean 202.7, new mean 187.3, diff -15.4 ms
Welch t = -2.72, p-value = 0.0075 -> reject H0 at 0.05
95% CI for new mean: 187.3 +/- 2.001*4.02 = [179.2, 195.3]
accuracy 0.850, bootstrap 95% CI [0.800, 0.900]
coverage of 2000 simulated 95% CIs: 0.949
```

- p = 0.0075: with no real difference, a gap this large would appear < 1% of the time → reject H_0.
- 2.001 is the t critical value for 59 **degrees of freedom** (roughly sample size minus quantities estimated); it approaches 1.96 for large samples.
- **Bootstrap CI** for test accuracy: resample the 200 test predictions with replacement 5,000 times, take the 2.5th and 97.5th percentiles. 85% on 200 examples really means "about 80–90%" — remember this before celebrating a 1-point gain on a small test set.
- 94.9% of 2,000 simulated 95% intervals contained the true mean.

Common mistakes

- **"p = 0.03 means a 3% chance H_0 is true."** No: p assumes H_0 and measures how surprising the data is.
- **Significant ≠ important.** Huge samples make trivial differences significant; report the **effect size** ("15 ms faster") with its CI.
- **Multiple comparisons / p-hacking:** testing 20 useless features at α = 0.05 yields ~one "significant" by chance. Correct (Bonferroni: α/20) or pre-register (write down the test before seeing data).
- "Peeking": stopping an A/B test as soon as p < 0.05 inflates false positives.

In the interview

> **30-second answer:** "A p-value is the probability of seeing data at least as extreme as ours if the null hypothesis were true. If it's below a pre-chosen α, like 0.05, we reject the null. It's not the probability the null is true, and significance isn't practical importance, so I report effect sizes with confidence intervals and correct for multiple comparisons."

- *Statistical power?* — Probability of detecting a real effect; rises with sample size and effect size.

- *Comparing two models' accuracy?* — Same test set, paired test (e.g. McNemar's, comparing examples one model gets right and the other wrong) or bootstrap the difference.

Self-check

1. p = 0.2, α = 0.05? → Fail to reject H_0; consistent with no effect (doesn't prove it).
2. Accuracy CI [0.80, 0.90]? → True accuracy plausibly anywhere in range; small differences inside it may be noise.

TL;DR

Set α, compute p; p < α → reject H_0. p is not P(H_0 true). Report effect sizes with CIs; beware multiple testing.

2.12 Information theory: entropy, cross-entropy and KL divergence

What it is

- **Entropy** $H(p) = -\Sigma\, p(x) \log_2 p(x)$: average **uncertainty** (surprise) in bits. Fair coin 1 bit; 90/10 coin 0.469; certain outcome 0.
- **Cross-entropy** $H(p, q) = -\Sigma\, p(x) \log q(x)$: average surprise when reality follows p but you predict q. Always ≥ H(p), equal only when q = p.
- **KL divergence** $KL(p \| q) = \Sigma\, p(x) \log (p(x)/q(x)) = H(p, q) - H(p)$: *extra* surprise from using q. ≥ 0, 0 only when q = p, **not symmetric**.

Why it matters

- **Classification loss.** With a **one-hot** true distribution (probability 1 on the right class), cross-entropy = −log q(correct class). Averaged, this is **log loss**, minimised by logistic regression and nearly every neural classifier, including language models predicting the next **token** (word or word-piece).
- **Decision trees** pick splits that most reduce entropy (**information gain**).
- **KL** appears in variational autoencoders (networks that compress data and generate samples), **knowledge distillation** (a small model copying a big model's probabilities), and **RLHF** (RL from human feedback), where it keeps a fine-tuned language model close to its original.

Worked numbers

True p = [0.7, 0.2, 0.1], model q = [0.5, 0.3, 0.2] (base 2):

- $H(p) = 0.360 + 0.464 + 0.332 =$ **1.157 bits**
- $H(p, q) = -(0.7 \log_2 0.5 + 0.2 \log_2 0.3 + 0.1 \log_2 0.2) = 0.700 + 0.347 + 0.232 =$ **1.280 bits**
- $KL(p \| q) =$ **0.123 bits**

Log loss, labels [1, 0, 1, 1], predicted P(class 1) = [0.9, 0.2, 0.6, 0.99] (natural log): −¼ [ln 0.9 + ln 0.8 + ln 0.6 + ln 0.99] = **0.2123**.

```
manual = -np.mean(y_true * np.log(p_pred) + (1 - y_true) * np.log(1 - p_pred))
log_loss(y_true, p_pred)   # scikit-learn gives the same number
```

Output of `code/information_theory.py`:

```
entropy of fair coin            = 1.000 bits
entropy of biased coin 0.9/0.1  = 0.469 bits
entropy of certain              = 0.000 bits
entropy of fair 8-sided die     = 3.000 bits

H(p) = 1.157, H(p, q) = 1.280, KL(p||q) = 0.123, H(p) + KL = 1.280
KL(q||p) = 0.133  (KL is not symmetric)

log loss by hand = 0.2123 | sklearn = 0.2123
  true class predicted with p=0.99: loss = 0.010
  true class predicted with p=0.6 : loss = 0.511
  true class predicted with p=0.1 : loss = 2.303
  true class predicted with p=0.01: loss = 4.605
```

Log loss punishes **confident mistakes**: giving the true class 1% costs 460× more than giving it 99%.

Common mistakes

- Mixing log bases: $\log_2$ (bits) vs natural log (nats, used by ML libraries); only the scale changes.
- Treating KL as a distance: it is asymmetric and violates the triangle inequality (going A→C directly is never longer than via B).

- Predicting exact 0 or 1: log(0) = −∞, so libraries clip probabilities (e.g. to 1e-15 … 1 − 1e-15).

Self-check

1. Loss when the true class gets 0.5? → −ln 0.5 ≈ 0.693.

TL;DR

Entropy = uncertainty; cross-entropy = surprise predicting with q; KL = the gap. Classifiers minimise cross-entropy (log loss), which punishes confident errors hardest.

Chapter TL;DR

Area	Key idea	Where it shows up
Vectors	Data point = vector; dot product = multiply and sum	Linear models, embeddings, similarity search
Matrices	Dataset = matrix; predictions = Xw + b	Every model; neural-net layers; GPUs
Transpose / inverse / det	Swap axes; undo a matrix; det = 0 → no inverse	Normal equation, gradients, covariance
Eigen / SVD	Main directions and their strengths	PCA, compression, recommenders
Derivatives / gradients	Slope of the loss per parameter	How every model learns
Chain rule	Multiply slopes through nested functions	Backpropagation
Gradient descent	$w \leftarrow w - \eta \nabla L$; tune η, scale features	Training almost everything
Distributions	Bernoulli, binomial, normal; mean, variance	Labels, noise, A/B tests
Bayes	Posterior ∝ likelihood × prior; mind the base rate	Naive Bayes, rare-event precision
Statistics	Median vs mean, standardisation, CLT, $\sigma/\sqrt{n}$	Preprocessing, trusting results
Correlation ≠ causation	Confounders; experiments for causes	Interpreting models
Tests and CIs	p-value, α, CI, bootstrap; significance ≠ importance	Model comparison, A/B tests
Information theory	Entropy, cross-entropy, KL	Log loss, decision trees, LLM training

Chapter 3 — The Python ML Stack

Practical ML happens in Python not because Python is fast (it isn't) but because a few libraries wrap fast C, C++ and Fortran code (*compiled* languages, translated to machine instructions ahead of time) behind a friendly interface. This chapter covers the five tools used in every later chapter — **NumPy** (arrays), **pandas** (tables), **matplotlib** (plots), **scikit-learn** (classic ML), **Jupyter** (interactive work) — and when to reach for the deep-learning frameworks (PyTorch, TensorFlow/Keras).

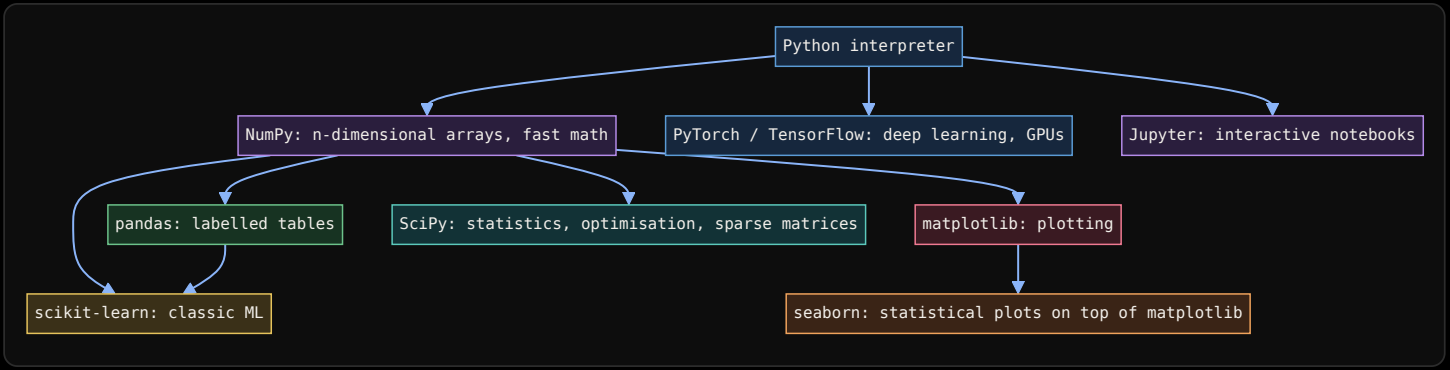
It assumes basic Python; if that is new, work through the **Python Handbook** first.

All code is in `code/`.

3.1 Setting up

What it is

The **stack** is a set of open-source libraries that build on each other:



How it works

1. **Install Python 3** (python.org or the OS package manager, e.g. `apt`, Homebrew); check with `python --version` (`python3` on macOS/many Linux). This book uses Python 3.12+.
2. **Create a virtual environment** (a project folder with its own Python and libraries) per project:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
```

3. **Install** with pip (Python's package installer, pulling from the Python Package Index):

```
pip install numpy pandas matplotlib scikit-learn scipy jupyter
pip install seaborn                  # optional, nicer statistical plots
pip install torch                    # only when you need deep learning
```

4. **Record versions:** `pip freeze > requirements.txt`, later `pip install -r requirements.txt`. conda, uv and Poetry do the same job (see the Python Handbook's packaging chapter).
5. **Standard aliases:** `import numpy as np`, `import pandas as pd`, `import matplotlib.pyplot as plt`.

Code here was run with NumPy 2.5, pandas 3.0, scikit-learn 1.9 and matplotlib 3.11.

Common mistakes

- `sudo pip` into the system Python (the copy the OS relies on). Use a virtual environment.
- Installing with one Python and running another; `python -m pip install ...` guarantees pip matches the interpreter.

TL;DR

One virtual environment per project; `pip install numpy pandas matplotlib scikit-learn`; freeze versions; import as `np`, `pd`, `plt`.

3.2 NumPy: fast arrays

What it is

NumPy's `ndarray` is an n-dimensional grid of same-typed numbers in one contiguous block of memory. 1-D = vector, 2-D = matrix (Chapter 2); higher dimensions hold colour images (height × width × 3) or batches.

Why it matters

A Python list stores pointers to separate objects and loops one interpreted instruction at a time; an array stores raw numbers and runs whole operations in compiled code — often 10–1000× faster and far more compact. pandas, scikit-learn, SciPy and matplotlib all speak NumPy, and PyTorch tensors (its arrays) convert easily.

How it works

Arrays, shapes and dtypes

Attribute / function	Meaning	Example
`np.array([1, 2, 3])`	Create from a list	
`a.dtype`	Element type: `int64`, `float64`, `float32`, `bool`, ...	Mixing ints and floats upcasts to float
`a.shape`	Size per dimension	`(3, 4)` = 3 rows, 4 columns
`a.ndim`, `a.size`, `a.nbytes`	Dimensions, element count, memory	
`np.zeros`, `np.ones`, `np.eye`, `np.arange`, `np.linspace`	Constructors	`np.linspace(0, 1, 5)` → 5 evenly spaced values
`a.reshape(3, 4)`, `a.reshape(-1)`, `a.T`	New shape (−1 = "work it out"), flatten, transpose	
`a[:, None]` or `a.reshape(-1, 1)`	Shape (n,) → column (n, 1)	scikit-learn wants 2-D X

dtype matters. `float64` = 8 bytes, `float32` = 4 (1 million values: 8 MB vs 4 MB); deep learning uses `float32` or smaller to save memory and speed up GPUs. Small integer types silently **wrap around** on overflow: `int8` 127 + 1 = −128. Floating point is approximate: `0.1 + 0.2 == 0.3` is `False`; use `np.isclose`.

Indexing

```
M = np.arange(12).reshape(3, 4)
M[1, 2]          # one element (row 1, column 2) -> 6
M[0]             # first row
M[:, 1]          # second column
M[:2, 1:3]       # rows 0-1, columns 1-2 (slices are start:stop, stop excluded)
M[M % 2 == 0]    # boolean mask: all even values
M[[2, 0]]        # "fancy" indexing with a list: rows 2 and 0
np.where(M > 5, 1, 0)   # vectorised if/else
```

Views vs copies. Basic slicing (`M[0]`, `M[:2, 1:3]`) returns a **view** onto the same memory — changing it changes the original. Boolean indexing (a True/False *mask*) and fancy indexing return **copies**. Call `.copy()` for independent data.

Axis operations

For 2-D arrays `axis=0` runs **down the rows** (one result per column), `axis=1` **across columns** (one per row): the named axis disappears.

```
scores = np.array([[80, 90, 70],     # rows = students, columns = exams
                   [60, 75, 85]])
scores.sum(axis=0)      # per exam:    [140 165 155]
scores.mean(axis=1)     # per student: [80. 73.33]
scores.argmax(axis=1)   # best exam per student: [1 2]
```

`keepdims=True` keeps the reduced axis as size 1 ((2, 1) not (2,)) so the result broadcasts back against the original.

Broadcasting

Broadcasting combines arrays of different shapes without copying, comparing dimensions from the *last* backwards: compatible if equal or one is 1; missing leading dimensions count as 1; size-1 dimensions stretch.

Shapes	Result	Why
(3, 2) and (2,)	(3, 2)	(2,) acts as (1, 2), repeated per row
(3, 1) and (1, 4)	(3, 4)	Both stretch: an outer "table"
(3, 2) and (3,)	**error**	Trailing 2 vs 3 clash; reshape to (3, 1) for "per row"

Most common ML use — **standardising every column at once**:

```
X = np.array([[1.0, 200.0], [2.0, 300.0], [3.0, 400.0]])   # (3, 2)
(X - X.mean(axis=0)) / X.std(axis=0)                        # (3, 2) - (2,) / (2,)
```

Output:

```
mean [  2. 300.] std [ 0.816 81.65 ]
standardised (X - mu) / sd =
 [[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
```

Advanced: squared distance from every point to every cluster centre (the core step of **k-means**, which repeatedly assigns points to their nearest centre) with no loops — shapes (4, 1, 2) and (1, 2, 2) broadcast to (4, 2, 2):

```
d2 = ((pts[:, None, :] - centres[None, :, :]) ** 2).sum(axis=2)   # (4 points, 2 centres)
d2.argmin(axis=1)                                                 # nearest centre per point
```

```
squared distances (points x centres):
 [[ 0. 50.]
 [ 2. 32.]
 [50.  0.]
 [61.  1.]]
nearest centre: [0 0 1 1]
```

Vectorisation vs loops

Vectorisation = whole-array operations instead of a Python loop.

```
def loop():
    total = 0.0
    for v in x:            # 1,000,000 Python-level iterations
        total += v * v
    return total

def vec():
    return float(x @ x)    # one call into optimised compiled code
```

Output from `code/numpy_basics.py`:

```
sum of squares of 1M floats: loop 58 ms, vectorised 0.06 ms -> ~930x faster (varies by machine)
same answer: True
```

A `for` loop over array rows usually has a faster NumPy function or broadcasting equivalent.

Random numbers

Use the **Generator** API with an explicit seed (fixes the whole random sequence) for reproducibility:

```
rng = np.random.default_rng(42)
rng.random(3)                # uniform [0, 1)
rng.normal(0, 1, 3)          # normal, mean 0, std 1
rng.integers(0, 10, 5)       # integers 0..9
rng.choice(["a", "b", "c"], size=4)
rng.permutation(10)          # shuffled indices, e.g. for a manual train/test split
```

```
uniform [0.774 0.439 0.859]
normal [ 0.941 -1.951 -1.302]
integers [7 7 7 7 5]
choice ['a' 'c' 'b' 'b']
shuffled indices for a train/test split: [7 0 4 2 3 5 8 1 9 6] -> train [7 0 4 2 3 5 8 1] test [9 6]
same seed, same numbers: [0.774 0.439 0.859]
```

The older `np.random.seed()` / `np.random.rand()` share one global state (any code can disturb it); NumPy recommends `default_rng` for new code.

Common mistakes

- Shape (n,) vs (n, 1): scikit-learn needs 2-D `X`, so a single feature is `x.reshape(-1, 1)`.
- Modifying a slice and changing the original (views).
- Silent wrong broadcasting: (n,) − (n, 1) gives an (n, n) matrix. Check shapes.
- `*` for matrix multiplication; use `@`.

In the interview

> **30-second answer:** "NumPy arrays are homogeneous (all elements the same type), contiguous blocks of memory, so operations run in compiled code instead of the Python interpreter. Vectorising, meaning whole-array operations plus broadcasting, is typically orders of magnitude faster than Python loops. Broadcasting aligns shapes from the trailing dimension and stretches size-1 dimensions."

- *View or copy?* — Basic slices are views; boolean and fancy indexing return copies.
- *axis=0?* — Down the rows, one result per column.

Self-check

1. Can (4, 3) and (4,) broadcast? → No (trailing 3 vs 4); use (4, 1).

TL;DR

ndarrays are typed, fixed-size and fast. Master shape, dtype, slicing (views!), `axis`, broadcasting and vectorisation; seed a `default_rng`.

3.3 pandas: tables with labels

What it is

pandas adds labels to NumPy arrays. A **Series** is a 1-D array with an **index** (row labels) and a name; a **DataFrame** is a table of Series (columns) sharing one index, each with its own dtype — a spreadsheet controlled by code.

Why it matters

Real data (CSV files, Excel, database queries) has mixed types, missing values and messy text; pandas is the standard tool to load, clean, explore and reshape it, and scikit-learn accepts DataFrames directly.

How it works

`code/pandas_basics.py` uses a small employee table.

Series and DataFrame

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="points")
s["b"]        # 20  (by label)
df = pd.DataFrame({"city": ["Paris", "Lima"], "pop_m": [2.1, 10.0]})
```

Load and inspect

```
df = pd.read_csv("employees.csv", parse_dates=["start_date"])
df.head(3)            # first rows
df.shape              # (rows, columns)
df.dtypes             # type of each column
df.isnull().sum()     # missing values per column
df.describe()         # count, mean, std, min, quartiles, max for numeric columns
df.info()             # dtypes, non-null counts, memory
```

Output:

```
   emp_id  name   dept    salary age start_date
0       1  Asha    Eng  120000.0  34 2019-03-01
1       2   Ben    Eng       NaN  29 2021-07-15
2       3  Chen  Sales   70000.0  41 2015-01-10

shape: (7, 6)
emp_id                int64
name                    str
dept                    str
salary              float64
age                     str
start_date    datetime64[us]

missing values per column:
 emp_id        0
name           0
dept           0
salary         2
age            0
start_date     0
```

`salary` became `float64` because integer columns cannot hold **`NaN`** ("not a number", the missing marker). `age` was read as text (`str`, pandas 3's default string dtype) because one row says `"unknown"`, so `isnull()` misses it. **Always check dtypes after loading.** Other readers: `read_excel`, `read_parquet` (Parquet: compressed, column-oriented format for large data), `read_json`, `read_sql`; `read_csv` also reads URLs and compressed files.

Clean: types and missing values

```
df["age"] = pd.to_numeric(df["age"], errors="coerce")     # "unknown" -> NaN
df["salary_missing"] = df["salary"].isna()                # remember what was missing
df["salary"] = df["salary"].fillna(df.groupby("dept")["salary"].transform("median"))
df["age"] = df["age"].fillna(df["age"].median())
```

Output:

```
age dtype after to_numeric: float64 | missing ages: 1
dropna() would keep 4 of 7 rows
   name   dept    salary  salary_missing   age
0  Asha    Eng  120000.0           False  34.0
1   Ben    Eng  127500.0            True  29.0
...
6   Gus     HR   60000.0            True  27.0
```

Strategy	Code	When
Drop rows	`df.dropna()`	Few missing rows, missing at random (here it would lose 3 of 7)
Drop a column	`df.drop(columns="x")`	Mostly empty or useless
Fill with a constant	`df["x"].fillna(0)`	0 has real meaning ("no purchases")
Fill with mean / median	`df["x"].fillna(df["x"].median())`	Numeric; median is outlier-robust
Fill per group	`groupby(...).transform("median")`	Groups differ (Ben gets the Eng median, 127,500)
Add a "was missing" flag	`df["x_missing"] = df["x"].isna()`	Missingness may be informative

In a modelling project, fill inside a scikit-learn `SimpleImputer` (*imputation* = filling missing values) in a pipeline (§3.5), so fill values are learned from training data only.

Select: `loc` vs `iloc`

	`df.loc[...]`	`df.iloc[...]`
Selects by	**Labels** (index values, column names) and boolean masks	**Integer positions**
Slice end	**Included** (`loc[0:2]` → 3 rows)	Excluded (`iloc[0:2]` → 2 rows)
Example	`df.loc[df["dept"] == "Eng", ["name", "salary"]]`	`df.iloc[:2, :3]`

`df["col"]` returns one column (Series); `df[["a", "b"]]` several (DataFrame).

Filter

```
df[(df["salary"] > 100_000) & (df["age"] < 40)]   # combine with & | ~ and parentheses
df.query("dept == 'Sales' and salary >= 65000")   # same idea, SQL-like string
df[df["dept"].isin(["HR", "Sales"])]
df.sort_values("years", ascending=False)
```

```
salary > 100k AND age < 40: ['Asha', 'Ben']
query(): ['Chen', 'Dana']
isin(): ['Chen', 'Dana', 'Eli', 'Gus']
```

Use `&`, `|`, `~` (not `and` / `or` / `not`) and parenthesise each condition, since `&` binds tighter than `>`.

groupby: split, apply, combine

`groupby` splits rows into groups, applies a function, combines results — like SQL's `GROUP BY`.

```
df.groupby("dept").agg(headcount=("emp_id", "count"),
                       avg_salary=("salary", "mean"),
                       max_age=("age", "max"))
df["dept"].value_counts()
df.pivot_table(index="dept", values="salary", aggfunc="median")
```

```
      headcount  avg_salary  max_age
dept
Eng           3    127500.0     45.0
HR            2     60000.0     38.0
Sales         2     67500.0     41.0
```

`agg` reduces each group to one row; `transform` returns a same-length result (used above to fill each row with its group median).

merge / join

`merge` combines tables on a key column (e.g. an ID), like SQL joins.

```
depts = pd.DataFrame({"dept": ["Eng", "Sales", "Legal"], "floor": [3, 1, 5]})
df.merge(depts, on="dept", how="inner")   # only matching keys
df.merge(depts, on="dept", how="left")    # all rows of df, NaN where no match
```

```
inner join rows: 5 (HR has no match) | left join rows: 7 (HR kept with floor NaN: True)
```

how=	Keeps
`"inner"`	Keys in both tables
`"left"`	All left rows
`"right"`	All right rows
`"outer"`	All keys from both

Check row counts after every merge: duplicate keys in the right table multiply rows ("many-to-many" explosion); `validate="many_to_one"` raises an error instead. `pd.concat([df1, df2])` stacks tables vertically.

apply vs vectorised

`df.apply(func, axis=1)` calls Python once per row: flexible but slow. Column arithmetic, `np.where`, `pd.cut` and the `.str` / `.dt` accessors (whole-column text and date methods) are vectorised.

```
big.apply(lambda r: r["price"] * r["qty"], axis=1)   # row by row
big["price"] * big["qty"]                            # whole columns at once
big["band"] = np.where(big["price"] > 50, "high", "low")
big["bucket"] = pd.cut(big["price"], bins=[0, 25, 75, 100], labels=["cheap", "mid", "premium"])
```

```
row-wise apply: 430 ms | vectorised: 0.21 ms -> ~2068x faster (varies by machine)
```

Use `apply` only when nothing vectorised exists (e.g. an external parser per value).

A pandas 3 pitfall: chained assignment

Since pandas 3.0 **copy-on-write** is always on: every indexing result behaves as a copy, so calling an in-place method on a selected column no longer changes the DataFrame:

```
df["A"].fillna(0, inplace=True)    # does NOT change df (pandas warns: ChainedAssignmentError)
df["A"] = df["A"].fillna(0)        # correct: assign the result back
```

```
after df['A'].fillna(0, inplace=True): [1.0, nan, 3.0]
after df['A'] = df['A'].fillna(0):    [1.0, 0.0, 3.0]
```

Prefer assignment over `inplace=True`; it is clearer and works in every version.

Common mistakes

- Not checking dtypes (numbers as text, dates as strings).
- `and` / `or` in filters, or missing parentheses.
- Confusing `loc` (labels, end inclusive) with `iloc` (positions, end exclusive).
- Row-wise `apply` on large data.
- Dropping rows with missing values without checking how many, or whether missingness relates to the target.

In the interview

30-second answer: "pandas gives labelled, column-typed tables on top of NumPy. My first steps on new data are `head`, `dtypes`, `info`, `describe` and `isnull().sum()`, then fixing types with `to_numeric` or `to_datetime`, handling missing values, and exploring with `groupby` and plots. I use `loc` for labels and masks, `iloc` for positions, and vectorised operations rather than `apply`."

- *`agg` vs `transform`?* — One row per group vs a same-length result aligned to the original rows.
- *Avoid a merge blowing up rows?* — Check key uniqueness; pass `validate="one_to_one"` or `"many_to_one"`.

Self-check

1. A numeric column shows dtype `str`. First step? → `pd.to_numeric(col, errors="coerce")`, then inspect the new NaNs.
2. Why fill missing values inside a scikit-learn pipeline rather than in pandas before splitting? → Fill values come from training data only (no **leakage** of test information into training).

TL;DR

Load → check `dtypes`, `isnull().sum()`, `describe()` → fix types → handle missing values deliberately → `loc`/`iloc`, masks, `groupby`, `merge`. Vectorise; assign instead of `inplace`.

3.4 matplotlib: seeing the data

What it is

matplotlib is Python's foundational plotting library. **seaborn** sits on top and makes statistical plots (distributions, pair plots — a scatter grid of every column pair — heatmaps) in one line; pandas' `.plot()` / `.hist()` also call matplotlib, so its core lets you customise them all.

Why it matters

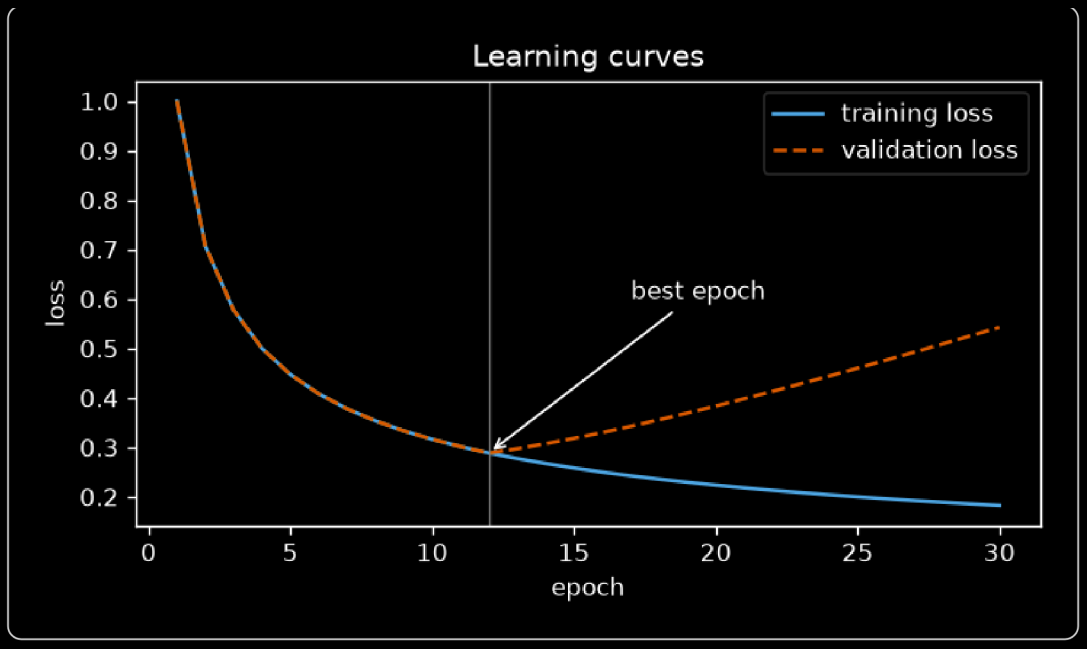
Plots reveal what summaries hide: skew, outliers, class overlap, non-linear relationships, data-entry errors, and **overfitting** (memorising training data) in **learning curves** (training and validation error over training time). Plot before you model.

How it works

Use the **object-oriented API**: create a figure (`fig`, the whole image) and axes (`ax`, individual plots) and call methods on `ax` — rather than relying on a hidden "current plot".

```
import matplotlib
matplotlib.use("Agg")                      # draw to files (scripts, servers); omit in Jupyter
import matplotlib.pyplot as plt

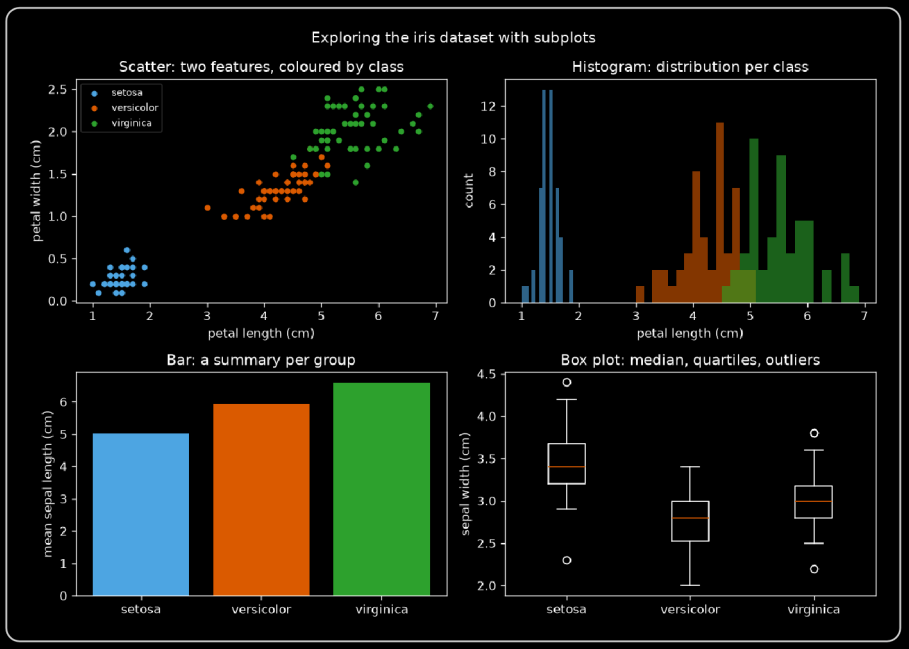
fig, ax = plt.subplots(figsize=(6, 3.5))
ax.plot(epochs, train, label="training loss")
ax.plot(epochs, val, "--", label="validation loss")
ax.set(xlabel="epoch", ylabel="loss", title="Learning curves")
ax.legend()
fig.tight_layout()
fig.savefig("img/line_learning_curves.png", dpi=120)
plt.close(fig)                             # free memory when making many figures
```



A **line plot** suits ordered x (time, **epochs** = full passes over the training data). This one shows the classic overfitting signature: validation loss turns up while training loss keeps falling.

Subplots: `fig, axes = plt.subplots(2, 2)` returns a 2 × 2 array of axes.

```
ax = axes[0, 0]
for sp, g in df.groupby("species"):
    ax.scatter(g["petal_length"], g["petal_width"], s=15, label=sp)
axes[0, 1].hist(g["petal_length"], bins=15, alpha=0.6)
axes[1, 0].bar(means.index, means.values)
axes[1, 1].boxplot([...], tick_labels=[...])
```

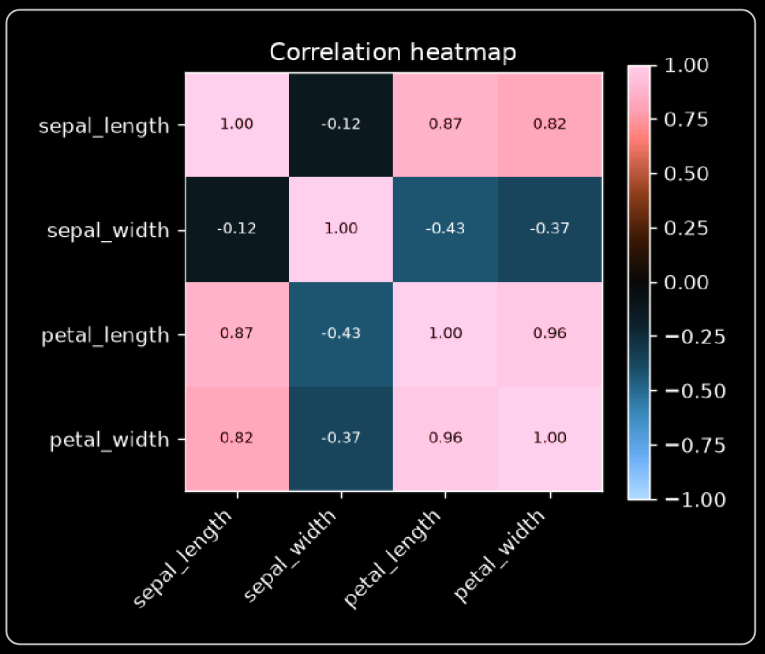


Plot	Shows	ML use
Scatter	Relationship of two numeric variables	Separable classes? Linear relationship?
Histogram	One variable's distribution	Skew, outliers, per-class overlap
Bar	A value per category	Class balance, group means
Box plot	Median, quartiles (25th/75th percentiles; the box), whiskers (typical range), outliers (dots)	Compare groups, spot outliers
Line	Trend over an ordered axis	Learning curves, time series
Heatmap	A matrix as colours	Correlation and confusion matrices

Petal measurements separate *setosa* perfectly and the others mostly — a simple classifier will do well.

Correlation heatmap (`sns.heatmap(corr, annot=True)` in seaborn; plain matplotlib here):

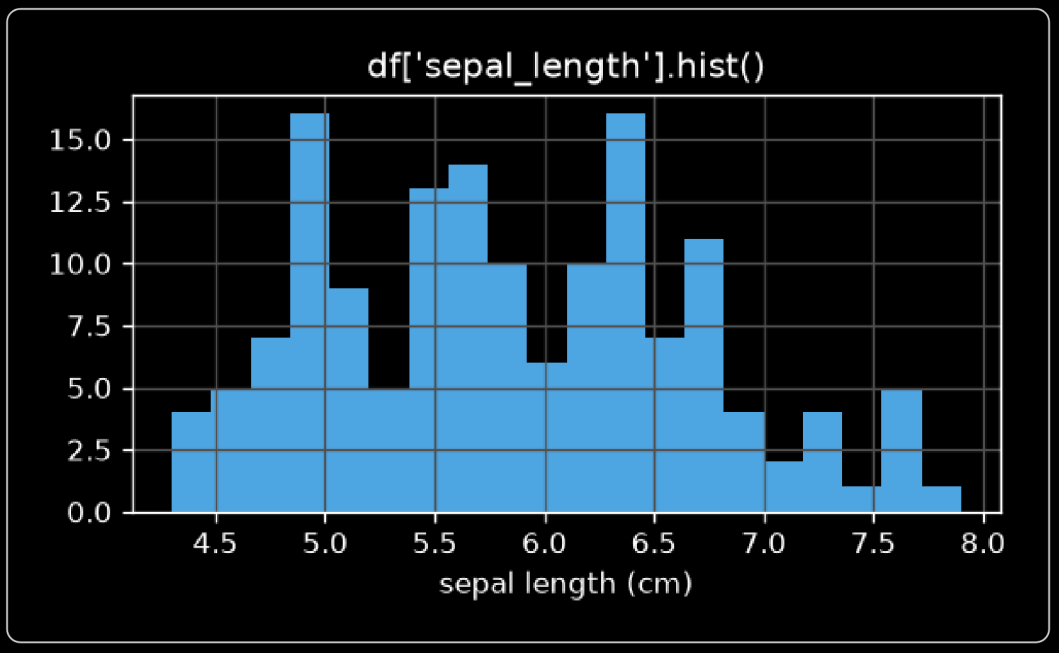
```
corr = df.drop(columns=["species", "target"]).corr()
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
fig.colorbar(im, ax=ax)
```



Petal length and width correlate at 0.96 — nearly the same information, which matters for linear models (multicollinearity makes weights unstable, Chapter 2) and **feature selection** (choosing which columns to keep).

pandas shortcut for a first look:

```
ax = df["sepal_length"].hist(bins=20)
```



Common mistakes

- Mixing state-based `plt.plot(...)` with multiple subplots; use `fig, ax = plt.subplots()`.
- No axis labels or units.
- Default 10 bins hiding structure; try several `bins`.
- No `plt.close(fig)` in loops (memory leak).
- `plt.show()` on a display-less server; use `savefig`.

TL;DR

`fig, ax = plt.subplots()`, `ax.plot/scatter/hist/bar/boxplot/imshow`, label everything, `savefig`. Plot distributions, relationships and class balance before modelling.

3.5 scikit-learn: the estimator API

What it is

scikit-learn is the standard Python library for classic ML: linear models, trees and forests, SVMs (support vector machines: widest-margin boundary between classes), k-NN (k-nearest neighbours: predicts from the most similar stored examples), clustering (grouping unlabelled rows), dimensionality reduction (compressing many features into a few), preprocessing, model selection and metrics. Its power is **one consistent interface** for every algorithm (API design paper: Buitinck et al., 2013).

How it works

Object type	Key methods	Learns in `fit`	Examples
Estimator (all objects)	`fit(X, y)`; hyperparameters in the constructor; `get_params()`, `set_params()`		Everything below
Transformer	`transform(X)`, `fit_transform(X)`	Statistics to transform (mean, categories, ...)	`StandardScaler`, `OneHotEncoder`, `SimpleImputer`, `PCA`
Predictor	`predict(X)`, often `predict_proba(X)`, `score(X, y)`	Model parameters	`LogisticRegression`, `RandomForestClassifier`, `KMeans`

Conventions:

1. **Hyperparameters** go in the constructor: `LogisticRegression(C=0.1)`.
2. `fit` **returns** `self`, so calls chain: `LogisticRegression().fit(X, y).predict(X_new)`.
3. **Learned attributes end with an underscore** (`scaler.mean_`, `model.coef_`, `model.classes_`) and exist only after fitting.
4. **X is 2-D** (n_samples × n_features); **y is 1-D**.
5. `score` = accuracy for classifiers, R^2 (share of target variation explained; 1 perfect) for regressors.

```
scaler = StandardScaler()
scaler.fit(X_train)                 # learn mean_ and scale_ from TRAINING data
scaler.transform(X_train)
scaler.transform(X_new)             # uses the training mean and std

clf = LogisticRegression().fit(X, y)
clf.predict([[1.0], [4.5]]); clf.predict_proba([[1.0], [4.5]]); clf.score(X, y)
```

Output from `code/sklearn_api.py`:

```
mean_ [  2. 200.] scale_ [ 0.816 81.65 ]
transform(new): [[2.449 2.449]] (uses the training mean/std)
hyperparameters via get_params(): {'copy': True, 'with_mean': True, 'with_std': True}
predict([[1.0],[4.5]]) = [0 1]
predict_proba = [[0.843 0.157]
 [0.096 0.904]]
score (accuracy) = 1.0 | coef_ [[1.121]] intercept_ [-2.802]
```

`fit_transform` on training data, `transform` only on validation/test/production data. Fitting on test data re-learns its statistics — leakage.

Pipeline

A `Pipeline` chains transformers and a final estimator into one estimator: `fit` fits each step in order on training data; `predict` runs the same transforms then predicts.

- **No leakage:** in **cross-validation** (split training data into k *folds*, train on k − 1, validate on the remaining one, rotate) every step is refit per training fold.
- **One object to tune** (`GridSearchCV` tries every listed hyperparameter combination with cross-validation, for any step) and **one object to save and deploy**, so training and serving preprocess identically.

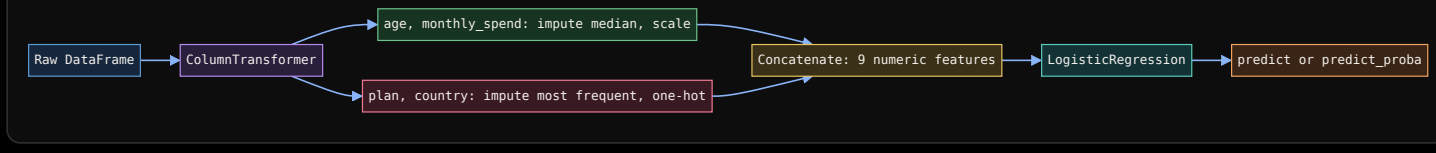
Parameters inside are addressed `step__parameter` (double underscore), nesting as deep as needed: `prep__num__impute__strategy`.

ColumnTransformer

Tables mix numeric and **categorical** columns (labels from a fixed set, like plan or country) needing different preprocessing. `ColumnTransformer` applies a transformer per column group and concatenates the results.

```
numeric = ["age", "monthly_spend"]
categorical = ["plan", "country"]
preprocess = ColumnTransformer([
    ("num", Pipeline([("impute", SimpleImputer(strategy="median")),
                      ("scale", StandardScaler())]), numeric),
    ("cat", Pipeline([("impute", SimpleImputer(strategy="most_frequent")),
                      ("onehot", OneHotEncoder(handle_unknown="ignore"))]), categorical),
])
model = Pipeline([("prep", preprocess), ("clf", LogisticRegression(max_iter=1000))])
model.fit(X_train, y_train)

grid = GridSearchCV(model, {
    "prep__num__impute__strategy": ["mean", "median"],
    "clf": [LogisticRegression(max_iter=1000), RandomForestClassifier(n_estimators=100, random_state=0)],
}, cv=5)
grid.fit(X_train, y_train)
```



Output on a synthetic 600-customer **churn** (customers leaving) dataset with missing values:

```
    age  monthly_spend   plan country
0  62.0          75.10  basic      DE
1   NaN          62.60  basic      IN
2  44.0          89.69    pro      DE
missing: {'age': 43, 'monthly_spend': 0, 'plan': 32, 'country': 0} | churn rate: 0.372
output features: ['num__age', 'num__monthly_spend', 'cat__plan_basic', 'cat__plan_family', 'cat__plan_pro', 'cat__country_BR',
'cat__country_DE', 'cat__country_IN', 'cat__country_US']
test accuracy: 0.827 | baseline (always 'no churn'): 0.627
CV accuracy (5 folds): 0.773
P(churn) for a customer with unseen categories: 0.933
best: {'clf': 'LogisticRegression', 'prep__num__impute__strategy': 'mean'} | CV 0.773 | test 0.827
```

- `get_feature_names_out()` lists the 9 columns the model sees: 2 scaled numerics plus one 0/1 column per category (**one-hot encoding**).
- `handle_unknown="ignore"` scores a customer with unseen plan (`"enterprise"`) and country (`"JP"`) by zeroing those one-hot columns instead of crashing in production.
- `GridSearchCV` treated the whole pipeline as one estimator and even swapped the final model; logistic regression won (the data was generated from a logistic formula).
- Test 0.827 vs CV 0.773: with 150 test rows the gap is within sampling noise (Chapter 2's bootstrap CI).

Other scikit-learn tools you will use constantly

Module	Tools
`sklearn.model_selection`	`train_test_split`, `cross_val_score`, `cross_validate`, `KFold`, `StratifiedKFold`, `GridSearchCV`, `RandomizedSearchCV`
`sklearn.preprocessing`	`StandardScaler`, `MinMaxScaler`, `OneHotEncoder`, `OrdinalEncoder`, `PolynomialFeatures`, `FunctionTransformer`
`sklearn.impute`	`SimpleImputer`, `KNNImputer`
`sklearn.metrics`	`accuracy_score`, `precision_score`, `recall_score`, `f1_score`, `roc_auc_score`, `mean_squared_error`, `confusion_matrix`, `classification_report`
`sklearn.datasets`	Toy datasets (`load_iris`, `load_breast_cancer`, `load_digits`, `load_diabetes`), generators (`make_classification`, `make_blobs`)
`sklearn.pipeline`, `sklearn.compose`	`Pipeline`, `make_pipeline`, `ColumnTransformer`, `make_column_transformer`

`set_output(transform="pandas")` makes transformers return DataFrames with named columns (helps debugging). Save fitted pipelines with `joblib.dump` (Chapter 1) and load only trusted files.

Common mistakes

- `fit`/`fit_transform` on the test set; preprocessing the full dataset in pandas before cross-validating (put it in the pipeline).
- 1-D `X`; reshape to (n, 1).
- No `handle_unknown="ignore"` → crash on a new category in production.
- `LabelEncoder` on input features; it is for targets. Use `OrdinalEncoder` or `OneHotEncoder`.

In the interview

> **30-second answer:** "Every scikit-learn object is an estimator with `fit`; transformers add `transform`, predictors add `predict`. Hyperparameters go in the constructor and learned attributes end in an underscore. I wrap preprocessing and the model in a Pipeline, with a ColumnTransformer for mixed column types, so cross-validation refits everything per fold without leakage, and I can tune and deploy one object."

- *`fit_transform` vs `transform`?* — Learn statistics and apply them (training only) vs apply already-learned statistics.
- *Tune inside a pipeline?* — `GridSearchCV` with `step__param` keys.

TL;DR

Constructor = hyperparameters; `fit` learns (attributes_); `transform`/`predict` apply. Pipeline + ColumnTransformer = leak-free, tunable, deployable preprocessing plus model.

3.6 Jupyter workflow

What it is

Jupyter notebooks (`.ipynb`) mix code cells, outputs and Markdown notes, running against a live Python process, the **kernel**; they keep data in memory and show results inline, ideal for exploratory work. Use JupyterLab (`jupyter lab`), VS Code, or hosted Google Colab.

How it works: a reproducible routine

1. **Start in the project's virtual environment** (`source .venv/bin/activate && jupyter lab`); install from a notebook with `%pip install package` (the kernel's environment), not `!pip`.
2. **First cell: imports, settings, seeds** (`rng = np.random.default_rng(42)`).
3. **Explore top to bottom:** load → inspect → clean → plot → model, a Markdown heading per stage.
4. **Magics** (`%` commands Jupyter understands): `%timeit expr` (benchmark), `%time` (one run), `%load_ext autoreload` + `%autoreload 2` (reload edited modules); a last-line `df.head()` displays a table; plots show inline by default.
5. **Restart & Run All before trusting or sharing.** Out-of-order execution leaves hidden state (a variable from a deleted cell) — the top cause of "works on my notebook" bugs.
6. **Move stable code into `.py` modules** and import it. `jupyter nbconvert --to script notebook.ipynb` exports code; `jupyter nbconvert --to html --execute` produces a freshly executed report.
7. **Version control** (tracking every change, usually with git): clear large outputs before committing, or pair notebooks with scripts (jupytext).

Trade-offs

	Notebook	`.py` script / module
Exploration, plots, teaching	Excellent	Clumsy
Reproducibility	Risky (out-of-order execution)	Good
Testing, code review, diffs	Poor	Good
Production pipelines	Avoid	Yes

Common mistakes

- Trusting out-of-order cells; giant notebooks; committing megabytes of output or printed secrets (passwords, API keys).

TL;DR

Notebooks for exploration, scripts for anything that must run again. Seed first, run top to bottom, Restart & Run All before sharing.

3.7 Deep-learning frameworks: when scikit-learn is not enough

What it is

Framework	Origin	Focus	Style
scikit-learn	Community project, 2007	Classic ML on tabular data: regression, classification, clustering, preprocessing, evaluation	High level `fit` / `predict`; CPU
PyTorch	Meta (Facebook) AI Research, 2016; now the PyTorch Foundation	Deep learning; dominant in research, wide in industry	Define-by-run ("dynamic graph"): the model is ordinary Python run each step; automatic differentiation; GPUs
TensorFlow + Keras	Google, 2015	Deep learning with strong deployment tooling (TF Serving for servers, TF Lite / LiteRT for mobile)	Keras: high-level API (`model.compile`, `model.fit`); since Keras 3 runs on TensorFlow, JAX (Google's numerical library) or PyTorch backends

How the styles differ

`code/frameworks_compared.py` trains the same logistic-regression classifier on the breast-cancer data both ways:

```
# scikit-learn: the library owns the training loop
sk = LogisticRegression(max_iter=1000).fit(X_tr, y_tr)

# PyTorch: you define the model, loss and loop
model = torch.nn.Linear(X_tr.shape[1], 1)               # w.x + b
loss_fn = torch.nn.BCEWithLogitsLoss()                  # sigmoid + cross-entropy
opt = torch.optim.SGD(model.parameters(), lr=0.1, weight_decay=1e-3)
for epoch in range(300):
    opt.zero_grad()
    loss = loss_fn(model(Xt), yt)
    loss.backward()        # autograd: the chain rule from Chapter 2, automatically
    opt.step()             # gradient-descent update from Chapter 2
```

Output:

```
scikit-learn accuracy: 0.982
  epoch   0 loss 0.6904
  epoch 299 loss 0.0738
PyTorch accuracy:      0.974
device available: mps
```

Similar accuracy. PyTorch is longer but every piece is replaceable: swap `nn.Linear` for a 50-layer network, the loss for any differentiable function, and move tensors to a GPU (`"cuda"` on NVIDIA, `"mps"` on Apple silicon, as detected here) without changing the loop.

Which one to choose

- **scikit-learn** for tabular data, small-to-medium datasets, baselines, and whenever a linear model or tree ensemble may suffice. (XGBoost, LightGBM and CatBoost follow its API and are often strongest on tables.)
- **PyTorch or TensorFlow/Keras** for images, audio, text and other unstructured data, very large datasets, GPU training and custom architectures. PyTorch is more common in current research and many companies; TensorFlow/Keras remains widespread in production and mobile.
- They combine: scikit-learn splitting and metrics around a PyTorch model is common.

Real-world examples

- scikit-learn: its testimonials page quotes Spotify, J.P. Morgan and Booking.com.
- PyTorch: most open research code and Hugging Face Transformers (library of pre-trained models); Andrej Karpathy, then Tesla's AI director, described training Autopilot vision networks with it at the 2019 PyTorch Developer Conference.
- TensorFlow: many Google products; TensorFlow Lite (renamed LiteRT in 2024) for mobile and embedded devices.

Self-check

1. `loss.backward()`? → Computes gradients of the loss for every parameter via automatic differentiation.

> **TL;DR**
>
> scikit-learn for classic ML on tables; PyTorch or TensorFlow/Keras for neural networks, unstructured data and GPUs. They complement each other.

Chapter TL;DR

- **Setup:** one virtual environment per project; `pip install numpy pandas matplotlib scikit-learn`; `np`, `pd`, `plt`.
- **NumPy:** know shape and dtype; slices are views; `axis=0` = down rows; broadcasting aligns trailing dimensions; vectorise; seed `default_rng`.
- **pandas:** `read_csv` → `head`, `dtypes`, `isnull().sum()`, `describe` → `to_numeric(errors="coerce")`, missing values → `loc`/`iloc` → masks with `&`/`|` → `groupby().agg()` → `merge(how=...)`. Vectorise; assign rather than `inplace` (copy-on-write in pandas 3).
- **matplotlib:** `fig, ax = plt.subplots()`; line, scatter, histogram, bar, box, heatmap; label; `savefig`.
- **scikit-learn:** constructor = hyperparameters, `fit` learns attributes_, `transform`/`predict` apply; `Pipeline` + `ColumnTransformer` = leak-free, tunable, deployable.
- **Jupyter:** exploration; seed first, Restart & Run All, move stable code to modules.
- **Frameworks:** scikit-learn for classic ML; PyTorch or TensorFlow/Keras for deep learning.

Chapter 4 — Data, EDA and Feature Engineering

Most of an ML project's time goes into this chapter's work: finding labels, looking at data, cleaning it, turning raw columns into numbers, splitting honestly, and wrapping it all in one reproducible **pipeline** (a fixed chain of processing steps). A mediocre model on well-prepared data usually beats a clever model on messy data.

All code is in `code/` (Python 3.14, scikit-learn 1.9, pandas 3.0, fixed seeds; outputs are real, long ones trimmed with "..."). Most scripts use a synthetic "subscription churn" table from `code/make_data.py`: 3,000 customers plus 25 duplicate rows, a skewed (most values small, long tail of large ones) `income`, a 60-level `city`, dates, short reviews, missing values and typos (ages of 999, incomes of 9,999,999). **Churn** = a customer cancelling; about 12% churn.

4.1 Where training data comes from

What it is

Supervised learning needs **labels**: the correct answer (the **target**, y) attached to each example's **features** (inputs, X). A **label pipeline** produces those (features, label) pairs. Each label source has its own cost, delay and **bias** (a systematic tilt, as opposed to random error).

Why it matters

The first design question is "where does y come from, and when does it arrive?" Label quality caps model quality: a model can't be more accurate than the labels it's graded against.



Figure — label sources differ mainly in delay and cost (course diagram)

How it works — the label sources

Source	Example	Delay	Main weakness
Natural (outcome) labels — the world records the answer	Delivery took 34 min; loan defaulted; card payment reversed (a **chargeback**)	Minutes to weeks	Late; outcome ≠ truth (a chargeback is evidence of fraud, not proof)
Behavioural labels / implicit feedback — actions stand in for judgments	Clicks, skips, saves, purchases, watch time	Seconds	Noisy (a click is not satisfaction); biased toward what the system showed
Human annotation — reviewers follow a written guideline	Is this post harmful? Is this document relevant?	Hours to days	Costly (cents to dollars per judgment); reviewer quality needs managing
Weak supervision — many cheap, noisy rules vote	"Contains 'wire transfer' and sender is new → spam"	Instant once written	Rules noisy and correlated; a model must combine them
LLM-assisted labeling — an LLM (large language model) applies the guideline	Tag support tickets by topic	Seconds	Systematic errors (the same blind spot everywhere)

Explicit vs implicit feedback. *Explicit* = the user tells you (star rating); *implicit* = inferred from behaviour (watched 90%). Implicit is plentiful but biased: a non-click may mean "never shown" (**exposure bias**), and top positions get clicked for being on top (**position bias**). Modelling it: Chapter 13, section 13.2.

Label delay. Outcome labels arrive late (chargebacks can take ~6 weeks), so train only on examples with "mature" labels and use faster **proxy labels** (rule hits, analyst decisions) for recent data. Joining delayed labels and monitoring them live: Chapter 16, section 16.3.

Weak supervision in one paragraph

Instead of hand-labeling 100,000 items, experts write dozens of **labeling functions** — small heuristics that output a label or "abstain". A **label model** estimates each function's accuracy from their agreements and disagreements and combines votes into a probabilistic label; a normal model trained on those labels usually generalises beyond the rules. This is Stanford's **Snorkel** idea.

Human labeling, and measuring agreement

Treat annotation as a system: **versioned guidelines** (a label means what guideline v3 says, stored with the data), **gold questions** (known-answer items mixed into queues to track reviewer accuracy), **active learning** (spend labels on items the model is least sure about), and reuse of decisions the business already makes (fraud analysts' verdicts). Production details: Chapter 16 (16.3).

Inter-annotator agreement (IAA) — several reviewers label the same items. Raw agreement flatters (random guessers agree sometimes); **Cohen's kappa** corrects for chance:

$\kappa = (p_o - p_e) / (1 - p_e)$

p_o = observed agreement; p_e = chance agreement = Σ over labels of (A's share × B's share). κ = 1 perfect, 0 chance, negative worse than chance.

Worked example: two reviewers agree on 80 of 100 emails ($p_o = 0.8$); each says "spam" half the time, so $p_e = 0.5 \cdot 0.5 + 0.5 \cdot 0.5 = 0.5$ and $\kappa =$ **0.6** — "moderate" on the Landis–Koch scale (0.41–0.60 moderate, 0.61–0.80 substantial). Low agreement usually means an unclear guideline or genuinely ambiguous items; high agreement doesn't prove labels *correct* (reviewers can share a mistake).

LLMs in the labeling loop

An LLM can label at scale from the guideline, inside a hierarchy: humans review a random sample plus low-confidence and disputed cases, and accuracy is measured against human-labeled slices **per important subgroup** (Chapter 16, section 16.3).

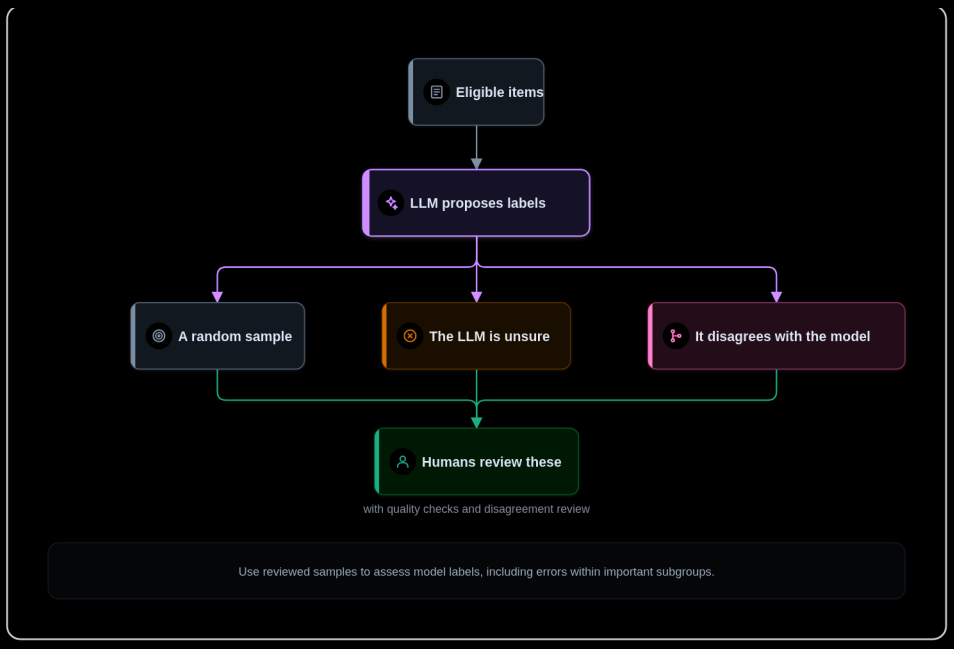


Figure — LLM-assisted labeling with human audit (course diagram)

Negatives for retrieval, and privacy

- **Manufactured negatives.** Logs record clicks, never the millions of items a user would *not* click, so retrieval models (which pull a shortlist from a huge catalogue) create **negatives**: random, **in-batch** (other users' positives in the same training batch) and **hard** (shown but not clicked; some are **false negatives** the user would have liked). See Chapter 13, section 13.7.
- **Privacy.** Collect only what the task needs, restrict access, set retention (how long data is kept); a hashed ID (scrambled by a one-way formula) is still linkable across tables. More in Chapter 16, section 16.3.

Common mistakes

- Assuming labels "exist" in the warehouse (central analytics database) without asking who creates them, when, how reliably; treating clicks as ground truth.
- Unversioned guidelines; raw agreement instead of κ; LLM labels with no human-audited slices.

Real-world examples

- **YouTube**'s 2016 recommendation paper (Covington et al.) trains ranking on implicit feedback — impressions (times a video was shown) clicked or not — weighting clicked impressions by watch time.
- Content-moderation teams run the guidelines + reviewers + gold questions + agreement loop; payment companies use chargebacks (slow) and analyst decisions (fast) as fraud labels.

In the interview

> **30-second answer** — "How will you get labels for this model?": *"I'd list the candidate sources — observed outcomes, user behaviour, human review, rules, LLMs — with their cost, delay and bias. I'd check label quality (agreement with κ, gold questions), train only on matured labels, and join them to features as they were at prediction time."*

- *"Two annotators agree 90% of the time — good?"* — Not necessarily. If each says "not spam" 90% of the time, $p_e = 0.9^2 + 0.1^2 = 0.82$, so $\kappa = (0.90 - 0.82)/(1 - 0.82) \approx 0.44$ — only moderate.
- *"When is weak supervision worth it?"* — When experts can state rules faster than label thousands of items and labels are needed fast or at scale; audit on a small hand-labeled set.
- *"Why is a click a weak label?"* — It measures the old system's exposure and position as much as preference; a non-click is not a dislike.

TL;DR

Labels come from outcomes (late), behaviour (fast, biased), people (costly), rules (noisy) or LLMs (scalable, correlated errors). Measure quality with κ, version the guideline, audit machine labels against human ones.

4.2 Exploratory data analysis (EDA)

What it is

EDA is examining data with summary tables and plots before modeling, to learn its shape, spot problems and form hypotheses.

Why it matters

It catches what silently wrecks models: impossible values, **sentinel codes** (9,999,999 meaning "unknown"), duplicates, skew, leaky columns (secretly containing the answer, 4.8) and imbalance (4.9), and tells you which transformations and models make sense.

How it works — a checklist

1. **Shape and types** — numeric / categorical / dates / text.
2. **Missingness** — share per column; related to the target?
3. **Distributions** — histograms (skew, outliers, spikes at round numbers); category counts (**cardinality** = number of distinct values; rare levels).
4. **Target** — class balance, or its distribution for regression.
5. **Relationships** — feature vs target (rate per bucket, box plots) and feature vs feature (correlation matrix).
6. **Sanity checks** — duplicates, impossible ranges, IDs that predict the target (leakage).

Correlation. Pearson measures *linear* association (−1 to 1). **Spearman** is Pearson on *ranks*, capturing any **monotonic** (always-increasing or always-decreasing) relationship, robust to outliers and skew — a good EDA default. Neither captures a U shape, and correlation is not causation.

```
# code/eda.py (key parts)
df = make_churn()
print(df.isna().mean().loc[lambda s: s > 0].round(3))          # missing share
print(df[num_cols].describe().T[["mean", "50%", "std", "min", "max"]])
print(df[["plan", "device", "city"]].nunique())                   # cardinality
print(df.duplicated().sum())                                      # exact duplicates
corr = num.corr(method="spearman")                                # rank correlation
```

Output (trimmed):

```
--- missing values (share) ---
age       0.03
income    0.08
device    0.02

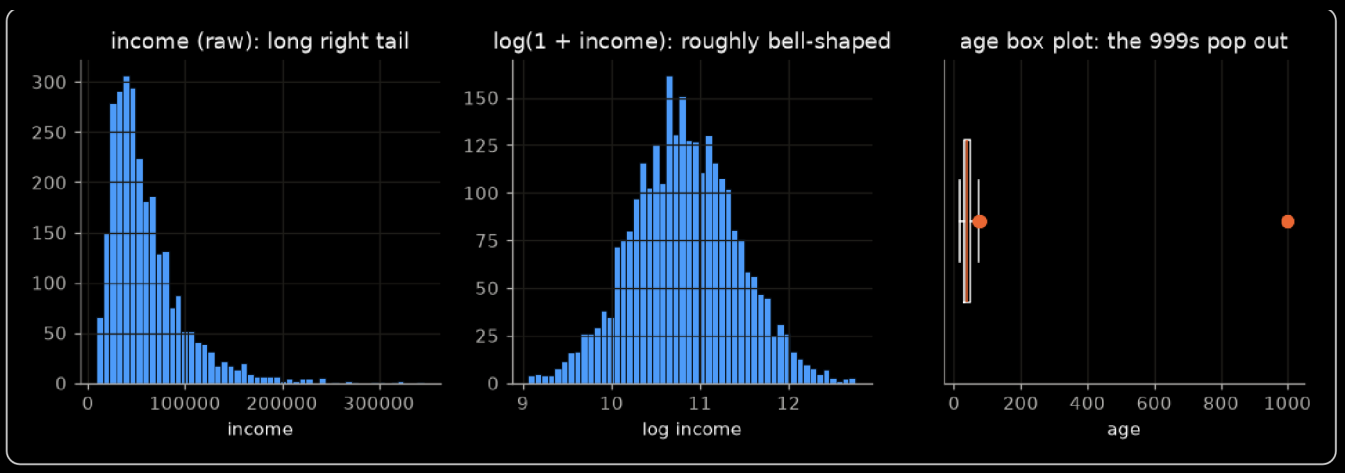
--- numeric summary ---
                    mean      50%       std     min        max
age                 41.1     39.0      37.2    18.0      999.0
income           70763.8  49100.0  328637.8  8600.0  9999999.0
tenure_months       23.3     23.0      13.5     1.0       47.0
support_tickets      1.2      1.0       1.1     0.0        6.0
usage_hours         12.0     10.1       8.4     0.1       58.9

--- categorical cardinality ---
plan 3 | device 3 | city 60

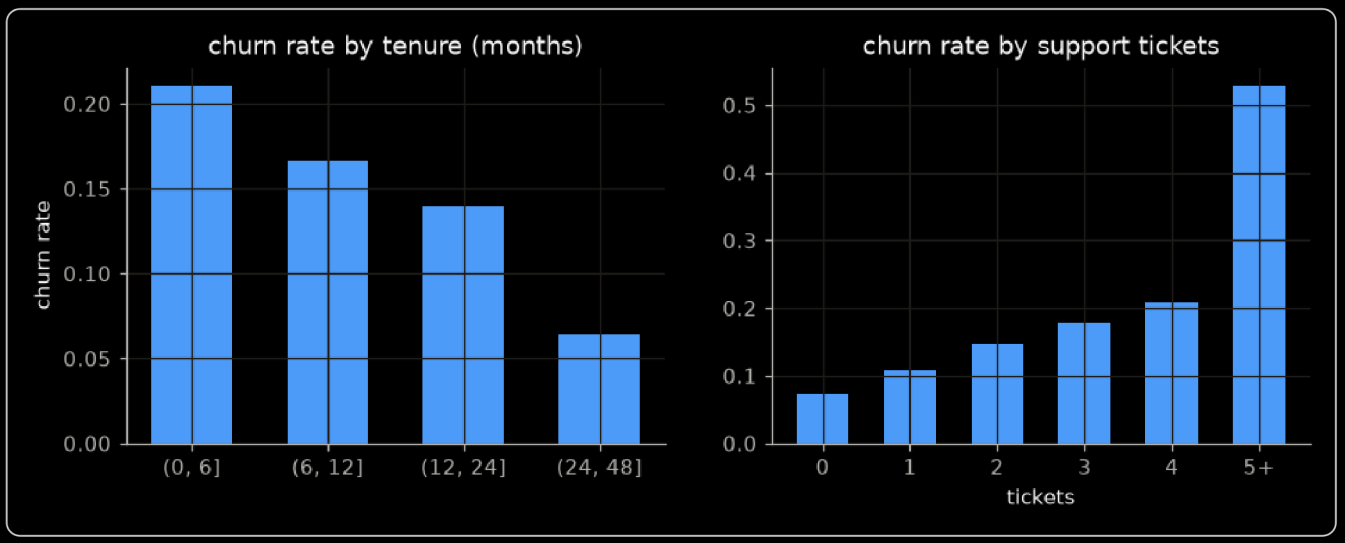
churn rate: 0.117
exact duplicate rows: 25

--- Spearman correlation with churn ---
tenure_months     -0.18
usage_hours       -0.08
...
support_tickets    0.12
```

`age` has max 999 and a std of 37 years — typos. `income` mean ≫ median (70.8k vs 49.1k): a long right tail plus a 9,999,999 sentinel. Churn is 11.7% (mildly imbalanced). Tenure (−) and support tickets (+) relate to churn as expected.







The bar charts show *shape* a single correlation can't: churn drops steeply in the first year, then flattens.

Common mistakes

- Detailed EDA including the test set (features then fit test quirks) — use the training split.
- Trusting `mean` on skewed columns; reading near-zero correlation as "no relationship" (it only rules out a *monotonic* one).

Tools: `pandas` (`describe`, `value_counts`, `corr`), `seaborn` pair plots, `ydata-profiling`.

Self-check

1. A customer-ID column strongly predicts the target? → Leakage (IDs assigned in a way that encodes outcome or time).

TL;DR

Look before you model: types, missingness, distributions, target balance, relationships, duplicates, impossible values. Plots reveal shapes single numbers hide.

4.3 Data cleaning

What it is

Fixing or removing **duplicates**, **impossible values**, **outliers** and **missing values** so the model learns from reality, not errors.

Why it matters

Duplicates leak between train and test and overweight examples; typos distort means and scales; many **estimators** (scikit-learn models, e.g. `LinearRegression`, `SVC`) reject NaNs (the missing-value marker) outright.

How it works

1. Duplicates. `df.drop_duplicates()`, then check the entity key (`customer_id`) is unique. Near-duplicates (different whitespace, a resized image) need fuzzy matching.

2. Impossible values are errors, not outliers. Age 999 or income 9,999,999 is a typo or sentinel: convert to NaN and use the missing-value strategy.

3. Outliers are legitimate-but-extreme values: detect, then keep, cap, transform or remove.

Method	Rule	Good for	Caveat
IQR fences	Flag x < Q1 − 1.5·IQR or x > Q3 + 1.5·IQR (Q1, Q3 = 25th/75th percentiles; IQR = Q3 − Q1)	Any distribution, robust	Over-flags skewed data
z-score	Flag |x − mean| / std > 3	Roughly normal data	Mean and std are pulled by outliers
Transform first	Log, then z-score	Right-skewed positive data	—
Winsorize / clip	Cap at e.g. 1st/99th percentiles	Keep rows, limit influence	Choose caps on training data only
Model-based	`IsolationForest`, `LocalOutlierFactor`	Multivariate outliers (odd only in combination)	More to tune

4. Missing values. First ask *why*:

- **MCAR** (missing completely at random) — a sensor randomly drops readings. Any sensible **imputation** (filling in) is fine.
- **MAR** (missing at random given other columns) — older customers skip the income field; other columns predict the value.
- **MNAR** (missing not at random) — the value causes the missingness (high earners hide income); add a **missing-indicator** column, since missingness is signal.

Strategy	What it does	Use when
Drop rows / column	Remove incomplete rows or a mostly-empty feature	Very few missing rows (MCAR) / low-value column
Mean / median / mode (`SimpleImputer`)	Fill with a column statistic	Default baseline; median for skew; mode (`most_frequent`) for categories
Constant / "missing" category	Fill with `"missing"`	Categoricals — keeps the absence information
Missing indicator (`add_indicator=True`)	Extra 0/1 "was missing" column	Missingness may be informative (MNAR)
KNN (`KNNImputer`)	Average of the k most similar rows	Other features predict this one; scale first
Iterative / MICE (`IterativeImputer`)	Models each column from the others, round-robin	MAR data with correlated columns (experimental: first `from sklearn.experimental import enable_iterative_imputer`)
Leave it to the model	`HistGradientBoosting*`, XGBoost, LightGBM handle NaN natively	Gradient-boosted trees

```python
# code/cleaning.py (key parts)
df = df.drop_duplicates()
df.loc[df["age"] > 110, "age"] = np.nan                 # impossible -> missing
df.loc[df["income"] >= 9_999_999, "income"] = np.nan    # sentinel  -> missing

q1, q3 = inc.quantile([0.25, 0.75]); iqr = q3 - q1       # IQR fences (z-scores similar)

# Evaluate imputers honestly: hide 300 KNOWN incomes, impute them, measure the error
X_masked[hide, 1] = np.nan
for name, imp in {"mean": SimpleImputer(strategy="mean"), "median": SimpleImputer(strategy="median"),
                  "KNN (k=5)": KNNImputer(n_neighbors=5),
                  "iterative (MICE-like)": IterativeImputer(random_state=0)}.items():
    filled = imp.fit_transform(X_masked)[hide, 1]
    print(name, np.abs(filled - truth).mean())
```

Output:

```
duplicates removed: 25  -> rows now 3000
duplicate customer_ids left: 0
impossible ages -> NaN: 4, sentinel incomes -> NaN: 3

IQR fences: [-29,300, 137,100]  -> 141 flagged
|z| > 3                     -> 50 flagged
|z| > 3 on log(income)      -> 4 flagged  (skew made raw z-score over-flag)
winsorize (clip) to 1st/99th pct: [12,800, 207,332]

missing per column: {'age': 94, 'income': 243, 'device': 60}
churn rate | income missing: 0.152  | present: 0.114

imputer                 MAE on hidden incomes
drop rows (listwise)    n/a  (would keep only 79% of rows)
mean                    32,096
median                  29,903
KNN (k=5)               35,635
iterative (MICE-like)   32,096

device with 'missing' level : {'android': 1314, 'ios': 1036, 'web': 590, 'missing': 60}
...
```

- On skewed income, IQR flags 141 normal high earners and raw z flags 50; after a log only 4 are extreme. **Skew is not the same as outliers.**
- Dropping incomplete rows would discard 21% of the data.
- **Fancy imputers are not automatically better.** Income was generated independently of other columns, so KNN and MICE have nothing to learn (MICE collapses to the mean) and the median wins. The lesson is the *method*: hide known values and measure.
- Churn is 15.2% with income missing vs 11.4% present, but missingness was random by construction: the gap is chance (≈1.8 **standard errors** — the typical size of chance wobble — with 243 rows). Test informative missingness rather than assume it; an indicator lets the model use it if real.

Trade-offs

Simple imputation is fast, stable and easy to reproduce at serving time (when the deployed model answers live requests); model-based imputation can be more accurate but is slower and one more thing to monitor. Any imputation shrinks variance, since every filled value is "typical".

Common mistakes

- Computing imputation statistics (mean, median, clip percentiles) on all data before splitting — leakage (4.8).
- Deleting real, important outliers (fraud *is* the outlier).
- Filling categoricals with the mode and losing the "was missing" signal.

In the interview

30 seconds: *"Dedupe on the entity key, turn impossible values into missing, and treat outliers by whether they're errors or real extremes — often a log transform or clipping at training-set percentiles. For missing data, start with median plus a missing indicator, compare KNN or iterative imputation by masking known values, and fit it all inside a pipeline."*

- *"Mean or median?"* — Median for skewed columns.
- *"Did imputation help?"* — Mask known values and measure error; compare downstream CV scores.

Self-check

1. Why convert 999 ages to NaN rather than clip to 85? → They're errors; clipping invents a specific wrong value.

TL;DR

Dedupe, impossible values → NaN, handle outliers deliberately (log/clip, don't blindly delete), impute with training-set statistics (median + indicator is a strong default), measure imputation quality.

4.4 Encoding categorical variables

What it is

Encoding turns categories like `"ios"` or `"city_17"` into numbers; the choice tells the model how categories relate.

How it works — the four main encoders

Encoder	Output	Example	Use for	Risk
One-hot (`OneHotEncoder`, `pd.get_dummies`)	One 0/1 column per category	`device=ios` → `[0,1,0,0]`	**Nominal** (unordered) categories, low–moderate cardinality	Width explodes with many levels
Ordinal (`OrdinalEncoder`)	One integer column	`basic/standard/premium` → 0/1/2	Truly ordered categories; fine for trees	Imposes false order and equal spacing on nominal data for linear models
Target encoding (`TargetEncoder`)	Category → (smoothed) mean of y	`city_17` → 0.19	High cardinality (hundreds/thousands of levels)	**Target leakage** if computed on the rows the model trains on
Hashing (`FeatureHasher`)	hash(value) mod m buckets	`city_17` → column 13	Huge or unbounded ID spaces, streaming data	**Collisions** (unrelated values share a bucket); not reversible

- **Label encoding** = "category → integer code"; scikit-learn's `LabelEncoder` is for the *target* y — use `OrdinalEncoder` for features.
- **Dummy-variable trap:** with an intercept, k one-hot columns always sum to 1, so they are perfectly collinear with it and coefficients aren't unique. Classical **OLS** (ordinary least squares, unpenalised linear regression, e.g. statsmodels) needs `drop="first"`; scikit-learn's `LinearRegression` still fits (section 5.1); regularised models and trees don't care.
- **Embeddings:** in deep learning each ID gets a learned **dense vector** (e.g. 32 mostly non-zero numbers) — standard for user/item IDs (Chapter 13).
- Unseen categories at prediction time: `OneHotEncoder(handle_unknown="ignore")` outputs all zeros; `TargetEncoder` uses the global mean.

Target encoding and its leakage trap

Naive target encoding uses each category's mean target on the training rows. For a 2–3-row category that mean is essentially *those rows' own labels*: the feature copies the answer, shining on training data and failing on new data.

The fix (scikit-learn's `TargetEncoder`):

1. **Cross-fitting** — in `fit_transform`, split training data into k folds; encode each fold with means from the *other* folds, so no row sees its own label.
2. **Smoothing / shrinkage** — blend each category mean with the global mean by category size, so rare categories stay near the global rate.
3. At `transform` time, use encodings learned on the full training set.

CatBoost (Yandex's gradient-boosting library) instead uses "ordered target statistics": each row is encoded only from rows before it in a random permutation.

```
# code/encoding.py (key parts)
ohe = OneHotEncoder(handle_unknown="ignore", sparse_output=False).fit(df[["device"]].fillna("missing"))
ord_enc = OrdinalEncoder(categories=[["basic", "standard", "premium"]])      # explicit order
fh = FeatureHasher(n_features=16, input_type="string")                        # 16 buckets

df["noise_id"] = rng.integers(0, 1000, len(df)).astype(str)                   # pure noise, 1000 levels
means = pd.Series(y[tr]).groupby(df["noise_id"].iloc[tr].to_numpy()).mean()  # naive: leaky
naive_tr = df["noise_id"].iloc[tr].map(means)
te_enc = TargetEncoder(target_type="binary", cv=KFold(5, shuffle=True, random_state=0))
cf_tr = te_enc.fit_transform(df[["noise_id"]].iloc[tr], y[tr])                # cross-fitted
```

Output:

```
one-hot columns: ['device_android', 'device_ios', 'device_missing', 'device_web']
encode ['ios'], ['tablet'(unseen)]:
 [[0. 1. 0. 0.]
 [0. 0. 0. 0.]]

ordinal plan: [2. 0. 1.]

hashed city (16 buckets), row 0 non-zeros at: [13] [-1.]

Feature = target-encoded 'noise_id' (contains NO real signal):
naive target mean (leaky)              train AUC 0.941 | test AUC 0.519
TargetEncoder (cross-fitted, smoothed) train AUC 0.533 | test AUC 0.518

Feature = target-encoded 'city' (60 levels, real signal):
TargetEncoder(city)                    train AUC 0.632 | test AUC 0.555
OneHot(city)                           train AUC 0.722 | test AUC 0.554
```

AUC (area under the ROC curve, Chapter 6): 0.5 = random ranking, 1.0 = perfect. Naive encoding of pure noise scores **0.94 on training** but 0.52 on test — a mirage cross-fitting removes. On real `city` signal, target encoding matches one-hot on test with **one column instead of 60**. The hashed value is −1 because `FeatureHasher` uses a second hash for the sign, so collisions cancel on average.

When not to use them

- No one-hot of a 100,000-level ID for a linear model — hash, target-encode or embed.
- No ordinal encoding of nominal categories for linear models or k-NN.
- No target encoding without cross-fitting, nor with validation/test statistics.

Real-world examples

Feature hashing ("the hashing trick") was popularised in large-scale spam filtering and ad click prediction; built into Vowpal Wabbit and scikit-learn's `FeatureHasher` / `HashingVectorizer`. CatBoost exists largely to handle categoricals safely.

In the interview

30 seconds: *"Low-cardinality nominal → one-hot. Ordered → ordinal. High-cardinality → target encoding (cross-fitted and smoothed), hashing if the ID space is unbounded, or embeddings in a neural model. Target encoding's risk is a row's own label leaking into its feature, so I encode out-of-fold (using statistics from the other folds only)."*

- *"Hashing vs target encoding?"* — Hashing needs no labels or state and handles new values, but collides and isn't interpretable; target encoding is compact and informative but needs leakage protection and can go stale.
- *"Why is ordinal OK for trees?"* — Threshold splits can isolate any level with a few extra splits.

TL;DR

One-hot for small unordered sets, ordinal for ordered, target encoding (out-of-fold, smoothed) or hashing for huge ones, embeddings in deep models. Target encoding is the most powerful and the easiest to leak.

4.5 Feature scaling

What it is

Scaling puts numeric features on comparable ranges so one measured in thousands (income; proline in the wine dataset) doesn't dominate one in fractions.

Scaler	Formula	Result	Sensitive to outliers?
Standardization (`StandardScaler`)	z = (x − mean) / std	mean 0, std 1	Yes
Min-max (`MinMaxScaler`)	(x − min) / (max − min)	range [0, 1]	Very (one outlier squashes the rest)
Robust (`RobustScaler`)	(x − median) / IQR	median 0, IQR 1	No
Log / power (`np.log1p`, `PowerTransformer`)	Reshapes the distribution	Less skew	Reduces their influence

Scaling changes *units*, not *shape*: a skewed feature stays skewed after `StandardScaler`; use log/power to change shape.

Why it matters — and which models care

- **Distance-based** (k-NN; k-means; RBF-kernel SVM, which measures similarity with a bell-shaped function of distance): the widest feature dominates distance.

- **Gradient-descent** models (linear/logistic regression, neural nets) converge much faster on similar scales (section 5.4).
- **Regularised** models (penalty on large weights: Ridge, Lasso, logistic regression's default L2): coefficient size depends on units.
- **PCA** (compresses features into summary directions) finds largest variance — unscaled, just the biggest-unit column.
- **Trees don't care**: "income ≤ 50,000" works in dollars or thousands.

```python
# code/scaling.py (key parts)
x = np.array([[10.0], [12], [11], [13], [12], [11], [100]])
for s in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
    print(type(s).__name__, s.fit_transform(x).ravel().round(2))

X, y = load_wine(return_X_y=True)     # 13 features on very different scales
for name, m in models.items():
    accs = [cross_val_score(m, X, y, cv=5).mean()]
    for sc in [StandardScaler(), MinMaxScaler(), RobustScaler()]:
        accs.append(cross_val_score(make_pipeline(sc, m), X, y, cv=5).mean())
```

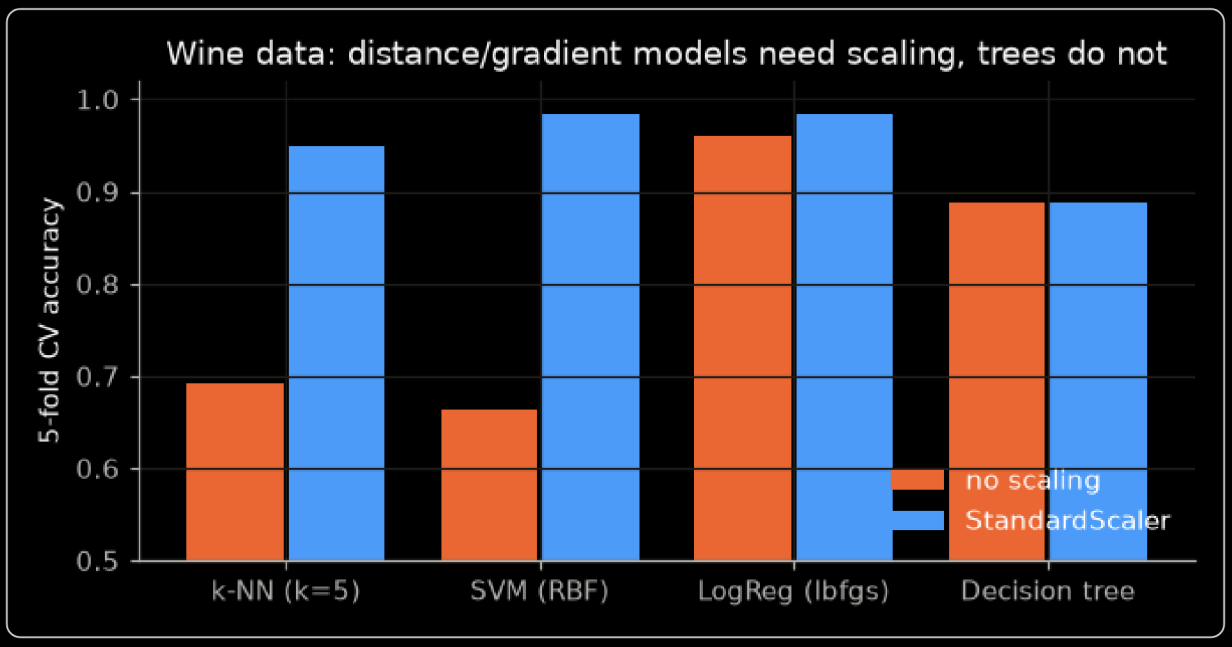
Output:

```
raw       : [ 10.  12.  11.  13.  12.  11. 100.]
StandardScaler : [-0.46 -0.39 -0.42 -0.36 -0.39 -0.42  2.45]
MinMaxScaler   : [0.   0.02 0.01 0.03 0.02 0.01 1.  ]
RobustScaler   : [-1.33  0.   -0.67  0.67  0.   -0.67 58.67]

feature ranges: min std = 0.124, max std = 314.0

model             no scaling  standard  min-max  robust
k-NN (k=5)           0.691     0.949     0.950     0.938
SVM (RBF)            0.663     0.983     0.978     0.983
LogReg (lbfgs)       0.961     0.983     0.978     0.989
Decision tree        0.888     0.888     0.888     0.888
```

Robust check: median 12, Q1 11, Q3 12.5, IQR 1.5, so 100 → 58.67; the six normal values keep a useful spread (−1.33 to 0.67), whereas min-max squeezes them into [0, 0.03].



Trade-offs / when to use which

StandardScaler by default (linear models, SVMs, neural nets, PCA); **MinMaxScaler** for a bounded range (pixels to [0, 1]) without big outliers; **RobustScaler** with outliers you keep; **nothing** for tree ensembles.

Common mistakes

- Fitting the scaler before splitting (leakage, 4.8) — fit on train, `transform` the rest.
- Scaling one-hot columns is harmless but unnecessary; scaling a regression *target* requires inverting predictions (`TransformedTargetRegressor`).
- Expecting scaling to fix skew.

In the interview

30 seconds: *"Scale for anything that uses distances, gradients, or penalties — k-NN, SVM, linear models, neural nets, PCA, k-means. Standardize by default, robust-scale if there are outliers, skip it for trees. Always fit the scaler on the training fold only."*

- *"Does logistic regression need scaling?"* — Not for correctness; yes for convergence speed and fair regularisation.

TL;DR

Standardize for distance/gradient/penalty models, robust-scale with outliers, nothing for trees, log for skew — fit on training data only.

4.6 Feature creation

What it is

Feature engineering creates inputs that make the pattern easier to learn. A linear model draws straight lines in the features you give it; a good feature bends the problem so a straight line works.

How it works — the core techniques

1. **Interactions and polynomials.** If the target depends on a *product* (revenue = price × quantity), add it. `PolynomialFeatures(degree=2)` adds all squares and pairwise products.
2. **Binning (discretization).** Cut a numeric feature into intervals (equal-width, quantile = equal rows per bin, or domain-chosen) and one-hot them (`KBinsDiscretizer`), so a linear model can fit non-monotonic shapes (risk high for both young and old).
3. **Log/power transforms for heavy tails** (most values small, a few enormous: income, counts, prices). `log1p(x) = log(1 + x)` compresses the tail and handles zeros.
4. **Date/time features.** Parts (month, day of week, hour, is_weekend, is_holiday), durations ("days since signup"), and **cyclical encoding** for periodic values: month m → (sin(2πm/12), cos(2πm/12)), so December and January are neighbours, not 11 apart.
5. **Text: bag-of-words and TF-IDF. Bag-of-words** counts words per document. **TF-IDF** (term frequency × inverse document frequency) down-weights words common across documents. scikit-learn default: tf = raw count, idf(t) = ln((1 + n) / (1 + df(t))) + 1 (n = documents, df(t) = documents containing t), then each row scaled to unit L2 length.
6. **Aggregates and ratios.** "Tickets per month of tenure", "spend in the last 7 days", "average order value" — often the strongest business features.

```
# code/features.py (key parts)
revenue = price * qty + rng.normal(0, 1, n)
lin   = cross_val_score(LinearRegression(), X, revenue, cv=5, scoring="r2").mean()
inter = cross_val_score(make_pipeline(PolynomialFeatures(2, interaction_only=True, include_bias=False),
                                      LinearRegression()), X, revenue, cv=5, scoring="r2").mean()

p = 1 / (1 + np.exp(-(0.004 * (age - 50) ** 2 - 2)))       # U-shaped risk
binned = make_pipeline(KBinsDiscretizer(n_bins=6, encode="onehot", strategy="quantile",
                                        quantile_method="averaged_inverted_cdf"), LogisticRegression())

"month_sin": np.sin(2 * np.pi * ts.dt.month / 12), "month_cos": np.cos(2 * np.pi * ts.dt.month / 12)

docs = ["app is slow and expensive", "love the app, fast and easy",
        "slow slow slow, will cancel", "easy to use and helpful support"]
M = TfidfVectorizer().fit_transform(docs)
```

Output:

```
revenue ~ price, qty          R^2 = 0.893
revenue ~ price, qty, price*qty R^2 = 0.997
PolynomialFeatures(2) names: ['price' 'qty' 'price^2' 'price qty' 'qty^2']

U-shaped age effect: raw age AUC = 0.531 | 6 quantile bins AUC = 0.747

 signup_date  year  month  dayofweek  is_weekend  days_since  month_sin  month_cos
 2025-10-09  2025     10          3           0          84     -0.866        0.5
 2024-12-11  2024     12          2           0         386      0.000        1.0
 ...

vocabulary: ['and', 'app', 'cancel', 'easy', 'expensive', 'fast', 'helpful', 'is', 'love', 'slow', 'support', 'the', 'to', 'use',
'will']
doc 2 non-zero weights: {'cancel': 0.363, 'slow': 0.858, 'will': 0.363}
idf('slow') = 1.511, idf('and') = 1.223
churn ~ TF-IDF(review) alone: AUC = 0.736
```

TF-IDF by hand, doc 2 ("slow slow slow, will cancel"), n = 4:

- idf(slow) (2 docs) = ln(5/3) + 1 = 1.511; idf(will), idf(cancel) (1 doc) = ln(5/2) + 1 = 1.916; "and" (3 docs) = 1.223 — common words get low weight.
- tf·idf: slow 3 × 1.511 = 4.533; will 1.916; cancel 1.916.
- L2 norm = √27.89 = 5.281 → slow 0.858, will 0.363, cancel 0.363. ✓

The interaction lifts **R²** (share of target variation explained) from 0.893 to 0.997 because the truth is a product; binning lifts AUC from 0.53 (one slope can't draw a U) to 0.75.

Feature groups in production systems

Brainstorm in four groups so no family is missed. For a food-delivery **ETA** (estimated time of arrival) at checkout:

- **Entity** — one participant, changing rarely (restaurant's typical kitchen time, courier's vehicle).
- **Behavioural aggregate** — count/average over a recent window (orders in the past 30 minutes); worthless unless fresh.
- **Context** — the request moment (hour, weekday, rain, trip distance).
- **Cross** — a pair of participants (this courier's past speed at this restaurant).

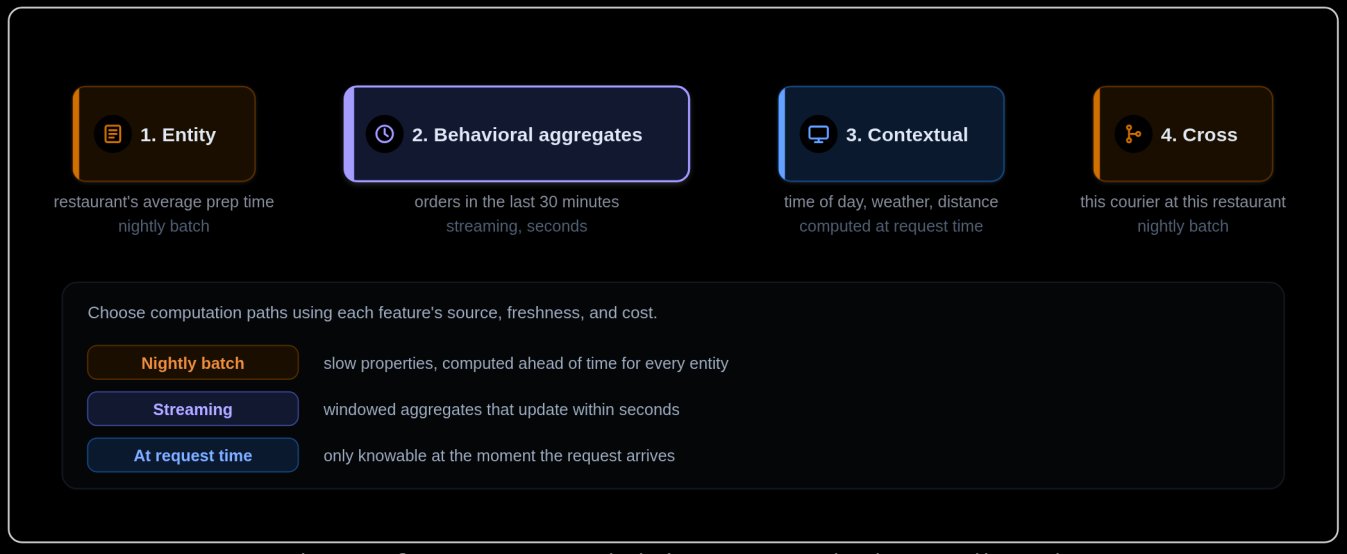


Figure — feature groups and their compute paths (course diagram)

Rule from day one: **point-in-time availability** — every feature must be computable *at prediction time* from data that existed then (4.8). How each group is computed and served (nightly batch vs streaming, the offline/online **feature store** — serving identical values to training and live prediction — latency budgets, see System Design Fundamentals, and fallbacks) is in Chapter 16, section 16.4.

Trade-offs

More features capture more signal but add overfitting risk, compute, latency and maintenance (each is a pipeline that can break); degree-2 polynomials on 100 features give 5,150 columns. Trees and neural nets learn interactions themselves, so explicit engineering matters most for linear models and for domain knowledge (ratios, time windows) no model can invent.

Common mistakes

- Features using future information (a 30-day average including days *after* the prediction).
- Month or hour as a plain number.

- Fitting `TfidfVectorizer` or bin edges on all data.

Real-world examples

- **Uber** and **DoorDash** have described ETA models combining entity, real-time aggregate and context features.
- Search engines long ranked with TF-IDF-style scores; **BM25**, its refined descendant, is still a first-stage retriever (picks a few hundred candidates before a slower ranker).
- Google's **Wide & Deep** recommender (2016) used hand-crafted cross features in its "wide" linear part.

In the interview

30 seconds: *"I'd brainstorm features by group — entity, behavioural aggregates, request context, crosses — check each is available at prediction time, then transform them: log for heavy tails, one-hot or embeddings for categories, cyclical encoding for time, TF-IDF or embeddings for text, products or bins where a linear model needs curvature."*

- *"Why cyclical encoding?"* — So hour 23 and hour 0 are neighbours.

Self-check

1. Encode hour = 6 cyclically. → (sin(2π·6/24), cos(2π·6/24)) = (1, 0).
2. Why is "orders in the last 30 minutes" risky to compute nightly? → Useful only when fresh; stale it is noise (16.4).

TL;DR

Give the model the problem's shape: products for interactions, bins/polynomials for curves, logs for tails, sin/cos for cycles, TF-IDF for text, aggregates for behaviour — all available at prediction time.

4.7 Train, validation and test splits

What it is

Splits make the reported score reflect **unseen** data:

- **Training set** — learn parameters.
- **Validation set** — compare models and tune **hyperparameters** (settings chosen before training, like tree depth).
- **Test set** — touched **once**, at the end, for the unbiased estimate.

How it works

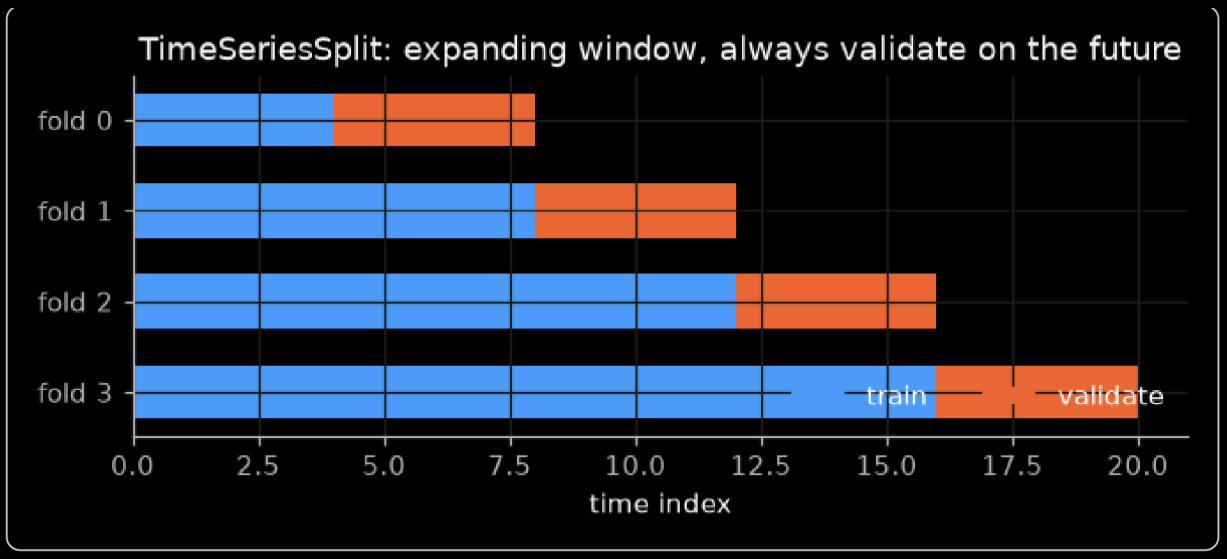
1. **Random split** — shuffle, then e.g. 60/20/20, or 80/20 with cross-validation on the 80% instead of a fixed validation set (Chapter 6).
2. **Stratified split** — same class proportions in every split (`stratify=y`); essential for imbalanced or small data.
3. **Time-based split** — train on the past, validate/test on the future (`TimeSeriesSplit` or a date cut); needed whenever the model predicts the future (almost every production system).
4. **Group split** — all rows of one entity (customer, patient, device) in one split (`GroupKFold`, `GroupShuffleSplit`), when the model must work for *new* entities.

```
# code/splits_and_leakage.py (key parts)
X_tmp, X_test, y_tmp, y_test = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)
X_train, X_val, y_train, y_val = train_test_split(X_tmp, y_tmp, test_size=0.25, stratify=y_tmp,
                                                  random_state=0)      # 0.25 of 80% = 20%
for tr, va in TimeSeriesSplit(n_splits=4).split(np.arange(20)): ...
GroupKFold(n_splits=5).split(X, groups=customer_ids)
```

Output:

```
sizes train/val/test: 600 200 200
positive rate  all/train/val/test: 0.045 0.045 0.045 0.045
WITHOUT stratify, test positive rate over 200 random splits: min 0.005  max 0.080
fold 0: train t=0..3  validate t=4..7
...
fold 3: train t=0..15  validate t=16..19
GroupKFold val customers: [4]  train customers: [0, 1, 2, 3]
...
```

With 4.5% positives and 200 test rows, an unstratified split can leave 1 positive (0.5%) or 16 (8%), so the metric swings wildly; stratification pins it at 4.5%.



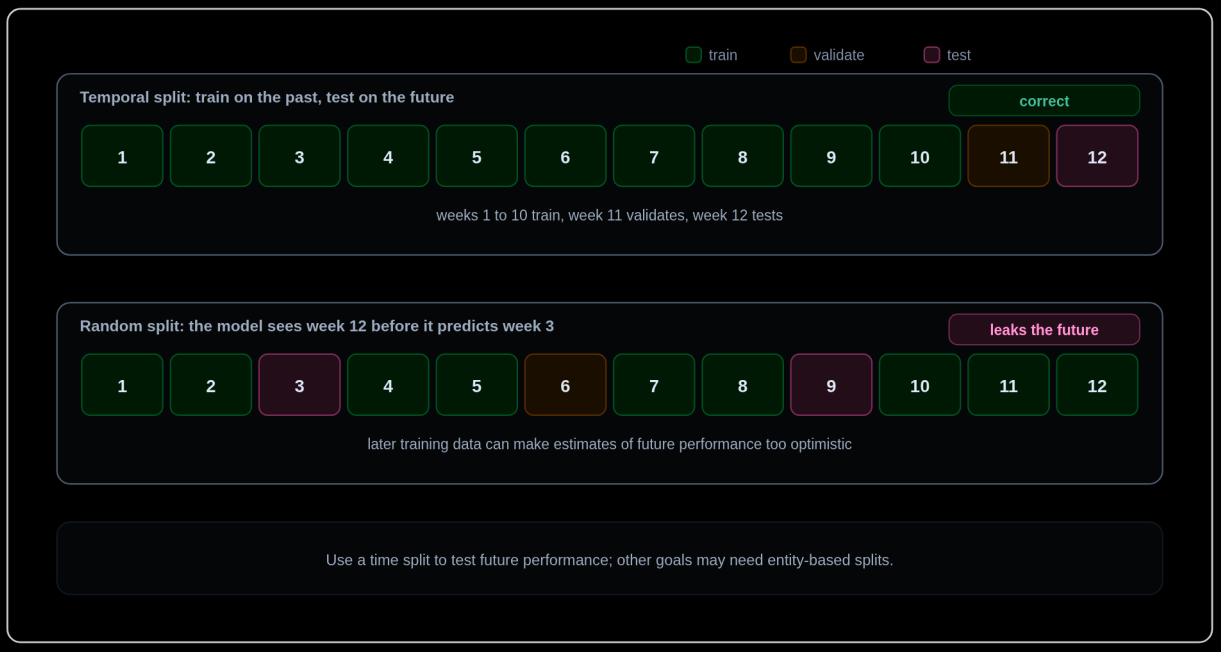


Figure — temporal vs random split (course diagram)

For "future behaviour of *known* users" the same user may appear in train and test (at different times); for "generalise to *new* users", split by user.

Trade-offs

Split	Pros	Cons
Random	Simple, data-efficient	Optimistic with time order or repeated entities
Stratified	Stable class ratios	Classification labels (or binned targets) only
Time-based	Matches deployment; exposes **drift** (data changing over time)	Less training data in early folds; must respect label delay
Group	Honest for new entities	Fewer effective samples; uneven group sizes

Sizes: with millions of rows 98/1/1 is fine (1% = tens of thousands); with a few thousand, cross-validate instead of one validation set.

Common mistakes

- Tuning on the test set ("peeking") — biased upward.
- Random splits on time-ordered data — training on the future.
- Splitting *after* oversampling (4.9) or botched deduplication, so copies land in train and test.

TL;DR

Train to learn, validate to choose, test once to report. Stratify for classification; split by time to predict the future; by group to predict for new entities.

4.8 Data leakage

What it is

Data leakage is information unavailable at prediction time sneaking into training or evaluation: the model looks brilliant offline (on historical data) and disappoints in production. It is the most common cause of "too good to be true" results (introduced in Chapter 1, section 1.10).

The main kinds — and how to prevent each

Kind	Example	Prevention
Target leakage — feature is a consequence or proxy of the label	`refund_issued` or `case_resolution_code` filled only *after* churn/fraud	Ask of every feature: *would I know this at the moment of prediction?*
Future data in aggregates (not point-in-time correct)	Per-merchant fraud rate whose window contains the transaction being scored	**Point-in-time joins**: each feature's value as of prediction time
Train–test contamination — preprocessing fit on all data	Scaler, imputer, feature selection or target encoder fit before the split	Split **first**; every learned step inside a `Pipeline`, refit per fold
Duplicates across splits	Same document, image or patient in train and test	Deduplicate (incl. near-duplicates) *across* splits; group splits
Temporal leakage — random split on time-ordered data	Training on next month to predict this month	Split by time
Leaky identifiers	IDs assigned in an order correlated with the outcome	Drop IDs; suspect any feature with huge importance

Famous case: in the KDD Cup 2008 breast-cancer challenge (a long-running data-mining competition), patient IDs predicted the label because of how the data was assembled — a leak, not medicine.

Three runnable demonstrations

```
# (a) Feature selection BEFORE cross-validation on pure noise (100 rows, 5,000 random features)
leaky_X = SelectKBest(f_classif, k=20).fit_transform(Xn, yn)         # looked at ALL labels
leaky  = cross_val_score(LogisticRegression(max_iter=1000), leaky_X, yn, cv=5).mean()
honest = cross_val_score(make_pipeline(SelectKBest(f_classif, k=20),
                                       LogisticRegression(max_iter=1000)), Xn, yn, cv=5).mean()

# (b) 'refund_issued' is set for 90% of churners, 2% of others - but only AFTER churn
# (c) Drifting data: random split vs train-on-past / test-on-future
```

Output:

```
(a) pure-noise data: select-then-CV accuracy = 0.90 | select-inside-CV = 0.57
(b) with refund_issued: offline AUC 0.975 | without: 0.751 | with it at serving time (field is empty): 0.750
    recall at threshold 0.5: offline 0.89 -> in production 0.00
(c) drifting data: random-split AUC 0.751 | train-on-past, test-on-future AUC 0.551
```

- **(a)** Honest accuracy on noise is ~0.5 (0.57 is chance on 100 rows); selecting with *all* labels reports **90%** because selection saw the validation labels. Selection inside the pipeline fixes it.
- **(b)** The leaky feature lifts offline AUC from 0.75 to 0.975; in production the field is always empty, so at the offline-tuned **threshold** (probability cut-off for "will churn") the model catches **0%** of churners.
- **(c)** With drift, a random split mixes future rows into training and reports 0.75; the honest forward-in-time estimate is 0.55.

Be suspicious of any score far above published baselines — investigate before celebrating.

In the interview

30 seconds: *"Leakage is training on information you won't have at prediction time. I prevent it with point-in-time-correct features, time or entity splits that match deployment, all preprocessing inside the CV pipeline, deduplication across splits, and treating suspiciously strong features as bugs until proven otherwise."*

- *"Offline AUC is 0.99 on fraud. What first?"* — Feature timestamps vs label time (post-outcome fields, aggregates including the current event), split method, duplicates, top **feature importances** (how much the model relies on each feature).
- *"Is fitting a StandardScaler on all data really a problem?"* — Usually a small bias, but the same mistake becomes huge with target encoding or feature selection, so never do it.

Self-check

1. A label-adjacent feature for fraud? → Chargeback flag, refund flag, case-resolution code.

TL;DR

If the model couldn't know it at prediction time, it mustn't see it in training. Split first, pipeline everything, split by time/group, dedupe across splits, distrust scores that look too good.

4.9 Class imbalance

What it is

Class imbalance: one class is much rarer — fraud (e.g. 0.2%, 1 in 500), disease, churn, ad clicks — and it is usually the one you care about.

Why it matters

Accuracy lies: at 99.8% legitimate, "always legitimate" is 99.8% accurate and catches no fraud. **Learning is harder**: negatives dominate the **loss** (the error training minimises), so models predict low probabilities everywhere. Millions of easy negatives **waste compute**.

How it works — the toolbox

Technique	What it does	Effect on probabilities
Better metrics	Precision, recall, F1, PR-AUC instead of accuracy (Chapter 6)	—
Threshold moving	Keep the model; lower the threshold from 0.5	None
Class weights (`class_weight="balanced"`)	Positives count more in the loss (weight ∝ 1/class frequency)	Inflated — recalibrate
Random oversampling	Duplicate minority rows	Inflated; risk overfitting repeats
Random undersampling / negative downsampling	Keep all positives, a fraction w of negatives	Inflated; training set shrinks to ~w, so ~1/w times cheaper
SMOTE	*Synthetic* minority points between a minority point and one of its k nearest minority neighbours	Inflated; can blur boundaries
Recalibration	Map scores back to real-world probabilities	Fixes the above

> **Key terms:** *Precision* = share of flagged rows truly positive; *recall* = share of real positives caught; *F1* is high only when both are. *ROC-AUC* = how well positives rank above negatives (0.5 random, 1 perfect); *PR-AUC* summarises precision–recall across thresholds and, unlike ROC-AUC, is very sensitive to positive rarity — the more honest ranking metric for rare classes. Chapter 6 covers them.

SMOTE (Synthetic Minority Over-sampling Technique, Chawla et al., 2002): for minority point a, pick one of its k nearest minority neighbours b, create a + u·(b − a), u random in [0, 1]. Library: `imbalanced-learn` (`imblearn.over_sampling.SMOTE`); written from scratch here.

Recalibrating after negative downsampling. Keeping a fraction w of negatives inflates the **odds** p/(1 − p) by 1/w. Multiply back by w:

odds_true = w · p/(1 − p) → p_true = odds_true / (1 + odds_true) = p·w / (p·w + (1 − p))

Worked example: w = 0.1, p = 0.5 → p_true = 0.05 / 0.55 = **0.091**. (Keeping 1 in 10 negatives at a 0.2% base rate raises the positive share to ~2%.) General recalibration (Platt scaling, isotonic regression: mapping raw scores to honest probabilities) is in Chapter 6.

Resample only the training data, after splitting — validation and test must reflect the real population.

```python
# code/imbalance.py (key parts) — 20,000 rows, ~2.2% positives, gradient-boosted trees
def smote(X_min, n_new, k=5, seed=0):
    rng = np.random.default_rng(seed)
    _, idx = NearestNeighbors(n_neighbors=k + 1).fit(X_min).kneighbors(X_min)
    base = rng.integers(0, len(X_min), n_new)              # random minority points a
    neigh = idx[base, rng.integers(1, k + 1, n_new)]       # one of their k neighbours b
    return X_min[base] + rng.random((n_new, 1)) * (X_min[neigh] - X_min[base])   # a + u(b - a)

report("GBM class_weight=bal.", Model(class_weight="balanced").fit(X_tr, y_tr), X_te, y_te)
keep = np.r_[pos, rng.choice(neg, int(len(neg) * 0.10), replace=False)]   # downsample negatives
p_corr = p_s * w / (p_s * w + (1 - p_s))                                   # prior correction
```

Output:

```
positive rate: train 0.0221, test 0.0222  (about 1 in 45)

method                        acc   prec    rec     F1 ROC-AUC  PR-AUC   mean p
majority-class baseline     0.978  0.000  0.000  0.000   0.500   0.022   0.0000
GBM, no fix                 0.980  0.604  0.218  0.320   0.905   0.402   0.0151
GBM class_weight=bal.       0.969  0.376  0.579  0.456   0.906   0.402   0.0668
random oversampling         0.967  0.353  0.571  0.437   0.910   0.412   0.0659
SMOTE (from scratch)        0.936  0.209  0.684  0.320   0.901   0.384   0.1196
downsample negatives 1:10   0.958  0.283  0.571  0.378   0.916   0.368   0.0729

after downsampling, mean predicted p = 0.0729 vs true rate 0.0222
after prior correction p' = p*w / (p*w + 1 - p): mean p' = 0.0208 (ranking unchanged: ROC-AUC 0.916)
plain model, threshold  0.5: precision 0.604  recall 0.218
plain model, threshold  0.2: precision 0.490  recall 0.376
plain model, threshold  0.1: precision 0.426  recall 0.496
```

- The do-nothing baseline has **97.8% accuracy** and zero recall.
- **Ranking metrics barely change** (ROC-AUC ≈ 0.90–0.92, PR-AUC ≈ 0.37–0.41): resampling and weighting mostly *move the operating point* (the precision/recall balance at a given threshold) — which threshold moving on the plain model does for free.
- Every rebalancing method inflates mean predicted probability (0.07–0.12 vs 0.022); the prior correction restores 0.021 with ranking untouched.
- SMOTE isn't magic: most recall, worst precision, slightly lower PR-AUC.

Trade-offs

Downsampling is fastest (a real win at billions of rows) but discards data; class weights keep all data at more compute; oversampling/SMOTE enlarge the training set and risk overfitting copies or inventing unrealistic points (SMOTE works poorly with categorical or high-dimensional features); threshold moving is free and keeps **calibrated** probabilities (a predicted 0.3 really means ~30%) but doesn't help the model *learn* the minority. All but threshold moving need recalibration if scores are used as probabilities.

When not to use it

If the model only ranks (top-k alerts) or you'll pick the threshold on a precision–recall curve anyway, rebalancing is often unnecessary. Gradient boosting (sequential tree ensemble, each tree fixing earlier errors) on moderately imbalanced data (5–20% positives) usually does fine — just use the right metric.

Common mistakes

- Oversampling **before** splitting (copies of a positive in train and test); resampling the test set.
- Reading a downsampled model's scores as real probabilities (ad auctions, expected-cost calculations).

Real-world examples

- Facebook's 2014 ad click-prediction paper (He et al.) used negative downsampling and recalibrated with q = p / (p + (1 − p)/w) — the same formula rearranged. Production version with Platt/isotonic calibration: Chapter 16, section 16.3.

In the interview

30 seconds: *"First, change the metric — precision, recall and PR-AUC, not accuracy. Then reweight the loss or downsample negatives for speed, and recalibrate scores to the true base rate if they're used as probabilities. Pick the threshold from the business cost of each error. SMOTE is an option, but I'd compare it with class weights on a stratified validation set."*

- *"Downsample or reweight?"* — Downsample when data is huge and compute matters; reweight when modest. Both need recalibration.

- *"Does class_weight improve ROC-AUC?"* — Often barely; it mainly shifts where 0.5 lands. Keep it if it helps the chosen validation metric.

Self-check

1. Recalibrate p = 0.2 with w = 0.1. → 0.02/(0.02 + 0.8) = 0.024.

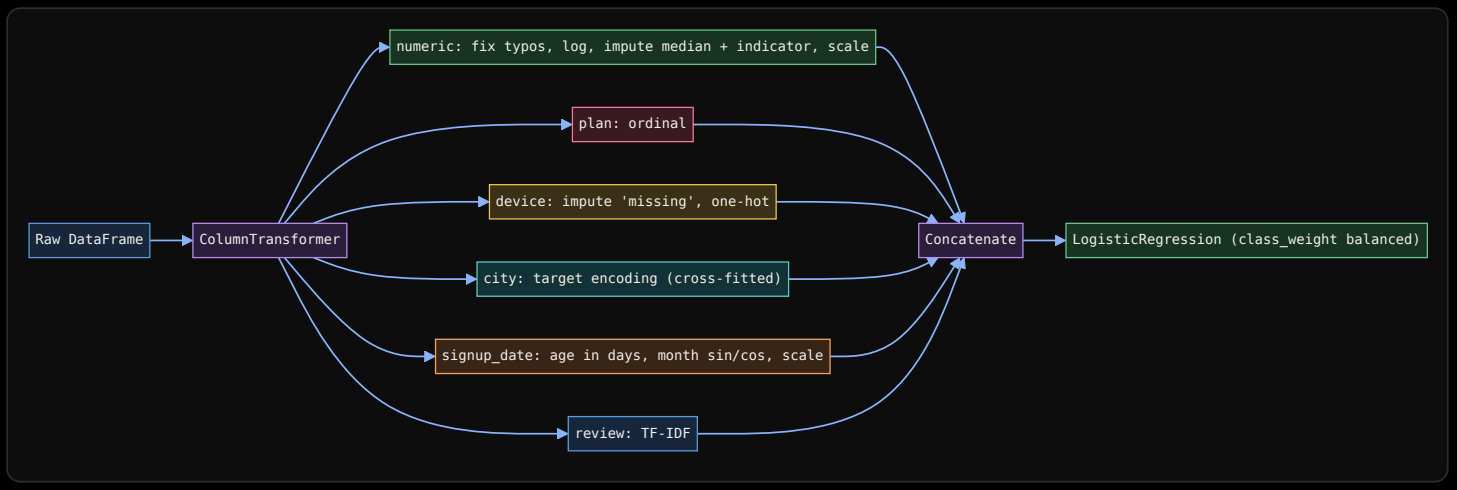
TL;DR

Imbalance breaks accuracy, not learning per se. Use precision/recall/PR-AUC; weight, downsample or oversample the training data; move the threshold to business costs; recalibrate to the real base rate.

4.10 End-to-end: `Pipeline` + `ColumnTransformer`

What it is

A `Pipeline` chains transformers and a final model into one estimator; a `ColumnTransformer` applies different preprocessing per column group and concatenates (API basics: Chapter 3, section 3.5). Together they turn a notebook of manual steps into **one object** you fit, tune, save and serve.



Why it matters

No leakage (every learned step — imputer medians, scaler means, TF-IDF vocabulary, target encodings — refit per training fold), **no training–serving skew** (same transform code at training and prediction), **tuning across steps** (`GridSearchCV` via `step__param`), **one artifact to ship** (`joblib.dump(pipe, "model.joblib")`).

```
# code/pipeline_end_to_end.py (key parts)
def fix_impossible(X):                      # stateless rule: no fitting, safe anywhere
    X = X.copy()
    X["age"] = X["age"].where(X["age"] <= 110)
    X["income"] = np.log1p(X["income"].where(X["income"] < 9_999_999))
    return X

numeric_pipe = make_pipeline(FunctionTransformer(fix_impossible, feature_names_out="one-to-one"),
                             SimpleImputer(strategy="median", add_indicator=True), StandardScaler())
pre = ColumnTransformer([
    ("num", numeric_pipe, numeric),
    ("plan", OrdinalEncoder(categories=[["basic", "standard", "premium"]]), ["plan"]),
    ("device", make_pipeline(SimpleImputer(strategy="constant", fill_value="missing"),
                             OneHotEncoder(handle_unknown="ignore")), ["device"]),
    ("city", TargetEncoder(target_type="binary", cv=KFold(5, shuffle=True, random_state=0)), ["city"]),
    ("date", make_pipeline(FunctionTransformer(date_parts, feature_names_out=date_names),
                           StandardScaler()), ["signup_date"]),
    ("text", TfidfVectorizer(min_df=2), "review"),     # string, not list: TF-IDF wants 1-D text
])                                                     # remainder="drop" drops customer_id
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=2000, class_weight="balanced"))])

X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=42)
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1.0, 10.0, 100.0]},
                      cv=StratifiedKFold(5, shuffle=True, random_state=0), scoring="average_precision")
search.fit(X_tr, y_tr)
p = search.best_estimator_.predict_proba(X_te)[:, 1]      # test set used once
```

Output:

```
best C: 10.0 | CV PR-AUC: 0.414
features after ColumnTransformer: 27
most negative coefs:
 text__fast    -1.12
...
most positive coefs:
 ...
text__cancel    0.76
city__city      4.94
scaled numeric coefs: {'num__age': 0.0, 'num__income': 0.05, 'num__tenure_months': -0.65, 'num__support_tickets': 0.41,
'num__monthly_charges': 0.43, 'num__usage_hours': -0.49}

TEST  model   : ROC-AUC 0.836 | PR-AUC 0.413
TEST  baseline: ROC-AUC 0.500 | PR-AUC 0.117  (PR-AUC of a no-skill model = positive rate 0.117)

raw new row with unseen city + missing device -> p(churn) = 0.368
```

- Real drivers recovered: tenure and usage lower churn; tickets and negative review words raise it; age and income (noise here) ≈ 0. `city` looks huge only because a target-encoded value is a probability (0 to ~0.3), not standardized — compare coefficients only on the same scale.
- PR-AUC 0.41 vs 0.117 no-skill: a large lift. Always report a baseline.
- Best `C` (inverse regularisation strength: smaller = stronger penalty on large weights) is 10, scores flat for C ≥ 10 (0.414 at 10 and 100). Had the winner sat at the grid edge with scores still rising, extend the grid (Chapter 6).
- A raw row with an unseen city and missing device still gets a sensible prediction.

Useful API details

- `remainder="drop"` is the default (`"passthrough"` keeps untouched columns). Pass a list `["plan"]` for 2-D transformers, a string `"review"` for text vectorizers expecting 1-D input.
- `pipe.named_steps["pre"].get_feature_names_out()` names every output column (custom `FunctionTransformer`s need `feature_names_out`); `set_output(transform="pandas")` returns DataFrames (not with **sparse** TF-IDF output, which stores only non-zero entries).
- Custom pipeline functions must be importable (module-level) for `joblib` save/load.

Common mistakes

- Preprocessing in pandas *before* the split; forgetting `handle_unknown="ignore"`.
- Data-dependent logic ("clip at the 99th percentile") in a `FunctionTransformer` recomputes per batch; use a fitted transformer.

In the interview

30 seconds: *"One sklearn Pipeline with a ColumnTransformer per column type; tune it with cross-validation so every fold re-fits the preprocessing; evaluate once on a held-out test set against a baseline; ship the whole pipeline as one artifact so training and serving share code."*

- *"How does this prevent leakage?"* — `fit` runs only on training folds; validation folds only see `transform`.
- *"Where does this break at scale?"* — Streaming data and real-time aggregates need a feature pipeline and feature store (Chapter 16); the idea stays.

TL;DR

One `Pipeline` (with a `ColumnTransformer`) = no leakage in CV, no training–serving skew, joint tuning, one deployable artifact.

Chapter TL;DR

- **Labels**: outcomes (late), implicit feedback (exposure and position bias), humans (costly; κ, versioned guidelines), weak supervision, LLMs (audit with human slices).
- **EDA and cleaning**: look first (training split); impossible values → NaN; outliers → log/clip, not blind deletion; median + missing indicator, measured by masking known values.
- **Encoding**: one-hot (small nominal), ordinal (ordered), target encoding (cross-fit + smooth), hashing (unbounded IDs), embeddings (deep models).
- **Scaling**: for distance/gradient/penalty models, not trees; fit on train only.
- **Features**: interactions, bins, logs, sin/cos dates, TF-IDF, aggregates — available at prediction time.
- **Splits and leakage**: stratify; split by time or group to match deployment; split first and pipeline everything.
- **Imbalance**: not accuracy; weight/resample training data or move the threshold; recalibrate $p \cdot w / (p \cdot w + 1 - p)$.

- **Pipeline + ColumnTransformer**: one leak-proof, tunable, deployable object.

Chapter 5 — Linear and Logistic Regression

Linear and logistic regression are ML's "hello world" and still everywhere in production: as **baselines** (simple reference models fancier ones must beat), as the final layer of neural networks, in click-through-rate models (share of shown items clicked), credit scores and A/B-test analyses (experiments comparing two versions on random user groups). They are also the clearest place to learn the machinery every model uses: a **model**, a **loss function**, and an **optimizer** (gradient descent) that lowers the loss.

What you'll learn: the supervised workflow; linear regression fitted by the normal equation and by gradient descent (NumPy vs scikit-learn); GD variants and feature scaling; assumptions and residual plots; polynomial regression and under/overfitting; Ridge, Lasso, ElasticNet; logistic regression from scratch; one-vs-rest and softmax; interpreting coefficients.

All code is in `code/` (Python 3.14, scikit-learn 1.9, NumPy 2.5). Outputs are real; timings vary by machine.

5.1 The supervised-learning workflow

What it is

Supervised learning learns from **labeled data** (examples with the correct answer attached) and predicts answers for **unseen** examples. **Regression** predicts a number; **classification** a category. (Overview: Chapter 1.)

How it works — the steps

1. **Get data** — public datasets (Kaggle, the UCI repository, `sklearn.datasets`) or your own: CSV, Excel, database tables, text.
2. **Inspect and clean** — drop mostly-empty unimportant rows/columns or **impute** (fill in) missing values with mean, median, mode or a default; some models refuse NaNs.
3. **Encode categories** — **label (integer) encoding** (Red → 0, Blue → 1, Green → 2) is fine for trees but tells a linear model Green > Blue > Red; **one-hot encoding** (`is_Red`, `is_Blue`, `is_Green`) avoids the fake order (all encoders: Chapter 4).
4. **Split** into train and test (80/20 or 70/30).
5. **Choose and fit** a model on the training set.
6. **Evaluate** on test with a suitable metric against a trivial baseline (e.g. always predict the average).
7. **Iterate** on features, model, **hyperparameters** (settings chosen before training, e.g. tree depth; Chapter 6).

Choosing between linear regression and a decision tree

	Linear regression	Decision tree
Task	Regression (logistic regression for classification)	Regression and classification
How it works	Fits a line / plane / **hyperplane** (flat surface in many dimensions) minimising squared error	Splits data with if-then questions into groups with similar outcomes
Strengths	Simple, fast, interpretable coefficients, **extrapolates** a trend beyond the data	Non-linear patterns and **interactions** (one feature's effect depending on another), mixed feature types, easy to visualise
Weaknesses	Misses non-linearity without engineered features	**Overfits** (memorises noise) if too deep; can't extrapolate
Pick it when	Roughly linear (weight vs height)	Clear thresholds or if-then structure

(Trees in depth: Chapter 7.)

The workflow in code

Predict **petal length** from the other iris measurements plus one-hot species:

```python
# code/supervised_walkthrough.py
iris = load_iris(as_frame=True)                          # bundled with sklearn, no download
df = iris.frame.rename(columns=lambda c: c.replace(" (cm)", "").replace(" ", "_"))
df["species"] = iris.target_names[df.pop("target")]
print(df.head(3).to_string(), "\n")
print("missing values (total):", df.isna().sum().sum())

X = pd.get_dummies(df.drop(columns="petal_length"), columns=["species"], dtype=int)
y = df["petal_length"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, random_state=42)   # 80/20

lin = LinearRegression().fit(X_tr, y_tr)
print("coefficients:", {c: round(float(w), 3) for c, w in zip(X.columns, lin.coef_)})
print("intercept:", round(float(lin.intercept_), 3))
for name, m in [("linear regression", lin),
                ("tree, no depth limit", DecisionTreeRegressor(random_state=0)),
                ("tree, max_depth=3", DecisionTreeRegressor(max_depth=3, random_state=0))]:
    m.fit(X_tr, y_tr)
    print(name, mean_squared_error(y_tr, m.predict(X_tr)), mean_squared_error(y_te, m.predict(X_te)))
```

Output:

```
   sepal_length  sepal_width  petal_length  petal_width species
0           5.1          3.5           1.4          0.2  setosa
1           4.9          3.0           1.4          0.2  setosa
2           4.7          3.2           1.3          0.2  setosa

missing values (total): 0
model inputs: ['sepal_length', 'sepal_width', 'petal_width', 'species_setosa', 'species_versicolor', 'species_virginica']

coefficients: {'sepal_length': 0.585, 'sepal_width': -0.145, 'petal_width': 0.64, 'species_setosa': -1.144, 'species_versicolor':
0.315, 'species_virginica': 0.829}
intercept: 0.019

y range: 1.0 to 6.9 cm, std 1.77
model                        train MSE  test MSE
baseline (predict mean)                     3.302
linear regression                0.061     0.088
tree, no depth limit             0.001     0.142
tree, max_depth=3                0.078     0.121
```

Interpreting the result. **MSE** (mean squared error, in cm²) only means something against the target's scale and a baseline: predicting the mean gives 3.3, so 0.088 (≈ √0.088 ≈ 0.3 cm typical error) is good; MSE 5 would be worse than guessing the mean. The unlimited tree scores ~0 on training but worse on test: it **overfit**. Limiting `max_depth` (or raising `min_samples_split`) trades training accuracy for **generalisation** (performance on unseen data).

The three species columns always sum to 1, duplicating the intercept (the **dummy-variable trap**). `LinearRegression` still works — it returns the **minimum-norm solution** (of all equally good weight sets, the smallest), which is why the species coefficients sum to 0 — but they are only meaningful relative to each other. Use `drop="first"` (`OneHotEncoder`) or `drop_first=True` (`get_dummies`) to read each against a reference species.

Self-check

1. Train MSE 0.001, test 0.142? → Overfitting.
2. Why one-hot rather than integer-encode `species` for linear regression? → Integers impose fake order and equal spacing.

TL;DR

Get data → clean → encode → split → fit → evaluate against a baseline → iterate. Linear models for roughly linear trends; depth-limited trees for thresholds and interactions.

5.2 Linear regression: model and loss

What it is

Linear regression predicts a number as a weighted sum of features plus a constant:

$\hat{y} = w_1 x_1 + w_2 x_2 + \ldots + w_d x_d + b = \mathbf{w \cdot x + b}$

- $\hat{y}$ ("y-hat") — prediction; x — the **feature vector** of one example.
- w — **weights** (coefficients): change in $\hat{y}$ when that feature rises by 1, others fixed.
- b — **bias** / **intercept**: the prediction when all features are 0.

One feature → line; two → plane; more → hyperplane. Trick used in all the maths: add a constant feature $x_0 = 1$ so b becomes w_0 and $\hat{y} = \theta^T x$ (θ = all parameters).

The loss: mean squared error

For m examples: $MSE(\theta) = (1/m)\ \Sigma_i\ (\hat{y}_i - y_i)^2$

Worked example: y = (3, 5, 7), ŷ = (2.5, 5, 8): errors (−0.5, 0, 1), squared (0.25, 0, 1), MSE = **0.417**; **RMSE** = √0.417 = 0.65 (target units).

Why squares? Errors become positive, big errors are punished more, and the loss is a smooth **convex** bowl with one minimum calculus can find. Minimising MSE is also **maximum likelihood** (parameters under which the data is most probable) when noise is Gaussian.

Self-check

1. Errors (1, −1, 2): MSE? → 2.

TL;DR

ŷ = w·x + b; fit by minimising average squared error — a convex bowl with one minimum.

5.3 Fitting it: the normal equation vs gradient descent

What it is

1. **Normal equation** — solve for θ directly with linear algebra (read the lowest point off a map).
2. **Gradient descent (GD)** — repeatedly step downhill (walk down in fog by feeling the slope). Fundamentals: Chapter 2, section 2.5.

How it works

Normal equation. At the bottom the gradient $(2/m)\ X^T(X\theta - y)$ is zero, giving $X^TX\theta = X^Ty$; if X^TX is invertible:

$\theta = (X^TX)^{-1}\ X^Ty$

X is the m × (d+1) matrix (with the column of 1s), y the targets. Never compute the inverse: solve $X^TX\ \theta = X^Ty$ (`np.linalg.solve`) or use least squares (`np.linalg.lstsq`, which uses the SVD and also works when X^TX is singular). scikit-learn's `LinearRegression` calls SciPy's least-squares solver.

Gradient descent. $\nabla MSE(\theta) = (2/m)\ X^T(X\theta - y)$. Why: by the **chain rule** (multiply the slopes of nested functions), the derivative of one term $(\theta^Tx_i - y_i)^2$ w.r.t. θ_j is $2(\theta^Tx_i - y_i)\cdot x_{ij}$ — "twice the error times that feature"; average and stack. Update $\theta \leftarrow \theta - \eta\ \nabla MSE(\theta)$ (η = **learning rate**); a full pass over the data is an **epoch**.

One step by hand: x = (1, 2), y = (2, 4), w = b = 0, η = 0.1: errors (−2, −4); $\partial/\partial b = -6$, $\partial/\partial w = (1\cdot-2 + 2\cdot-4) = -10$; new b = 0.6, w = 1.0 — heading toward y = 2x.

```python
# code/linreg_from_scratch.py (key parts)
def add_bias(X):
    return np.c_[np.ones(len(X)), X]                  # column of ones -> intercept is w0

def normal_equation(X, y):
    Xb = add_bias(X)
    return np.linalg.solve(Xb.T @ Xb, Xb.T @ y)       # solves (X^T X) theta = X^T y

def batch_gd(X, y, lr=0.1, epochs=1000):
    Xb = add_bias(X)
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        err = Xb @ theta - y
        theta -= lr * (2 / len(y)) * Xb.T @ err       # gradient step on MSE
    return theta

x = 2 * rng.random((100, 1)); y = 4 + 3 * x[:, 0] + rng.normal(0, 1, 100)    # truth: b=4, w=3
# ... then the 10-feature diabetes dataset, standardized, compared with sklearn
```

Output:

```
normal equation  [b, w]: [3.949 3.038]
batch GD         [b, w]: [3.949 3.038] (MSE 51.39 -> 0.959)
sklearn          [b, w]: [3.949 3.038]

method            test MSE   max |diff vs sklearn|      time
normal eq.          3424.3               1.46e-12    0.07ms
batch GD            3424.6               2.68e-01   35.06ms
sklearn             3424.3               0.00e+00
baseline (predict train mean) test MSE: 5134.8
GD after 5000 epochs, coef for s1, s2 (correlated pair): [-31.   15.1] exact: [-31.2  15.3]

largest standardized effects (+1 std in feature -> change in disease score):
    s5:   +35.8
    s1:   -31.2
   bmi:   +27.5
    s2:   +15.3
intercept = mean training target: 151.6 = 151.6
```

All three agree on the toy problem (3.949, 3.038 vs true 4 and 3; the gap is noise). On diabetes the normal equation matches scikit-learn to 12 decimals; GD is still slightly off after 5,000 epochs on `s1` / `s2`, two strongly correlated blood-serum measurements that make a long narrow valley in the **loss surface** where GD crawls (5.4). With standardized (mean 0) features the intercept equals the mean training target.

Trade-offs

	Normal equation / least squares	Gradient descent
Result	Exact, one shot	Approximate, iterative
Hyperparameters	None	Learning rate, epochs (batch size)
Cost	~$O(m \cdot d^2)$ to form X^TX + $O(d^3)$ to solve — fine for d up to a few thousand	$O(m \cdot d)$ per epoch; scales to huge m and d
Memory	Needs X^TX (d × d)	Can stream mini-batches from disk
Feature scaling	Not needed	Needed for fast convergence
Other losses	Only squared error (and Ridge)	Any **differentiable** (smooth, has a slope) loss — logistic regression, neural nets

(Big-O: how cost grows with m rows and d features; $O(d^3)$ means doubling features ≈ 8× slower solve.)

When not to use each

- Normal equation: not for very large d (tens of thousands) or data that doesn't fit in memory.
- Hand-rolled GD: not when a **closed form** (direct formula) or tuned library solver exists — use it for learning and for losses without closed forms.

Common mistakes

- `np.linalg.inv(X.T @ X)` explicitly — slower and numerically less stable (more rounding error) than `solve` / `lstsq`.
- Forgetting the column of ones; running GD on unscaled features and concluding "GD doesn't work".

Real-world examples

- scikit-learn `LinearRegression` (SciPy least squares); `statsmodels` **OLS** (ordinary least squares) for statistical inference (p-values, confidence intervals).
- `SGDRegressor` / `SGDClassifier` and Vowpal Wabbit fit linear models batch by batch with SGD, so data never needs to fit in RAM.
- Every neural network trains with GD variants (SGD with momentum, Adam: smoother step directions, per-weight step sizes).

In the interview

30 seconds: *"Linear regression minimises MSE. You can solve it exactly with the normal equation, $\theta = (X^TX)^{-1}X^Ty$ — in practice with a least-squares solver — which is cheap when there are few features. Gradient descent iterates $\theta \leftarrow \theta - \eta \cdot (2/m)X^T(X\theta - y)$; it scales to huge data, works for any differentiable loss, but needs a learning rate and scaled features."*

- *"When would the normal equation fail?"* — X^TX singular (duplicate/perfectly collinear features, or more features than rows); use the **pseudo-inverse** (generalised inverse via SVD, still least-squares) or Ridge, which makes $X^TX + \alpha I$ invertible (I = identity matrix).
- *"Is MSE convex?"* — Yes, for linear regression; GD with a small enough η reaches the global minimum.

Self-check

1. GD update for linear regression? → $\theta \leftarrow \theta - \eta\ (2/m)\ X^T(X\theta - y)$.
2. Why did GD lag on `s1` / `s2`? → Correlated features make a narrow valley.

TL;DR

Normal equation: exact, no tuning, $O(d^3)$ — great for small d. Gradient descent: iterative, scalable, general — needs a learning rate and scaled features.

5.4 Gradient descent in practice

What it is

Three flavours differing only in **how many examples compute each gradient**:

Variant	Examples per update	Updates per epoch (m = 100)	Behaviour
Batch GD	All m	1	Smooth, exact; slow per step on big data
Mini-batch GD	e.g. 16–512	m / batch size (100/16 → 7)	Noisy but efficient on vectorised hardware (GPUs); deep-learning default
Stochastic GD (SGD)	1	m (100)	Very noisy, cheap; can escape shallow local minima; never settles unless η decays

How it works

Shuffle each epoch; step on each batch's gradient; optionally decay η with a **learning-rate schedule**, e.g. $\eta_t = \eta_0 / (1 + k \cdot t)$, so noisy steps shrink.

```python
# code/gd_variants.py (key parts)
def run(Xb, y, lr, epochs, batch_size, seed=0, lr_decay=False):
    rng = np.random.default_rng(seed)
    theta, step = np.array([-1.0, -1.0]), 0
    for epoch in range(epochs):
        idx = rng.permutation(len(y))                    # reshuffle every epoch
        for start in range(0, len(y), batch_size):
            b = idx[start:start + batch_size]
            eta = lr / (1 + 0.05 * step) if lr_decay else lr
            theta = theta - eta * mse_grad(Xb[b], y[b], theta)
            step += 1
    return theta

run(Xb, y, lr=0.1, epochs=50, batch_size=100)            # batch
run(Xb, y, lr=0.1, epochs=8,  batch_size=16)             # mini-batch
run(Xb, y, lr=0.1, epochs=1,  batch_size=1, lr_decay=True)  # stochastic
```

Output:

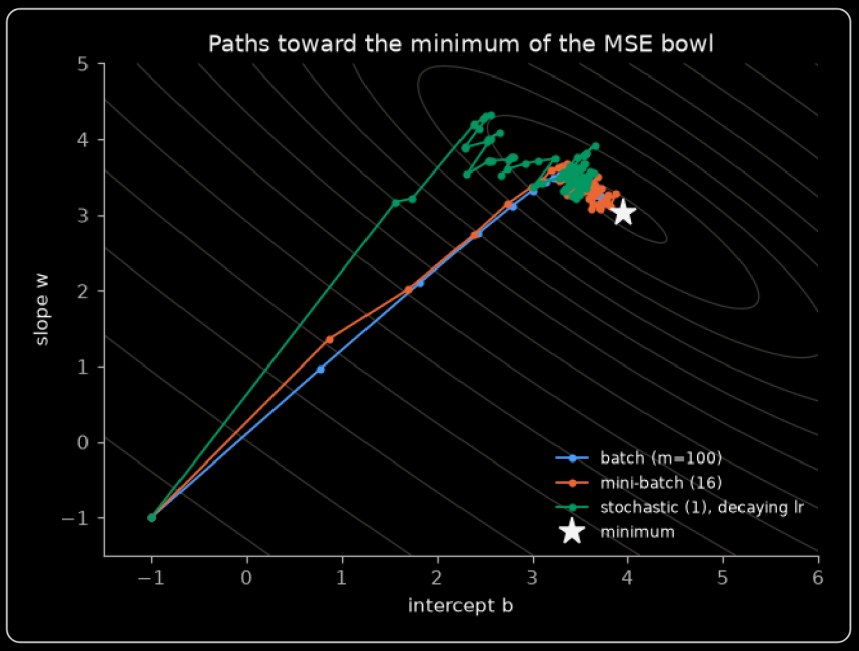
```
lr=0.01  MSE after 60 epochs: 1.37
lr=0.1   MSE after 60 epochs: 0.963
lr=0.6   MSE after 60 epochs: 1.92e+23

exact (normal eq.): [3.949 3.038]
batch (m=100)               updates=  50  final=[3.762 3.203]
mini-batch (16)             updates=  56  final=[3.814 3.109]
stochastic (1), decaying lr  updates= 100  final=[3.574 3.434]
unscaled      condition number        36  lr=1.36e-02  iterations to converge: 485
standardized  condition number         1  lr=4.86e-01  iterations to converge: 5
```

The learning rate

η = 0.01 crawls (MSE 1.37 after 60 epochs); 0.1 reaches the noise floor (≈ 0.96); 0.6 overshoots further every step and explodes to 10^{23} (**divergence**). For MSE, GD converges if $\eta < 2/\lambda_{max}$, where λ_{max} is the largest eigenvalue of the **Hessian** (curvature matrix, here $(2/m)X^TX$). In practice: try a log scale (0.001, 0.01, 0.1, …) and watch the loss curve.

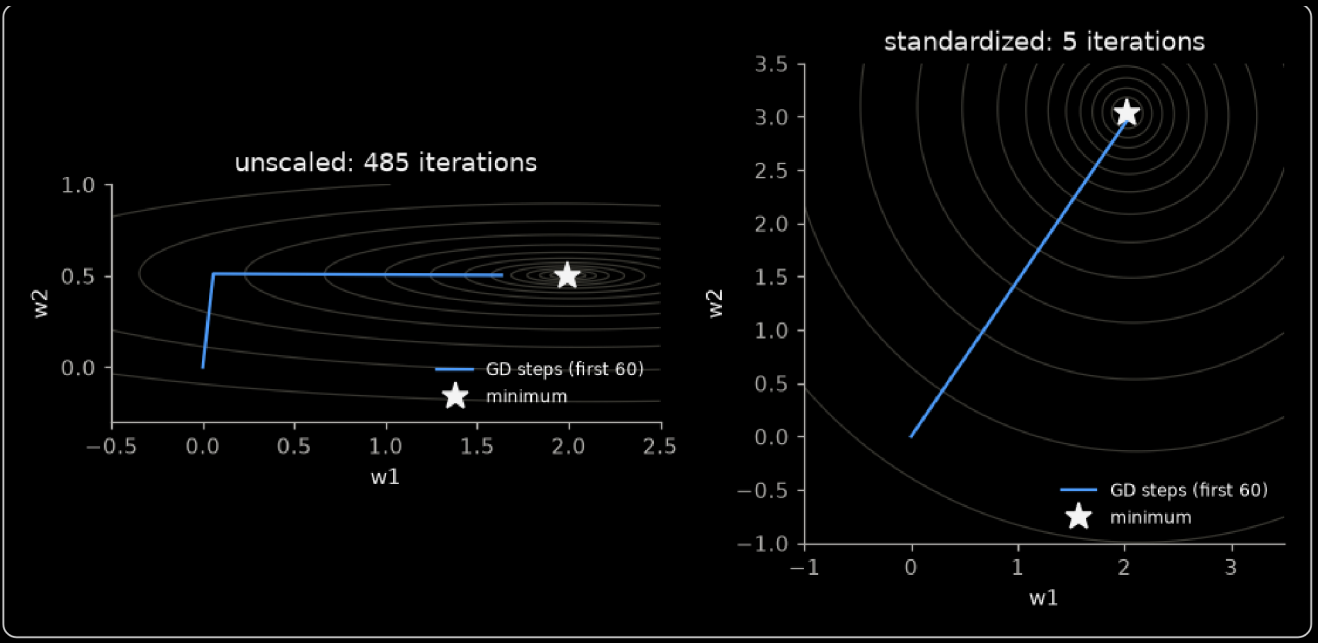




Batch moves smoothly, mini-batch wiggles, SGD bounces and settles only because η decays. With equal update counts batch GD saw 50× more data — per unit of *compute*, SGD and mini-batch are far cheaper on large data, which is why they dominate.

Why feature scaling matters for GD

Unequal spreads turn the MSE bowl into a narrow valley: η must be small for the steep direction, so progress along the flat one crawls. The **condition number** (λ_max / λ_min) measures the stretch; GD's step count grows roughly with it.



A 6× spread gives condition number 36 and **485** iterations; standardized, the bowl is round (≈ 1) and GD needs **5**. At 100× spread, well over 100,000. Correlated features (`s1` / `s2`) stretch the bowl too; scaling doesn't fully fix that — regularization helps.

Trade-offs / common mistakes

- Larger batches: smoother gradients, better hardware use, fewer updates per epoch; very large ones may need a larger η or **warm-up** (ramp η up over the first steps).
- Not shuffling → unrepresentative batches (e.g. sorted by class).
- Judging SGD by its last iterate without η decay — it keeps bouncing.
- Fixed epoch counts instead of monitoring validation loss (**early stopping**, Chapter 6).

In the interview

30 seconds: *"Batch GD uses all data per step — smooth but expensive; SGD uses one example — cheap and noisy, needs a decaying learning rate; mini-batch is the practical middle ground and fits GPUs. The learning rate is the key hyperparameter: too small is slow, too large diverges. Scale features first — otherwise the loss surface is a narrow valley and GD zig-zags or crawls."*

- *"Why does SGD sometimes generalise better?"* — Its noise acts like a **regularizer** (anything discouraging overfitting; 5.7) and can escape sharp minima — an empirical observation, not a guarantee.
- *"How do you pick a learning rate?"* — Log-scale search on the training-loss curve, or a learning-rate range test (raise η each step, note where loss blows up); then add a schedule.

Self-check

1. m = 1,000, batch 50: updates per epoch? → 20.

TL;DR

Batch = exact, slow; SGD = noisy, cheap; mini-batch = default. Tune η on a log scale, decay it for SGD, scale features.

5.5 Assumptions of linear regression

What it is

Linear regression always returns *some* line; whether predictions are good and **coefficients and p-values** trustworthy depends on assumptions:

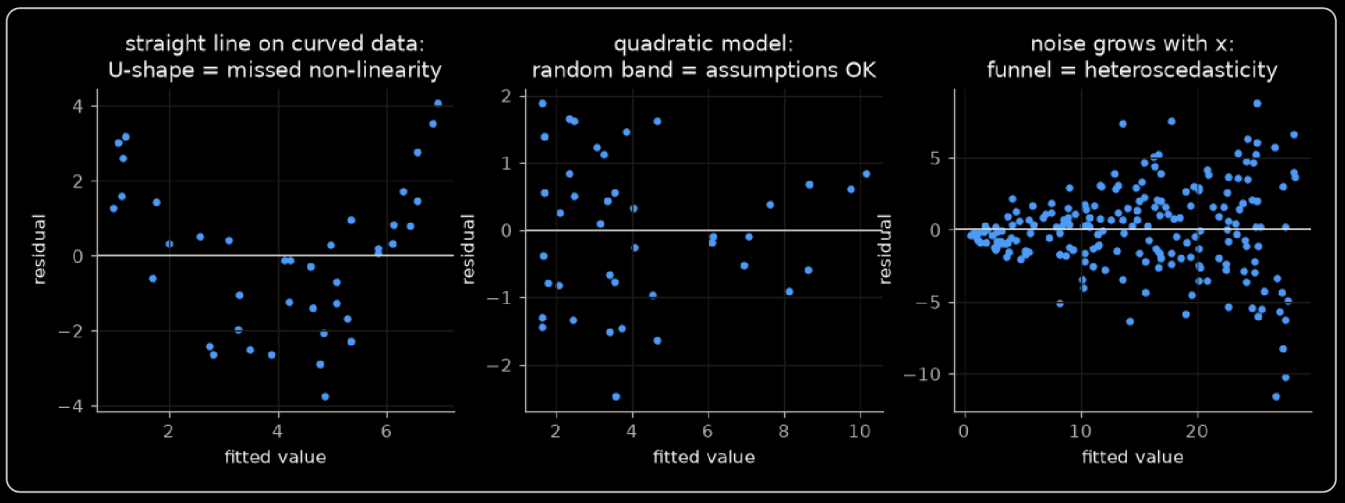
Assumption	Meaning	If violated
Linearity	y is linear in the (possibly transformed) features	Systematic errors; add polynomial/log terms or go non-linear
Independence	Row errors unrelated	Common in time series or repeated measures (several rows per subject); **standard errors** (coefficient uncertainty) too small
Homoscedasticity	Constant error spread	"Funnel" residuals; coefficients still unbiased but standard errors wrong; try log(y) or weighted least squares (noisier rows count less)
Normal errors	Residuals roughly normal	Matters mainly for small-sample CIs and p-values
No (perfect) multicollinearity	Features aren't near-copies	Unstable, uninterpretable coefficients (predictions can be fine)

For prediction, linearity and a stable data distribution matter most; the rest matter when interpreting coefficients or reporting **statistical significance**.

How to check: residual plots

A **residual** is $y_i - \hat{y}_i$. Against fitted values, a good model leaves a structureless band around 0.

```
# code/polynomial_and_assumptions.py (residual part)
lin = LinearRegression().fit(x[:, None], y)            # straight line on quadratic data
resid = y - lin.predict(x[:, None])
ax.scatter(lin.predict(x[:, None]), resid); ax.axhline(0)
```



- **U-shape:** missed curvature — add x^2 or transform.
- **Random band:** the quadratic model captures the pattern.
- **Funnel:** noise grows with x (heteroscedasticity) — log y, weighted least squares, or **robust standard errors** (valid under varying spread).

Common mistakes

- Checking only **R^2** (share of target variance explained) and never plotting residuals.
- Reading coefficients causally from **observational data** (recorded as it happened, not from an experiment).

In the interview

30 seconds: *"Linear regression assumes a linear relationship, independent errors with constant variance, roughly normal errors, and no strong multicollinearity. For prediction, linearity and a stable distribution matter most; the rest matter when you interpret coefficients or p-values. I check with a residuals-vs-fitted plot."*

- *"Funnel-shaped residuals?"* — Heteroscedasticity; log y, weighted least squares or robust standard errors.

Self-check

1. Does heteroscedasticity bias coefficients? → No; it makes their standard errors wrong.
2. Does multicollinearity hurt predictions? → Usually not much; it destabilises individual coefficients.

TL;DR

Plot residuals vs fitted: a U means missing non-linearity, a funnel non-constant variance; correlated features make coefficients unreliable.

5.6 Polynomial regression

What it is

Linear regression on polynomial features (degree 2, one feature: $\hat{y} = w_2x^2 + w_1x + b$). Still "linear" — in the *weights* — so the same solvers work; only inputs are expanded (`PolynomialFeatures`).

Why it matters

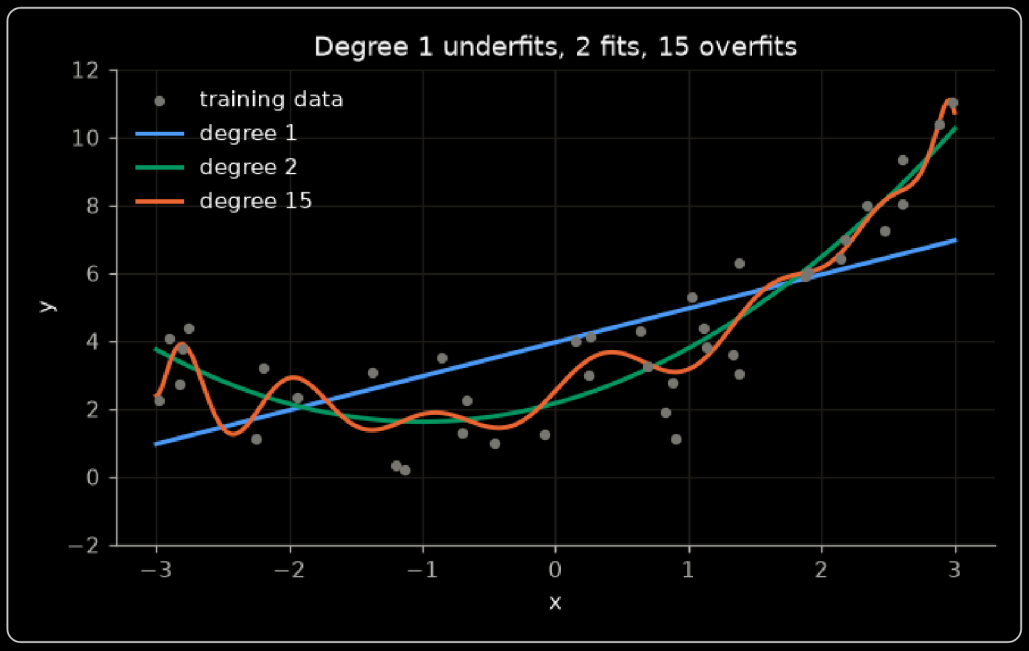
The simplest way to fit curves and the clearest demonstration of:

- **Underfitting** — too simple: high error on training *and* test (high **bias**: systematic error from a too-rigid model).
- **Overfitting** — fits the noise: low training, higher test error (high **variance**: fit changes a lot with the training sample).

```
# code/polynomial_and_assumptions.py (key parts) — truth: y = 0.5x² + x + 2 + noise, 40 training points
for deg in [1, 2, 15]:
    model = make_pipeline(PolynomialFeatures(deg, include_bias=False), StandardScaler(), LinearRegression())
    model.fit(x[:, None], y)
```

Output:

```
degree  train MSE  test MSE
     1       3.73      2.83
     2       1.14      1.04
    15       0.90      1.35
degree-2 learned: y = 0.54 x^2 + 1.08 x + 2.16   (truth 0.5, 1, 2)
```



Degree 2 recovers the true coefficients. Degree 15 has the *lowest training error* but higher test error, wiggling between points and shooting off at the edges (test points here lie inside the training range; outside it, degree 15 is wildly wrong). Noise variance is 1.0, so test MSE ≈ 1.0 is the best possible.

Trade-offs / when not to use it

- Features explode: d features at degree k → C(d+k, k) − 1 columns (10 at degree 2 → 65; degree 3 → 285).
- High degrees are unstable at the edges and extrapolate badly; prefer low degrees, **splines** (smooth curves of short low-degree pieces joined end to end; `SplineTransformer`) or trees/boosting.
- Scale after expanding: x^{15} dwarfs x.

In the interview

30 seconds: *"Polynomial regression expands the inputs with powers and products and fits an ordinary linear model — linear in the weights. The degree is a complexity knob: too low underfits, too high overfits and extrapolates wildly. I'd choose the degree, or add Ridge, using validation data."*

Self-check

1. Degree-2 features from 3 inputs (no bias)? → 3 + 3 squares + 3 pairwise = 9.

TL;DR

Expand features with powers and products, fit a linear model; pick the degree (or regularize) on validation data.

5.7 Regularization: Ridge, Lasso, ElasticNet

What it is

Regularization adds a weight-size penalty to the loss, so the model only "spends" weight where data strongly justifies it.

Model	Penalty	scikit-learn objective	Effect
Ridge (L2)	$\alpha \sum w_j^2$	$\|y - Xw\|^2 + \alpha\|w\|_2^2$	Shrinks all weights smoothly toward 0, never exactly 0; splits weight among correlated features
Lasso (L1)	$\alpha \sum \|w_j\|$	$(1/2m)\|y - Xw\|^2 + \alpha\|w\|_1$	Sets some weights **exactly 0** → automatic feature selection; picks one of a correlated group
ElasticNet	Both	$(1/2m)\|y - Xw\|^2 + \alpha \cdot r\|w\|_1 + \frac{1}{2}\alpha(1 - r)\|w\|_2^2$	**Sparse** (many exact zeros) *and* stable with correlated features; r = `l1_ratio`

- **α** = strength: 0 = plain least squares; large → weights toward 0. Ridge's and Lasso's α are on different scales in scikit-learn (sum vs divided by 2m), so the same α shrinks differently.
- The **intercept is not penalized**. **Scale features first** — the penalty treats all weights equally.
- Logistic regression and SVMs (Chapter 8) use **C = 1/strength**: *smaller* C = *stronger* regularization.

Why it matters

- Reduces overfitting with many features relative to rows, or correlated features.
- Makes $X^TX + \alpha I$ always invertible, so Ridge has a closed form $\theta = (X^TX + \alpha I)^{-1}X^Ty$ (intercept unpenalized).
- Lasso gives compact, interpretable models.

Why L1 gives exact zeros and L2 doesn't: geometrically, the L1 "budget" is a diamond with corners on the axes, and loss contours usually first touch it at a corner (some weights 0); the L2 circle has no corners. Computationally, L1's slope is a constant ±α however small the weight, pushing it all the way to 0; L2's slope 2αw fades as w shrinks.

```python
# code/regularization.py (key parts) — diabetes data, standardized
ridge_path = np.array([Ridge(alpha=a).fit(X, y).coef_ for a in np.logspace(-2, 4, 60)])
lasso_path = np.array([Lasso(alpha=a, max_iter=50_000).fit(X, y).coef_ for a in np.logspace(-2, 2, 60)])

X_dup = np.c_[X[:, [2]], X[:, [2]] + rng.normal(0, 0.01, (len(X), 1))]    # bmi twice (near-copy)

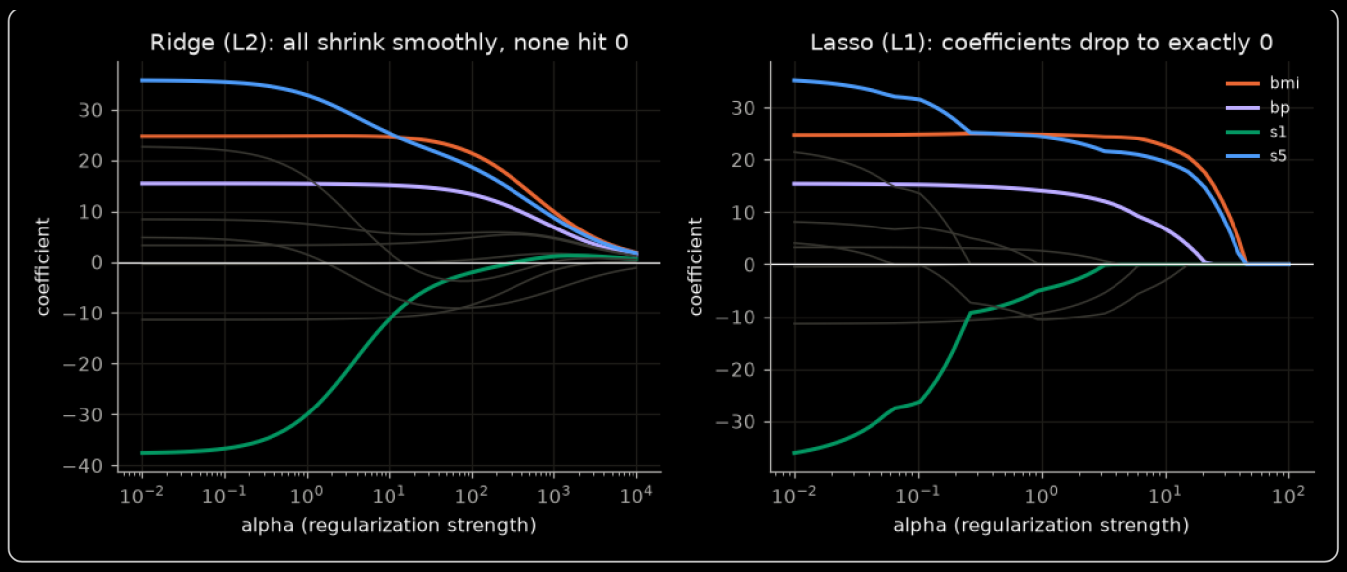
poly = PolynomialFeatures(2, include_bias=False)                          # 65 features, ~330 rows
models = {"OLS": make_pipeline(poly, StandardScaler(), LinearRegression()),
          "RidgeCV": make_pipeline(poly, StandardScaler(), RidgeCV(alphas=np.logspace(-3, 3, 50))),
          "LassoCV": make_pipeline(poly, StandardScaler(), LassoCV(cv=5, random_state=0)),
          "ElasticNetCV": make_pipeline(poly, StandardScaler(),
                                        ElasticNetCV(l1_ratio=[0.2, 0.5, 0.8], cv=5, random_state=0))}
```

Output:

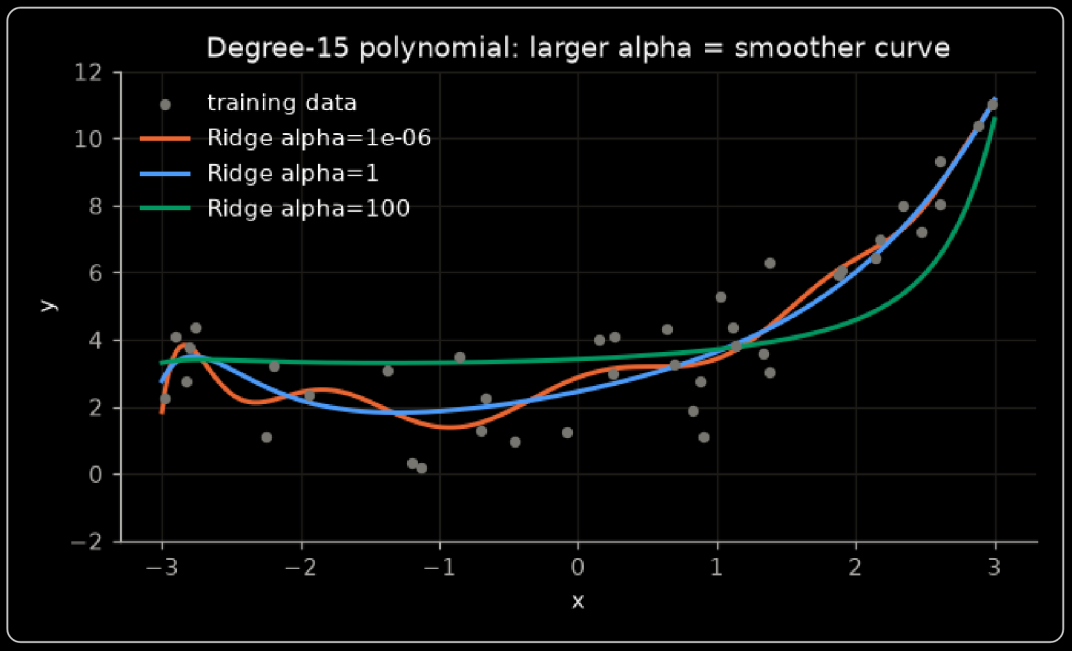
```
Lasso alpha=0.1 :  9 of 10 coefficients non-zero
Lasso alpha=1   :  7 of 10 coefficients non-zero
Lasso alpha=5   :  5 of 10 coefficients non-zero
Lasso alpha=20  :  3 of 10 coefficients non-zero

bmi duplicated -> OLS: [ 434.3 -389.1]  Ridge(1): [31.5 13.5]  Lasso(1): [44.2  0. ]

model          test MSE    alpha l1_ratio non-zero
OLS                4112      nan      nan       65
RidgeCV            3291   59.636      nan       65
LassoCV            3113    4.256      nan       11
ElasticNetCV       3178    0.754      0.8       41
```



- **Sparsity:** as Lasso's α grows, 9 → 7 → 5 → 3 non-zero; the last survivors (`bmi`, `s5`) are the strongest predictors.
- **Correlated features:** with `bmi` duplicated, OLS gives absurd opposite-signed weights (+434, −389) that nearly cancel; Ridge shares (31.5 + 13.5 ≈ 45); Lasso keeps one (44.2).
- **Generalisation:** 65 polynomial features on ~330 rows: OLS overfits (4112); CV-tuned Ridge/Lasso/ElasticNet ≈ 3100–3300, Lasso keeping 11 of 65. (Honest footnote: OLS on the *original* 10 features scores ~2900 — regularization rescues an over-expanded model but doesn't make needless features useful.)



Trade-offs

	Ridge	Lasso	ElasticNet
Feature selection	No	Yes	Yes (less aggressive)
Correlated features	Shares weight — stable	Picks one arbitrarily — unstable	Groups them — stable
Closed form	Yes	No (coordinate descent: one weight at a time)	No
When p > n (more features than rows)	Works	Selects at most n features	Works
Good default when	Many small effects	Few features truly matter	Correlated groups + want sparsity

When not to use it

With plenty of rows per feature it matters little (diabetes' raw 10 features: all four within ~2%, test MSE ≈ 2900–2950). Don't treat Lasso's selected set as "the true causes" — with correlated features its choice flips between

resamples.

Common mistakes

- Regularizing unscaled features; choosing α on the test set (use `RidgeCV` / `LassoCV` / `GridSearchCV`); forgetting C in `LogisticRegression` is the *inverse* strength.

Real-world examples

- Ridge is the classic multicollinearity fix in econometrics; L2 "weight decay" is standard in neural-network training.
- L1-regularized logistic regression for sparse, high-dimensional text and click models (Google's 2013 FTRL-Proximal ad click model, an online method updating per example, used L1 for sparsity).
- Genomics (thousands of genes, few patients) uses Lasso/ElasticNet to select predictive genes.

In the interview

30 seconds: *"Regularization adds a weight penalty to fight overfitting. Ridge (L2) shrinks all weights smoothly and handles correlated features; Lasso (L1) zeroes some weights, doing feature selection; ElasticNet mixes both. Scale features first and tune α with cross-validation."*

- *"Why does Lasso give zeros?"* — Constant L1 slope pushes small weights to 0; the diamond constraint has corners on the axes.
- *"Two identical features?"* — Ridge splits the weight equally; Lasso picks one.
- *"Effect on bias and variance?"* — More regularization → more bias, less variance.

Self-check

1. `LogisticRegression` C = 0.01: strong or weak? → Strong.

TL;DR

Penalize big weights: L2 shrinks, L1 selects, ElasticNet both. Scale first, tune α (or C) by cross-validation.

5.8 Logistic regression

What it is

Logistic regression is a linear model for **binary classification**. It computes z = w·x + b (the **logit**) and squashes it with the **sigmoid** (logistic) function:

p = P(y = 1 | x) = σ(z) = 1 / (1 + e^−z)

Despite the name, it's a classifier (named after the logistic function).

How it works

1. Sigmoid maps reals to (0, 1): σ(−2) = 0.119, σ(0) = 0.5, σ(2) = 0.881.

2. Log-odds. Inverting: z = ln(p / (1 − p)); p/(1 − p) is the **odds** (p = 0.8 → 4 to 1). The model is linear in **log-odds**: +1 in x_j adds w_j to the log-odds, **multiplying the odds by e^{w_j}** (the **odds ratio**).

3. Decision boundary. Predict 1 when p ≥ threshold (default 0.5). p = 0.5 exactly when z = 0, so the **decision boundary** is w·x + b = 0 — a line (2-D) or hyperplane. Changing the threshold shifts it parallel (choosing thresholds: Chapter 6).

4. Log-loss (binary cross-entropy). Squared error with a sigmoid is non-convex (several dips to get stuck in), so:

Loss = −(1/m) Σ [y_i ln p_i + (1 − y_i) ln(1 − p_i)]

Only one term is active per example: −ln p if y = 1, −ln(1 − p) if y = 0.

True y	Predicted p	Loss	Reading
1	0.9	−ln 0.9 = 0.105	confident and right: tiny loss
1	0.5	−ln 0.5 = 0.693	unsure
1	0.1	−ln 0.1 = 2.303	confident and wrong: big loss
0	0.1	−ln 0.9 = 0.105	confident and right

It is the negative log-likelihood under a **Bernoulli** model (one yes/no trial, 1 with probability p), so minimising it is maximum likelihood. Convex in the weights (GD finds the global minimum) but no closed form — solved iteratively.

5. Gradient — same form as linear regression's: $\nabla \text{Loss} = (1/m)\, X^T(\sigma(X\theta) - y)$, "features × (prediction − truth)". For one example ($p = \sigma(z)$, $z = \theta^T x$): $d\text{Loss}/dp = -y/p + (1-y)/(1-p)$; $dp/dz = p(1-p)$; so $d\text{Loss}/dz = \mathbf{p - y}$, and $dz/d\theta_j = x_j$. (No factor 2: no square.)

```python
# code/logreg_from_scratch.py (key parts)
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def fit_logreg_gd(X, y, lr=0.5, epochs=5000):
    Xb = np.c_[np.ones(len(X)), X]
    theta = np.zeros(Xb.shape[1])
    for _ in range(epochs):
        p = sigmoid(Xb @ theta)
        theta -= lr * Xb.T @ (p - y) / len(y)            # gradient of mean log-loss
    return theta

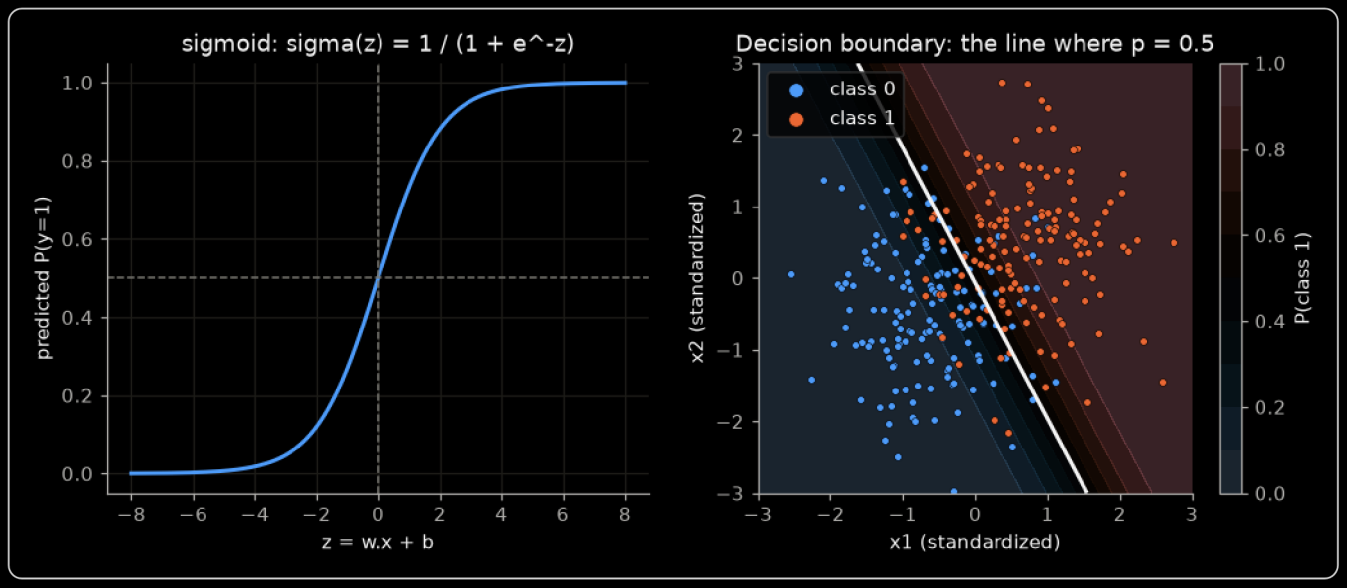
X, y = make_blobs(n_samples=400, centers=[[-1, -0.5], [1, 0.8]], cluster_std=1.1, random_state=0)
# ... stratified split, StandardScaler fitted on train
theta = fit_logreg_gd(X_tr, y_tr)
sk = LogisticRegression(C=np.inf).fit(X_tr, y_tr)        # C=inf -> no penalty (sklearn >= 1.8)
```

Output:

```
sigmoid(-2) = 0.119
sigmoid(+0) = 0.500
sigmoid(+2) = 0.881
y=1, p=0.9: loss = 0.105
y=1, p=0.5: loss = 0.693
y=1, p=0.1: loss = 2.303
y=0, p=0.1: loss = 0.105

from scratch [b, w1, w2]: [0.092 2.454 1.297]
sklearn      [b, w1, w2]: [0.092 2.454 1.298]
test log-loss scratch 0.4048 | sklearn 0.4048
test accuracy scratch 0.850 | sklearn 0.850
w1 = 2.454: +1 std in x1 multiplies the odds of class 1 by e^w1 = 11.64
```

Ten lines of GD match scikit-learn's **L-BFGS** solver (a faster optimiser using a curvature estimate) to ~three decimals.



Probability changes smoothly across the boundary; points near it get ≈ 0.5 — honest uncertainty about overlapping classes.

scikit-learn details (1.8+). `LogisticRegression` is L2-regularized by default (`C=1.0`, solver `lbfgs`). The old `penalty=` argument is deprecated: `C=np.inf` = no penalty; `l1_ratio=1` (with `solver="saga"` or `"liblinear"`) = L1; values in (0, 1) = ElasticNet.

Trade-offs

Strengths	Weaknesses
Fast to train and serve, tiny model	Linear boundary only (needs feature engineering or kernels)
Probabilities often reasonably **calibrated** (predicted 0.7 comes true ~70% of the time)	Sensitive to correlated features and outliers in X
Coefficients readable as odds ratios	On **perfectly separable** data, unregularized weights grow without bound
Convex: one global optimum	Usually beaten by gradient boosting (Chapter 7) on complex tabular data

When not to use it

Strongly non-linear boundaries without good features (use trees/boosting or neural nets); high-cardinality interactions you can't enumerate.

Common mistakes

- `LinearRegression` on a 0/1 target — predictions leave [0, 1], fit dominated by easy points.
- Forgetting the default is regularized (C = 1): compare coefficients only with the same C and scaling.
- Assuming 0.5 is the right threshold on imbalanced data.

Real-world examples

- **Credit scoring** scorecards — regulators and loan officers must read the coefficients.
- **Ad click-through rate** at Google and Facebook (Facebook's 2014 paper fed boosted-tree features into logistic regression).
- Medical risk scores; the final layer of every neural binary classifier (sigmoid on a linear layer).

In the interview

30 seconds: *"Logistic regression computes a linear score $z = w \cdot x + b$, turns it into a probability with the sigmoid, and is trained by minimising log-loss — convex, so gradient descent finds the global optimum. The decision boundary is the hyperplane $w \cdot x + b = 0$. Each coefficient is a change in log-odds, so e^{w} is an odds ratio."*

- *"Why not MSE for classification?"* — With a sigmoid it's non-convex and gives weak gradients for confident mistakes; log-loss is convex and punishes them hard.
- *"Perfectly separable data?"* — Loss keeps falling as weights grow, so they diverge; regularization keeps them finite.
- *"Calibrated?"* — Often roughly, since log-loss is a **proper scoring rule** (lowest only at the true probabilities); check with a **reliability curve** (predicted probability vs observed frequency; Chapter 6).

Self-check

1. Loss for y = 1, p = 0.25? → 1.386.
2. w = 0.7: effect on odds? → × $e^{0.7} \approx 2.01$ per unit.

TL;DR

Linear score → sigmoid → probability; log-loss (convex, gradient $X^T(p - y)/m$); boundary $w \cdot x + b = 0$; coefficients are log-odds.

5.9 Multiclass classification: one-vs-rest and softmax

What it is

Two ways to handle K > 2 classes:

1. **One-vs-rest (OvR)** — K binary classifiers ("class k vs the rest"); predict the most confident.
2. **Softmax (multinomial) regression** — one model, a weight vector per class, scores $z_k = w_k \cdot x + b_k$ turned into probabilities summing to 1 by **softmax**: $P(y = k \mid x) = e^{z_k} / \Sigma_j e^{z_j}$.

Worked example: scores (2, 1, 0.1) → $e^2 = 7.39$, $e^1 = 2.72$, $e^{0.1} = 1.11$, total 11.21 → **(0.659, 0.242, 0.099)**.

The loss is **cross-entropy** $-(1/m) \Sigma_i \ln P(y_i \mid x_i)$ (negative log-probability of the correct class); with K = 2 it is exactly binary logistic regression. Gradient: $X^T(P - Y)/m$, Y = one-hot labels.

```python
# code/multiclass.py (key parts)
def softmax(Z):
    Z = Z - Z.max(axis=1, keepdims=True)                # subtract row max: avoids overflow
    E = np.exp(Z)
    return E / E.sum(axis=1, keepdims=True)

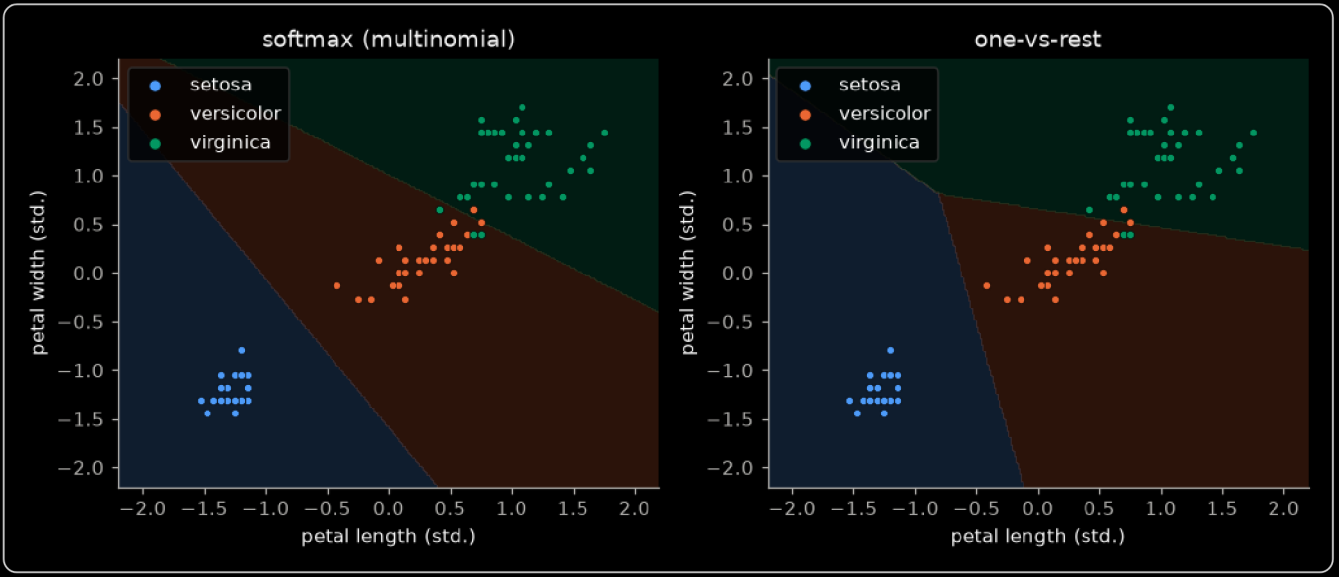
def fit_softmax_gd(X, y, k, lr=0.5, epochs=3000, l2=0.0):
    Xb = np.c_[np.ones(len(X)), X]
    W = np.zeros((Xb.shape[1], k)); Y = np.eye(k)[y]      # one-hot targets
    for _ in range(epochs):
        grad = Xb.T @ (softmax(Xb @ W) - Y) / len(y)
        grad[1:] += l2 * W[1:]
        W -= lr * grad
    return W

softmax_sk = LogisticRegression(C=10).fit(X_tr, y_tr)                  # multinomial by default
ovr = OneVsRestClassifier(LogisticRegression(C=10)).fit(X_tr, y_tr)     # 3 binary models
```

Output:

```
softmax([2, 1, 0.1]) = [[0.659 0.242 0.099]]  sum = 1.0

test accuracy  softmax(sklearn) 0.978 | OvR 0.978 | softmax(scratch) 0.978
one flower, P(setosa, versicolor, virginica):
  softmax sklearn : [[0.    0.001 0.999]]
  softmax scratch : [0.    0.001 0.999]
  OvR raw scores  : [0.    0.175 0.998] (sum 1.174) -> normalized [[0.    0.149 0.851]]
sklearn softmax coef_ shape: (3, 2) (one weight vector per class)
```



Same 97.8% accuracy, different probabilities: independently trained OvR scores sum to 1.174 and are simply rescaled (0.851 for virginica vs softmax's 0.999); its boundaries come from three separate problems.

Trade-offs

	One-vs-rest	Softmax
Models	K binary	One joint
Probabilities	Not naturally normalised	Sum to 1, jointly trained — better calibrated
Parallel training / per-class tuning	Easy	No
Use with	Any binary classifier (e.g. SVMs); multi-label	Mutually exclusive classes; neural output layers

One-vs-one trains K(K−1)/2 pairwise classifiers (used inside `SVC`). For **multi-label** problems (several labels per item, e.g. photo tags) use independent sigmoids, not softmax. scikit-learn 1.9 has no `multi_class` argument: `LogisticRegression` fits softmax for K > 2; wrap in `OneVsRestClassifier` for OvR.

In the interview

30 seconds: *"For K classes you can train K one-vs-rest binary models, or one softmax model that outputs K scores and normalises them with e^z / Σe^z, trained with cross-entropy. Softmax gives coherent probabilities for mutually*

exclusive classes and is the output layer of neural classifiers; OvR works with any binary model and suits multi-label problems."

- *"Why subtract the max in softmax?"* — e^1000 overflows; subtracting a constant from all scores doesn't change the result.
- *"Softmax with 2 classes?"* — Logistic regression on the score difference.

Self-check

1. softmax(0, 0, 0)? → (1/3, 1/3, 1/3).
2. Classifiers for 10 classes? → OvR 10; one-vs-one 45.

TL;DR

OvR = K binary models; softmax = one model, normalised probabilities, cross-entropy. Softmax for exclusive classes, sigmoids for multi-label.

5.10 Interpreting coefficients

What it is

Coefficients are a linear model's explanation; reading them depends on units, scaling and correlations.

How it works

- **Raw units:** w_j = change in ŷ per unit of x_j, **others fixed**. Switch m² → km² and the number changes a millionfold; the model doesn't.
- **Standardized features:** w_j = change per 1 standard deviation — comparable across features, a rough importance ranking.
- **Logistic:** w_j = change in log-odds; e^w_j = **odds ratio** (> 1 raises odds, < 1 lowers).
- **Intercept:** prediction at all-zero features (standardized: at average values).

```
# code/interpret_coefficients.py (key parts)
price = 50 + 3.0 * size - 1.5 * age + rng.normal(0, 20, n)     # $1000s, size in m², age in years
LinearRegression().fit(np.c_[size, age], price)
LinearRegression().fit(np.c_[size / 1e6, age], price)            # size in km²
make_pipeline(StandardScaler(), LinearRegression()).fit(np.c_[size, age], price)

clf = make_pipeline(StandardScaler(), LogisticRegression(C=0.1, max_iter=5000)).fit(cancer.data, cancer.target)

x2 = x1 + rng.normal(0, 0.05, 300)                               # near-duplicate feature
# fit on 3 bootstrap resamples
```

Output:

```
raw units      : price = 52.6 + 2.98*size_m2 -1.49*age_years
size in km^2   : coef for size = 2984148  (same model, different units -> different number)
standardized   : +1 std of size -> +137.6, +1 std of age -> -20.8  (comparable importance)

breast cancer (target 1 = benign), standardized features, C=0.1:
  worst texture            coef -0.60  odds ratio e^coef = 0.55
  worst radius             coef -0.54  odds ratio e^coef = 0.58
  worst concave points     coef -0.53  odds ratio e^coef = 0.59
  fractal dimension error  coef +0.27  odds ratio e^coef = 1.31
  compactness error        coef +0.27  odds ratio e^coef = 1.31

bootstrap coefficients for two near-duplicate features (true: 1.0 and 1.0):
  resample 0: w1 = +1.07, w2 = +0.95, sum = 2.02
  resample 1: w1 = +2.91, w2 = -0.92, sum = 2.00
  resample 2: w1 = -1.03, w2 = +3.04, sum = 2.01
```

- Each m² adds ~$2,980, each year of age subtracts ~$1,490. Raw, age looks half as big as size; standardized, size matters ~6.6× more (137.6 vs 20.8) because sizes vary much more.
- One std more "worst texture" multiplies the odds of *benign* by 0.55 (about halves them).
- Near-duplicate features: across **bootstrap** resamples (rows redrawn at random with replacement) individual coefficients swing wildly, even flipping sign, while their **sum** stays ≈ 2. Predictions are fine; individual numbers are meaningless. Diagnose with correlations or the **variance inflation factor (VIF)** (how much a coefficient's uncertainty is inflated by correlation with other features); fix by dropping/combining features or Ridge.

Common mistakes

- Comparing raw coefficients across different units; reading them causally from observational data.
- Interpreting one coefficient of a correlated group alone; forgetting regularization biases coefficients toward 0.

In the interview

30 seconds: *"A coefficient is the change in prediction per unit of that feature, holding the others constant. Standardize features to compare magnitudes; in logistic regression exponentiate (compute e to the power of the coefficient) to get odds ratios. Coefficients are unstable under multicollinearity and are associations, not causal effects."*

Self-check

1. Logistic coefficient −0.69? → × e^−0.69 ≈ 0.5: halves the odds.
2. Why are correlated coefficients unstable? → Many weight combinations predict almost identically; small data changes flip between them.

TL;DR

Coefficients = effect per unit (or per std if standardized), others fixed; exponentiate logistic ones for odds ratios; distrust individual coefficients of correlated features and never read them causally.

Chapter TL;DR

- **Workflow:** data → clean → encode (one-hot for nominal) → split → fit → evaluate against a baseline → iterate. MSE only means something relative to the target's scale and a baseline.
- **Linear regression:** $\hat{y} = w \cdot x + b$, minimise MSE (convex). **Normal equation** $\theta = (X^T X)^{-1} X^T y$ via a solver — exact, $O(d^3)$, no scaling. **Gradient descent** $\theta \leftarrow \theta - \eta(2/m)X^T(X\theta - y)$ — scalable, general, needs η and scaled features.
- **GD variants:** batch (smooth, costly), stochastic (noisy, decaying η), mini-batch (default). Too-large η diverges (MSE: $\eta < 2/\lambda_{max}$). Unscaled features → elongated bowl → many more iterations.
- **Assumptions:** linearity, independent errors, constant variance, roughly normal errors, no strong multicollinearity; check residual plots (U = missing curvature, funnel = heteroscedasticity).
- **Polynomial regression:** linear model on expanded features; degree controls under/overfitting.
- **Regularization:** Ridge (L2, shrink, stable with correlated features), Lasso (L1, exact zeros), ElasticNet (both). Scale first; tune α (or C = 1/strength) by CV.
- **Logistic regression:** $p = \sigma(w \cdot x + b)$, log-loss (convex, gradient $X^T(p - y)/m$), boundary $w \cdot x + b = 0$, e^w = odds ratio. sklearn ≥ 1.8: `C=np.inf` = no penalty.
- **Multiclass:** OvR (K binary models) vs softmax ($e^{z_k}/\Sigma e^{z_j}$ + cross-entropy; coherent probabilities).
- **Coefficients:** per-unit effects holding others fixed; standardize to compare; unstable under multicollinearity; associations, not causes.

Chapter 6 — Evaluation and Tuning

Training is the easy part. The hard questions come after: *Is it any good? At what? Compared to what? Will it hold on next month's data? Could other settings do better?* This chapter covers the right metric for the job, reliable performance estimates, over- and underfitting, and systematic **hyperparameter tuning** (searching for the best settings chosen before training).

What you'll learn: baselines and offline vs business metrics; regression metrics (MAE, MSE, RMSE, R^2, percentile and per-cohort errors); classification metrics and threshold choice; ranking metrics (precision@k, recall@k, MAP, MRR, NDCG); calibration; cross-validation (k-fold, stratified, time-series, group, nested); bias–variance, learning and validation curves; grid, random, successive halving and Bayesian (Optuna) tuning; early stopping.

All code is in `code/` (scikit-learn 1.9). Outputs are real.

6.1 Baselines, and offline vs business metrics

What it is

An **evaluation metric** summarises how well predictions match reality. A **baseline** is the score of a trivially simple predictor. An **offline metric** is computed on **held-out** (never trained on) historical data; an **online** or **business metric** is what the product cares about (revenue, retention, fraud losses, delivery promises kept), measured on live traffic.

Why it matters

A metric alone is meaningless. "MAE of 6 minutes" says nothing until you know the naive and current production scores: if "same as last week" gives 8, 6 is a real gain; if it gives 6.2, the model is an expensive way to save 12 seconds.

How it works — baselines to always compute

Task	Naive baseline	scikit-learn
Classification	Majority class; random at class rates	`DummyClassifier(strategy="most_frequent" / "prior" / "stratified")`
Regression	Mean or median of training target	`DummyRegressor(strategy="mean" / "median")`
Time series	Last value; same hour last week (seasonal naive)	pandas `shift`
Ranking / recommendation	Most popular items	`value_counts()`
Any	Current production model; a simple rule or linear model	—

Offline metric vs business metric

Offline metrics are cheap, repeatable **proxies** for comparing candidates and deciding what deserves a live test, but can disagree with the business outcome when:

- The label is a proxy (clicks ≠ satisfaction; chargebacks ≠ all fraud).
- The test set no longer resembles live traffic (**drift**, new users, new items).
- Serving differs from training (feature delays, timeouts, fallbacks to default values).
- The metric ignores costs: latency (see System Design Fundamentals), review workload, fairness across groups.

Hence two gates (Chapter 16, section 16.7): **offline evaluation** to shortlist, then an **online experiment** (A/B test: random users get the new vs old model; Chapter 16) to confirm business impact. Pick an offline metric whose improvement *likely* improves the business metric; report it with the dataset, base rate and threshold.



Figure — mapping problem types to metric families (course diagram)

In the interview

30 seconds: *"I'd pick the offline metric from the task and the cost of each kind of error — precision/recall and PR-AUC for rare-event classification, recall@k and NDCG for ranking, MAE or RMSE plus tail and per-segment errors for regression — check calibration separately if scores are used as probabilities, and always compare against a naive baseline and the production model before going to an A/B test."*

- *"Offline improved, A/B flat — why?"* — The proxy drifted from the goal: proxy labels, stale test set, training/serving differences. Also check the experiment's **statistical power** (chance of detecting a real effect of the expected size) and logging.

Self-check

1. Churn PR-AUC 0.41 — good? → Unknown until compared: no-skill scores the churn rate (e.g. 0.12), and production scores something too.

TL;DR

No number means anything without a baseline and the production model beside it. Offline metrics shortlist; online experiments decide.

6.2 Regression metrics

What it is

Summaries of errors $e_i = \hat{y}_i - y_i$.

Metric	Formula	Units	Reads as
MAE (mean absolute error)	$(1/n)\ \Sigma\ \lvert e_i \rvert$	y	Typical error size
MSE (mean squared error)	$(1/n)\ \Sigma\ e_i^2$	y^2	Average squared error; usual training loss
RMSE (root MSE)	$\sqrt{\text{MSE}}$	y	Like MAE, big errors weigh more
R^2 (coefficient of determination)	$1 - \Sigma e_i^2 / \Sigma(y_i - \bar{y})^2$	none	Share of target variance explained; 1 perfect, 0 = predicting the mean, < 0 worse
MAPE (mean absolute % error)	$(100/n)\ \Sigma\ \lvert e_i / y_i \rvert$	%	Relative error; breaks near y = 0
Percentile error (e.g. p90)	90th percentile of $\lvert e_i \rvert$	y	90% of predictions at least this good

How it works — a worked example

Delivery times $y = (30, 25, 40, 35, 50)$, $\hat{y} = (28, 27, 35, 35, 60)$: $e = (-2, 2, -5, 0, 10)$. MAE = **3.8** min; MSE = 133/5 = **26.6** min^2, RMSE = **5.16**; $\bar{y} = 36$, $\Sigma(y - \bar{y})^2 = 370$, $R^2 = 1 - 133/370 = \mathbf{0.641}$.

```python
# code/regression_metrics.py (key parts)
from sklearn.metrics import mean_absolute_error, mean_squared_error, root_mean_squared_error, r2_score
y, yhat = np.array([30, 25, 40, 35, 50]), np.array([28, 27, 35, 35, 60])
print(mean_absolute_error(y, yhat), mean_squared_error(y, yhat), root_mean_squared_error(y, yhat), r2_score(y, yhat))

base = np.full(10, 10.0); one_big = base.copy(); one_big[0] = 40      # one miss grows from 10 to 40 min
# ... then diabetes data: Ridge vs mean/median DummyRegressor, p50/p90 error, error per cohort
```

Output:

```
errors: [-2  2 -5  0 10]
MAE  = mean(|e|)      = 3.80   sklearn 3.80
MSE  = mean(e^2)      = 26.60  sklearn 26.60
RMSE = sqrt(MSE)      = 5.16   sklearn 5.16
R^2  = 1 - 133/370 = 0.641  sklearn 0.641
ten 10-min errors        MAE  10.0   RMSE  10.0
one of them is 40 min    MAE  13.0   RMSE  15.8

model                     MAE   RMSE    R^2
mean baseline            59.5   71.4 -0.000
median baseline          59.1   72.5 -0.029
Ridge                    44.3   55.2  0.402
p50 abs error 40.6 | p90 abs error 90.6
cohort A: n= 74  MAE 47.8
cohort B: n= 59  MAE 39.9
```

MAE vs RMSE — how much should a big miss count? Squared, a 40-minute miss weighs 16× a 10-minute one in MSE (1600 vs 100); in MAE just 4×. Turning one of ten 10-minute errors into 40 lifts MAE 30% (10 → 13) but RMSE 58% (10 → 15.8) (not 16× — the root and averaging dampen it).

- **RMSE** when large misses are disproportionately bad (a very late delivery loses the customer).
- **MAE** when cost grows linearly, or label outliers shouldn't dominate.
- The constant minimising MSE is the **mean**; minimising MAE, the **median**.

Averages hide things — add two views.

1. **Tail percentiles:** median error 41 but p90 91 — one prediction in ten is off by 90+. Products often promise on the tail ("on time for 90%+ of orders").
2. **Cohorts:** error per business segment (city, device, customer type, new vs mature market). Cohort A's MAE is 20% worse than B's; a model can look fine on average while failing a new region.

The mean baseline has $R^2 \approx 0$ by construction (slightly negative on test, since the training mean isn't the test mean). Ridge (penalised linear regression, Chapter 5) explains 40% of the variance and cuts MAE 26% — real but modest, the honest way to report it.

Common mistakes

- R^2 alone — it depends on the test set's target variance, so it isn't comparable across datasets.
- MAPE with targets near zero, or ignoring its asymmetry (penalises over-prediction more for positive targets).
- Comparing RMSE across targets in different units.

In the interview

30 seconds: *"MAE is the typical error in the target's units; RMSE squares errors first, so big misses count more — use it when large misses are disproportionately costly. R^2 compares against predicting the mean. I'd add a tail percentile like p90 and per-segment errors, because averages hide the failures that matter."*

- *"Why might RMSE ≫ MAE?"* — A few large errors inflate RMSE; the gap signals **heavy-tailed** errors (mostly small plus a few huge).

Self-check

1. Errors (1, −1, 4): MAE, RMSE? → 2; √(18/3) = 2.45.
2. $R^2 = -0.2$? → Worse than always predicting the mean.

TL;DR

MAE = typical error; RMSE = punishes big misses; R^2 = variance explained vs the mean. Add p90 and per-cohort errors; compare with a baseline.

6.3 Classification metrics

What it is

Every binary prediction lands in one cell of the **confusion matrix**:

	Predicted positive	**Predicted negative**
Actually positive	**TP** — true positive (caught)	**FN** — false negative (missed)
Actually negative	**FP** — false positive (false alarm)	**TN** — true negative

Metric	Formula	Question it answers
Accuracy	(TP + TN) / all	What fraction is right?
Precision	TP / (TP + FP)	Of flagged items, how many are really positive?
Recall (sensitivity, true-positive rate)	TP / (TP + FN)	Of real positives, how many did I catch?
F1	2·P·R / (P + R)	**Harmonic mean** of P and R — high only if both are
Specificity (true-negative rate)	TN / (TN + FP)	Of real negatives, how many left alone?
FPR (false-positive rate)	FP / (FP + TN) = 1 − specificity	Share of negatives wrongly flagged
Fβ	(1 + β²)·P·R / (β²·P + R)	Recall weighted β times precision (F2 favours recall, F0.5 precision)

How it works — a worked example

1,000 transactions, 100 fraud; the model flags 80, 60 truly fraud: TP = 60, FP = 20, FN = 40, TN = 880. Accuracy **0.94**; precision 60/80 = **0.75** (three in four alerts real); recall **0.60** (40% slips through); F1 = 0.9/1.35 = **0.667**; FPR = 20/900 = **0.022**.

The trap: flagging *nothing* scores accuracy **0.90** with zero recall; at a 0.2% fraud rate "never fraud" is 99.8% accurate. **Accuracy misleads whenever classes are imbalanced.**

Tiny tutorial case: y_true = [0, 1, 0, 1], y_pred = [0, 1, 1, 1] → accuracy 0.75, precision 0.667, recall 1.0, F1 0.8.

```python
# code/classification_metrics.py (key parts)
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
                             roc_auc_score, average_precision_score, roc_curve, precision_recall_curve)
y    = np.r_[np.ones(100), np.zeros(900)].astype(int)
pred = np.r_[np.ones(60), np.zeros(40), np.ones(20), np.zeros(880)].astype(int)
tn, fp, fn, tp = confusion_matrix(y, pred).ravel()        # sklearn's layout: [[TN FP] [FN TP]]
```

Output:

```
tiny: acc 0.75  precision 0.667  recall 1.00  F1 0.80

confusion matrix [[TN FP] [FN TP]] =
[[880  20]
 [ 40  60]]
TP=60 FP=20 FN=40 TN=880
accuracy  (TP+TN)/N      = 0.940
precision TP/(TP+FP)     = 0.750
recall    TP/(TP+FN)     = 0.600
F1        2PR/(P+R)      = 0.667
FPR       FP/(FP+TN)     = 0.022   specificity = 0.978
always-'legit' baseline accuracy = 0.900
```

Thresholds: the model gives scores, you choose the operating point

A **threshold** turns a score into yes/no. Raising it flags fewer items: recall falls, precision *usually* rises (not necessarily at every step). One model thus offers many **operating points**; quote precision and recall *at a stated threshold* ("90% precision at 70% recall"), or summarise across thresholds:

- **ROC curve** — recall (TPR) vs FPR. **ROC-AUC** = probability a random positive scores above a random negative (ties half): 0.5 random, 1.0 perfect ranking.
- **Precision–recall (PR) curve** — precision vs recall; **PR-AUC**, usually **average precision (AP)**. No-skill PR-AUC = the positive rate.

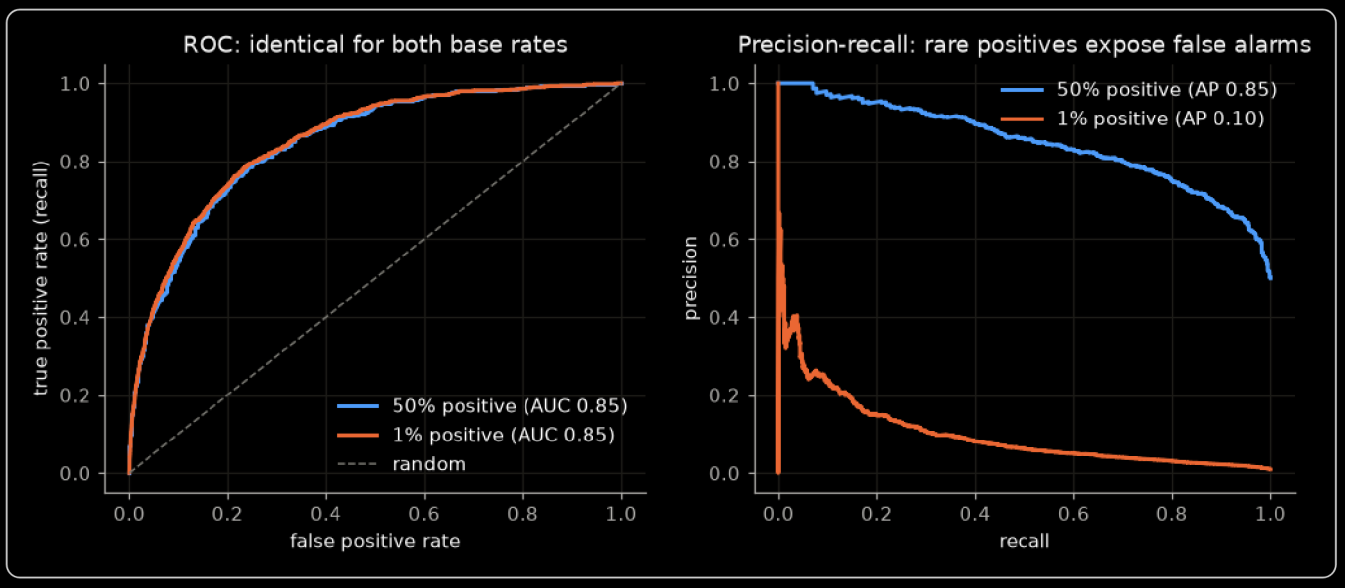
ROC-AUC by hand: positives (0.9, 0.7, 0.4), negatives (0.8, 0.3, 0.2): positives win 7 of 9 pairs → **0.778**.

Why PR-AUC for rare positives? FPR divides false positives by *all* negatives; with 10 million legitimate transactions, thousands of extra false alarms barely move it, so ROC stays flattering while analysts drown. Precision has false positives directly in its denominator.

```
pos = rng.normal(1.5, 1, 1000)                          # the SAME score model for positives
for n_neg in [1000, 99_000]:                            # 50% vs 1% positives
    neg = rng.normal(0, 1, n_neg)
    roc_auc_score(yy, ss), average_precision_score(yy, ss)
```

Output:

```
ROC-AUC by pair counting: 7/9 = 0.778  sklearn 0.778
positives  50.0%: ROC-AUC 0.850 | PR-AUC (avg precision) 0.846 | no-skill PR-AUC 0.500
positives   1.0%: ROC-AUC 0.853 | PR-AUC (avg precision) 0.099 | no-skill PR-AUC 0.010
```



Same model, same ROC-AUC (0.85), but at 1% positives AP drops from 0.85 to 0.10: at 50% recall precision is below 0.1, more than nine false alarms per real positive. Report the base rate with every precision number — precision shifts whenever the traffic's positive mix does.

Choosing the threshold by cost

The threshold prices a missed positive against a false alarm, so the cost owner (product, risk, operations) should own it. With **calibrated** probabilities (6.5), flag when $p \cdot \text{cost(FN)} > (1 - p) \cdot \text{cost(FP)}$, i.e.

$p > \text{cost(FP)} / (\text{cost(FP)} + \text{cost(FN)})$

Example: missed fraud \$100, reviewing a false alarm \$5 → flag when $p > 5/105 =$ **0.048**.

```
# 20,000 transactions, 5% fraud, calibrated probabilities p
for t in [0.01, 0.025, 0.048, 0.1, 0.2, 0.5]:
    cost = 100 * missed + 5 * false_alarms
```

Output:

```
threshold  precision  recall  flagged   cost($)
     0.01      0.075   0.970    13006    63,180
    0.025      0.107   0.894     8361    47,935
    0.048      0.151   0.784     5195    43,655
      0.1      0.219   0.573     2611    52,890
      0.2      0.335   0.367     1095    66,940
      0.5      0.595   0.103      173    90,050
cheapest threshold: 0.048  (theory for calibrated p: flag if p > 5/(5+100) = 0.048)
```

The default 0.5 costs twice the optimum. Mature fraud systems use **two thresholds**: below the low one approve, above the high one block, in between send to human review — the band's width set by reviewer capacity.

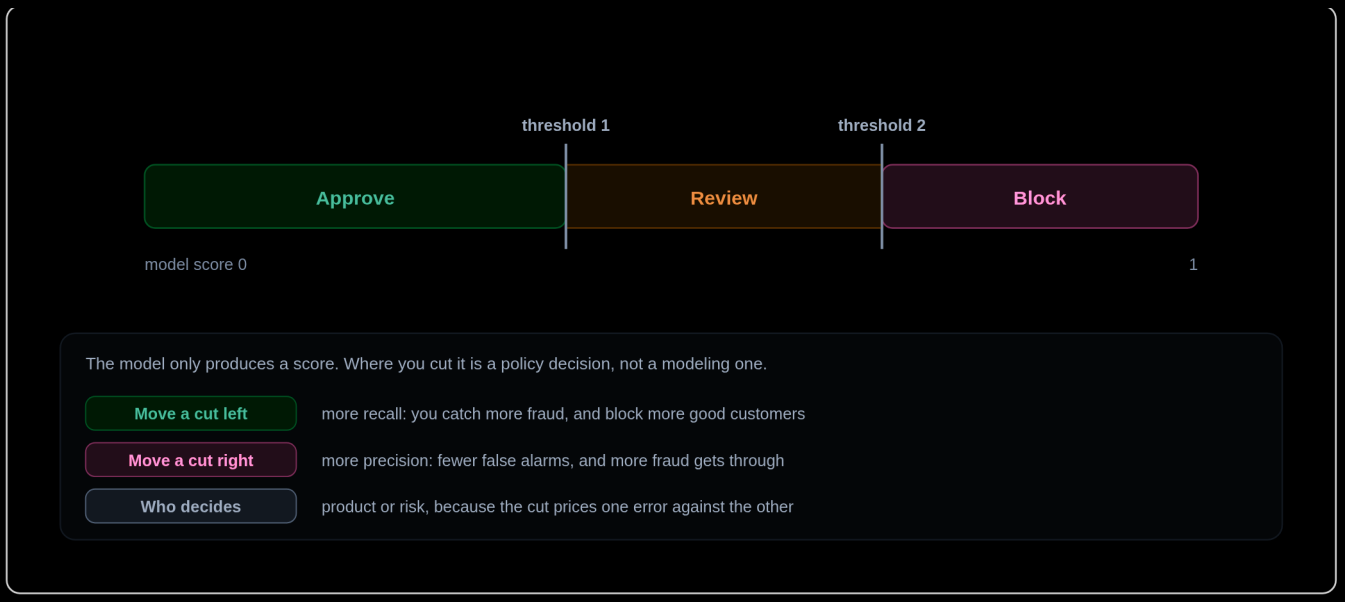


Figure — two thresholds on one score (course diagram)

Which metric when?

Situation	Use	Why
Balanced classes, equal error costs	Accuracy (plus confusion matrix)	Meaningful when no class dominates
False positives expensive (hiding real mail, blocking good customers)	Precision (at a recall floor)	Every alert must be trustworthy
Misses expensive (cancer screening, fraud, safety)	Recall (at a precision floor)	Catch as many as possible
One number balancing both, imbalanced	F1 or Fβ	Harmonic mean punishes a weak side
Compare rankers, moderate imbalance	ROC-AUC	Threshold-free, base-rate-insensitive
Rare positives (fraud, ads, disease)	PR-AUC / AP, plus P & R at the chosen threshold	Shows the false-alarm burden
Probabilities feed decisions (bids, expected cost)	Log-loss (punishes confident wrong probabilities), Brier score (mean squared probability error), calibration plot	Rewards honest probabilities
Multiclass	Per-class P/R/F1 with **macro** (unweighted class mean) or **weighted** averages; `classification_report`	Macro exposes weak minority classes

Common mistakes

- Accuracy on imbalanced data; precision without threshold and base rate; tuning the threshold on test (use validation); comparing F1 across different base rates.

Real-world examples

Spam filters (Gmail) favour very high precision (losing real mail is worse than some spam); medical screening favours recall, then a more precise second test; card-fraud systems tune approve/review/block bands to reviewer capacity and fraud losses.

In the interview

30 seconds: *"Start from the confusion matrix. Precision is 'how many alerts are real', recall 'how many positives we caught'; the threshold trades one for the other, and the right point comes from the cost of each error. For rare positives I'd report PR-AUC plus precision and recall at the chosen threshold, not accuracy or only ROC-AUC, because the flood of easy negatives hides extra false alarms inside ROC's FPR denominator."*

- *"PR-AUC or ROC-AUC at 0.2% base rate?"* — PR-AUC: it keeps precision, what reviewers and customers experience, on the axis.
- *"What threshold would you ship?"* — Not purely a modeling decision: present the PR curve as a menu for the cost owner; derive from expected cost if calibrated; for fraud, two thresholds with a review band.
- *"Why harmonic mean?"* — Stays low if either is low (P = 1, R = 0.01 → F1 ≈ 0.02 vs arithmetic 0.5).

Self-check

1. TP = 30, FP = 10, FN = 20? → P 0.75, R 0.6, F1 0.667.
2. ROC-AUC 0.8 means? → A random positive outranks a random negative 80% of the time.
3. FN $50, FP $10; threshold for calibrated p? → 10/60 = 0.167.

TL;DR

Confusion matrix → precision, recall, F1. The threshold sets the trade-off; choose it from error costs. ROC-AUC measures ranking; PR-AUC exposes false alarms for rare positives. Accuracy only for balanced problems.

6.4 Ranking metrics (an introduction)

What it is

Search, recommendation and feeds return an **ordered list**; ranking metrics reward relevant items **near the top**, usually in the first **k**. Relevance labels come from **judged sets** (raters score results against a rubric) or **behavioural signals** (clicks, purchases: cheaper but biased by what was shown and where).

Metric	Definition	Use when
Precision@k	Relevant in top k ÷ k	User sees exactly k slots
Recall@k	Relevant in top k ÷ all relevant	Candidate generation: did good items survive?
AP (average precision)	Mean of precision@i at each relevant item's rank (÷ all relevant)	Binary relevance, whole list
MAP	Mean AP over queries	Standard IR (information retrieval) benchmark
MRR (mean reciprocal rank)	Mean of 1/rank of first relevant (0 if none)	One good answer fast (Q&A, navigation)
NDCG@k	DCG@k ÷ ideal DCG@k, DCG = $\Sigma\ rel_i\ /\ \log_2(i+1)$	Graded relevance, position matters

How it works — worked example

Query A returns grades [3, 0, 2, 1, 0, 0] (3 = perfect … 0 = bad); the collection holds 4 relevant documents, one (grade 2) not returned.

- Precision@3 = 0.667; recall@3 = 0.5.
- AP: relevant at ranks 1, 3, 4 with precision 1, 2/3, 3/4 → (1 + 0.667 + 0.75)/4 = **0.604** (the missing one adds 0).
- Reciprocal rank = 1.
- DCG@5 = 3 + 2/$\log_2$4 + 1/$\log_2$5 = 3 + 1 + 0.431 = **4.431**; ideal [3, 2, 2, 1] = 5.693 → **NDCG@5 = 0.778**.

```python
# code/ranking_metrics.py (key parts)
def average_precision(rels, n_relevant):
    hits, total = 0, 0.0
    for i, r in enumerate(rels, start=1):
        if r > 0:
            hits += 1
            total += hits / i                # precision at this relevant item's rank
    return total / n_relevant

def dcg(rels, k):
    rels = np.asarray(rels[:k], dtype=float)
    return np.sum(rels / np.log2(np.arange(2, len(rels) + 2)))
```

Output:

```
query A graded list: [3, 0, 2, 1, 0, 0] | relevant in collection: 4
  precision@3 = 0.667   recall@3 = 0.500
  precision@5 = 0.600   recall@5 = 0.750
  AP  = (1/1 + 2/3 + 3/4) / 4 = 0.604
  RR  = 1/rank of first relevant = 1.000
  DCG@5 = 4.431, ideal DCG@5 = 5.693, NDCG@5 = 0.778

query B list: [0, 0, 1, 1, 0, 0] AP = 0.417  RR = 0.333
MAP over A and B = 0.510   MRR = 0.667

sklearn ndcg_score@5 on the returned list only: 0.930  (ours, same ideal): 0.930
note: sklearn builds the ideal ordering from the returned items only; with the unreturned grade-2
document included in the ideal list, NDCG@5 drops to 0.778
```

`sklearn.metrics.ndcg_score` builds the ideal ordering from the items you pass, so an unretrieved relevant document doesn't lower the score (0.930 vs 0.778). Some libraries use exponential gain ($2^{rel} - 1$); always state which NDCG you report.

The funnel assigns the metric

Recommenders narrow millions of items in stages: **candidate generation** (retrieval) is judged by **recall@k** (did the user's eventual choices survive the cut to, say, 500?); **ranking** by **NDCG** or MRR. If later stages re-score every candidate, retrieval's internal order matters little; if they truncate, it matters.

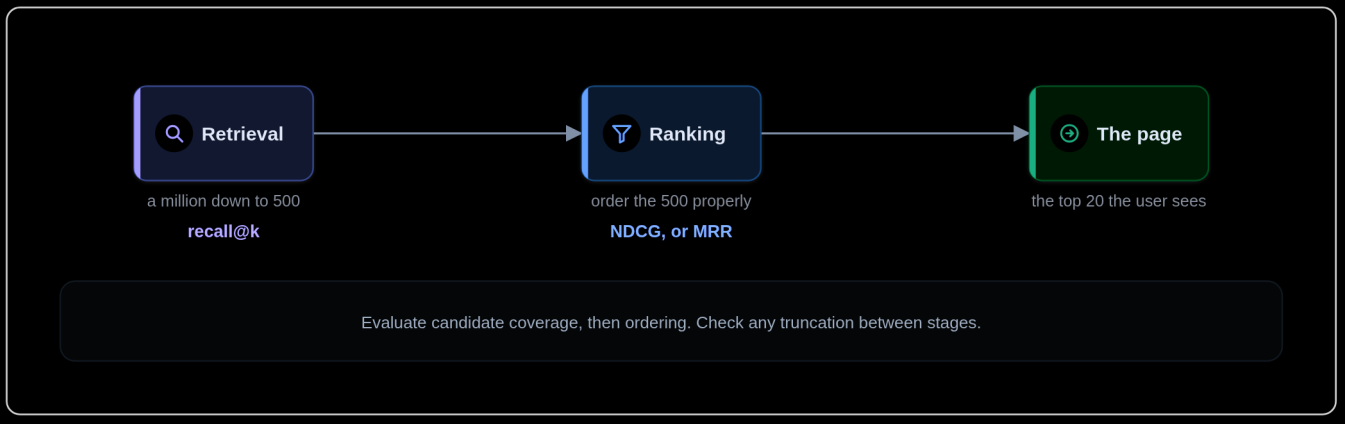


Figure — metrics per recommendation-funnel stage (course diagram)

Full treatment: Chapter 13 (recommenders) and Chapter 17 (search and feed ranking).

Common mistakes

- Comparing NDCG with different gains or different "ideal" lists.
- Evaluating only on items the old system showed (**exposure bias**: only shown items could get clicks — Chapter 13, sections 13.11–13.12).

In the interview

30 seconds: *"For a ranked list I'd use recall@k for the retrieval stage — did good items survive the cut — and NDCG@k or MRR for the ranking stage, because position matters. NDCG handles graded relevance with a log discount; MRR fits tasks with one right answer. Labels come from judged sets or clicks, and clicks carry position bias (items placed higher get more clicks just because of where they sit)."*

- *"MAP or NDCG?"* — MAP for binary labels; NDCG for graded relevance.

Self-check

1. First relevant at rank 4: reciprocal rank? → 0.25.
2. What does $\log_2(i + 1)$ do in DCG? → Discounts lower positions: rank 1 counts fully, rank 3 half.

TL;DR

Precision@k/recall@k for set quality, MAP for binary relevance, MRR for the first good answer, NDCG for graded relevance with position discount. Retrieval → recall@k; ranking → NDCG.

6.5 Calibration

What it is

A model is **calibrated** if its probabilities match reality: of cases scored 0.8, ~80% are positive. **Ranking well and being calibrated are different properties**: ROC-AUC depends only on score order (squaring every score leaves it unchanged), so a 0.95-AUC model can say "0.9" for events that happen 50% of the time.

Why it matters

Needed whenever downstream logic does arithmetic with the score rather than sorting: ad auctions (bid = value × predicted click probability), expected-cost thresholds (6.3), risk scores shown to people, combining models. After class re-weighting or negative downsampling, scores are systematically inflated and must be corrected (closed-form fix for downsampling: section 4.9; Platt vs isotonic at production scale: Chapter 16, section 16.3).

How it works

1. **Measure** with a **reliability diagram**: bin predictions (e.g. 10 bins), plot mean predicted probability vs observed positive fraction; perfect = diagonal. Summaries: **Brier score** = mean((p − y)²) and **log-loss**, both *proper scoring rules* (minimised by the true probabilities).
2. **Fix** with a score → probability map learned on held-out data:
   - **Platt scaling** (`method="sigmoid"`) — logistic regression on the score, p = σ(a·s + b). Two parameters, little data needed; assumes S-shaped distortion.
   - **Isotonic regression** (`method="isotonic"`) — any non-decreasing step function. More flexible; needs roughly ≥ 1,000 calibration examples or it overfits.
   - `CalibratedClassifierCV` uses cross-validation so the calibrator never sees the base model's training rows.
3. **Re-check** when the serving population changes — calibration drifts with the base rate.

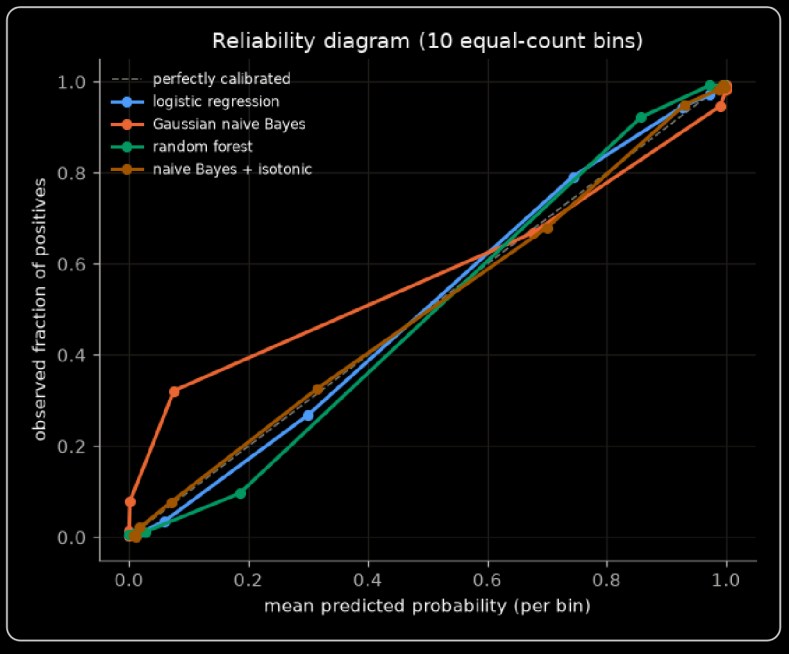
Brier by hand: y = (1, 0, 1, 1), p = (0.9, 0.2, 0.6, 0.3) → (0.01 + 0.04 + 0.16 + 0.49)/4 = **0.175**.

```
# code/calibration.py (key parts)
models = {"logistic regression": LogisticRegression(max_iter=1000), "Gaussian naive Bayes": GaussianNB(),
          "random forest": RandomForestClassifier(n_estimators=100, random_state=0),
          "naive Bayes + isotonic": CalibratedClassifierCV(GaussianNB(), method="isotonic", cv=5),
          "naive Bayes + Platt (sigmoid)": CalibratedClassifierCV(GaussianNB(), method="sigmoid", cv=5)}
frac_pos, mean_pred = calibration_curve(y_te, p, n_bins=10, strategy="quantile")
```

Output:

```
model                          ROC-AUC   Brier log-loss
logistic regression              0.980   0.048    0.176
Gaussian naive Bayes             0.975   0.069    0.335
random forest                    0.992   0.020    0.173
naive Bayes + isotonic           0.975   0.060    0.201
naive Bayes + Platt (sigmoid)    0.975   0.068    0.246

Brier by hand: mean((p - y)^2) = 0.175  sklearn 0.175
```



Naive Bayes (assumes features independent given the class; Chapter 8) ranks almost as well as logistic regression (0.975 vs 0.980) but its independence assumption double-counts redundant features, pushing scores toward 0 and 1 (log-loss 0.335 vs 0.176). Isotonic calibration keeps ROC-AUC and cuts log-loss to 0.201. Logistic regression, trained on log-loss, is near-calibrated out of the box. Random forests (averaged trees, Chapter 7) often show the opposite distortion (averaging pulls probabilities away from 0 and 1); boosted trees and neural nets can drift either way.

Common mistakes

Calibrating on training data (learns the overconfidence on memorised data); isotonic with a few hundred examples; assuming high AUC means trustworthy probabilities.

Real-world examples

Ad click prediction (Google, Meta): predicted CTRs enter the auction, so calibration errors cost money. Weather: "70% chance of rain" is the textbook calibrated forecast; the Brier score was invented for weather forecasts.

In the interview

30 seconds: *"Calibration means predicted probabilities match observed frequencies — separate from ranking quality. I'd check it with a reliability diagram and Brier or log-loss on held-out data, and fix it with Platt scaling or isotonic regression fitted on a separate calibration set, especially after resampling or class weighting, and recheck it when the population shifts."*

Self-check

1. Does calibration change ROC-AUC? → Not with a strictly increasing map (Platt); isotonic can create ties and change it slightly.
2. Brier for p = 0.5 everywhere? → 0.25.

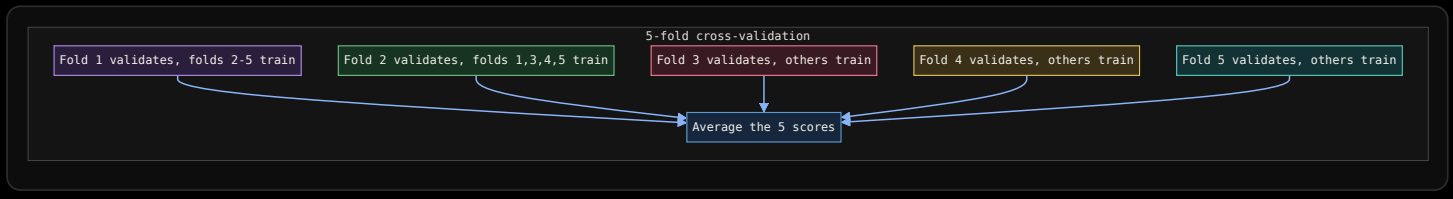
TL;DR

Calibration = probabilities you can take literally. Measure with reliability diagrams and Brier/log-loss; fix with Platt (small data) or isotonic (more data) on held-out data.

6.6 Cross-validation

What it is

Cross-validation (CV) trains and validates several times on different splits and averages. In **k-fold CV** the training data is cut into k **folds**; each takes one turn validating while the other k − 1 train.



Why it matters

- **One split is noisy:** 20 different 80/20 splits here give accuracy 0.947–0.991 — 4 points from the split alone.
- Several estimates: the mean is more reliable, the spread says how much to trust model differences.
- Every row trains and validates — important for small data.

How it works — the variants

Splitter	Folds	Use for
`KFold(k, shuffle=True)`	Random equal folds	Regression, **i.i.d.** data (independent rows from one distribution)
`StratifiedKFold`	Each fold keeps class proportions	Classification — `cross_val_score`'s default for classifiers
`GroupKFold` / `StratifiedGroupKFold`	All rows of a group (user, patient) in one fold	Correlated rows; new entities
`TimeSeriesSplit(gap=…)`	Train on the past, validate on the next block; optional gap	Time-ordered data; gap mimics label delay
`RepeatedStratifiedKFold`	Repeat with different shuffles	Small data, stabler estimates
`LeaveOneOut`	k = n	Tiny datasets only (expensive, high-variance estimate)

Typical k = 5 or 10: larger k → more training data per fold (less pessimistic) but more compute and more correlated estimates.

```
# code/cross_validation.py (key parts)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))   # preprocessing inside!
scores = cross_val_score(model, X, y, cv=5)
KFold(5)                                          # on label-sorted data: badly unbalanced folds
StratifiedKFold(5, shuffle=True, random_state=0)
TimeSeriesSplit(n_splits=3, gap=2, test_size=3)
cross_validate(model, X, y, cv=..., scoring=["accuracy", "roc_auc", "f1"], return_train_score=True)
inner = GridSearchCV(model, {"logisticregression__C": [0.01, 0.1, 1, 10, 100]}, cv=...)
outer = cross_val_score(inner, X, y, cv=...)              # nested CV
```

Output:

```
20 different 80/20 splits: accuracy from 0.947 to 0.991
5-fold CV scores: [0.982 0.982 0.974 0.974 0.991] mean 0.981 +/- 0.007
KFold (no shuffle), sorted y   positive rate per fold [0.   0.14 1.   1.   1.  ]  accuracy 0.968
StratifiedKFold                positive rate per fold [0.62 0.62 0.63 0.63 0.63]  accuracy 0.974
time split: train 0..3  (gap)  validate 6..8
time split: train 0..6  (gap)  validate 9..11
time split: train 0..9  (gap)  validate 12..14
accuracy  train 0.988 | validation 0.979
roc_auc   train 0.998 | validation 0.995
f1        train 0.991 | validation 0.983
non-nested best CV score 0.979 (optimistic) | nested CV 0.975
```

- Report **mean ± standard deviation** (0.981 ± 0.007); means differing by less than ~one std are not clearly different.
- Unshuffled `KFold` on label-sorted data gives 0%- or 100%-positive folds — nonsense; stratification gives ~63% everywhere.
- `return_train_score=True` exposes the train–validation gap (small here: little overfitting).
- **Nested CV:** the *best* CV score from tuning is optimistic (the maximum of several noisy estimates). Nested CV wraps tuning in an outer loop to estimate "tune then train" on new data (0.975 vs 0.979; the gap grows with search size). A test set used once serves the same purpose.

Common mistakes

- Preprocessing (scaling, imputation, feature selection, target encoding) outside the CV loop → leakage (Chapter 4); use a `Pipeline`.
- Shuffled k-fold on time series; reporting the selection CV score as final performance.

In the interview

30 seconds: *"k-fold CV trains k models, each validated on a different fold, and averages the scores — a lower-variance estimate than one split, with an error bar. I use stratified folds for classification, group folds when rows share an entity, and forward-in-time splits with a gap for temporal data. All preprocessing goes inside the pipeline, and a final untouched test set — or nested CV — gives the unbiased number."*

- *"Why not always leave-one-out?"* — n fits and a high-variance estimate; 5–10 folds is the sweet spot.
- *"CV on 100 million rows?"* — Usually unnecessary: one large well-designed (e.g. time-based) validation set already has low variance.

Self-check

1. 5-fold CV: fraction training each model? → 80%.
2. What does nested CV estimate? → The whole tuning-plus-training procedure.

TL;DR

k-fold CV = k train/validate rounds, averaged, with a spread. Stratify for classes, group for entities, forward in time for temporal data; preprocessing inside the folds.

6.7 The bias–variance trade-off

What it is

Expected test error = Bias² + Variance + Irreducible noise

- **Bias** — how far the *average* model (over many training sets) is from the truth. High bias = **underfitting**.
- **Variance** — how much the model changes with a different training sample. High variance = **overfitting** (chasing noise).
- **Irreducible noise** — label randomness no model removes.

Darts: high bias = tight cluster off-centre; high variance = scattered, centred on average.

How it works — a simulation you can run

k-nearest-neighbours regression (predict the average y of the k closest training points) on 300 training sets of 50 noisy points from y = sin(2πx) + noise (σ = 0.3, σ² = 0.09):

```
# code/bias_variance.py (key parts)
for k in [40, 25, 15, 10, 6, 4, 2, 1]:
    preds = []
    for _ in range(300):
        x = rng.uniform(0, 1, 50); y = true_f(x) + rng.normal(0, sigma, 50)
        preds.append(KNeighborsRegressor(n_neighbors=k).fit(x[:, None], y).predict(x_test[:, None]))
    preds = np.array(preds)
    bias2 = ((preds.mean(0) - true_f(x_test)) ** 2).mean()   # average model vs truth
    var = preds.var(0).mean()                                 # spread across training sets
```

Output:

```
  k   bias^2  variance  expected test MSE
 40    0.375     0.021              0.486
 25    0.069     0.017              0.176
 15    0.021     0.015              0.127
 10    0.007     0.018              0.115
  6    0.001     0.020              0.111
  4    0.000     0.028              0.118
  2    0.000     0.047              0.137
  1    0.000     0.095              0.185
```



k = 40 is almost flat (huge bias); k = 1 copies the nearest noisy point (zero bias, highest variance); total error is lowest at k ≈ 6–10. Every model family has such a **complexity knob**: tree depth, polynomial degree, regularization strength (inverse), number of boosting rounds, network size.

Over- and underfitting: symptoms and remedies

	Underfitting (high bias)	Overfitting (high variance)
Symptom	Training **and** validation both poor and close	Training much better than validation
More data helps?	No	Yes
Remedies	More flexible model (deeper trees, more features, polynomial/interaction terms, a neural net); **less** regularization; train longer; better features	More data; **more** regularization (L1/L2, dropout, weight decay); simpler model; early stopping; bagging/ensembles; data augmentation; leakage-free feature selection

Terms: *regularization* = any penalty/constraint against overly complex fits; *dropout* randomly switches off network units in training; *weight decay* = L2 penalty; *early stopping* halts when validation error stops improving (6.10); *bagging* averages models trained on random resamples; an *ensemble* combines several models; *data augmentation* creates examples by modifying existing ones (flipping images); *capacity* = how complex a pattern a model can represent.

Common mistakes

- Adding data to a high-bias model; judging overfitting from training score alone; assuming more complex = better.

In the interview

30 seconds: *"Test error = bias² + variance + noise. Simple models have high bias and underfit — both scores bad; flexible models have high variance and overfit — training much better than validation. I diagnose with learning curves: if the curves have converged at a low score, add capacity or features; if there's a big gap, add data or regularization."*

- *"Bagging: bias or variance?"* — Mainly variance. **Boosting** (sequential models fixing predecessors' errors) mainly reduces bias.
- *"More regularization?"* — Bias up, variance down.

Self-check

1. Train 99%, validation 80%? → Overfitting.
2. Train 70%, validation 69%, target 90%? → More capacity / features / less regularization.

TL;DR

Bias = systematically wrong; variance = unstable across samples; error is U-shaped in complexity. Big gap → regularize or add data; both low → add capacity.

6.8 Learning curves and validation curves

What it is

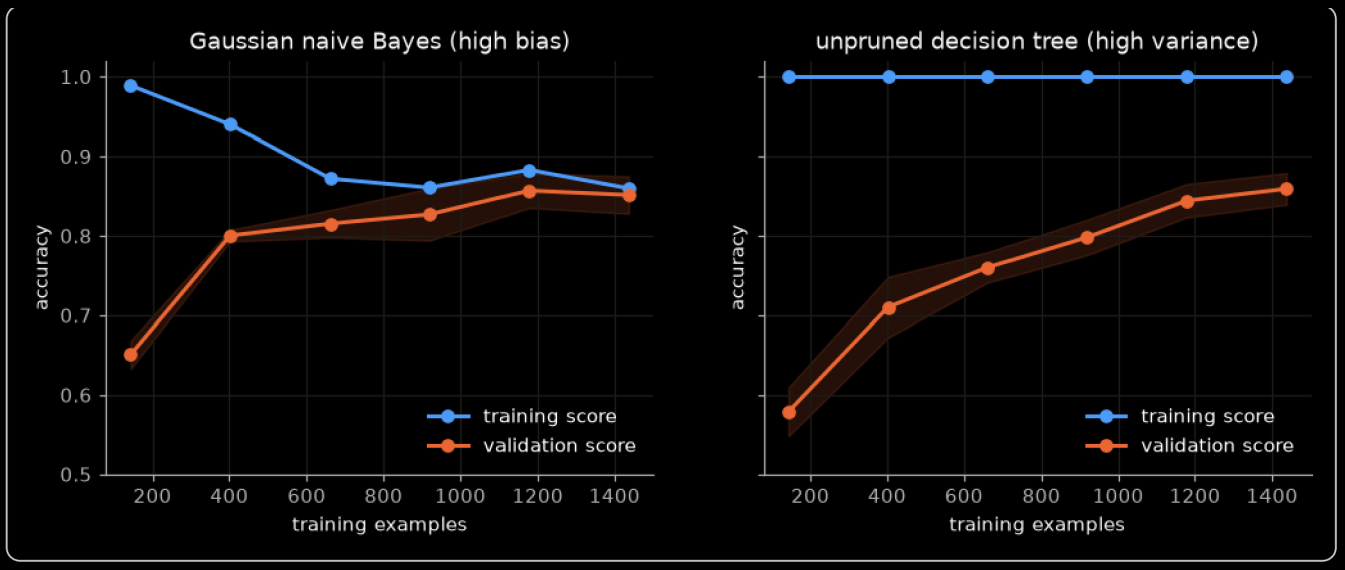
- **Learning curve:** train and validation score vs **training-set size** — *would more data help?*
- **Validation curve:** train and validation score vs **one hyperparameter** — *where is the sweet spot?*

```
# code/bias_variance.py (curves)
sizes, tr, va = learning_curve(model, X, y, cv=cv, train_sizes=np.linspace(0.1, 1.0, 6))
tr, va = validation_curve(make_pipeline(StandardScaler(), SVC()), X, y,
                          param_name="svc__gamma", param_range=np.logspace(-6, -1, 11), cv=cv)
```

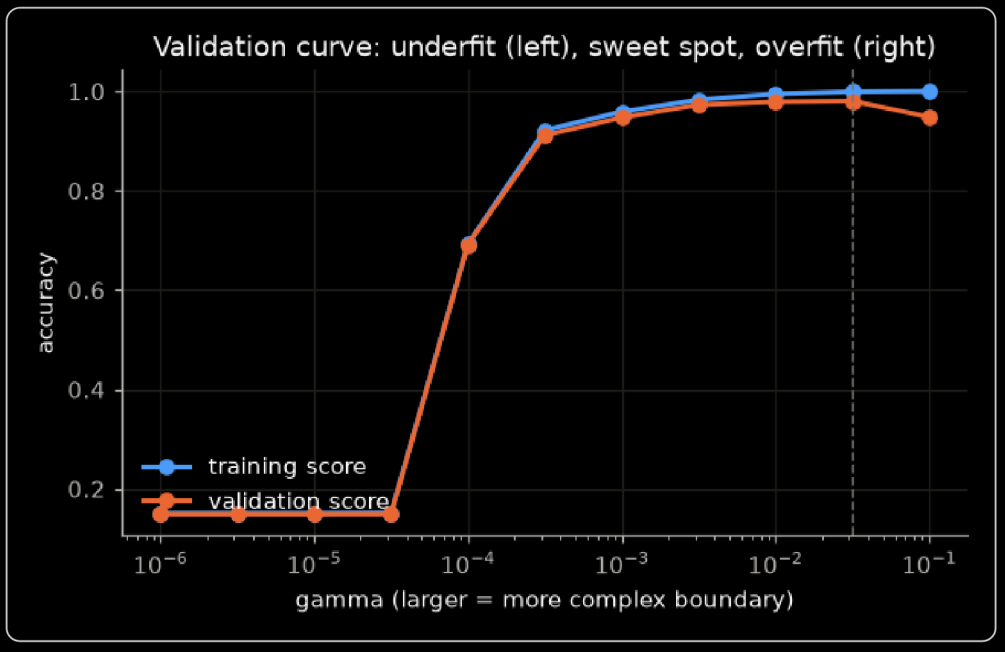
Output:

```
Gaussian naive Bayes (high bias)         n=1437: train 0.860  validation 0.851
unpruned decision tree (high variance)   n=1437: train 1.000  validation 0.859

validation curve: best gamma = 3e-02, validation acc 0.981
  gamma 1e-06: train 0.151  validation 0.149
  gamma 1e-05: train 0.151  validation 0.149
  gamma 1e-04: train 0.693  validation 0.692
  gamma 1e-03: train 0.959  validation 0.947
  gamma 1e-02: train 0.995  validation 0.979
  gamma 1e-01: train 1.000  validation 0.948
```



- **Naive Bayes:** both scores meet at ~0.85 and flatten — high bias; more data won't help; change model or features.
- **Unpruned tree** (splits until every leaf is pure): training always 1.0, validation climbs slowly — big gap = high variance. More data, pruning (`max_depth`, `min_samples_leaf`) or a random forest would help.



gamma sets the RBF kernel's reach (the SVM's similarity measure, falling off with distance; larger gamma = each point influences a tinier neighbourhood = wigglier boundary). Tiny gamma: both scores ~0.15 (near the 0.10 chance level for 10 digits) — underfit. Past ~0.03 training stays at 1.0 while validation falls — overfit. Pick the validation peak.

Common mistakes

- Plotting only the training curve; reading a small final gap as "no overfitting" while both curves still rise; trusting one noisy point (show the CV band).

In the interview

30 seconds: *"A learning curve plots train and validation score against training-set size: if both have converged low, the model is biased and more data won't help; if there's a big gap that narrows with more data, it's variance and more data or regularization will. A validation curve sweeps one hyperparameter to find the peak between under- and overfitting."*

TL;DR

Learning curve (score vs data size): will more data help? Validation curve (score vs hyperparameter): where is the sweet spot?

6.9 Hyperparameter tuning

What it is

Parameters are learned (weights, split thresholds); **hyperparameters** are set **before** training (`max_depth`, k in k-NN, C or α, learning rate, number of trees). **Tuning** searches for the best *validation* performance — CV or a validation set, never the test set.

How it works — the main strategies

1. Grid search (`GridSearchCV`) — try **every combination** of listed values with CV.

- *Pros:* systematic; finds the best point *on the grid*; easy to parallelise across cores.
- *Cons:* cost multiplies (3 × 3 values × 5 folds = 45 fits; 5 hyperparameters × 5 values = 3,125 combinations); the grid may miss the good region.

2. Random search (`RandomizedSearchCV`) — sample a fixed number of combinations from **distributions** (`randint` for integers, `loguniform` for log-scale values).

- *Pros:* budget control (`n_iter`); more distinct values per hyperparameter; usually as good as grid in far fewer trials when few hyperparameters matter (Bergstra & Bengio, 2012).
- *Cons:* seed-dependent; no guarantee of the exact optimum.

3. Successive halving / Hyperband (`HalvingGridSearchCV`, `HalvingRandomSearchCV` — experimental in scikit-learn 1.9: first `from sklearn.experimental import enable_halving_search_cv`) — evaluate many candidates cheaply (few samples or iterations), keep the best fraction, give survivors more resources, repeat. `factor=2`: keep half, double samples.

4. Bayesian optimisation — fit a **surrogate model** of hyperparameters → score from past trials (a Gaussian process, which predicts a value and its uncertainty, or **TPE**, Tree-structured Parzen Estimator), then pick the next trial by maximising an **acquisition function** such as **expected improvement**, balancing *exploiting* promising regions and *exploring* uncertain ones. Best when trials are expensive (minutes to hours). Tools: **Optuna** (TPE default, plus **pruning** of unpromising trials), Hyperopt, scikit-optimize, Ray Tune.

Search learning rates, C, α and gamma on a **log scale** (`loguniform(1e-4, 1e1)`).

```
# code/tuning.py (key parts)
grid = GridSearchCV(DecisionTreeClassifier(random_state=0),
                    {"max_depth": [3, 5, 7], "min_samples_split": [2, 5, 10]}, cv=3, scoring="f1")
rnd = RandomizedSearchCV(DecisionTreeClassifier(random_state=0),
                         {"max_depth": randint(3, 10), "min_samples_split": randint(2, 20),
                          "min_samples_leaf": randint(1, 10)}, n_iter=10, cv=3, scoring="f1", random_state=0)

svm = make_pipeline(StandardScaler(), SVC())
GridSearchCV(svm, {"svc__C": [0.1, 1, 10], "svc__gamma": [1e-4, 1e-3, 1e-2]}, cv=cv)
RandomizedSearchCV(svm, {"svc__C": loguniform(1e-2, 1e3), "svc__gamma": loguniform(1e-5, 1e-1)}, n_iter=9, cv=cv)
HalvingRandomSearchCV(svm, {...same distributions...}, n_candidates=32, factor=2, cv=cv)

# Tiny Bayesian optimisation over log10(gamma): GP surrogate + expected improvement
gp = GaussianProcessRegressor(Matern(nu=2.5) + WhiteKernel(1e-4), normalize_y=True).fit(xs, ys)
mu, sd = gp.predict(grid_pts, return_std=True)
z = (mu - max(ys)) / sd
ei = (mu - max(ys)) * norm.cdf(z) + sd * norm.pdf(z)       # expected improvement
next_x = grid_pts[ei.argmax()]
```

Output:

```
grid   best params: {'max_depth': 5, 'min_samples_split': 2} CV F1 0.942 | test F1 0.920
random best params: {'max_depth': 6, 'min_samples_leaf': 6, 'min_samples_split': 17} CV F1 0.937 | test F1 0.939

SVM, 9 evaluations each: grid best 0.9866 {'svc__C': 10, 'svc__gamma': 0.01}
                         random best 0.9844 {'svc__C': 1.22, 'svc__gamma': 0.00761}
halving: [32, 16, 8, 4, 2] candidates per round, samples per round [60, 120, 240, 480, 960] -> best 0.9698

Bayesian optimisation of gamma (9 evaluations):
  eval 1: gamma = 1e-2.45  CV accuracy 0.9861  (random start)
  eval 2: gamma = 1e-3.92  CV accuracy 0.9510  (random start)
  eval 3: gamma = 1e-4.84  CV accuracy 0.7284  (random start)
  eval 4: gamma = 1e-2.37  CV accuracy 0.9878
  eval 5: gamma = 1e-3.31  CV accuracy 0.9777
  eval 6: gamma = 1e-1.18  CV accuracy 0.9688
  eval 7: gamma = 1e-1.78  CV accuracy 0.9850
  eval 8: gamma = 1e-2.09  CV accuracy 0.9861
  eval 9: gamma = 1e-3.57  CV accuracy 0.9672
```

- **Grid vs random (trees):** best CV scores differ by 0.005 — noise — and random's pick tests better. Small differences between tuned configurations rarely matter; don't over-tune.

- **SVM:** 9 trials each, within 0.2%; random search's edge grows with more (especially unimportant) hyperparameters.
- **Halving** tried 32 candidates, most on only 60–240 rows; 2 finalists got 960. Its `best_score_` is on the last round's subsample, not comparable.
- **Bayesian:** after 3 random starts the first guided trial improves (0.9878), then expected improvement alternates refining and exploring (evals 6, 7, 9).

Optuna *(not installed in this book's environment — snippet not run)*:

```
import optuna

def objective(trial):
    params = {
        "learning_rate": trial.suggest_float("learning_rate", 1e-3, 0.3, log=True),
        "max_leaf_nodes": trial.suggest_int("max_leaf_nodes", 8, 128, log=True),
        "l2_regularization": trial.suggest_float("l2_regularization", 1e-6, 10, log=True),
    }
    model = HistGradientBoostingClassifier(**params, random_state=0)
    return cross_val_score(model, X_train, y_train, cv=5, scoring="roc_auc").mean()

study = optuna.create_study(direction="maximize")      # TPE sampler by default
study.optimize(objective, n_trials=50)                 # then study.best_params, study.best_value
```

Trade-offs

Method	Budget control	Many hyperparameters	Uses past results	Best for
Grid	No (product of grid sizes)	Poorly	No	1–2 hyperparameters, cheap models
Random	Yes (`n_iter`)	Well	No	Default for 3+
Successive halving	Yes	Well	Yes (early elimination)	Many candidates; scalable resource (rows, epochs)
Bayesian (Optuna, GP)	Yes	Moderately (tens of dims)	Yes (surrogate)	Expensive trials: boosting on big data, deep nets

Practical recipe

1. Start from library defaults and a baseline.
2. Tune the few hyperparameters that matter (learning rate + tree size for boosting; C for linear models/SVMs; depth/leaf size for trees).
3. Random search or Optuna on log scales, 20–100 trials, with CV.
4. If the best value sits at a range **edge**, extend the range.
5. Refit on all training data (`refit=True`), evaluate **once** on test.

Common mistakes

- Tuning on test, or reporting the best CV score as final (optimistic — use a test set or nested CV).
- Linear ranges for scale-type hyperparameters; grids so fine you fit CV noise.
- Tuning before fixing data and features — usually the bigger win.

In the interview

30 seconds: *"Hyperparameters are set before training, so I tune them with cross-validation: grid search for one or two parameters, random search for more — it's budget-controlled and explores each dimension better — and Bayesian optimisation like Optuna when each trial is expensive; successive halving to discard bad configurations early. Log-scale ranges, tune on validation folds only, and evaluate the winner once on the test set."*

- *"Why is random often better than grid?"* — Same budget, but a new value of every hyperparameter each trial, so important dimensions are explored more finely.
- *"How does Bayesian optimisation choose the next trial?"* — Maximises an acquisition function (e.g. expected improvement) on a surrogate, trading predicted score against uncertainty.

Self-check

1. 4 × 5 grid, 5-fold CV: fits before refit? → 100.
2. Distribution for a learning rate in 1e-4…1? → Log-uniform.

TL;DR

Grid for tiny spaces, random for most, halving to prune early, Bayesian (Optuna) for expensive trials. Log scales, CV only, test once.

6.10 Early stopping

What it is

For iteratively trained models (gradient boosting, neural networks, SGD), **early stopping** tracks a validation score and stops after it fails to improve for **patience** iterations — tuning the iteration count almost for free.

Why it matters

Training loss falls forever; validation loss bottoms out then rises as the model fits noise. Stopping near the bottom is a cheap regularizer that also saves compute.

```
# code/tuning.py (early stopping)
full = HistGradientBoostingClassifier(max_iter=400, learning_rate=0.2, early_stopping=True,
                                      n_iter_no_change=400, validation_fraction=0.2, random_state=0).fit(Xc, yc)
stop = HistGradientBoostingClassifier(max_iter=400, learning_rate=0.2, early_stopping=True,
                                      n_iter_no_change=10, validation_fraction=0.2, random_state=0).fit(Xc, yc)
```

Output:

```
full run: best validation loss at iteration 25 of 400
early stopping (patience 10) stopped after 35 iterations
```



Validation loss bottoms at 25; with patience 10 training stops at 35 instead of 400 (later rounds only overfit — validation loss doubles by the end). scikit-learn's `HistGradientBoosting*` keeps all iterations up to the stop; Keras's `EarlyStopping(restore_best_weights=True)` and XGBoost/LightGBM best-iteration tracking roll back to the best.

How to use it

- scikit-learn: `HistGradientBoosting*(early_stopping=True, validation_fraction=0.1, n_iter_no_change=10)` (on by default when n_samples > 10,000), `GradientBoosting*(n_iter_no_change=…)`, `SGDClassifier(early_stopping=True)`, `MLPClassifier(early_stopping=True)`.
- XGBoost / LightGBM: an evaluation set plus early-stopping rounds; Keras: the `EarlyStopping` callback (called by the training loop, e.g. after every epoch).
- Set a generous maximum and small learning rate; let early stopping choose the count.

Common mistakes

- Early-stopping on the **test** set (it becomes training).
- Patience too small on noisy curves (stops on a random bump).
- A randomly sampled internal validation split for a time-series problem.

In the interview

30 seconds: *"Early stopping holds out a validation set, tracks its loss each iteration or epoch, and stops once it hasn't improved for `patience` rounds — tuning the number of iterations for free and acting as a regularizer. Set a*

generous maximum, keep or restore the best iteration, and never early-stop on the test set."

Self-check

1. Why is early stopping regularization? → It limits how far the model fits the training data, like limiting complexity.

TL;DR

Watch validation loss; stop after `patience` rounds without improvement. Cheap regularization and compute savings — on validation, never test.

6.11 Putting it together: from ML metric to business decision

1. **Define the business goal and error costs** (fraud losses vs customer friction; late deliveries vs over-cautious promises).
2. **Pick the offline metric** tracking it (6.3 table; PR-AUC plus P/R at the operating point for rare events; recall@k/NDCG for ranking; MAE/RMSE + p90 + cohorts for regression), plus a **calibration** check if scores are probabilities.
3. **Split correctly** (stratified / group / time) and **cross-validate** the training part.
4. **Baselines:** naive + current production.
5. **Tune** with random search or Bayesian optimisation; early stopping for iterative models.
6. **Diagnose** with learning and validation curves.
7. **Evaluate once on test**; report metric with base rate, threshold, cohorts and uncertainty.
8. **Choose the threshold** with the cost owner.
9. **Confirm online** with an A/B test (Chapter 16), then monitor.

In the interview

- *"Which metric would you use?"* — Name the metric family, base rate and cost asymmetry; give an operating-point metric (precision at recall X) plus a threshold-free one (PR-AUC or NDCG); mention calibration if probabilities are consumed; compare with naive and production; confirm online.

Chapter TL;DR

- **Baselines first** (majority / mean / last value / most popular, plus production). Offline metrics are proxies; confirm online.
- **Regression:** MAE (typical error, median-optimal), RMSE (punishes big misses, mean-optimal), R^2 (variance explained vs the mean); add p90 and per-cohort errors.
- **Classification:** precision TP/(TP+FP), recall TP/(TP+FN), F1 (harmonic mean); accuracy only when balanced. Threshold from error costs (calibrated p > C_FP/(C_FP + C_FN)); fraud uses approve/review/block bands.
- **ROC-AUC vs PR-AUC:** ROC-AUC = P(positive outranks negative), base-rate-insensitive; PR-AUC exposes false alarms, the choice for rare positives (no-skill = base rate).
- **Ranking:** precision@k, recall@k (retrieval), MAP, MRR, NDCG — check which "ideal" your library uses.
- **Calibration:** separate from ranking; reliability diagram, Brier, log-loss; Platt (small data) or isotonic (more data) on held-out data.
- **Cross-validation:** k-fold mean ± std; stratified, group, time-series with gap; preprocessing inside; nested CV or a test set for final numbers.
- **Bias–variance:** error = bias² + variance + noise. Underfit → capacity/features, less regularization; overfit → data, regularization, simpler model, early stopping, ensembles.
- **Curves:** learning (score vs data size), validation (score vs hyperparameter).
- **Tuning:** grid (small), random (default), successive halving (prune early), Bayesian/Optuna (expensive). Log scales; never tune on test.
- **Early stopping:** stop when validation loss hasn't improved for `patience` rounds.

Chapter 7 — Decision Trees and Ensembles

Decision trees are the most "human" model: a chain of yes/no questions you can print and read. Alone they overfit easily, but combined — by **averaging** (Random Forests) or by **correcting each other's mistakes** (gradient boosting) — they win most real-world problems on **tabular data** (rows and columns, like a spreadsheet or SQL table).

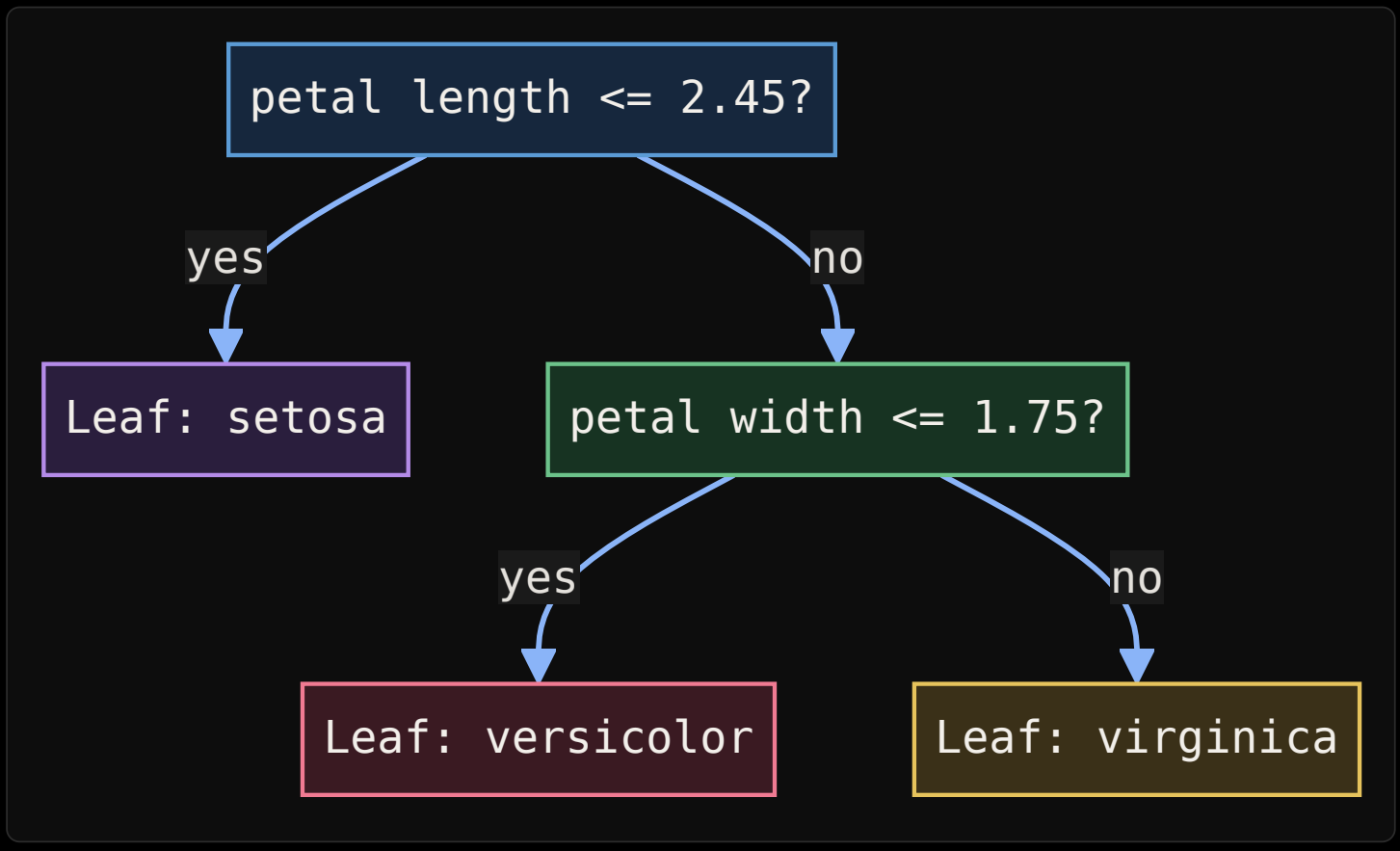
What you'll learn: how a tree picks each split (Gini, entropy, information gain); depth, pruning, regression trees; bagging and Random Forests (bootstrap, feature subsampling, out-of-bag score); feature importance and its pitfalls (permutation importance); AdaBoost and gradient boosting on residuals; XGBoost vs LightGBM vs CatBoost; voting and stacking; why trees beat neural nets on tables; a hyperparameter cheat sheet.

All code is in `code/` (scikit-learn 1.9). Outputs are real.

7.1 Decision trees

What it is

A **decision tree** predicts by asking questions about features ("Is petal length ≤ 2.45 cm?"). Each question is a **node**, each answer a **branch**, and a **leaf** holds the prediction (class or number). Like "20 Questions", the learning algorithm greedily asks whichever question rules out the most first.



Why it matters

- **Interpretable:** a shallow tree is a flowchart a doctor or loan officer can check.
- **No scaling needed:** "x ≤ 3.2" doesn't care about units.
- **Mixed data, non-linear effects and interactions** handled automatically ("high income AND young" is its own branch).
- The **building block** of Random Forests, XGBoost, LightGBM and CatBoost.

How it works — growing a tree (CART)

scikit-learn implements **CART** (Classification And Regression Trees), always with **binary** splits:

1. All training rows start at the root.
2. For every feature and candidate threshold (midpoints between sorted unique values), split rows left (≤) and right (>).
3. Score each split by how much it reduces **impurity** (label mixing); take the best.
4. Recurse on each child.

5. Stop when the node is pure, `max_depth` is reached, too few samples (`min_samples_split`, `min_samples_leaf`), or no split helps enough.
6. Leaves predict the majority class (storing class proportions for `predict_proba`) or the mean target.

This is **greedy** (best split now, never revisited); finding the optimal tree is **NP-hard** (no known fast exact algorithm).

Impurity measures

For class proportions p_k in a node:

Measure	Formula	Pure node	50/50 binary node
Gini impurity	$G = 1 - \sum p_k^2$	0	0.5
Entropy	$H = -\sum p_k \log_2 p_k$	0	1.0 bit
Misclassification error	$1 - \max_k p_k$	0	0.5 (not sensitive enough for growing)

- **Gini** = probability of mislabelling a random sample labelled randomly by the node's class mix.
- **Entropy** = average bits (yes/no answers) to identify the class (information theory).
- **Information gain** = entropy(parent) − weighted average entropy(children), weights = share of rows per child. The Gini version is called Gini decrease.

Worked example: Gini vs entropy

Ten loan applicants (1 = defaulted); split *income < 40k?*

Node	Labels	Gini	Entropy
Parent	4 defaults, 6 repaid	$1 - 0.4^2 - 0.6^2 =$ **0.480**	**0.971**
Left (income < 40k)	3 defaults, 1 repaid	0.375	0.811
Right (income ≥ 40k)	1 default, 5 repaid	0.278	0.650

Weighted children: Gini 0.4·0.375 + 0.6·0.278 = 0.317 → **Gini decrease 0.163**; entropy 0.715 → **information gain 0.256**. The tree tests every midpoint (`code/split_math.py`):

```
def gini(labels):
    n = len(labels)
    return 1 - sum((c / n) ** 2 for c in Counter(labels).values())

def entropy(labels):
    n = len(labels)
    return -sum((c / n) * math.log2(c / n) for c in Counter(labels).values())

def weighted(children, fn):          # impurity of a split
    n = sum(len(c) for c in children)
    return sum(len(c) / n * fn(c) for c in children)
# ... loop over every midpoint threshold and print both scores
```

Output:

```
parent [1, 1, 1, 0, 0, 0, 0, 0, 1, 0]  -> gini=0.480  entropy=0.971
left   [1, 1, 1, 0]         -> gini=0.375  entropy=0.811
right  [0, 0, 0, 0, 1, 0]     -> gini=0.278  entropy=0.650
weighted child gini    = 0.317  -> gini decrease   = 0.163
weighted child entropy = 0.715  -> information gain = 0.256

threshold  gini_decrease  info_gain
     27.5          0.080      0.144
     32.5          0.180      0.322
     36.5          0.309      0.557
     41.5          0.163      0.256
     ...
     85.0          0.036      0.079
```

Both measures pick **36.5** (left side 25, 30, 35 is pure). Gini and entropy almost always rank splits the same; Gini is scikit-learn's default because it skips the logarithm. Don't spend tuning time on it.

Reading a fitted tree

`code/decision_tree_demo.py` prints a depth-2 Iris tree with `export_text`:

```
tree = DecisionTreeClassifier(max_depth=2, random_state=42).fit(iris.data, iris.target)
print(export_text(tree, feature_names=list(iris.feature_names)))
```

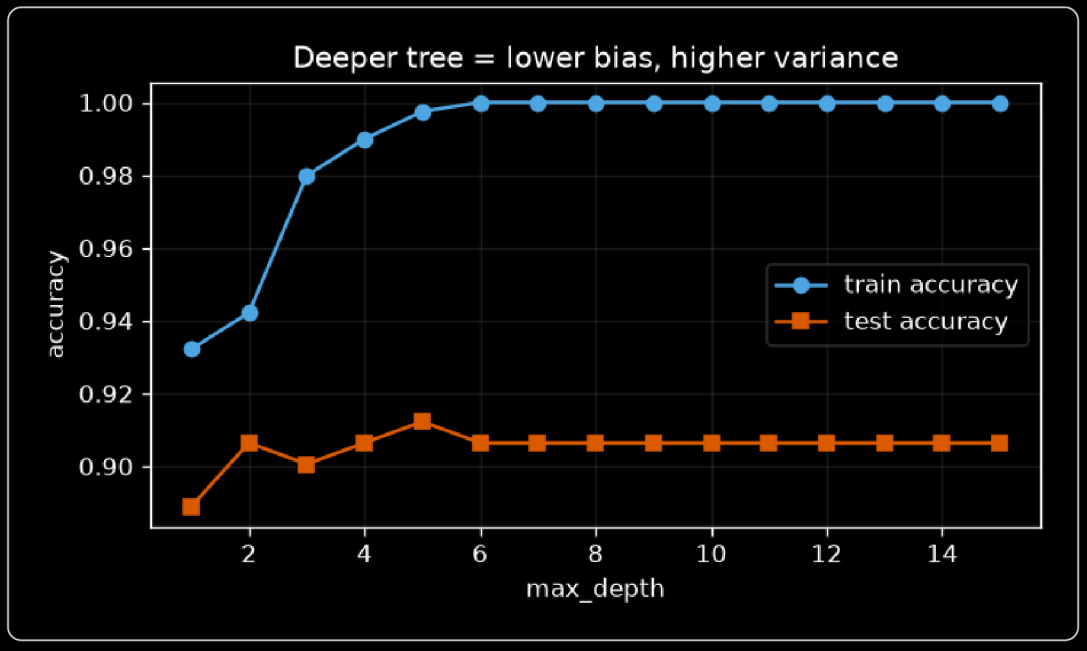
```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.75
|   |   |--- class: 1
|   |--- petal width (cm) >  1.75
|   |   |--- class: 2
```

Two questions classify ~96% of flowers — the interpretability selling point.

Depth and overfitting

Unlimited, a tree splits until leaves are pure and memorises training data. Breast-cancer dataset (30 features, 569 patients):

```
depth  train  test
    1  0.932  0.889
    2  0.942  0.906
    3  0.980  0.901
    4  0.990  0.906
    6  1.000  0.906
   10  1.000  0.906
   15  1.000  0.906
```



Training hits 100% at depth 6 while test stays ~0.90: extra splits fit noise.

Pruning

Kind	How	scikit-learn knobs
Pre-pruning (early stopping)	Stop growing early	`max_depth`, `min_samples_split`, `min_samples_leaf`, `max_leaf_nodes`, `min_impurity_decrease`
Post-pruning (cost-complexity pruning)	Grow fully, cut back branches that don't pay	`ccp_alpha`

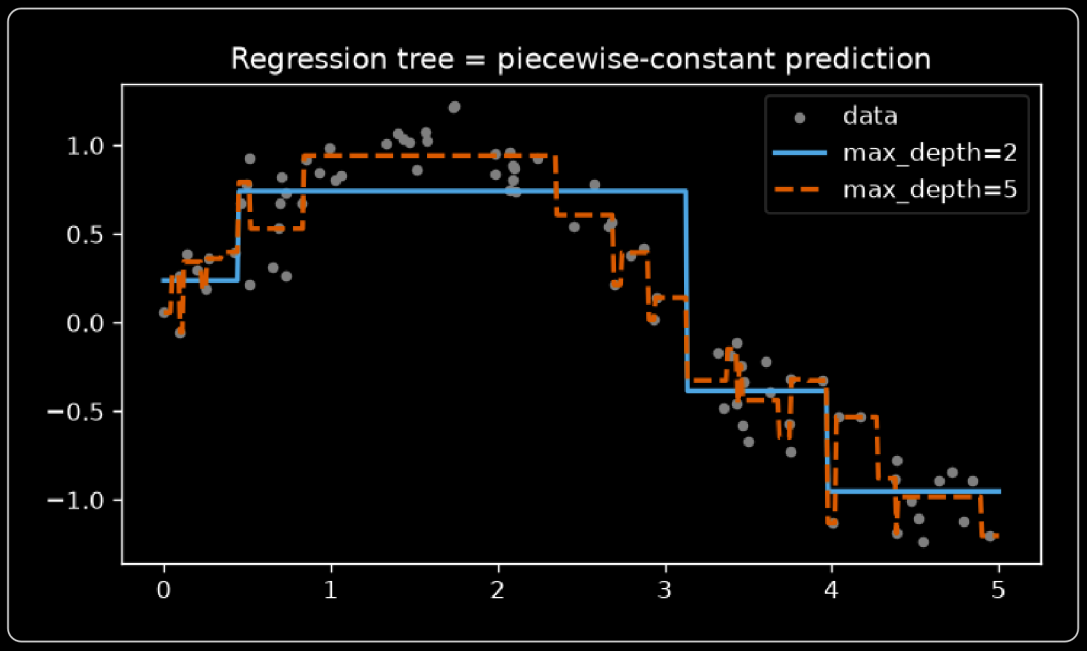
Cost-complexity pruning minimises $R_\alpha(T) = R(T) + \alpha \cdot |\text{leaves}(T)|$ (R = training impurity, α = price per leaf). $\alpha = 0$ keeps the full tree; larger α removes the weakest branches first. `cost_complexity_pruning_path` lists the α values where branches vanish; choose α by cross-validation.

```
full tree: 16 leaves, test acc 0.906
ccp_alpha=0.0    leaves= 16  test acc=0.906
ccp_alpha=0.005  leaves=  7  test acc=0.918
ccp_alpha=0.01   leaves=  5  test acc=0.918
ccp_alpha=0.02   leaves=  4  test acc=0.901
ccp_alpha=0.05   leaves=  2  test acc=0.889
```

A 5-leaf tree beats the 16-leaf one — simpler *and* better; 2 leaves underfits.

Regression trees

Splits minimise the **squared error** (equivalently the variance) of the target in the children; leaves predict the **mean** (`criterion="absolute_error"` uses the median, more outlier-robust). The result is a **step function**:




```
|--- x <= 3.13
|   |--- x <= 0.44
|   |   |--- value: [0.24]
|   |--- x >  0.44
|   |   |--- value: [0.74]
|--- x >  3.13
|   |--- x <= 3.98
|   |   |--- value: [-0.39]
|   |--- x >  3.98
|   |   |--- value: [-0.95]
```

Depth 2 = 4 flat steps (underfit); depth 5 chases noisy points (overfit). **A tree cannot extrapolate**: outside the training range it predicts the last leaf's value, flat forever.

Trade-offs

Strengths	Weaknesses
Readable rules	High variance: small data change → very different tree
No scaling; non-linearity and interactions	Overfits without depth limits or pruning
Fast prediction: O(depth)	**Axis-aligned** splits (one feature each): a diagonal boundary needs a staircase
Numeric + categorical data (scikit-learn ≥ 1.3 trees handle missing values natively)	Cannot extrapolate
	Greedy — may miss the best overall tree

When not to use it

- Accuracy over readability → forest or boosting.
- Smooth or linear relationships (a line needs 2 parameters; a tree a staircase).
- Forecasting beyond the training range (ever-rising prices).

Common mistakes

- `max_depth=None` and reporting training accuracy (~100%, meaningless).
- Scaling "for the tree" (harmless, pointless); over-reading one tree's structure (a resample may change the top split).
- One-hot encoding a thousands-level categorical into a tree (each split isolates one level); prefer target encoding (category → average target), ordinal encoding, or native categorical support.

Real-world examples

- Credit-scoring and medical-triage rules that regulators or clinicians must read.
- The **weak learner** (model only slightly better than guessing; 7.4) inside every gradient-boosting library.

In the interview

> **30-second answer:** "A decision tree recursively splits the data on the feature/threshold that most reduces impurity — Gini or entropy for classification, variance for regression. Leaves predict the majority class or the mean. Trees are interpretable and need no scaling, but they have high variance and overfit unless you limit depth or prune with cost-complexity pruning, which is why we usually use them inside ensembles."

- *Gini or entropy?* Agree on almost all splits; Gini is default (no log). Not worth tuning.
- *Why greedy?* The optimal tree is NP-hard; greedy top-down splitting is fast and works well.
- *A feature 1000× larger in scale?* Irrelevant — splits depend only on value order.

Self-check

1. 8 cats, 2 dogs: Gini? → $1 - 0.8^2 - 0.2^2 =$ **0.32**.
2. Increasing `ccp_alpha`? → Prunes more (smaller tree, higher bias, lower variance).
3. Why can't a tree predict a price above any seen? → Leaves are averages of training targets.

TL;DR

Trees ask the best yes/no question each step (max impurity decrease), are readable and scale-free, but overfit — limit depth or prune by cost complexity.

7.2 Bias–variance of trees

What it is

(Bias–variance in general: Chapter 6, section 6.7.) A **shallow** tree (a depth-1 **stump**) has high bias, low variance; a **deep** tree has low bias and **very high variance** — a different **bootstrap sample** (random resample of rows; 7.3) gives a noticeably different tree.

Why it matters

Deep trees are accurate *on average* but noisy — exactly what averaging fixes. Averaging B models, each with variance σ^2 and pairwise error correlation ρ (0 = unrelated, 1 = identical):

$$\mathbf{Var(average) = \rho\sigma^2 + (1 - \rho)\sigma^2/B}$$

As B grows the second term vanishes but $\rho\sigma^2$ stays: **averaging helps only as much as the models disagree**.

- **Bagging** averages many deep trees (shrinks the second term).
- **Random Forests** also **de-correlate** trees (lower ρ, shrinking the first).
- **Boosting** starts from high-bias shallow trees and adds them sequentially to **reduce bias**.

Method	Base trees	Mainly reduces	Trees built
Bagging / Random Forest	Deep (low bias, high variance)	Variance	Independently, in parallel
Boosting	Shallow (high bias, low variance)	Bias (some variance via **shrinkage**, a small learning rate)	Sequentially

TL;DR

Deep trees = low bias, high variance → average them (bagging/RF). Shallow trees = high bias → chain them (boosting).

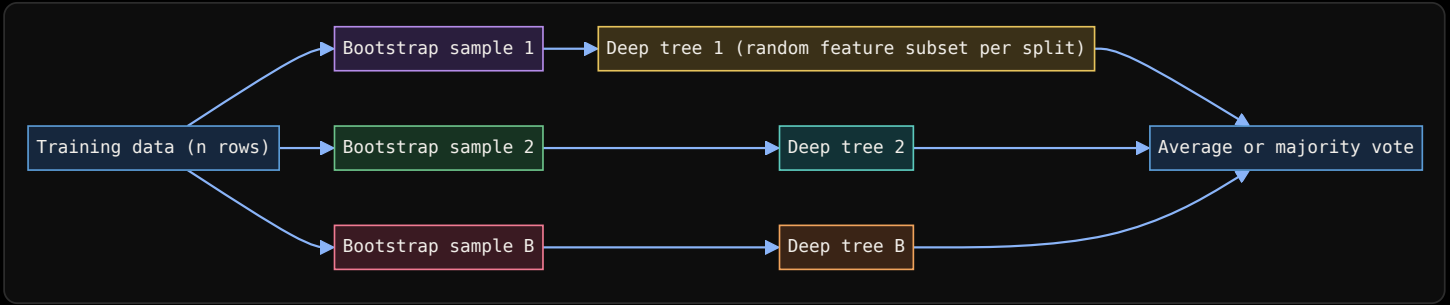
7.3 Bagging and Random Forests

What it is

Bagging (Bootstrap AGGregatING, Breiman 1996) trains copies of one model on different **bootstrap samples** — n rows drawn **with replacement** from n (some repeat, some missing) — and averages them (majority vote or averaged probabilities; mean for regression).

A **Random Forest** (Breiman 2001) = bagged decision trees **plus** a **random subset of features** considered at each split (`max_features`).

Analogy: 500 doctors from the same school reading the same chart add little; if each saw different tests (bootstrap) and focused on different symptoms (feature subsampling), their vote is far more reliable.



How it works

1. For b = 1..B (`n_estimators`, e.g. 100–500): draw a bootstrap sample; grow a deep (usually unpruned) tree, searching each split **only among** `max_features` random features.
2. Predict by averaging the trees' probabilities (scikit-learn uses **soft voting**) or values.

Why feature subsampling helps: one very strong feature would top every bagged tree, making them nearly identical (high ρ); hiding it from some splits forces other trees to find other patterns.

Defaults (scikit-learn): `max_features="sqrt"` (√p) for classification, `1.0` (all features = plain bagging) for regression; the textbook regression suggestion is p/3 — try a few.

Out-of-bag (OOB) evaluation

A row is **never** drawn in n draws with probability $(1 - 1/n)^n \approx 1/e \approx$ **36.8%**, so each tree misses about a third of rows — its **out-of-bag** rows. Predicting each row with only the trees that didn't see it gives a free validation score without a hold-out set (`oob_score=True`).

Code — single tree vs bagging vs Random Forest

`code/forest_demo.py`:

```
models = {
    "single tree": DecisionTreeClassifier(random_state=0),
    "bagged trees (100)": BaggingClassifier(DecisionTreeClassifier(), n_estimators=100, random_state=0),
    "random forest (100)": RandomForestClassifier(n_estimators=100, random_state=0),
}
for name, m in models.items():
    s = cross_val_score(m, X, y, cv=5)

rf = RandomForestClassifier(n_estimators=300, oob_score=True, random_state=0, n_jobs=-1)
rf.fit(X_tr, y_tr)
print(rf.oob_score_, rf.score(X_te, y_te))
```

Output:

```
5-fold CV accuracy (mean +/- std)
  single tree          0.917 +/- 0.016
  bagged trees (100)   0.958 +/- 0.029
  random forest (100)  0.963 +/- 0.022

OOB accuracy  = 0.967  (free validation estimate, no hold-out used)
test accuracy = 0.953
expected out-of-bag fraction per tree = 0.367  (1/e = 0.368)
```

Averaging adds ~4 points; feature subsampling a little more; OOB is close to the test score.

Feature importance and its pitfalls

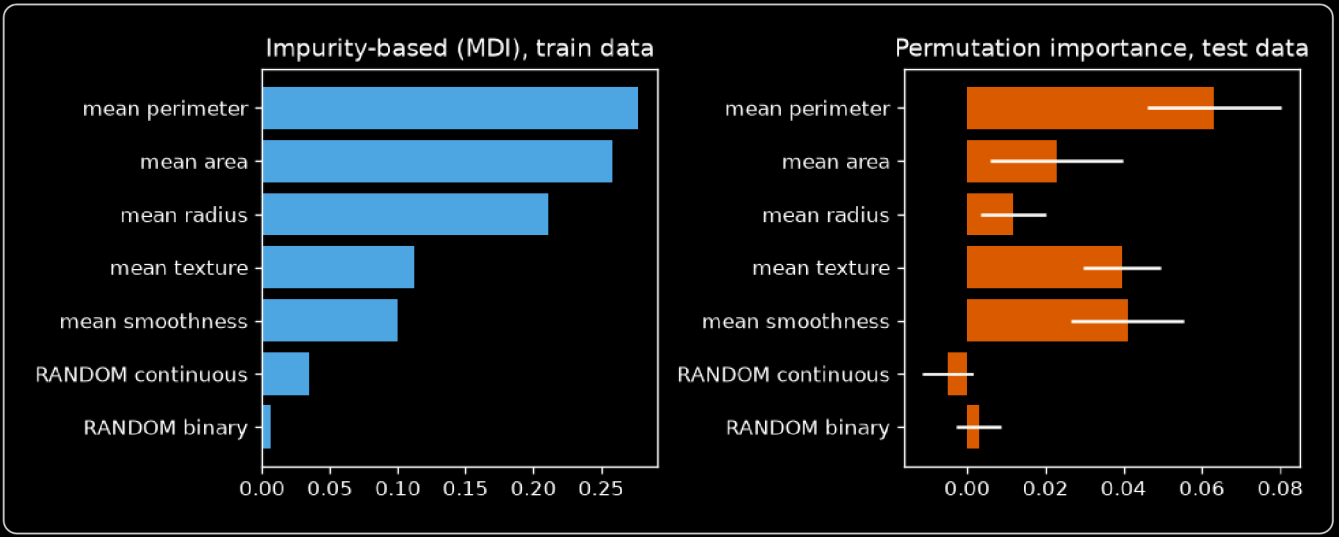
Impurity-based importance (Mean Decrease in Impurity, **MDI**; `feature_importances_`): sum each feature's impurity decreases over its splits, weighted by the share of samples reaching the node, averaged over trees, normalised to 1. Pitfalls:

1. **Biased toward high-cardinality features** (many distinct values): continuous features or IDs offer many thresholds and find chance splits; pure noise gets non-zero importance.
2. **Computed on training data** — rewards features used to *overfit*.
3. **Correlated features share credit** unpredictably (radius, perimeter, area each look less important than the concept).

Permutation importance fixes the first two: on a **held-out set**, shuffle one column (breaking its link to the target), measure the score drop, repeat. Model-agnostic. Caveat: with correlated features the unshuffled twin remains, so **both** can look unimportant — group or drop correlated features first.

```
X = np.hstack([X, noise_cont, noise_bin])   # add a random continuous and a random binary column
rf = RandomForestClassifier(n_estimators=300, random_state=0, n_jobs=-1).fit(X_tr, y_tr)
perm = permutation_importance(rf, X_te, y_te, n_repeats=20, random_state=0, n_jobs=-1)
```

```
feature              impurity(MDI)  permutation(test)
  mean perimeter             0.278     0.063 +/- 0.017
  mean area                  0.258     0.023 +/- 0.017
  mean radius                0.212     0.012 +/- 0.008
  mean texture               0.112     0.039 +/- 0.010
  mean smoothness            0.100     0.041 +/- 0.014
  RANDOM continuous          0.034    -0.005 +/- 0.006
  RANDOM binary              0.006     0.003 +/- 0.006
```



(1) MDI gives the random continuous column ~6× the random binary one — pure cardinality bias; permutation puts both ≈ 0. (2) Radius, perimeter and area are nearly one measurement: MDI ranks them top-3, but permutation ranks texture and smoothness above radius and area, since shuffling one size feature barely hurts while its twins remain.

Better still: SHAP values (per-prediction attributions from game-theory Shapley values, which split credit fairly among features; `shap` library with a fast exact tree algorithm) explain individual predictions and aggregate to global importance.

Trade-offs

Strengths	Weaknesses
Strong accuracy, almost no tuning	Larger, slower than one tree; not readable
More trees never hurt accuracy, only time	Usually slightly below well-tuned boosting
Trees train in parallel (`n_jobs=-1`)	Cannot extrapolate
Free OOB score; robust to outliers and noisy features	Memory-heavy with deep trees on big data

When not to use it

- Very high-dimensional **sparse** data (mostly zeros, e.g. word counts) — linear models are faster and often better.
- Latency- or memory-critical deployment (see System Design Fundamentals) with hundreds of deep trees.
- When you need the last percent — try gradient boosting.

Common mistakes

- Treating `feature_importances_` as causal or "true".
- Tuning `n_estimators` as if more trees overfit (they don't; stop when the score plateaus).
- Forgetting `n_jobs=-1` (one core); OOB with `bootstrap=False` (no OOB rows; scikit-learn errors).

Real-world examples

- Microsoft Kinect body-part recognition used randomized decision forests on depth-image features (Shotton et al., 2011).

- Standard strong baseline in credit risk, fraud, churn and bioinformatics (gene expression).

In the interview

> **30-second answer:** "A Random Forest trains many deep trees on bootstrap samples and, at each split, considers only a random subset of features. Averaging reduces variance; feature subsampling de-correlates the trees so averaging helps more. About 37% of rows are out-of-bag for each tree, which gives a free validation score. It works well out of the box and rarely overfits by adding trees."

- *Bagging vs Random Forest?* RF = bagging + random feature subset per split, to reduce tree correlation.
- *Why can MDI mislead?* Training-data based and favours high cardinality; use permutation importance on held-out data, or SHAP.
- *Does adding trees overfit?* No — test error converges; depth and leaf size control overfitting.

Self-check

1. Which term of $\rho\sigma^2 + (1-\rho)\sigma^2/B$ does feature subsampling target? → $\rho\sigma^2$.
2. Two near-duplicate features: permutation importances? → Both can look small (the twin remains).

TL;DR

Random Forest = many de-correlated deep trees, averaged. Great default, free OOB score. Explain with permutation importance or SHAP, not raw MDI.

7.4 Boosting

What it is

Boosting builds an ensemble **sequentially**, each new **weak learner** (typically a shallow tree or stump) focusing on what the ensemble gets wrong — like a student who re-studies only the questions missed on each practice exam.



AdaBoost (intuition)

AdaBoost (Adaptive Boosting, Freund & Schapire 1997) re-weights *samples*:

1. Every sample gets weight $1/n$.
2. Fit a stump on the weights; compute weighted error ε.
3. Its vote $\alpha = \tfrac{1}{2}\cdot\ln((1-\varepsilon)/\varepsilon)$: accurate stumps vote more; $\varepsilon = 0.5$ (coin flip) → $\alpha = 0$.
4. Multiply weights of **misclassified** samples by e^{α}, correct ones by $e^{-\alpha}$; renormalise.
5. Repeat; predict sign of $\Sigma\ \alpha_m \cdot h_m(x)$.

Worked: $\varepsilon = 0.2$ → $\alpha \approx 0.693$, so wrong samples ×2, right ×0.5; total weight 0.4 vs 0.4 → the misclassified 20% become **half** of the next stump's problem.

(scikit-learn's `AdaBoostClassifier` uses multi-class **SAMME**: $\alpha = \ln((1-\varepsilon)/\varepsilon) + \ln(K-1)$ for K classes — twice the above when $K = 2$ — scaled by `learning_rate`.) AdaBoost equals gradient boosting with **exponential loss**, making it sensitive to outliers and label noise: a mislabelled point keeps getting heavier.

Gradient boosting, step by step on residuals

Gradient boosting (Friedman 2001) re-fits *targets* instead of re-weighting samples. For squared error:

1. $F_0(x) = \text{mean}(y)$.
2. For $m = 1..M$: residuals $r_i = y_i - F_{m-1}(x_i)$; fit a small regression tree h_m to r; update $F_m(x) = F_{m-1}(x) + \eta\cdot h_m(x)$, with **learning rate** (shrinkage) η, e.g. 0.1.
3. Predict with F_M.

Why "gradient"? For $L = \tfrac{1}{2}(y-F)^2$ the negative gradient w.r.t. F is exactly the residual. For other losses fit the tree to the **negative gradient** ("pseudo-residuals"; for log-loss: $y - p$). Gradient boosting is gradient descent in *function space*: each tree is one downhill step.

Worked example (4 houses, size in 100 m² → price k$, $\eta = 0.5$, stumps; `code/boosting_demo.py`):

```
round 0: predict mean = 295.0 for every house
round 1: residuals [-95. -35.   5. 125.] -> new preds [274.2 274.2 274.2 357.5]  MSE=2568.8
round 2: residuals [-74.2 -14.2  25.8  62.5] -> new preds [252.1 252.1 296.2 379.6]  MSE=1105.7
round 3: residuals [-52.1   7.9   3.8  40.4] -> new preds [226.  260.8 304.9 388.3]  MSE=427.6
```

Round 1's stump splits size ≤ 2.5: left mean residual (−95 − 35 + 5)/3 = −41.7, right 125; half of each is added (295 − 20.8 = 274.2, 295 + 62.5 = 357.5). Residuals and MSE shrink each round.

The whole algorithm:

```python
class TinyGradientBoosting:
    def fit(self, X, y):
        self.f0 = y.mean()                      # step 0: constant prediction
        pred = np.full(len(y), self.f0)
        self.trees = []
        for _ in range(self.n):
            residual = y - pred                 # = negative gradient of 1/2 (y - pred)^2
            t = DecisionTreeRegressor(max_depth=self.depth, random_state=0).fit(X, residual)
            pred += self.lr * t.predict(X)      # take a small step toward the residuals
            self.trees.append(t)
        return self

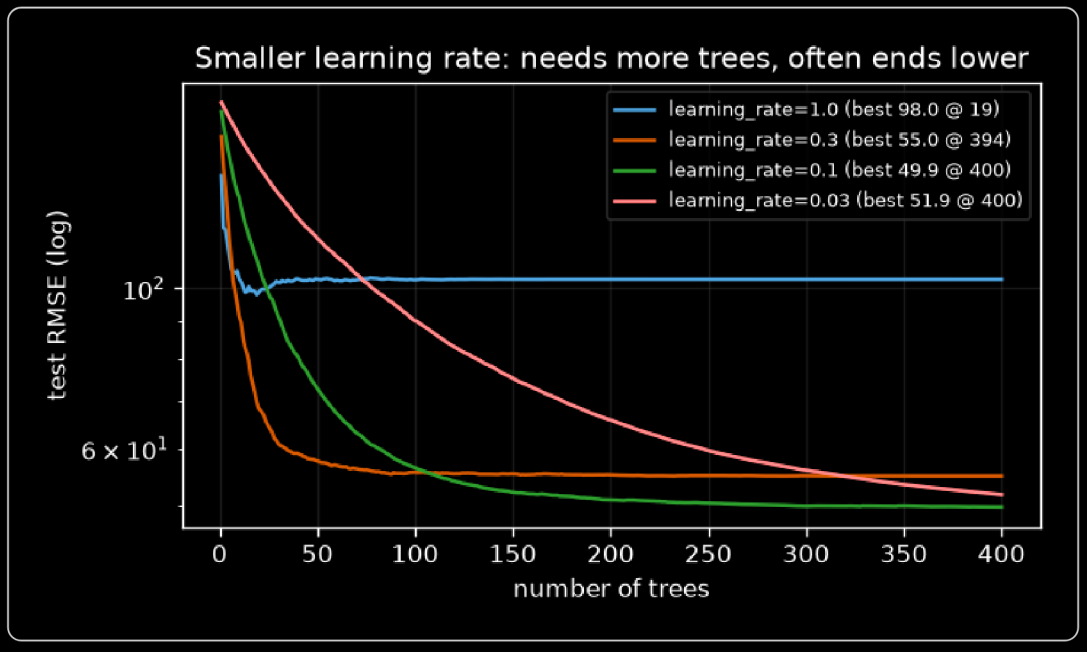
    def predict(self, X):
        return self.f0 + self.lr * sum(t.predict(X) for t in self.trees)
```

```
from-scratch GB test RMSE = 50.86
sklearn GB      test RMSE = 51.01
```

It matches scikit-learn's `GradientBoostingRegressor` with the same settings.

Learning rate and number of trees

Smaller η corrects less per tree, needing more trees, but usually generalises better (shrinkage is regularisation).



- η = 1.0: fast drop, then overfits and stalls (RMSE 98).
- η = 0.3: good quickly, plateaus at 55.
- η = 0.1: best (49.9) by 400 trees.
- η = 0.03: still improving at 400.

Recipe: η ≈ 0.05–0.1, large M, **early stopping** on validation (e.g. stop after 50 rounds without improvement). scikit-learn: `n_iter_no_change` + `validation_fraction` (`GradientBoostingClassifier`) or `early_stopping=True` (`HistGradientBoosting*`, automatic above 10,000 samples); XGBoost/LightGBM: `early_stopping_rounds` / an early-stopping callback. Other regularisers: shallow trees (depth 3–8 or `num_leaves` ≤ 31–255), **row subsampling** (`subsample` < 1 = *stochastic* gradient boosting), column subsampling, min samples per leaf, L1/L2 on leaf values.

Comparison on a real dataset

```
models = {
    "AdaBoost (stumps)": AdaBoostClassifier(n_estimators=200, random_state=0),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=0, n_jobs=-1),
    "HistGradientBoosting": HistGradientBoostingClassifier(random_state=0),
}
```

```
5-fold CV accuracy
  AdaBoost (stumps)      0.977 +/- 0.009
  RandomForest           0.960 +/- 0.019
  HistGradientBoosting   0.965 +/- 0.018
```

On a small clean dataset all are within ~2 points and the ranking changes with the seed; family differences show on larger, messier data.

XGBoost vs LightGBM vs CatBoost (and scikit-learn)

All are gradient-boosted decision trees (**GBDT**), differing in engineering and defaults:

	XGBoost (2014)	**LightGBM (Microsoft, 2017)**	**CatBoost (Yandex, 2017)**	**scikit-learn `HistGradientBoosting`**
Split finding	Histogram (`tree_method="hist"`, default since 2.0); exact available	Histogram	Histogram	Histogram (≤ 255 bins), LightGBM-inspired
Tree growth	Level-wise by default; `grow_policy="lossguide"` for leaf-wise	**Leaf-wise** — often faster/more accurate, can overfit small data	**Symmetric (oblivious)** trees: one split per level → very fast prediction, strong regularisation	Leaf-wise (`max_leaf_nodes=31`)
Regularisation	In the objective: γ per leaf, L1/L2 on leaf weights; 1st + 2nd derivatives (Newton step)	L1/L2, min gain, min data in leaf; **GOSS** (keep large-gradient rows, sample small) and **EFB** (bundle mutually exclusive sparse features) for speed	**Ordered boosting** against target leakage	L2, min samples per leaf
Categorical features	Native (`enable_categorical=True`)	Native	**Best-in-class**: ordered target statistics, feature combinations	Native (`categorical_features="from_dtype"` default)
Missing values	Native (learns default direction)	Native	Native	Native
GPU	Yes	Yes	Yes	No
Best when	All-rounder, huge ecosystem	Large data, speed	Many categoricals, good defaults	Mid-size data, no extra dependency

> **Key terms:** *Histogram* split finding buckets each feature into a few hundred bins so only bin edges are tested — much faster. *Level-wise* growth splits every node at a depth before going deeper; *leaf-wise* always splits the single leaf with the biggest loss reduction. A *Newton step* uses second derivatives (curvature) as well as first to size updates. *Target leakage* here = a row's own label influencing its features. A *GPU* runs many calculations in parallel; a *dependency* is an extra library to install.

XGBoost's objective: minimise Σ loss(y_i, $\hat{y}_i$) + Σ_trees (γ·T + ½λ·‖w‖²) (T = leaves, w = leaf values); with gradient sum G and second-derivative sum H in a leaf, the optimal leaf value is $w^* = -G/(H + \lambda)$.

All have scikit-learn-compatible APIs (not run — packages not installed here):

```python
# pip install xgboost lightgbm catboost   (not run)
from xgboost import XGBClassifier
from lightgbm import LGBMClassifier
from catboost import CatBoostClassifier

xgb = XGBClassifier(n_estimators=2000, learning_rate=0.05, max_depth=6,
                    subsample=0.8, colsample_bytree=0.8, early_stopping_rounds=100)
xgb.fit(X_tr, y_tr, eval_set=[(X_val, y_val)])

lgbm = LGBMClassifier(n_estimators=2000, learning_rate=0.05, num_leaves=31)
cat = CatBoostClassifier(iterations=2000, learning_rate=0.05, cat_features=["city", "device"])
```

Trade-offs: bagging vs boosting

	Random Forest	Gradient boosting
Accuracy on tabular data	Very good	Usually best
Tuning needed	Little	Moderate (η, trees, depth/leaves, subsampling)
Overfits by adding trees?	No	**Yes** — use early stopping
Training	Parallel across trees	Sequential across trees (parallel within)
Sensitivity to noise/outliers	Low	Higher (esp. AdaBoost)

When not to use it

- Unstructured data (images, audio, raw text) — deep learning wins.
- Tiny datasets with strong domain priors — a regularised linear model may match it, more explainably.
- Predictions outside the training range — combine with a linear model or model the trend separately.

Common mistakes

- `n_estimators` without early stopping; learning rate 1.0 "because it's faster".
- Same set for early stopping *and* final reporting (keep a separate test set).
- AdaBoost on noisy labels; deep trees (depth 15) in boosting — it wants weak learners.

Real-world examples

- GBDT dominates Kaggle tabular competitions; XGBoost's 2016 paper noted it in 17 of 29 winning Kaggle solutions published in 2015.
- Web search: **LambdaMART** (gradient-boosted trees for **learning-to-rank**, Microsoft Research) powered Bing's ranker, and a LambdaMART ensemble won the 2010 Yahoo! Learning to Rank Challenge.
- Fraud, credit scoring, click-through rate, demand forecasting; Yandex uses CatBoost in search and recommendation.

In the interview

30-second answer: "Boosting builds trees sequentially, each one correcting the current ensemble. In gradient boosting, you start from a constant, and each new shallow tree is fitted to the negative gradient of the loss — for squared error that's just the residuals — and added with a small learning rate. It mainly reduces bias. It's usually the most accurate model on tabular data but it can overfit as you add trees, so you use a small learning rate with early stopping."

- *AdaBoost vs gradient boosting?* AdaBoost re-weights misclassified samples (exponential loss); gradient boosting fits the negative gradient of any differentiable loss — AdaBoost is a special case.
- *Why a small learning rate?* Each tree fixes only part of the error, which regularises and generalises better, at the cost of more trees.
- *LightGBM vs XGBoost?* LightGBM: leaf-wise, GOSS/EFB for speed; XGBoost: depth-wise by default, regularised second-order objective. Both histogram-based; often similar after tuning.

Self-check

1. Negative gradient for log-loss? → y − p.
2. Learning rate 0.1 → 0.01: what else changes? → ~10× more trees (early stopping).
3. Which library uses symmetric (oblivious) trees? → CatBoost.

TL;DR

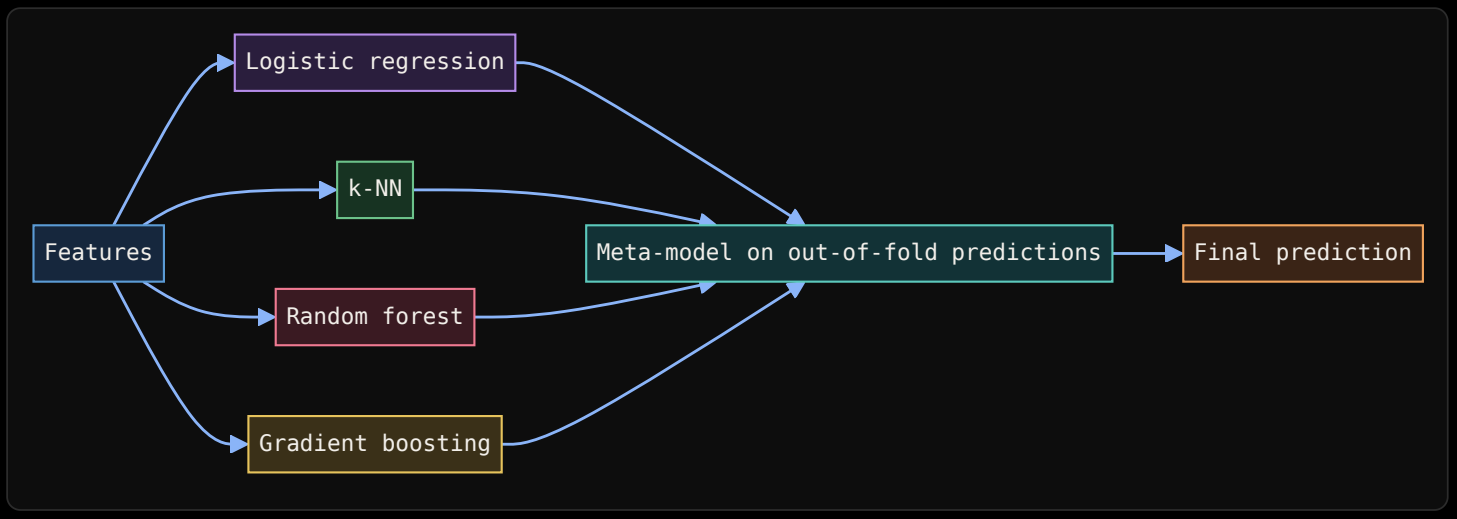
Boosting = shallow trees in sequence, each fitting the previous errors (residuals/gradients), scaled by a learning rate. Most accurate on tabular data; always early-stop.

7.5 Voting and stacking

What it is

Ensembles of **different** model families:

- **Hard voting:** majority of predicted classes.
- **Soft voting:** average predicted probabilities; usually better with **calibrated** models (probabilities match observed frequencies).
- **Stacking** (stacked generalisation, Wolpert 1992): a **meta-model** (often logistic/linear regression) trained on the base models' **out-of-fold** predictions (each from a model that didn't see that row), avoiding leakage; scikit-learn does this via `cv`.



Why it matters

Different families make **different errors** (low ρ → averaging helps, as in 7.2); stacking learns *how much to trust* each.

Code

`code/voting_stacking.py`:

```
base = [
    ("lr", make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))),
    ("knn", make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7))),
    ("rf", RandomForestClassifier(n_estimators=200, random_state=0, n_jobs=-1)),
    ("hgb", HistGradientBoostingClassifier(random_state=0)),
]
models["soft voting"] = VotingClassifier(base, voting="soft")
models["stacking (meta = LR)"] = StackingClassifier(
    base, final_estimator=LogisticRegression(max_iter=1000), cv=5)
```

```
10-fold CV accuracy
  lr                    0.981 +/- 0.015
  knn                   0.967 +/- 0.020
  rf                    0.965 +/- 0.028
  hgb                   0.968 +/- 0.022
  hard voting           0.977 +/- 0.021
  soft voting           0.975 +/- 0.016
  stacking (meta = LR)  0.979 +/- 0.019
```

Honest result: ensembles beat three of four base models but **not** the best (scaled logistic regression — the data is nearly **linearly separable**). Ensembles reduce the risk of picking wrong; they aren't guaranteed to win.

Trade-offs / when not to use it

- More models = more training time, latency and maintenance; Netflix didn't deploy the full Prize-winning blend of 100+ models because the gain didn't justify the engineering cost.
- Little gain when base models are highly correlated (five boosting libraries on the same features).

Common mistakes

- Meta-model on in-sample base predictions (it learns to trust overfit models).
- Soft voting with uncalibrated scores (e.g. SVM) without calibration.

In the interview

> **30-second answer:** "Voting averages diverse models' predictions — hard voting by majority, soft voting by averaged probability. Stacking trains a meta-model on out-of-fold predictions of the base models so it learns how to weight them. It helps when base models make different errors."

- *Why out-of-fold?* An overfit model's in-sample predictions look perfect, so the meta-model would over-trust it.

TL;DR

Combine diverse models: soft voting for simplicity, stacking on out-of-fold predictions for the last bit of accuracy.

7.6 When trees beat neural networks

On **tabular data**, tree ensembles (especially GBDT) remain the default winner over deep learning at typical sizes. Grinsztajn, Oyallon and Varoquaux (NeurIPS 2022, *"Why do tree-based models still outperform deep learning on typical tabular data?"*) benchmarked 45 medium-sized datasets (~10K samples):

Property of tabular data	Why trees handle it	Why neural nets struggle
Irregular targets (sharp jumps, thresholds)	Splits make sharp steps	Networks are biased toward smooth functions
Many **uninformative features**	Splits ignore them	**MLPs** (basic fully connected networks) are hurt by them
Columns have individual meaning (not rotation-invariant)	Axis-aligned splits respect each column	MLPs are rotation-invariant (they blend columns), wasting that structure
Mixed types, skew, missing values	No scaling; native categoricals/missing	Need careful normalisation and encoding

GBDTs also train in minutes on a CPU, need less tuning and are easier to explain (SHAP). Neural nets win on **unstructured** data, huge datasets, or end-to-end learning with **embeddings** (learned vectors for IDs, words, images) — e.g. recommenders with millions of IDs, or tabular + text + images. Tabular foundation models (pre-trained on many datasets; e.g. TabPFN for small data) are narrowing the gap, but a tuned GBDT is the baseline to beat.

Rule of thumb: tabular → start with a GBDT (LightGBM/XGBoost/CatBoost/`HistGradientBoosting`) and a Random Forest baseline.

7.7 Hyperparameter cheat sheet

Goal	Decision tree (sklearn)	Random Forest (sklearn)	GBDT (sklearn `HistGradientBoosting`)	XGBoost	LightGBM
Number of trees	–	`n_estimators` (100–500; more never hurts accuracy)	`max_iter` (use early stopping)	`n_estimators`	`n_estimators` / `num_iterations`
Step size	–	–	`learning_rate` (0.1)	`learning_rate` / `eta` (0.3)	`learning_rate` (0.1)
Tree size	`max_depth`, `max_leaf_nodes`	`max_depth` (None)	`max_leaf_nodes` (31), `max_depth`	`max_depth` (6)	`num_leaves` (31), `max_depth` (−1 = no limit)
Min data per leaf	`min_samples_leaf`	`min_samples_leaf`	`min_samples_leaf` (20)	`min_child_weight` (1)	`min_child_samples` (20)
Row sampling	–	`bootstrap`, `max_samples`	–	`subsample` (1.0)	`bagging_fraction` / `subsample` (1.0)
Feature sampling	`max_features`	`max_features` ("sqrt")	`max_features`	`colsample_bytree`	`feature_fraction` / `colsample_bytree`
Penalties	`ccp_alpha`, `min_impurity_decrease`	`ccp_alpha`	`l2_regularization`	`reg_alpha`, `reg_lambda`, `gamma`	`reg_alpha`, `reg_lambda`, `min_split_gain`
Class imbalance	`class_weight`	`class_weight`	`class_weight`	`scale_pos_weight`	`is_unbalance` / `scale_pos_weight`

(Defaults in parentheses: scikit-learn 1.9, XGBoost 2.x, LightGBM 4.x.)

GBDT tuning order: (1) learning rate 0.05–0.1 + early stopping; (2) tree size and min samples per leaf; (3) subsampling; (4) L1/L2 if still overfitting; (5) lower the learning rate and let early stopping add trees.

Chapter TL;DR

- **Decision tree:** greedy splits maximising impurity decrease (Gini ≈ entropy); leaves predict majority/mean. Readable, scale-free, high variance — limit depth or prune with `ccp_alpha`. Cannot extrapolate.
- **Bagging/Random Forest:** average deep trees on bootstrap samples; RF also samples features per split to de-correlate. ~37% OOB rows give a free validation score. Robust default.
- **Feature importance:** MDI is biased (training data, high cardinality, correlated features) — prefer permutation importance on held-out data or SHAP.
- **Boosting:** shallow trees in sequence fitting residuals/negative gradients, scaled by a learning rate. Best tabular accuracy; early-stop. AdaBoost = re-weighting, exponential loss.
- **XGBoost/LightGBM/CatBoost:** histogram GBDTs differing in tree growth (depth-wise / leaf-wise / symmetric) and categorical handling (CatBoost best).
- **Voting/stacking:** combine diverse models; stacking needs out-of-fold predictions.
- **Tabular data → start with gradient-boosted trees.**

Chapter 8 — k-Nearest Neighbours, Support Vector Machines, Naive Bayes

Three classic algorithms, three different ideas of "learning":

- **k-NN** builds no model — it remembers the training data and asks "what did similar examples look like?"
- **SVM** draws the boundary staying as far as possible from both classes, and with the **kernel trick** draws curved boundaries without computing curved features.
- **Naive Bayes** counts: Bayes' theorem (updating a probability after evidence) plus a bold simplifying assumption — surprisingly good at text.

The chapter ends with a table comparing **every classic supervised algorithm** in the book.

What you'll learn: k-NN distance metrics, scaling, choosing k, the curse of dimensionality; SVM margin, support vectors, soft margin and C, kernels and the kernel trick, SVR; Naive Bayes variants and a spam filter by hand; when to use which classic algorithm.

Code is in `code/`; every output is real (scikit-learn 1.9).

8.1 k-Nearest Neighbours (k-NN)

What it is

k-NN finds the k training points closest to a new point and takes a **majority vote** of their labels (classification) or the **average** of their targets (regression) — like pricing a flat from the 5 most similar recent sales.

It is a **lazy** (instance-based) learner — "training" just stores the data, all work happens at prediction — and **non-parametric**: no fixed weights; the model *is* the dataset.

Why it matters

- Zero training time, one **hyperparameter**, no assumption about the **decision boundary** (the line between predicted classes), which can be arbitrarily curved.
- The idea behind **similarity search** ("customers like you", "similar products") and **vector search** in modern retrieval, e.g. finding documents closest to a query **embedding** (a list of numbers representing meaning) in **RAG** systems (retrieval-augmented generation: a chatbot looks up relevant documents, then answers from them).
- Cover & Hart (1967): with infinite data, 1-NN's error is at most twice the best possible (Bayes) error. Simple ≠ weak.

How it works

1. Choose k and a distance metric.
2. Store the training set (optionally in a **KD-tree** or **ball tree**, indexes that partition space so most points can be skipped).
3. For a query, compute distances to all training points; keep the k smallest.
4. Classification: majority vote (odd k avoids ties in binary problems). Regression: mean of the k targets.
5. Optional `weights="distance"`: weight = 1/distance.

Distance metrics

For a = (1, 2, 3), b = (4, 6, 3) — `code/knn_demo.py`:

Metric	Formula	Value	Use it for
Euclidean (L2)	$\sqrt{\Sigma(a_i - b_i)^2}$	**5.000**	Default, continuous features
Manhattan (L1)	$\Sigma\lvert a_i - b_i\rvert$	**7.000**	Grid-like movement; more robust to one large difference
Chebyshev (L∞)	$\max_i \lvert a_i - b_i\rvert$	**4.000**	"Worst single feature" matters
Minkowski (Lp)	$(\Sigma\lvert a_i - b_i\rvert^p)^{(1/p)}$	p=1 → L1, p=2 → L2	scikit-learn default: `metric="minkowski", p=2`
Cosine distance	$1 - (a\cdot b)/(\lVert a\rVert\lVert b\rVert)$	1 − 0.855 = **0.145**	Text/embeddings, direction over length
Hamming	Fraction of differing positions	—	Binary or categorical vectors

Scaling is mandatory

Distances add raw differences, so large-unit features **drown out** the others. In Wine, proline has standard deviation 314 vs 0.12 for another feature:

```
wine feature ranges: min std 0.12, max std 314.0 (proline)
k-NN (k=5) CV accuracy  raw features: 0.691   standardized: 0.949
```

Standardising lifts accuracy from 69% to 95%. Put the scaler **inside a pipeline** so it is fitted on training folds only:

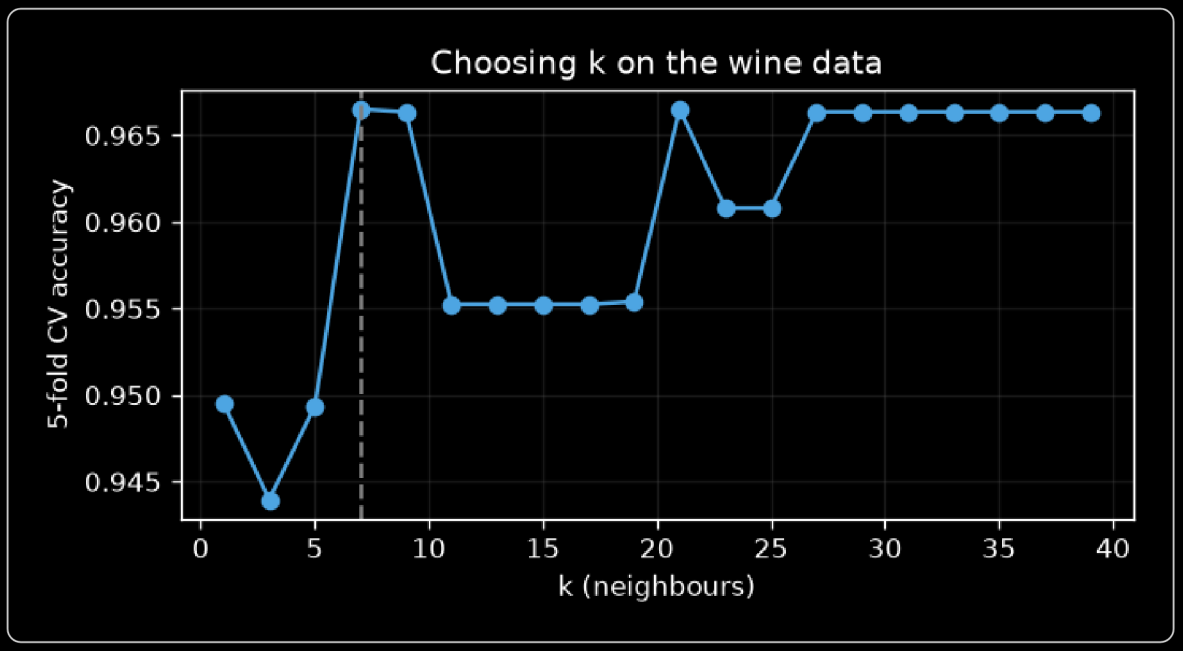
```
make_pipeline(StandardScaler(), KNeighborsClassifier(5))
```

Choosing k

- **Small k** (k = 1): jagged boundary, noise-sensitive → low bias, high variance.
- **Large k**: smooth, can blur real structure → high bias, low variance; k = n always predicts the majority.
- Choose by **cross-validation**; odd k for binary; k ≈ √n is a starting guess only.

```
ks = list(range(1, 41, 2))
scores = [cross_val_score(make_pipeline(StandardScaler(), KNeighborsClassifier(k)), X, y, cv=5).mean()
          for k in ks]
```

```
best k by 5-fold CV = 7 (accuracy 0.967);  k=1: 0.950, k=39: 0.966
```



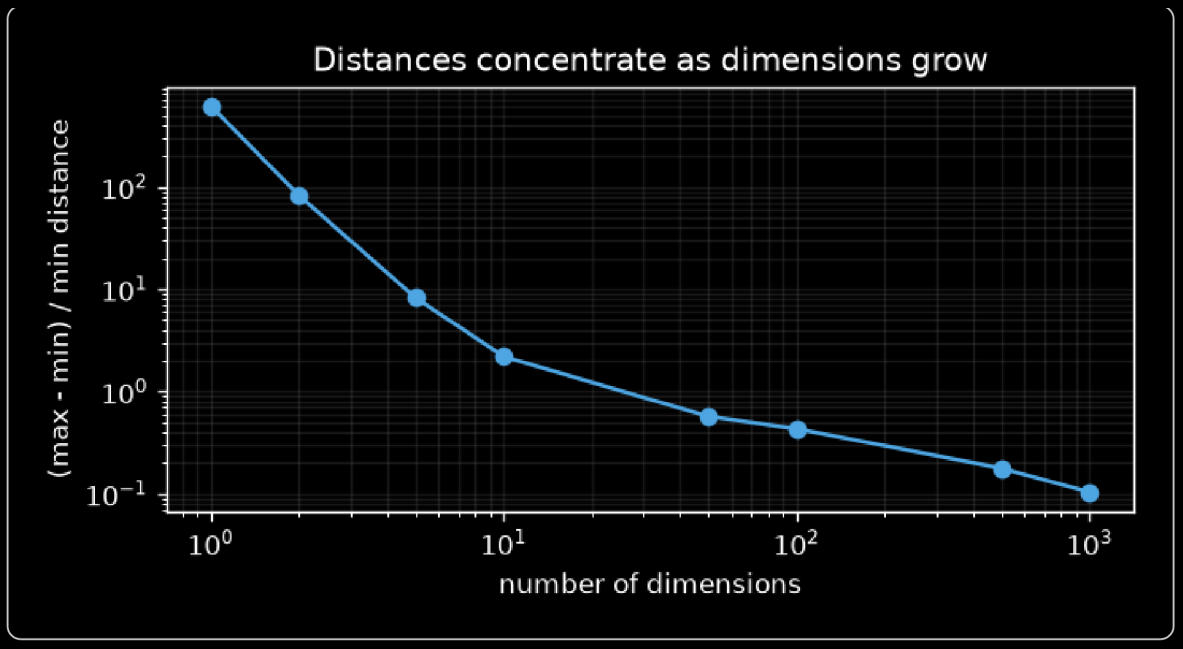
The curse of dimensionality

As the number of features d grows, distance-based methods break:

1. **Space empties out.** Capturing 10% of uniform data in a unit hypercube needs a sub-cube of edge 0.1^(1/d): 0.1 in 1-D, **0.79** in 10-D — the "neighbourhood" spans nearly every feature's whole range.
2. **Distances concentrate.** Nearest and farthest points become almost equally far (500 random points, `curse_of_dimensionality()`):

```
dim  (farthest - nearest) / nearest distance, 500 uniform points
   1   615.806
   2    82.019
  10     2.190
 100     0.432
1000     0.104
```

In 1000-D the farthest point is only ~10% farther than the nearest.



3. **Data needed grows exponentially** to keep the same density.

Remedies: feature selection, dimensionality reduction (PCA, Chapter 9), learned embeddings where data lies on a lower-dimensional surface, or a non-distance model.

Trade-offs

Strengths	Weaknesses
No training; add data instantly	Slow, memory-hungry prediction: brute force is O(n·d) per query
Any boundary shape; naturally multi-class	Needs scaling and a good metric
One intuitive hyperparameter	Bad in high dimensions and with irrelevant features
Classification and regression	Sensitive to class imbalance (majority floods the vote)

KD-trees and ball trees speed exact search in low dimensions; for millions of high-dimensional vectors production uses **approximate nearest neighbour (ANN)** indexes such as **HNSW** graphs (layered neighbour graphs that search hops through) or **FAISS** (Meta's open-source similarity-search library).

When not to use it

Large datasets needing fast predictions, many features (> ~20–50 raw without reduction), many irrelevant features, or when you need a compact model or explanations beyond "these examples were similar".

Common mistakes

- Not scaling, or scaling before the train/test split (leakage).
- Even k in binary problems; ID-like or irrelevant columns adding noise to every distance.
- Euclidean on one-hot or text counts where Hamming or cosine fit better.

Real-world examples

- "Similar items" / "users like you" (item-based **collaborative filtering** — recommending from similar users' or items' behaviour — is nearest-neighbour).
- **Vector databases** and semantic search (approximate k-NN over embeddings) — the retrieval step in RAG.
- `KNNImputer` fills missing values from the nearest rows.

In the interview

> **30-second answer:** "k-NN stores the training set and predicts by majority vote or average of the k closest points under a distance metric. It has no training cost but prediction is O(n·d) without an index. You must scale features, pick k by cross-validation — small k overfits, large k underfits — and it degrades in high dimensions because distances concentrate."

- *Why "lazy"?* All computation is deferred to query time.
- *Speed it up?* KD-tree/ball tree in low dimensions; ANN (HNSW, FAISS) at scale; reduce dimensions first.

- *k-NN regression?* (Distance-weighted) average of the neighbours' targets.

Self-check

1. Why scale for k-NN but not trees? → k-NN adds raw feature differences; trees compare values within one feature.
2. k = 1 on noisy data? → Memorises noise (100% training accuracy, high variance).

TL;DR

k-NN predicts from the k most similar examples. Scale, tune k by CV; avoid in high dimensions or when predictions must be fast.

8.2 Support Vector Machines (SVM)

What it is

A **Support Vector Machine** finds the separating **hyperplane** (line in 2-D, plane in 3-D, …) with the **largest margin** — the widest "street" between classes. Points on the street's edge are **support vectors**; they alone define the boundary. Like building the widest possible road between two villages, so a slightly shifted house doesn't land on the wrong side.

Why it matters

- A large margin generalises well, especially with many features and few samples.
- **Kernels** let the same algorithm learn curved boundaries.
- Before deep learning (~1995–2012) SVMs led text classification, image recognition and bioinformatics; still a strong small-to-medium baseline.

How it works — the margin

Predict with the sign of $f(x) = w \cdot x + b$; the boundary is $f(x) = 0$.

1. Scale w, b so the closest points of each class have $f(x) = \pm 1$ (the support vectors).
2. The gap between $f = +1$ and $f = -1$ is $2/\|w\|$.
3. **Hard-margin SVM:** minimise $\frac{1}{2}\|w\|^2$ subject to $y_i(w \cdot x_i + b) \geq 1$, $y_i \in \{-1, +1\}$.
4. A **convex** quadratic optimisation (squared objective, linear constraints): one global optimum.

Soft margin and C

Real data overlap, and one outlier can force a narrow margin. **Soft-margin** SVM adds a slack $\xi_i \geq 0$ per point (how far it breaks the margin):

minimise $\frac{1}{2}\|w\|^2 + C \cdot \Sigma\, \xi_i$ subject to $y_i(w \cdot x_i + b) \geq 1 - \xi_i$

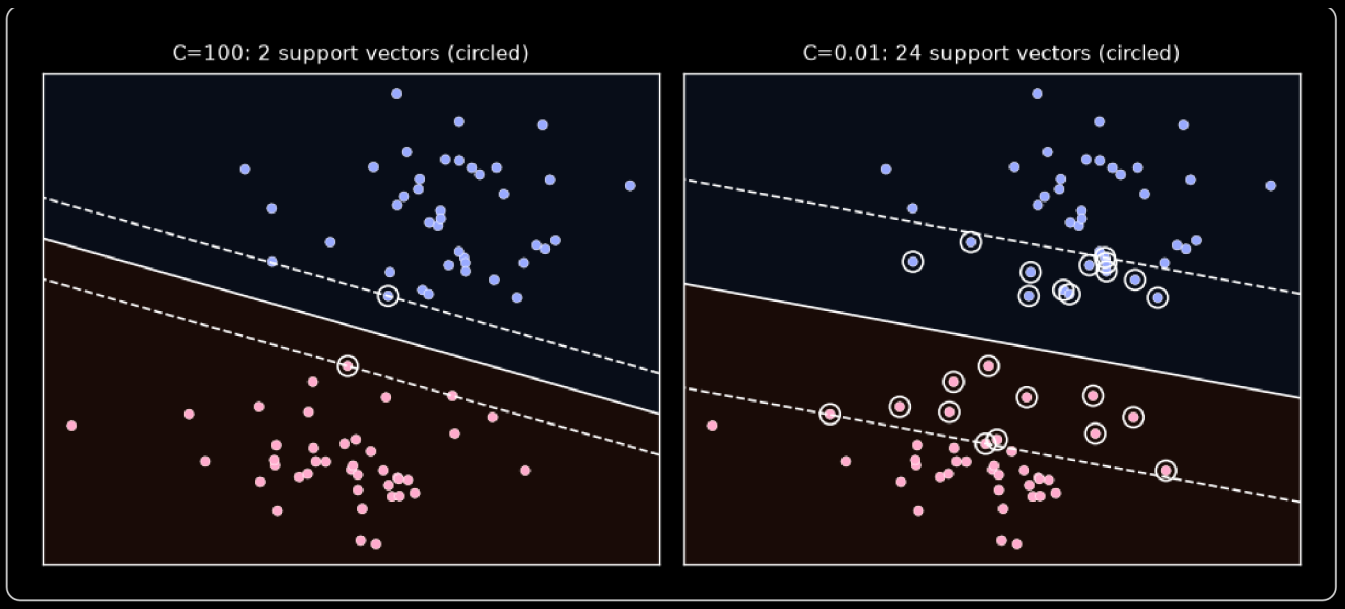
Equivalently: **hinge loss** $\max(0, 1 - y_i \cdot f(x_i))$ plus an L2 penalty on w. Correct points beyond the margin cost nothing; points inside it or misclassified cost linearly.

- **Large C** → violations expensive → narrow margin, tight fit (low bias, higher variance).
- **Small C** → wide margin, more support vectors, smoother (higher bias, lower variance).

C is the inverse of a regularisation strength (as in logistic regression). `code/svm_demo.py`:

```
for C in (100, 0.01):
    m = SVC(kernel="linear", C=C).fit(X, y)
    w = m.coef_[0]
    print(len(m.support_), 2 / np.linalg.norm(w), m.score(X, y))
```

```
linear SVM on 2 blobs
  C=100    support vectors=  2  margin width 2/||w|| = 1.85  train acc=1.000
  C=0.01   support vectors= 24  margin width 2/||w|| = 4.94  train acc=1.000
```



C = 100: 2 points define the boundary; C = 0.01: a 2.7× wider street resting on 24 points.

Kernels and the kernel trick

Some data (one class inside a ring of the other) can't be split by a line. Map points to a higher-dimensional space $\varphi(x)$ where a hyperplane *does* separate them — the ring becomes separable with feature $x_1^2 + x_2^2$.

Why only dot products appear. Solving with Lagrange multipliers (folding constraints into the objective, one weight per constraint — the **dual form**) gives a weight α_i per training point, $0 \le \alpha_i \le C$, and $w = \Sigma\, \alpha_i\, y_i\, x_i$, so

$f(x) = \Sigma_i\, \alpha_i\, y_i\, (x_i \cdot x) + b$

Most α_i are exactly 0; points with $\alpha_i > 0$ are the support vectors. Data enter only through dot products.

The kernel trick. Replace each $x_i \cdot x_j$ by a **kernel** $K(x, z) = \varphi(x) \cdot \varphi(z)$ computed **without building φ**: $f(x) = \Sigma_i\, \alpha_i\, y_i\, K(x_i, x) + b$ — the power of a huge (even infinite) feature space at the cost of computing K.

Check (`kernel_trick_check()`): for 2-D points, $K(x, z) = (x \cdot z)^2$ equals the dot product in $\varphi(v) = (v_1^2, \sqrt{2} \cdot v_1 v_2, v_2^2)$:

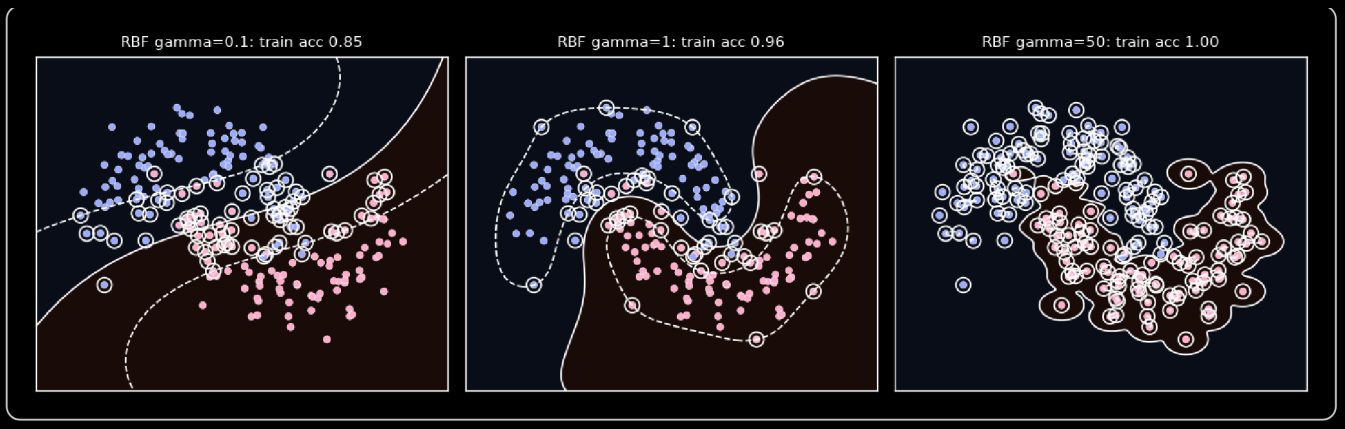
```
(x.z)^2 = 121.0   phi(x).phi(z) = 121.0
```

x = (1, 2), z = (3, 4): $x \cdot z = 11$, $11^2 = 121$ — without building the 3-D vectors.

Kernel	Formula	Hyperparameters	Use
Linear	$x \cdot z$	C	Many features (text), linearly separable data
Polynomial	$(\gamma \cdot x \cdot z + r)^d$	C, degree d, γ, r (`coef0`)	Interactions of known order
RBF (Gaussian)	$\exp(-\gamma \lVert x - z \rVert^2)$	C, γ	**Default** non-linear
Sigmoid	$\tanh(\gamma \cdot x \cdot z + r)$ (S-shaped)	C, γ, r	Rarely used

RBF intuition: K = 1 at x = z, decaying with distance; predictions are weighted "bumps" on the support vectors. **γ** sets bump width: small γ = wide bumps, smooth boundary; large γ = narrow bumps, wiggly boundary that can memorise.

```
5-fold CV accuracy      moons   circles
  linear              0.845   0.555
  poly (degree 3)     0.940   0.995
  RBF                 0.940   1.000
```



γ = 0.1 underfits (nearly linear), γ = 1 follows the moons, γ = 50 draws islands around points (100% train accuracy — overfit). Tune **C and γ together** on a log grid:

```
grid = GridSearchCV(make_pipeline(StandardScaler(), SVC()),
                    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]}, cv=5)
```

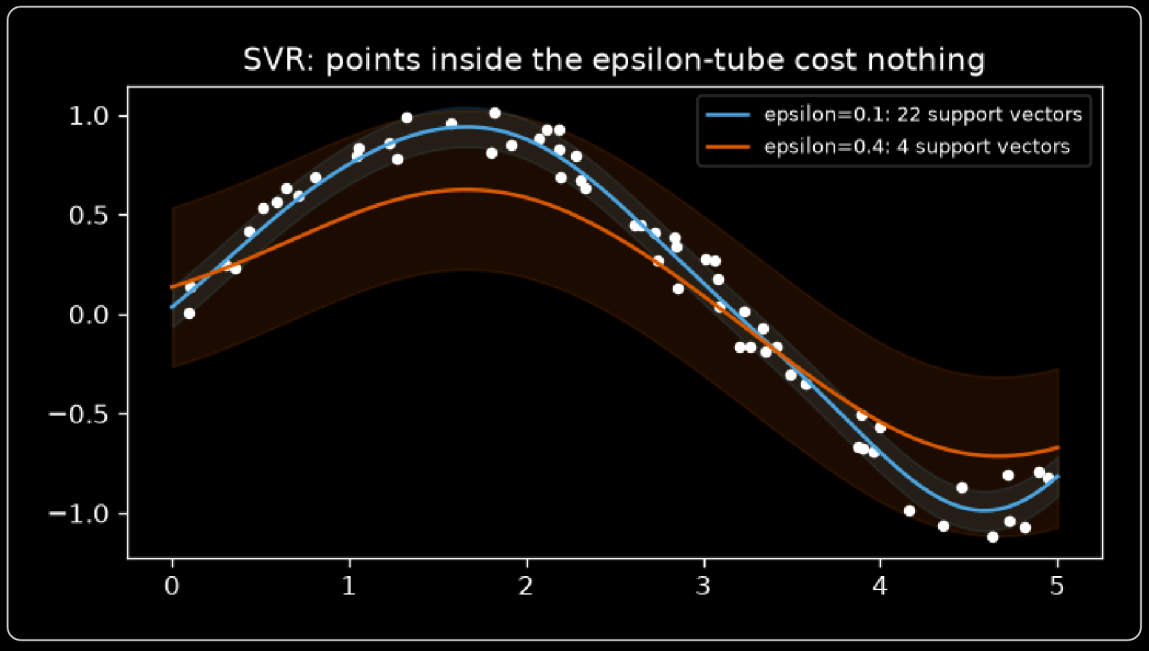
```
grid search best params {'svc__C': 0.1, 'svc__gamma': 10}  CV acc 0.960
```

(A large γ is balanced by a small C — the knobs interact.)

SVR — Support Vector Regression

SVR fits a function with a tube of half-width **ε**: points **inside cost nothing** (**ε-insensitive loss**); only points outside are penalised (weighted by C) and become support vectors.

```
SVR epsilon=0.1: 22 of 60 points are support vectors
SVR epsilon=0.4: 4 of 60 points are support vectors
```



A wider tube ignores more noise with fewer support vectors, but ε = 0.4 is too loose and flattens the curve.

Practical notes

- **Always scale** — RBF and polynomial kernels use distances/dot products.
- `SVC` training scales at least quadratically in samples (2× rows → ~4× slower or worse; impractical beyond tens of thousands). Large linear problems: `LinearSVC` or `SGDClassifier(loss="hinge")`; large non-linear: kernel approximation (`Nystroem`, `RBFSampler`) + a linear model.
- SVMs output **scores** (signed distances), not probabilities. Wrap in `CalibratedClassifierCV(SVC(), ensemble=False)` to fit **Platt scaling** (a small logistic regression on scores; `method="sigmoid"`, default) on cross-validated scores. `SVC(probability=True)` did this internally but is deprecated since scikit-learn 1.9 (removal planned for 1.11). From `svm_probabilities()`:

```
svc = make_pipeline(StandardScaler(), SVC())
cal = CalibratedClassifierCV(svc, method="sigmoid", ensemble=False).fit(X_tr, y_tr)
cal.predict_proba(X_te[:3])[:, 1]
```

Output:

```
scores  [-1.33  1.12  1.48]
P(y=1)  [0.04 0.92 0.96]
calibrated SVC test accuracy 0.900
```

Calibration: §6.5.

- Multi-class: `SVC` uses one-vs-one internally; `LinearSVC` one-vs-rest. Imbalance: `class_weight="balanced"`.

Trade-offs

Strengths	Weaknesses
Maximum margin → good generalisation, strong with many features / few samples	Slow on large n (≥ ~100k)
Convex: unique optimum, no random restarts	Needs scaling and joint C/γ tuning
Kernels give flexible boundaries	No native probabilities; hard to interpret with kernels
Only support vectors matter → compact	Struggles with heavy noise and overlap

When not to use it

Very large datasets; need for calibrated probabilities or explanations; raw images/audio/text sequences (deep learning); messy mixed-type tabular data (gradient boosting).

Common mistakes

- Not scaling; tuning C and γ separately.
- RBF on high-dimensional sparse text, where linear is faster and usually as good.
- Treating `decision_function` values as probabilities.

Real-world examples

- Linear SVMs led document and spam classification (Joachims, 1998).
- Bioinformatics with specialised kernels (string kernels compare letter sequences directly).
- 2000s face detection and handwriting recognition, often on **HOG** (histogram of oriented gradients, hand-crafted edge-direction features) or pixel features.

In the interview

30-second answer: "An SVM finds the hyperplane with the largest margin between classes; only the support vectors — points on or inside the margin — determine it. The soft-margin version uses C to trade margin width against violations: large C fits tightly, small C regularises. Kernels like RBF implicitly map data into a higher-dimensional space through the kernel trick, since the algorithm only needs dot products. Scale features and tune C and γ together."

- *Kernel trick?* Replace φ(x)·φ(z) by K(x, z) computed in the original space.
- *γ in RBF?* Reach of each support vector: high → local, wiggly (overfit); low → smooth (underfit).
- *SVM vs logistic regression?* Both linear with L2 penalties; SVM's hinge loss ignores well-classified points (max margin), LR's log-loss uses every point and gives probabilities.

Self-check

1. Why only support vectors matter? → Others have zero hinge loss ($\alpha_i = 0$).
2. A point inside the SVR ε-tube? → Zero loss; not a support vector.

TL;DR

SVM = widest-street classifier; C trades width for errors; kernels (RBF default) bend the street via the kernel trick. Scale, tune C and γ jointly, avoid on huge data.

8.3 Naive Bayes

What it is

Naive Bayes is a family of probabilistic classifiers applying **Bayes' theorem** with the "naive" assumption that **features are independent given the class**.

Bayes' theorem

P(class | data) = P(data | class) · P(class) / P(data)

Term	Name	Meaning
P(class)	Prior	How common the class is beforehand
P(data | class)	Likelihood	How typical the data is for the class
P(data)	Evidence	Same for every class — normalises
P(class | data)	Posterior	What we want

The worked medical-test example (1% prevalence, 90% sensitivity, 5% false positives → P(sick | positive) = **0.154**, the base-rate effect) is in Chapter 2, section 2.7; `bayes_theorem()` in the code reproduces it:

```
P(disease | positive) = 0.0090 / 0.0585 = 0.154
```

The naive assumption

The exact likelihood $P(x_1, \ldots, x_d \mid \text{class})$ needs a probability for every combination of values — impossible to estimate. Assuming independence given the class:

$P(x_1, \ldots, x_d \mid c) \approx P(x_1 \mid c) \cdot P(x_2 \mid c) \cdot \ldots \cdot P(x_d \mid c)$

$\hat{y} = \text{argmax}_c\ P(c) \cdot \Pi\ P(x_i \mid c)$ (argmax_c = the class making this largest; Π = product)

In practice **sum logs** ($\log P(c) + \Sigma \log P(x_i \mid c)$): multiplying hundreds of small probabilities **underflows** to 0 in floating point.

Gaussian NB, worked. `GaussianNB` models a continuous feature with the class's normal density $N(x; \mu_c, \sigma_c^2) = \exp(-(x-\mu_c)^2/(2\sigma_c^2)) / \sqrt{(2\pi\sigma_c^2)}$. Petal length $\mu = 1.5$, $\sigma = 0.2$ (class A) vs $\mu = 4.5$, $\sigma = 0.5$ (class B), equal priors, flower length 2.0:

- A: $z = 2.5 \rightarrow e^{(-3.125)/(0.2\cdot\sqrt{(2\pi)})} \approx 0.0439/0.501 \approx 0.088$
- B: $z = -5 \rightarrow e^{(-12.5)/(0.5\cdot\sqrt{(2\pi)})} \approx 3.7\times10^{-6}/1.253 \approx 3.0\times10^{-6}$

A wins overwhelmingly; with several features, add one log-density per feature.

The assumption is almost always false ("free" and "cash" co-occur in spam), yet NB classifies well because only the *ranking* of classes must be right. Side effect: **over-confident** probabilities — calibrate if you need real ones.

The three main variants

Variant	Feature type	$P(x_i \mid c)$ modelled as	Typical use
`GaussianNB`	Continuous	Normal with per-class mean, variance	Sensor readings, quick numeric baseline
`MultinomialNB`	Counts (word counts; TF-IDF also works in practice)	Word frequency within the class	Text classification, topic labelling
`BernoulliNB`	Binary (present/absent)	Probability the feature is present	Short texts, binary flags; also penalises *absent* words
(`ComplementNB`)	Counts	Statistics from all *other* classes	Imbalanced text — often beats Multinomial
(`CategoricalNB`)	Categories	Per-category frequency	Categorical tabular data

Worked example — a spam filter by hand

Training data:

Spam	Ham
win cash now	meeting at noon
win free prize now	lunch meeting tomorrow
free cash offer	project update at noon

("Ham" = legitimate email.) Vocabulary 13 words; 10 words per class. New message: **"free cash for the meeting"**.

1. **Priors:** P(spam) = P(ham) = 0.5.
2. **Laplace (add-one) smoothing:** P(word | c) = (count in c + 1) / (words in c + vocabulary size); without it one unseen word gives 0 and wipes out the product.
   - Spam: free 3/23, cash 3/23, meeting 1/23
   - Ham: free 1/23, cash 1/23, meeting 3/23
   - "for", "the" never seen → ignored (as scikit-learn's vectoriser does).
3. **Scores:** spam ∝ $0.5 \cdot 9/23^3$, ham ∝ $0.5 \cdot 3/23^3$ → 9 : 3.
4. **Normalise:** P(spam | message) = **0.75**.

`code/naive_bayes_demo.py`:

```
vec = CountVectorizer()
X = vec.fit_transform(texts)
nb = MultinomialNB(alpha=1.0).fit(X, labels)       # alpha = Laplace smoothing
proba = nb.predict_proba(vec.transform([message]))[0]
```

```
vocabulary size = 13
by hand : P(spam|'free cash for the meeting') = 0.750, P(ham) = 0.250
sklearn : P(spam|'free cash for the meeting') = 0.750, P(ham) = 0.250
```

On numeric data:

```
GaussianNB on iris, 5-fold CV accuracy = 0.953
BernoulliNB on binarised iris          = 0.753
```

Matching the variant to the feature type matters: binarising throws away detail and costs 20 points.

Trade-offs

Strengths	Weaknesses
Extremely fast: one counting pass; millions of documents	Poor probability estimates
Works with little data and very many features	Can't learn interactions ("not good")
Multi-class, incremental (`partial_fit`)	Usually beaten by logistic regression / linear SVM with enough data
Robust to irrelevant features	Gaussian variant assumes bell-shaped features

When not to use it

Strongly dependent features with enough data for a **discriminative** model (learns P(class | features) directly, e.g. logistic regression); need for calibrated probabilities; interactions matter.

Common mistakes

- No smoothing (`alpha=0` → zero probabilities); multiplying raw probabilities (underflow).
- Negative values into `MultinomialNB` (counts must be ≥ 0; standardised features break it).
- Trusting `predict_proba` as real probabilities.

Real-world examples

- Spam filtering: Paul Graham's 2002 *A Plan for Spam* popularised it; SpamAssassin includes a Bayesian component.
- Baselines for sentiment analysis, topic tagging, ticket routing, language detection.

In the interview

30-second answer: "Naive Bayes applies Bayes' theorem — posterior ∝ prior × likelihood — and assumes features are independent given the class, so the likelihood is a product of per-feature probabilities. Multinomial NB for word counts, Bernoulli for binary features, Gaussian for continuous. Use Laplace smoothing and log probabilities. It's fast and strong on text with little data, but its probabilities are over-confident."

- *Why does it work if the assumption is wrong?* Only the argmax must be right; probability errors often don't change the ranking.
- *Laplace smoothing?* Add α (usually 1) to every count so unseen words don't give zero.
- *Generative vs discriminative?* NB is **generative** (models P(x | y) and P(y)); logistic regression is discriminative (P(y | x)). NB learns faster from little data; LR usually wins with more.

Self-check

1. Word seen 0 times in spam; α = 1, spam has 10 words, vocabulary 13: P(word | spam)? → 1/23.

TL;DR

Naive Bayes = Bayes' theorem + "features independent given class". Counting-fast, great text baseline; smooth, use logs, don't trust its probabilities.

8.4 Choosing a classic supervised algorithm

Every classic classifier in the book on four datasets, defaults, 5-fold CV accuracy (`code/compare_classifiers.py`):

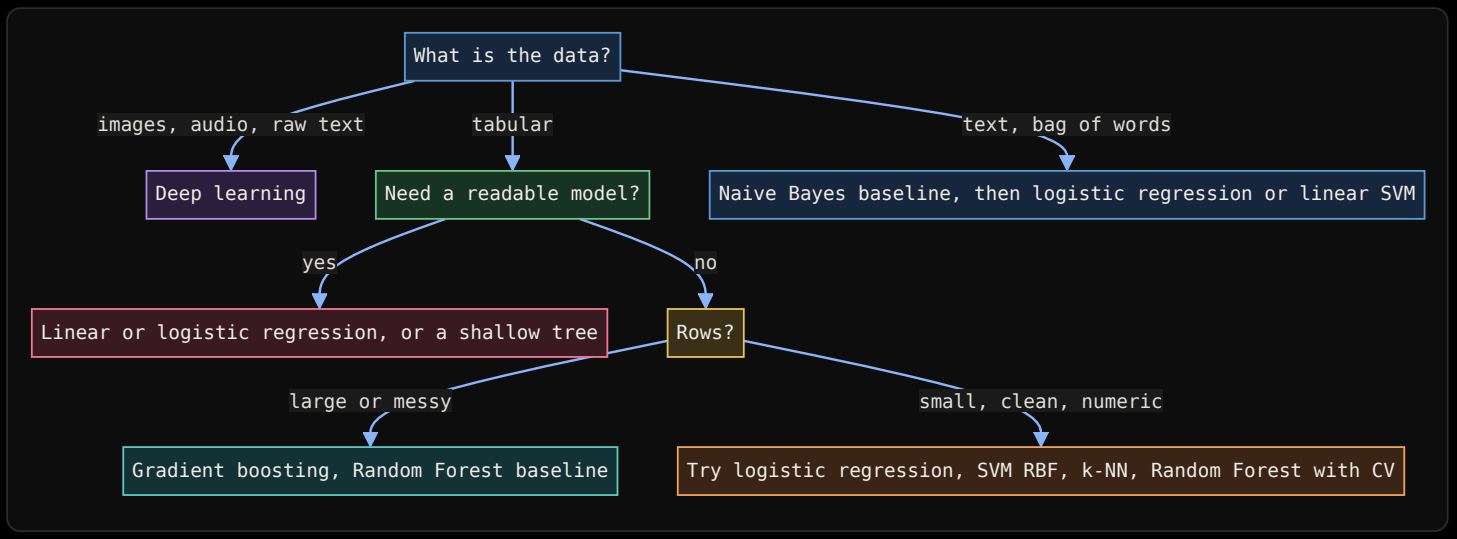
model	breast_cancer	wine	digits	moons (nonlinear)
LogisticRegression	0.981	0.983	0.920	0.844
k-NN (k=5)	0.965	0.949	0.944	0.886
SVM (RBF)	0.974	0.983	0.946	0.898
GaussianNB	0.939	0.966	0.807	0.840
DecisionTree	0.917	0.888	0.786	0.850
RandomForest	0.963	0.983	0.938	0.876
HistGradBoosting	0.965	0.955	0.935	0.872

No single winner ("no free lunch": no algorithm is best everywhere). Logistic regression wins the nearly linear breast-cancer data; RBF SVM wins curved moons and digits; a single tree is last almost everywhere; ensembles stay near the top. On small clean data the gaps are a few points; on large messy tabular data gradient boosting usually pulls ahead.

The comparison table

Algorithm	Type	Scale features?	Interpretable?	Training speed	Prediction speed	Non-linear?	Best for	Avoid when
Linear regression	Regression	Yes (if regularised)	High (coefficients)	Very fast	Very fast	No (unless features added)	Baselines, linear relationships, explaining effects	Strong non-linearity, outliers (use robust loss)
Logistic regression	Classification	Yes	High	Very fast	Very fast	No	Baseline classifier, calibrated probabilities, high-dim sparse text	Complex interactions without feature engineering
k-NN	Both	**Yes**	Medium (show neighbours)	None	**Slow** ($O(n \cdot d)$)	Yes	Small low-dim data, similarity search, recommendations	Many features, big data, real-time latency (System Design Fundamentals)
SVM (RBF)	Both (SVC/SVR)	**Yes**	Low	Slow for large n	Medium ($\propto$ support vectors)	Yes (kernels)	Small/medium data, many features, clear margins	> ~100k rows, need probabilities
Linear SVM	Classification	Yes	High	Very fast	Very fast	No	Text classification at scale	Non-linear problems
Naive Bayes	Classification	No	Medium	**Fastest**	Very fast	No (per-feature only)	Text, tiny data, streaming, baseline	Dependent features, need probabilities
Decision tree	Both	No	**High** (rules)	Fast	Very fast	Yes	Explainable rules, quick look at data	Accuracy matters (high variance)
Random Forest	Both	No	Low–medium (importances)	Medium (parallel)	Medium	Yes	Strong default, little tuning, robust	Tight latency/memory, extrapolation
Gradient boosting	Both	No	Low–medium (SHAP per-feature contributions)	Medium (sequential)	Fast	Yes	**Best accuracy on tabular data**	Unstructured data, tiny data, no time to tune
Neural network (MLP) (multi-layer perceptron)	Both	**Yes**	Low	Slow	Fast	Yes	Huge data, images/text/audio (CNNs, Transformers; Chapter 10)	Small tabular data

A decision flow



Practical rule: start with a simple baseline (logistic/linear regression, or Naive Bayes for text), then a Random Forest and a gradient-boosting model, compared with cross-validation.

Chapter TL;DR

- **k-NN:** predict from the k closest examples. Scale, choose k by CV, cosine for text/embeddings. Lazy: free training, slow prediction. Breaks in high dimensions (distances concentrate).
- **SVM:** maximise margin $2/\|w\|$; support vectors define it. C = strictness (large → tight fit). Kernel trick = dot products in a hidden space; RBF default, γ = each point's reach. Scale; tune C and γ together; SVR uses an ε-tube. Slow beyond tens of thousands of rows.
- **Naive Bayes:** posterior ∝ prior × product of per-feature likelihoods. Multinomial (counts), Bernoulli (binary), Gaussian (continuous). Laplace smoothing + logs. Fast, good for text; over-confident probabilities.
- **Choosing:** no universal winner — baseline first, compare with CV; gradient boosting for tabular, deep learning for unstructured data.

Chapter 9 — Unsupervised Learning

Unsupervised learning works with **no labels**: the goal is to find structure in the data itself. The four big jobs (each algorithm is explained in its own section):

Job	Question it answers	Main algorithms
Clustering	Which examples belong together?	k-means, hierarchical, DBSCAN, Gaussian mixtures
Dimensionality reduction	Can I describe the data with fewer numbers, or see it in 2-D?	PCA, t-SNE, UMAP
Anomaly detection	Which examples don't fit?	z-score, Isolation Forest, One-Class SVM, LOF
Association rules	Which items occur together?	Apriori, FP-Growth

The hard part is always **evaluation** — without labels, what is "good"? — covered last.

What you'll learn: k-means, k-means++, elbow and silhouette, k-means' limits; agglomerative clustering and dendrograms; DBSCAN; Gaussian mixtures and EM; PCA; t-SNE and UMAP; anomaly detection (statistics, Isolation Forest, One-Class SVM, LOF); Apriori; evaluating unsupervised models.

All code is in `code/`, figures in `img/`; outputs are real (scikit-learn 1.9, SciPy 1.18).

9.1 k-means clustering

What it is

k-means splits data into k **clusters**, each point belonging to the nearest **centroid** (cluster mean); you choose k. Like placing k pizza shops so every household walks to its nearest one with minimal total walking: move each shop to the middle of its customers, reassign, repeat.

Why it matters

Simple, fast (~O(n·k·d) per iteration: points × clusters × features), scales to millions, and the default first clustering method: customer segmentation, document grouping, colour quantisation (compressing an image's colours), "visual words" or **codebooks** (fixed lists of representative patterns), initialising other models.

How it works (Lloyd's algorithm)

k-means minimises **inertia** (within-cluster sum of squares, **WCSS**): total squared distance from points to their centroids.

1. **Initialise** k centroids (random points or k-means++).
2. **Assign** each point to its nearest centroid.
3. **Update** each centroid to the mean of its points.
4. **Repeat** 2–3 until assignments stop changing.

Each step can only lower inertia, so it converges — to a **local** minimum (no small change improves it, but a better solution may exist) that depends on the start.

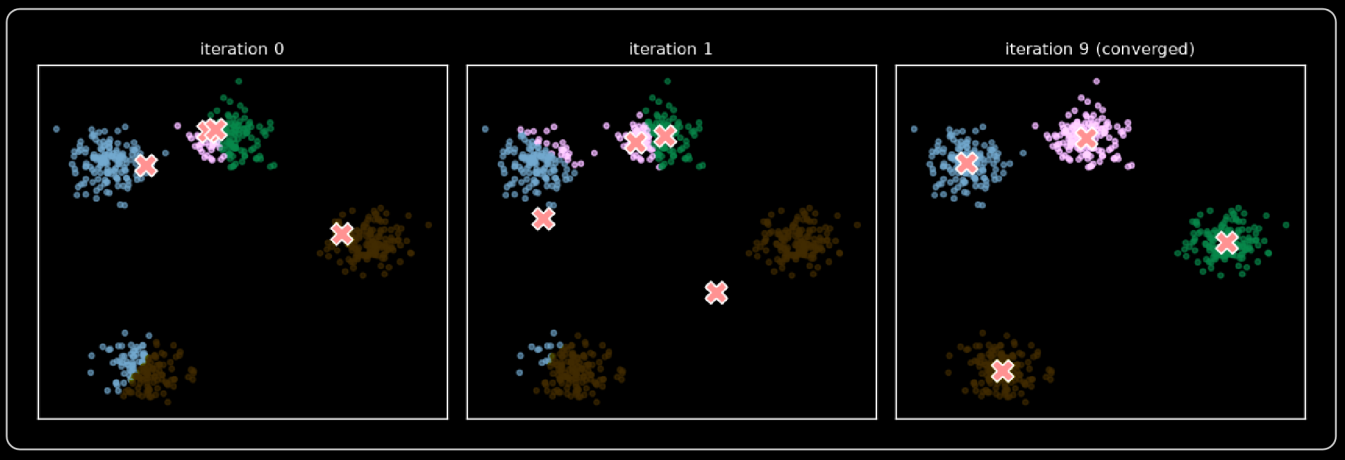
Worked example (1-D). Points 1, 2, 3, 10, 11, 12; k = 2; start at 1 and 2.

- Assign {1} and {2, 3, 10, 11, 12}; update $c_1 = 1$, $c_2 = 7.6$.
- 3 is 2 from c_1 and 4.6 from c_2 → {1, 2, 3}, {10, 11, 12}; update $c_1 = 2$, $c_2 = 11$.
- No change → converged. Inertia = (1 + 0 + 1) + (1 + 0 + 1) = **4**.

From scratch (`code/kmeans_demo.py`):

```
def kmeans_scratch(X, k, rng, max_iter=100):
    centroids = X[rng.choice(len(X), k, replace=False)]          # 1. random init
    for it in range(max_iter):
        d = np.linalg.norm(X[:, None, :] - centroids[None, :, :], axis=2)
        labels = d.argmin(axis=1)                                # 2. assign to nearest centroid
        new = np.array([X[labels == j].mean(axis=0) for j in range(k)])  # 3. move centroids to means
        if np.allclose(new, centroids):                          # 4. stop when nothing moves
            break
        centroids = new
    inertia = ((X - centroids[labels]) ** 2).sum()
    return labels, centroids, inertia
```

```
from scratch: converged in 9 iterations, inertia = 1160
sklearn     : inertia = 1160
```



Two starting centroids fell in one blob and another blob had none; it recovered in 9 iterations — not always that lucky.

k-means++ initialisation

k-means++ (Arthur & Vassilvitskii, 2007) spreads initial centroids:

1. First centroid uniformly at random.
2. For each point, D(x) = distance to the nearest chosen centroid.
3. Next centroid drawn with probability **∝ D(x)²** — far points much more likely.
4. Repeat to k, then run normal k-means.

Guarantee: expected inertia within O(log k) of optimal. On 8 well-separated blobs, one run per seed:

```
init          inertia over 20 single-run seeds, 8 true clusters (min / mean / max)
random            1673 /     9002 /    32908
k-means++          704 /      704 /      704
```

Random init never found the answer in 20 tries; k-means++ every time. scikit-learn defaults to `init="k-means++"`, and `n_init` restarts keep the best run.

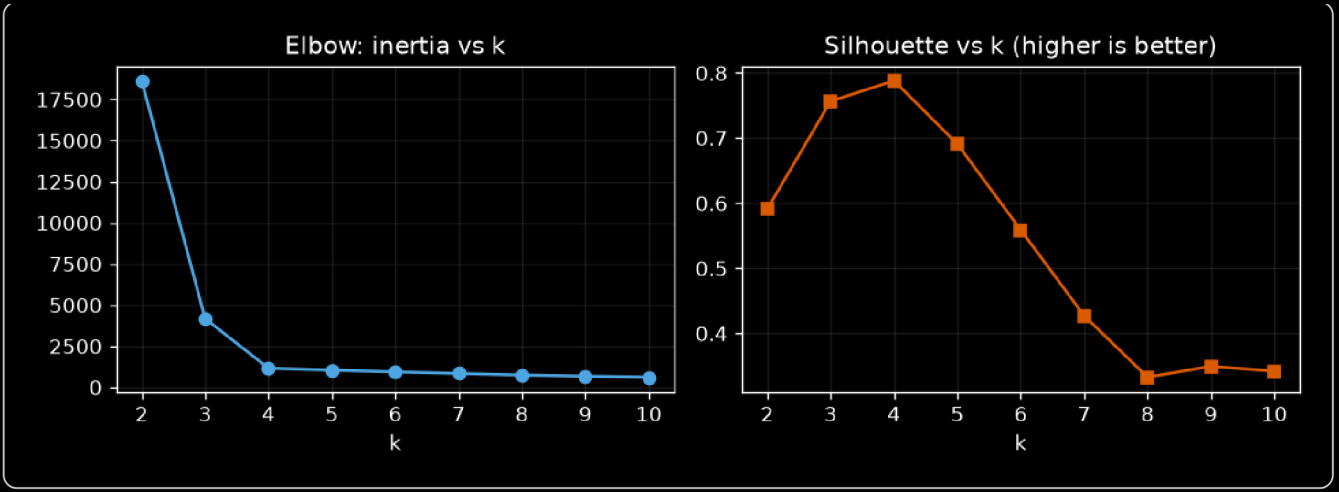
Choosing k: elbow and silhouette

Elbow method: plot inertia vs k (it always falls; k = n gives 0) and look for the bend where an extra cluster stops helping.

Silhouette per point: a = mean distance to its own cluster (cohesion), b = mean distance to the **nearest other** cluster (separation), s = (b − a) / max(a, b) ∈ [−1, 1]. s ≈ 1 well inside, ≈ 0 on a border, < 0 probably misassigned; the **silhouette score** averages s. In the 1-D example point 3 has a = 1.5, b = 8, s = 0.81.

On 4 true blobs:

```
k   inertia   silhouette
2     18638   0.591
3      4179   0.757
4      1160   0.788
5      1046   0.691
6       938   0.559
```



Both point to k = 4. On real data the elbow is often vague — prefer silhouette, domain knowledge ("marketing can handle 5 segments") and stability across runs.

Limits of k-means

It assumes **round (spherical), similar-sized clusters separated by straight boundaries**. It fails on:

- **Non-convex shapes** (moons, rings) — §9.4 comparison.
- **Elongated or differently spread clusters** — a diffuse cluster gets cut up.
- **Outliers** (drag the means); **unscaled features** (always standardise).
- **Categorical data** — means are meaningless (use k-modes or k-prototypes).
- You must choose k.

Common mistakes

- Not scaling; trusting one random-init run.
- Running the elbow on data with no clusters — k-means returns k groups even from uniform noise.

In the interview

> **30-second answer:** "k-means picks k centroids, assigns each point to the nearest, moves each centroid to the mean of its points, and repeats until nothing changes. It minimises within-cluster squared distance and converges to a local minimum, so we use k-means++ initialisation and multiple restarts. We choose k with the elbow or silhouette score. It assumes round, similar-sized clusters and needs scaled features."

- *Why k-means++?* Spread starts (∝ squared distance) → faster, better, more consistent results.
- *Complexity?* O(n·k·d) per iteration; **Mini-Batch k-means** (updates from small random batches) for huge data.
- *Always converges?* Yes (inertia never rises, finitely many assignments), but only to a local optimum.

Self-check

1. Why does inertia always drop as k grows? → More centroids sit closer to points; k = n gives 0.
2. Silhouette with a = 2, b = 6? → 0.67.

TL;DR

k-means = assign to nearest mean, move means, repeat. Scale, k-means++ with restarts, pick k by silhouette/elbow; assumes round clusters.

9.2 Hierarchical (agglomerative) clustering

What it is

Agglomerative clustering starts with every point as a cluster and repeatedly **merges the two closest** until one remains. The merge history is a **dendrogram** — a tree whose height shows the merge distance; cut it at a height to get clusters.

How it works

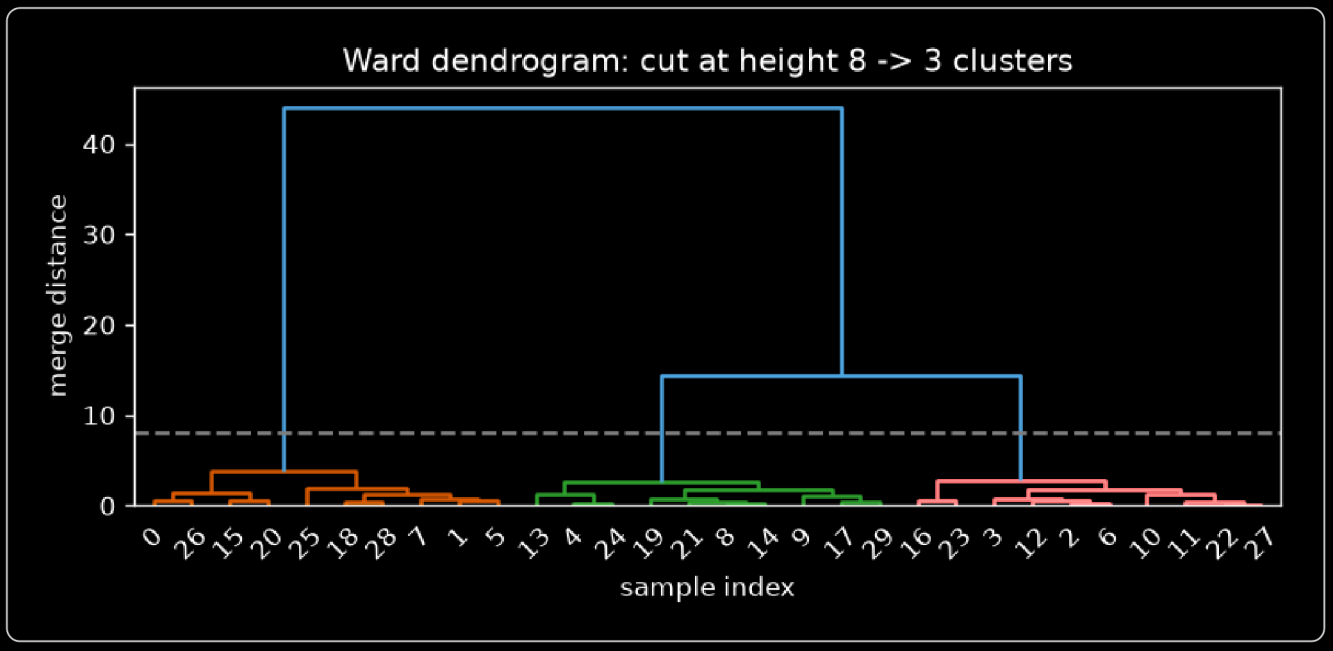
1. Compute pairwise distances; each point is a cluster.
2. Merge the pair with the smallest **linkage** distance.
3. Update distances; repeat to one cluster.

4. Cut the dendrogram at a height (or request n clusters).

Linkage	Distance between clusters A and B	Behaviour
Single	Closest pair of points	Follows chains — finds snakes and moons, but a thin noise bridge merges clusters ("chaining")
Complete	Farthest pair	Compact clusters of similar diameter
Average	Mean of all pairwise distances	Compromise
Ward (scikit-learn default)	Increase in total within-cluster variance from the merge	Like k-means: round, balanced

```
Z = linkage(X, method="ward")          # SciPy: the full merge history
dendrogram(Z, color_threshold=8)
```

```
last 3 merge heights: [ 3.74 14.37 44.1 ]  (big jump before the final merges = natural cut)
```



Merges below ~4 are within groups; jumps to 14.37 and 44.1 join genuinely different groups. Cutting in the gap (e.g. 8) gives 3 clusters.

Trade-offs

Strengths	Weaknesses
No k in advance — one tree, any number of clusters	$O(n^2)$ memory for distances; slow beyond ~10–20k points
Interpretable picture of nested structure	Greedy merges, never undone
Any metric; linkage adapts to shapes	Noise-sensitive (single) or biased to round shapes (Ward)

Real-world examples

Phylogenetic (evolutionary) trees, clustered gene-expression heatmaps, document/product taxonomies.

In the interview

30-second answer: "Agglomerative clustering starts with each point alone and repeatedly merges the closest pair of clusters, where 'closest' is defined by the linkage — single, complete, average or Ward. The dendrogram records the merges; cutting it at a height gives the clusters. It's great for small data and nested structure, but it's $O(n^2)$ in memory."

- *Linkage for moons?* Single (it chains), if there's little noise between them.

Self-check

1. Dendrogram merge height? → Linkage distance at which the clusters joined.
2. Linkage most like k-means? → Ward.

TL;DR

Merge closest clusters bottom-up; cut the dendrogram at a big gap; linkage decides shapes. Small data only.

9.3 DBSCAN — density-based clustering

What it is

DBSCAN (Density-Based Spatial Clustering of Applications with Noise, Ester et al. 1996) defines clusters as **dense regions separated by sparse ones**: any shape, number found automatically, isolated points labelled **noise** (−1) — like festival crowds versus lone wanderers.

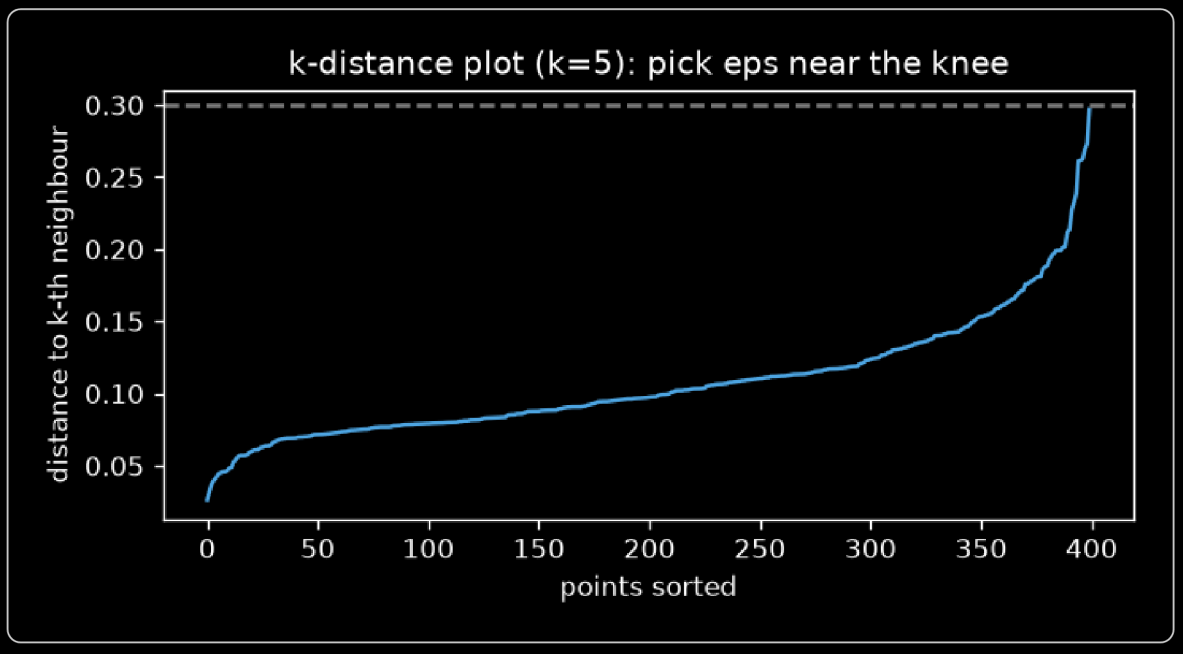
How it works

Parameters **eps** (neighbourhood radius) and **min_samples** (points to count as dense; in scikit-learn including the point itself).

1. **Core point:** ≥ `min_samples` points within eps.
2. **Border point:** within eps of a core point, not core itself.
3. **Noise point:** neither.
4. From an unvisited core point, the cluster is everything **density-reachable** (chains of core points each within eps of the next, plus their border points). Repeat.

Choosing eps: the k-distance plot

Sort each point's distance to its k-th neighbour (k = min_samples) and plot; pick eps near the "knee" where the curve shoots up (dense-region points vs outliers).




```
DBSCAN on moons (min_samples=5)
  eps=0.1  clusters=28  noise points=84
  eps=0.2  clusters=2  noise points=1
  eps=0.3  clusters=2  noise points=0
  eps=0.5  clusters=2  noise points=0
  eps=1.0  clusters=1  noise points=0
```

Too small shatters the data (28 fragments plus noise); too large merges everything. Rule of thumb: min_samples ≈ 2 × number of features (at least 3–5), more for noisy data.

Trade-offs

Strengths	Weaknesses
Any shape; no k	One global density: fails with **different densities**
Built-in noise/outlier detection	Sensitive to eps; hard to choose in high dimensions
Deterministic for core points	Needs scaling; suffers the curse of dimensionality (Chapter 8)

HDBSCAN (`sklearn.cluster.HDBSCAN` since 1.3) handles varying densities and drops eps, needing mainly `min_cluster_size` — often the better modern choice. **OPTICS** is related.

Real-world examples

GPS points → places of interest; geospatial events (crime, taxi pickups); sensor outliers.

In the interview

> **30-second answer:** "DBSCAN groups points in dense regions: a core point has at least min_samples neighbours within eps; clusters are chains of core points plus their border points; everything else is noise. It finds arbitrary shapes and the number of clusters automatically, but struggles with clusters of different densities — HDBSCAN addresses that."

Self-check

1. A tight cluster next to a diffuse one? → One eps can't fit both densities.

TL;DR

Clusters = dense regions; noise labelled −1. eps from a k-distance plot; HDBSCAN for mixed densities.

9.4 Gaussian mixture models (GMM) and EM

What it is

A **Gaussian mixture model** assumes data come from k Gaussians, each with a **mean** (centre), **covariance** (shape and orientation — stretched, tilted ellipses) and **weight** (share of data). Unlike k-means' hard assignment it gives **soft assignments** ("43% A, 35% C, 21% B").

How it works — Expectation-Maximisation (EM), intuitively

We know neither the Gaussians' parameters nor which Gaussian produced each point; knowing one makes the other easy, so EM alternates:

1. **Initialise** means, covariances, weights (scikit-learn: from k-means).
2. **E-step:** each point's **responsibility** per component — P(component | point) by Bayes' rule (weight × density, normalised).
3. **M-step:** re-estimate each Gaussian from all points **weighted by responsibility** (weighted mean, covariance; weight = average responsibility).
4. Repeat until the **log-likelihood** (log-probability of the data under the model) stops improving.

EM never decreases likelihood but finds a local optimum (`n_init` > 1). k-means is the limiting case with hard assignments and identical spherical covariances.

Choosing k: BIC

The **Bayesian Information Criterion** (BIC) = likelihood penalised by parameter count; **lower is better** (`gmm.bic(X)`; also `aic`, Akaike, lighter penalty).

```
GMM model selection by BIC (lower is better)
  k=1  BIC=  4797.3
  k=2  BIC=  4514.9
  k=3  BIC=  4513.3
  k=4  BIC=  4550.7
soft assignments: 103 of 500 points have max membership < 0.9, e.g. [0.43 0.21 0.35]
EM converged in 4 iterations; mixing weights [0.33 0.4  0.27]
```

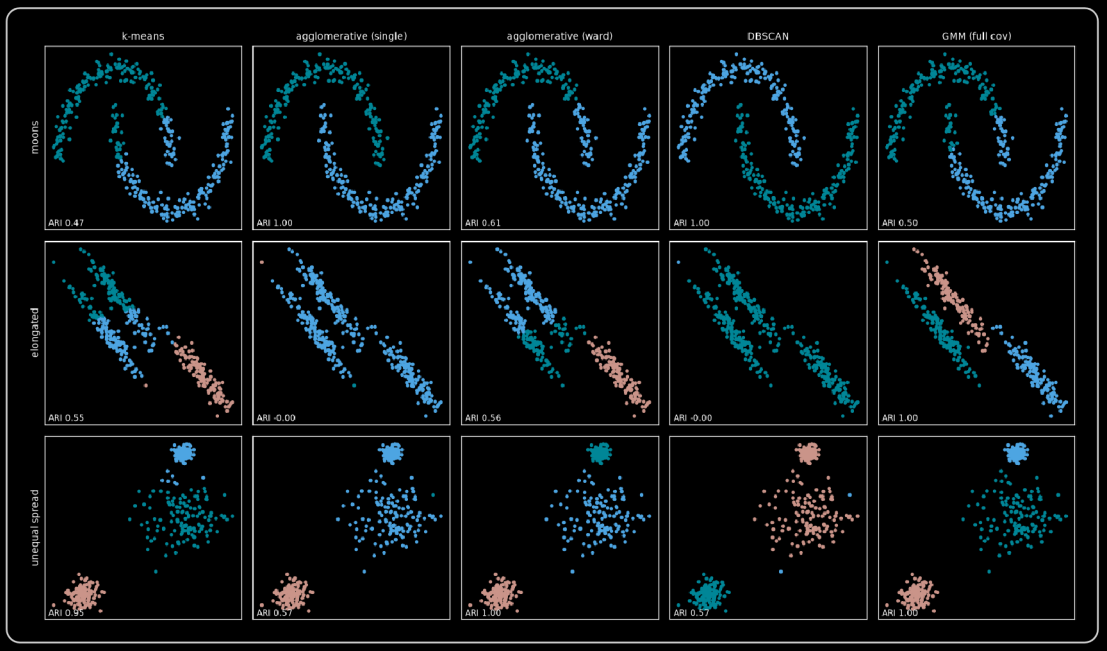
BIC picks k = 3 (the truth), narrowly over 2 because two blobs overlap; ~20% of points are genuinely ambiguous, as the soft assignments honestly show.

`covariance_type`: `"full"` (any ellipse per component), `"tied"` (one shared), `"diag"` (axis-aligned), `"spherical"` (circles, like k-means).

Comparing all four clusterers

`code/clustering_shapes.py` scores each method against the truth with the **Adjusted Rand Index** (ARI: 1 = perfect, ≈ 0 = random):

Adjusted Rand Index	moons	elongated	unequal spread
k-means	0.47	0.55	0.95
agglomerative (single)	1.00	-0.00	0.57
agglomerative (ward)	0.61	0.56	1.00
DBSCAN	1.00	-0.00	0.57
GMM (full cov)	0.50	1.00	1.00



- **Moons** (**non-convex**: a straight line between two cluster points can leave the cluster): only density/chaining methods (DBSCAN, single linkage) succeed.
- **Elongated** and close: only full-covariance GMM models the ellipses; DBSCAN and single linkage merge through the dense bridge.
- **Unequal spread:** GMM and Ward perfect; DBSCAN's one eps can't fit both.

No algorithm wins everywhere — choose by expected shapes, and look at the result.

When to use which clustering algorithm

Algorithm	Needs k?	Cluster shapes	Noise handling	Scales to	Pick it when
k-means	Yes	Round, similar size	None (outliers distort)	Millions (Mini-Batch)	Fast default, compact clusters
Agglomerative	Yes or cut height	Depends on linkage	Poor	~10k	Small data, want a hierarchy
DBSCAN / HDBSCAN	No	Any	Labels noise	~100k+ (with index)	Odd shapes, outliers, unknown k
GMM	Yes (by BIC)	Ellipses, overlapping	None	~100k	Soft probabilities, elongated clusters

In the interview

30-second answer: "A GMM models the data as a weighted mix of Gaussians, each with its own mean and covariance, and fits them with EM: the E-step computes each point's probability of belonging to each component, the M-step re-estimates the parameters with those weights. It gives soft assignments and elliptical clusters; you choose k with BIC. k-means is a special case with hard assignments and spherical clusters."

Self-check

1. E-step computes? → Responsibilities P(component | point).
2. GMM over k-means when? → Elliptical/overlapping clusters, or membership probabilities needed.

TL;DR

GMM = mixture of Gaussians fitted with EM; soft, elliptical clusters; k by BIC (lower better).

9.5 Dimensionality reduction: PCA

What it is

Principal Component Analysis finds new axes — **principal components**, linear combinations of the features — ordered so the first captures the **most variance**, the second the most of the rest while perpendicular, and so on. Keeping the top few compresses data with minimal loss — the most informative "camera angle" on a teapot.

Why it matters

- **Compression / speed** (fewer features); **noise reduction** (small-variance directions are often noise); **visualisation** in 2-D/3-D.
- **Removes correlation** (multicollinearity): components are uncorrelated.
- Fights the **curse of dimensionality** before distance-based methods.

How it works

(Eigenvectors and SVD: Chapter 2, section 2.3.)

1. **Centre** each feature; **standardise** if units differ.
2. Compute the **covariance matrix** C (d × d).
3. Its **eigenvectors** are the directions; **eigenvalues** the variance along each ($C \cdot v = \lambda \cdot v$).
4. Sort by eigenvalue; **explained variance ratio** of component i = λ_i / $\Sigma\lambda$.
5. **Project:** Z = X_centred · V_k (top-k eigenvectors as columns).

Libraries compute this via the **SVD** of the centred data, which is more numerically stable; scikit-learn does.

Worked example — hours studied vs exam score, 5 students (`code/dim_reduction.py`):

```
X = np.array([[2, 50], [4, 60], [6, 70], [8, 85], [10, 95]], dtype=float)
Xs = (X - X.mean(axis=0)) / X.std(axis=0, ddof=1)   # centre and scale
C = np.cov(Xs, rowvar=False)                          # covariance matrix
vals, vecs = np.linalg.eigh(C)                        # eigen-decomposition
z = Xs @ vecs[:, 0]                                   # project onto PC1
```

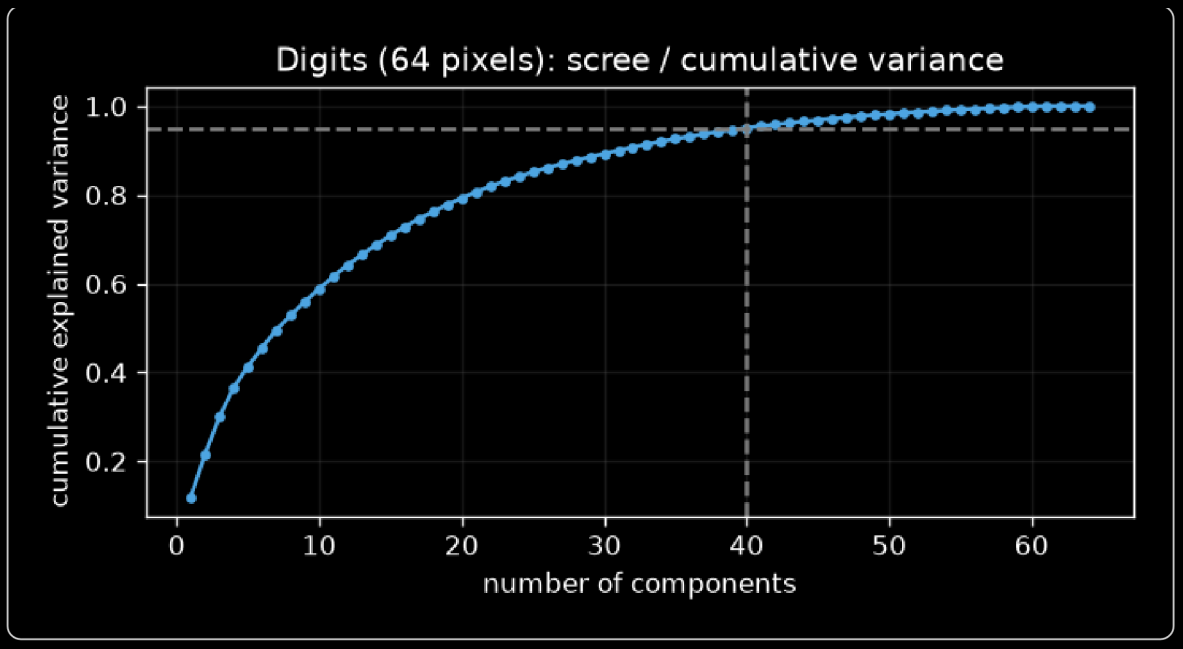
```
covariance (standardised):
 [[1.    0.997]
 [0.997 1.   ]]
eigenvalues: [1.997 0.003]  explained variance ratio: [0.999 0.001]
PC1 direction: [0.707 0.707]
students on PC1: [-1.75 -0.91 -0.08  0.95  1.79]
sklearn explained_variance_ratio_: [0.999 0.001]
```

For two standardised features with correlation r, eigenvalues are always 1 ± r with directions (1, ±1)/√2. Here r = 0.997, so PC1 ("overall diligence") holds 99.9%: two columns compress to one number per student. (Eigenvector sign is arbitrary; flipped so bigger = more.)

How many components? Explained variance

8×8 digit images (64 pixels):

```
components for 50% variance:  8 of 64
components for 80% variance: 21 of 64
components for 90% variance: 31 of 64
components for 95% variance: 40 of 64
components for 99% variance: 54 of 64
PCA(n_components=0.95) keeps 40 components, reconstruction MSE = 0.047
```



Keep 90–99% of variance (`PCA(n_components=0.95)`), find an elbow in the **scree plot** (eigenvalue vs component number), or cross-validate the count with the downstream model. `inverse_transform` maps back; the **reconstruction error** shows what was lost.

Variants

- **Incremental PCA:** data too big for memory (mini-batches).
- **Randomized PCA** (`svd_solver="randomized"`): fast approximate top components.
- **Kernel PCA:** non-linear via the kernel trick (Chapter 8).
- **TruncatedSVD:** PCA-like for sparse matrices (e.g. word counts), no centring.

Trade-offs / when not to use it

- **Linear** only — curved **manifolds** (surfaces the data lies on, like a "Swiss roll") need non-linear methods.
- Components mix all features → hard to interpret; sensitive to scaling and outliers.
- Max variance ≠ most predictive: a low-variance direction may separate classes (supervised alternative: **LDA**, linear discriminant analysis, picks directions separating known classes).

Common mistakes

- Not standardising mixed units (the large-unit feature becomes PC1).
- Fitting PCA before splitting (leakage) — put it in the pipeline.
- Over-reading a 2-component plot explaining only 20% of variance.

Real-world examples

Eigenfaces (Turk & Pentland, 1991); compressing embeddings before vector search; population-structure plots in genetics; decorrelating features before regression; image compression.

In the interview

> **30-second answer:** "PCA centres (and usually scales) the data, computes the covariance matrix, and takes its eigenvectors ordered by eigenvalue. Each eigenvalue is the variance along that direction, so the explained-variance ratio tells you how much information each component keeps. You project onto the top k — chosen for, say, 95% variance — to compress, denoise or visualise. It's linear, unsupervised, and sensitive to scaling."

- *PCA vs SVD?* PCA is usually computed via SVD of centred data: right singular vectors = principal directions; squared singular values/(n−1) = eigenvalues.
- *Helps every model?* No — trees don't need it, and it can drop low-variance predictive directions.

Self-check

1. Two standardised features, correlation 0.8: PC1's explained variance ratio? → (1 + 0.8)/2 = 0.9.

TL;DR

PCA rotates data onto uncorrelated axes of decreasing variance; keep the top ones. Scale first; choose k by explained variance; linear only.

9.6 t-SNE and UMAP — non-linear visualisation

What they are

t-SNE (t-distributed Stochastic Neighbor Embedding, van der Maaten & Hinton, 2008) and **UMAP** (Uniform Manifold Approximation and Projection, McInnes et al., 2018) are **non-linear** methods mainly for **visualising** high-dimensional data in 2-D/3-D, keeping **neighbours as neighbours**.

How t-SNE works (intuition)

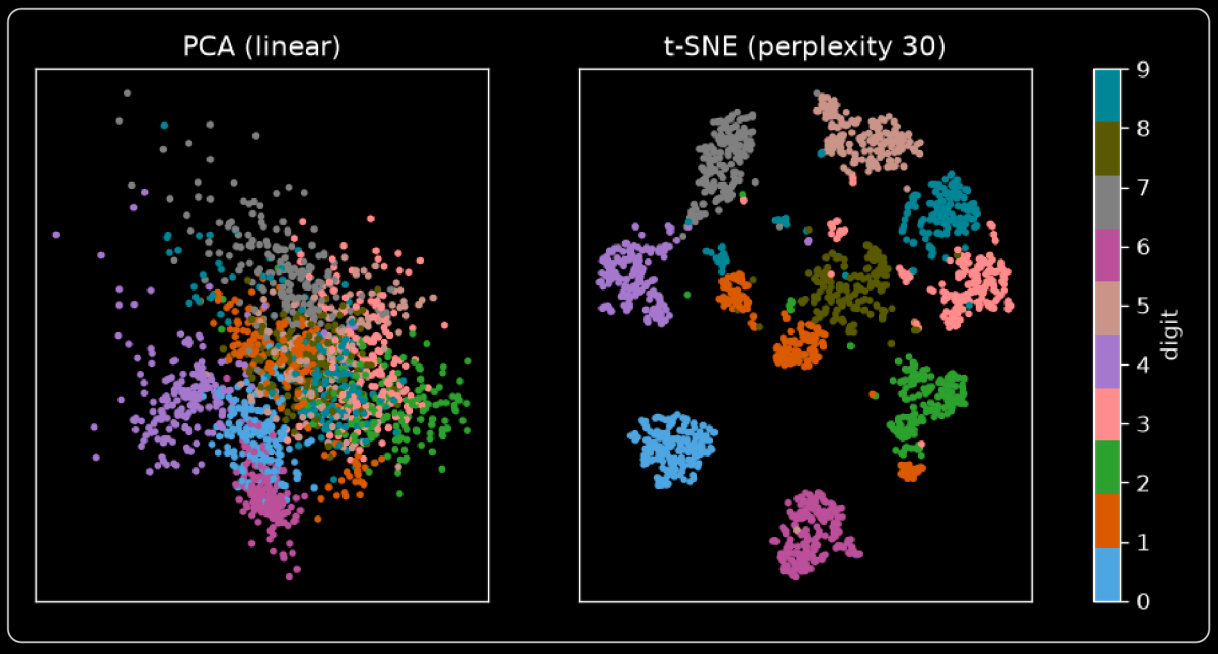
1. In the original space, turn distances into neighbour probabilities (Gaussian kernel); **perplexity** (typically 5–50) ≈ how many neighbours each point considers.
2. In 2-D, use a heavy-tailed **Student-t** distribution, letting moderately distant points push far apart instead of crowding the middle.
3. Move 2-D points by gradient descent to match the two distributions (minimise **KL divergence**).

UMAP builds a weighted k-nearest-neighbour graph (`n_neighbors`) and optimises a low-dimensional layout matching it (`min_dist` controls packing). Usually faster than t-SNE on large data, keeps more global arrangement, and can `transform` new points.

Code — PCA vs t-SNE on digits

```
Z_pca = PCA(n_components=2, random_state=0).fit_transform(Xs)
Z_tsne = TSNE(n_components=2, perplexity=30, init="pca", random_state=0).fit_transform(
    PCA(n_components=30, random_state=0).fit_transform(Xs))   # common trick: PCA to ~30-50 dims first
print(trustworthiness(Xs, Z, n_neighbors=5))
```

```
trustworthiness (k=5) PCA  : 0.818
trustworthiness (k=5) t-SNE: 0.992
```



PCA's best linear view mixes most digits; t-SNE separates all ten into islands without seeing labels (colour is only for the plot). **Trustworthiness** (0–1) = how many map neighbours were true neighbours.

UMAP (`umap-learn` not installed here — not run):

```
# pip install umap-learn   (not run)
import umap
reducer = umap.UMAP(n_neighbors=15, min_dist=0.1, random_state=0)
Z = reducer.fit_transform(Xs)
Z_new = reducer.transform(X_new_scaled)   # UMAP can embed new points; sklearn's TSNE cannot
```

Reading t-SNE/UMAP plots safely

- **Cluster sizes mean nothing** (t-SNE expands dense clusters, contracts sparse ones).
- **Inter-cluster distances mean little.**
- **Results change with perplexity/n_neighbors and seed** — try several.

- **Don't cluster on t-SNE output** or use it as features — it's for looking. (UMAP is sometimes used before clustering, with care.)

PCA vs t-SNE vs UMAP

	PCA	t-SNE	UMAP
Type	Linear	Non-linear	Non-linear
Preserves	Global variance	Local neighbourhoods	Local, plus more global than t-SNE
Speed	Very fast	Slow on big data (Barnes-Hut, O(n log n))	Fast
New points	`transform`	No (in scikit-learn)	`transform`
Deterministic	Yes	No (seed)	No (seed)
Use for	Compression, pre-processing, denoising, quick look	2-D cluster visualisation	Visualisation, sometimes pre-clustering
Key knobs	n_components	perplexity	n_neighbors, min_dist

Rule of thumb: PCA first (fast, interpretable, good pre-step to ~50 dims); t-SNE or UMAP to **look**; UMAP for large data or embedding new points.

In the interview

30-second answer: "PCA is a linear projection that keeps global variance — good for compression and pre-processing. t-SNE and UMAP are non-linear and keep local neighbourhoods, so they're great for 2-D visualisation of clusters, but cluster sizes and inter-cluster distances in the plot aren't meaningful and the output depends on hyperparameters. UMAP is faster and can transform new data."

Self-check

1. Which can embed new points out of the box? → PCA and UMAP.

TL;DR

PCA to compress; t-SNE/UMAP to see. Only local neighbourhoods in their plots are trustworthy.

9.7 Anomaly detection

What it is

Anomaly (outlier) detection finds points differing markedly from the majority: fraud, failing machines, intrusions, data-entry errors. Anomalies are usually **rare and unlabelled**, so it is mostly unsupervised. scikit-learn distinguishes **outlier detection** (training data contains the outliers) from **novelty detection** (train on clean data, flag new points).

Statistical baseline: z-score and robust z-score

z-score z = (x − mean)/std, flag |z| > 3 — but the outlier inflates mean and std (**masking**). Robust version: median and **MAD** (median absolute deviation), modified z = 0.6745·(x − median)/MAD, flag |modified z| > 3.5 (Iglewicz & Hoaglin). The box-plot **IQR rule** (outside Q1 − 1.5·IQR, Q3 + 1.5·IQR) is also robust.

```
amount   z-score   robust z
    42     -0.32      -0.54
    ...
    60     -0.25       1.89
   950      3.32     121.95  <- anomaly
mean=121.7 (dragged up by the outlier), median=46.0
```

The $950 payment barely passes z = 3: with population std, no point in a sample of n can exceed |z| = √(n − 1) — here √11 = 3.32 — so small samples **cannot** flag extremes reliably. The robust z says 122. These rules are univariate; for many features, use the methods below.

Isolation Forest

Idea (Liu, Ting & Zhou, 2008): anomalies are **few and different, so easy to isolate**. Random trees split on a random feature at a random value between its min and max; normal points in dense regions need many splits, anomalies few.

1. Build ~100–200 **isolation trees**, each on a small subsample (default 256 rows).
2. Measure each point's average **path length** to its leaf.
3. Convert to a score in (0, 1): near 1 = anomaly, ≤ ~0.5 = normal.

```
anomaly score of (6,6): 0.82   typical point (median): 0.43
```

(scikit-learn's `score_samples` returns the *negative*: lower = more abnormal.) Linear time, scales to large high-dimensional data, no distances, no scaling. Weakness: axis-parallel splits can miss anomalies visible only in a *combination* of features.

One-Class SVM

Learns a boundary around normal data (Schölkopf et al., 2001); with an RBF kernel, a smooth region holding most points; outside = anomaly. **ν** (`nu`) upper-bounds the fraction of training outliers (and lower-bounds the fraction of support vectors). Good for **novelty detection** on clean data; sensitive to γ and ν; needs scaling; poor past tens of thousands of points (`SGDOneClassSVM` is a scalable linear alternative).

Local Outlier Factor (LOF)

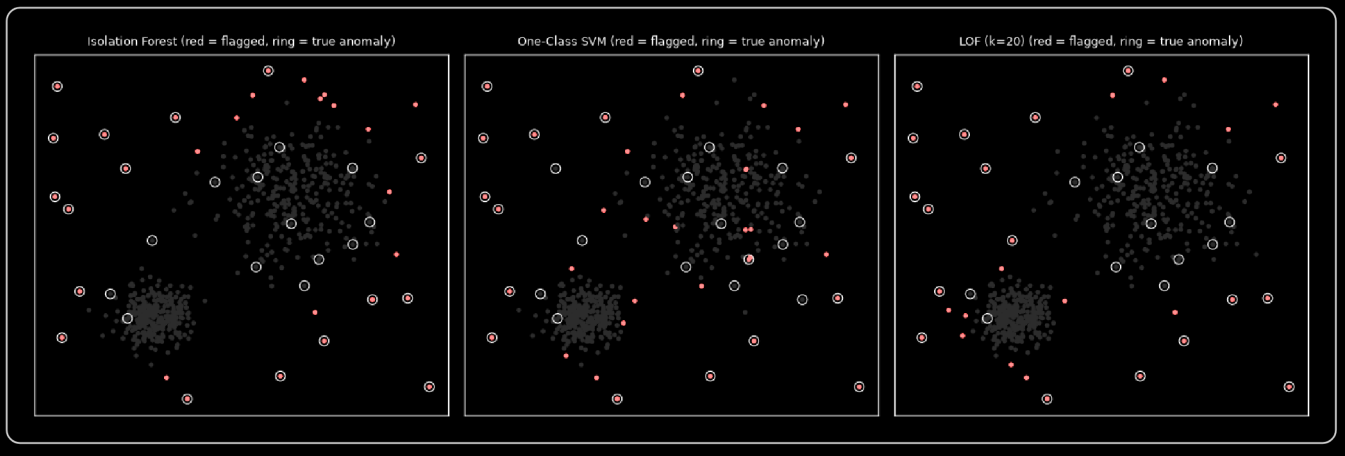
Compares a point's **local density** with its k neighbours' (Breunig et al., 2000): LOF ≈ 1 normal; LOF ≫ 1 much sparser than neighbours (outlier). Being **local**, it catches a point just outside a *tight* cluster that would look normal near a *diffuse* one. Scores training data by default; `novelty=True` scores new points.

Comparison

Tight cluster, diffuse cluster, 30 uniformly scattered anomalies (`code/anomaly_detection.py`):

```
detectors = {
    "Isolation Forest": IsolationForest(n_estimators=200, contamination=30 / 630, random_state=0),
    "One-Class SVM": OneClassSVM(kernel="rbf", gamma=0.3, nu=0.05),
    "LOF (k=20)": LocalOutlierFactor(n_neighbors=20, contamination=30 / 630),
}
pred = det.fit_predict(X)           # +1 inlier, -1 outlier
```

```
detector            ROC-AUC   avg precision   flagged   true anomalies caught
Isolation Forest      0.920           0.628        30   17 / 30
One-Class SVM         0.813           0.522        36   16 / 30
LOF (k=20)            0.886           0.617        30   18 / 30
```



About a third of the "anomalies" landed **inside** normal clusters — nothing can (or should) find those, so recall tops out near 18/30. `contamination` (expected outlier fraction) fixes the threshold; in practice you rarely know it, so rank by score and review the top. With a few labels, evaluate with **precision–recall** (average precision): anomalies are rare and ROC-AUC flatters under heavy imbalance.

Method	Idea	Strengths	Weaknesses
z-score / MAD / IQR	Distance from centre in std/MAD units	Trivial, explainable	One feature at a time; one "normal" centre
Isolation Forest	Anomalies isolate in few random splits	Fast, scales, high-dim, no scaling	Misses combination-only anomalies
One-Class SVM	Boundary around normal data	Flexible shapes, good for novelty	Slow on big n; γ, ν tuning; scaling
LOF	Density vs neighbours' density	Clusters of different density	$O(n^2)$-ish neighbour search; choice of k; scaling

Also: **autoencoders** (flag high reconstruction error; neural networks, later chapters), Elliptic Envelope (robust Gaussian fit), and time-series methods for **seasonal** data (repeating daily/weekly/yearly patterns).

Real-world examples

Card-fraud scoring, failing disks or turbines from sensor logs, network intrusion detection, data-quality checks before training.

In the interview

> **30-second answer:** "For one feature, a robust z-score or IQR rule is enough. For multivariate data, Isolation Forest is my default: anomalies get isolated by random splits in fewer steps, so short average path length means anomaly — it's fast and scales. LOF compares local density to neighbours and handles clusters of different density. One-Class SVM learns a boundary around clean data for novelty detection. Evaluate with precision–recall on whatever labels exist, and set the threshold from business cost."

- *Why not plain z-scores?* Masking (outlier inflates mean/std), and univariate.
- *Isolation Forest intuition?* Few-and-different points are cut off by random splits quickly.

Self-check

1. Isolation Forest: short or long path = anomaly? → Short.
2. LOF 1.0? → Same density as neighbours — normal.
3. Why is ROC-AUC misleading for rare anomalies? → A small false-positive *rate* over many negatives still means many false alarms.

TL;DR

Robust statistics for one feature; Isolation Forest as multivariate default; LOF for varying densities; One-Class SVM for novelty. Rank by score; judge with precision–recall.

9.8 Association rules (Apriori)

What it is

Association rule mining finds if-then patterns in transactions ($\{diapers\} \rightarrow \{beer\}$), classically **market-basket analysis**.

Measure	Formula	Meaning
Support(A)	Fraction of baskets containing A	How common
Confidence(A → B)	support(A ∪ B) / support(A)	How often B appears with A
Lift(A → B)	confidence(A → B) / support(B)	> 1: A makes B more likely; = 1 independent; < 1 less likely

How Apriori works

Apriori principle (anti-monotone): every subset of a frequent **itemset** must be frequent, so if {beer} is rare no superset is counted.

1. Count single items; keep support ≥ min_support (L1).
2. **Join** frequent (k−1)-itemsets into k-item candidates; **prune** any with an infrequent (k−1)-subset.
3. Count candidates in one pass; keep frequent ones (Lk); repeat until none.
4. Generate rules A → B from each frequent itemset; keep confidence ≥ min_confidence; rank by lift.

`code/apriori.py`, five baskets (min_support 0.6, min_confidence 0.7):

```
L1: {beer}=0.6, {bread}=0.8, {diapers}=0.8, {milk}=0.8
L2: {beer,diapers}=0.6, {bread,diapers}=0.6, {bread,milk}=0.6, {diapers,milk}=0.6

rule                        support  confidence  lift
           beer -> diapers       0.6        1.00   1.25
         diapers -> beer         0.6        0.75   1.25
           bread -> diapers      0.6        0.75   0.94
           bread -> milk         0.6        0.75   0.94
           ...
```

Every beer basket has diapers (confidence 1.0, lift 1.25): a real association. bread → milk has 75% confidence but lift 0.94 — milk is in 80% of baskets anyway, so bread makes it slightly *less* likely. **High confidence alone misleads — check lift.** (The only 3-item candidate, {bread, diapers, milk}, has support 0.4 and is dropped.)

FP-Growth finds the same itemsets without candidates (compressing data into a prefix tree storing shared basket beginnings once), usually faster; `mlxtend` provides both.

Real-world examples and limits

"Frequently bought together", product placement, cross-selling, co-occurring symptoms or page visits. (The beer-and-diapers story is partly legend.) Limits: many mostly trivial rules, arbitrary thresholds, association ≠ causation; modern recommenders mostly use collaborative filtering and embeddings.

In the interview

> **30-second answer:** "Association rules find co-occurrence patterns like A → B. Support is how common an itemset is, confidence is P(B | A), and lift is confidence divided by B's base rate — lift above 1 means a genuine positive association. Apriori finds frequent itemsets level by level and prunes candidates using the fact that all subsets of a frequent set must be frequent."

Self-check

1. Support(AUB) = 0.2, support(A) = 0.4, support(B) = 0.5? → Confidence 0.5, lift 1.0 (independent).

TL;DR

Support, confidence, lift; Apriori prunes by "subsets of frequent sets are frequent". Always check lift.

9.9 Evaluating unsupervised models

The problem

There is no **ground truth** to compare against, and clustering **always returns clusters**, even from noise — so evaluate from several angles.

Metrics

Kind	Metric	Range / better	Notes
Internal (no labels)	Silhouette	−1 to 1, higher	Cohesion vs separation; favours round, separated clusters
	Davies–Bouldin	≥ 0, **lower**	Average similarity of each cluster to its most similar one
	Calinski–Harabasz	≥ 0, higher	Between- vs within-cluster dispersion
	Inertia / BIC	Lower	Only to compare k within one algorithm
External (some labels)	Adjusted Rand Index (ARI)	≈ 0 random, 1 perfect	Pair-counting agreement, chance-corrected
	Normalized Mutual Information (NMI)	0 to 1, higher	Shared information between labelings
Dimensionality reduction	Explained variance, reconstruction error	Higher / lower	PCA-type methods
	Trustworthiness	0 to 1, higher	Are map neighbours true neighbours?
Anomaly detection	Precision@k (real anomalies in the top k flagged), average precision (small labelled set)	Higher	Rank-based; matches how analysts review alerts

Internal metrics can disagree with reality

`code/evaluate_clustering.py`:

```
iris (true k = 3 species)
  k-means k=2          silhouette= 0.582  DB= 0.59  CH=  251.3  ARI= 0.57  NMI= 0.73
  k-means k=3          silhouette= 0.460  DB= 0.83  CH=  241.9  ARI= 0.62  NMI= 0.66
  k-means k=4          silhouette= 0.387  DB= 0.87  CH=  207.3  ARI= 0.47  NMI= 0.60

moons (true k = 2, non-convex)
  k-means k=2          silhouette= 0.497  DB= 0.81  CH=  556.6  ARI= 0.47  NMI= 0.38
  DBSCAN eps=0.3       silhouette= 0.385  DB= 1.03  CH=  342.5  ARI= 1.00  NMI= 1.00
```

- Iris: all internal metrics prefer **k = 2** (two species overlap heavily, so "2 blobs" is geometrically cleaner); biology says 3.
- Moons: internal metrics prefer k-means' wrong split over DBSCAN's perfect one, since all three reward **convex** clusters.

A practical evaluation checklist

1. **Look at it:** plot clusters on a PCA/UMAP projection; read samples per cluster.
2. **Internal metrics** to compare settings of the *same* algorithm (k, eps), knowing their shape bias.
3. **Stability:** re-run with seeds or bootstrap resamples — do clusters persist (ARI between runs)?
4. **External check:** a small labelled sample or known categories, via ARI/NMI.
5. **Downstream value** — the real test: better campaigns, better model features, flagged anomalies confirmed as fraud?

In the interview

30-second answer: "Without labels I use internal metrics like silhouette, Davies–Bouldin and Calinski–Harabasz to compare settings — but they favour convex clusters. If some labels exist, ARI or NMI. I check stability across seeds and, most importantly, whether the result is useful for the downstream task. For dimensionality reduction: explained variance, reconstruction error, trustworthiness; for anomalies: precision on a reviewed sample."

Self-check

1. Why can silhouette prefer the wrong answer on moons? → It rewards compact, convex, separated clusters.

TL;DR

Internal metrics (shape-biased) + external labels when available + stability + downstream usefulness. Always look at the results.

Chapter TL;DR

- **k-means:** assign to nearest centroid, recompute means, repeat; k-means++ + restarts; k by silhouette/elbow; round clusters; scale.
- **Hierarchical:** merge closest clusters bottom-up; dendrogram; linkage sets shape; small data.
- **DBSCAN:** dense regions = clusters, sparse points = noise; any shape, no k; eps from a k-distance plot; HDBSCAN for varying density.
- **GMM + EM:** soft, elliptical clusters; E-step responsibilities, M-step weighted updates; k by BIC.
- **PCA:** covariance eigenvectors = max-variance directions; keep ~95% variance; linear; scale first.
- **t-SNE / UMAP:** non-linear neighbourhood-preserving maps for visualisation; ignore cluster sizes and gaps; UMAP faster, transforms new data.
- **Anomalies:** robust z/IQR univariate; Isolation Forest default; LOF for local density; One-Class SVM for novelty; precision–recall.
- **Apriori:** support, confidence, lift; subset pruning; trust lift over confidence.
- **Evaluation:** internal metrics are shape-biased — combine with external labels, stability and downstream usefulness.

Chapter 10 — Neural Networks and Deep Learning

A **neural network** is a function built from many tiny identical "neurons", each multiplying its inputs by numbers, adding them up and bending the result. Stack thousands in layers, tune the numbers on data with calculus, and you get systems that recognise faces, transcribe speech and write text. **Deep learning** is training networks with many layers.

This chapter builds it from zero — one neuron, a network, backpropagation with every number worked out, the engineering that makes deep nets train (optimizers, schedules, initialization, normalization, regularization), practical debugging habits — first in NumPy, then in PyTorch (the most popular deep-learning library).

All code is in `code/` and runs on CPU in seconds (PyTorch 2.x, scikit-learn); every output is real.

10.1 From the perceptron to the multi-layer perceptron

What it is

An **artificial neuron** turns several numbers into one:

1. **Weighted sum:** each input x_i times a **weight** w_i (how much and in which direction it matters), plus a **bias** b (a baseline shift): $z = w \cdot x + b$, the **pre-activation** or **logit**.
2. **Activation:** a fixed function bends z into $a = f(z)$ — e.g. ReLU keeps positives and zeroes negatives.

The output feeds the next layer or is the prediction.

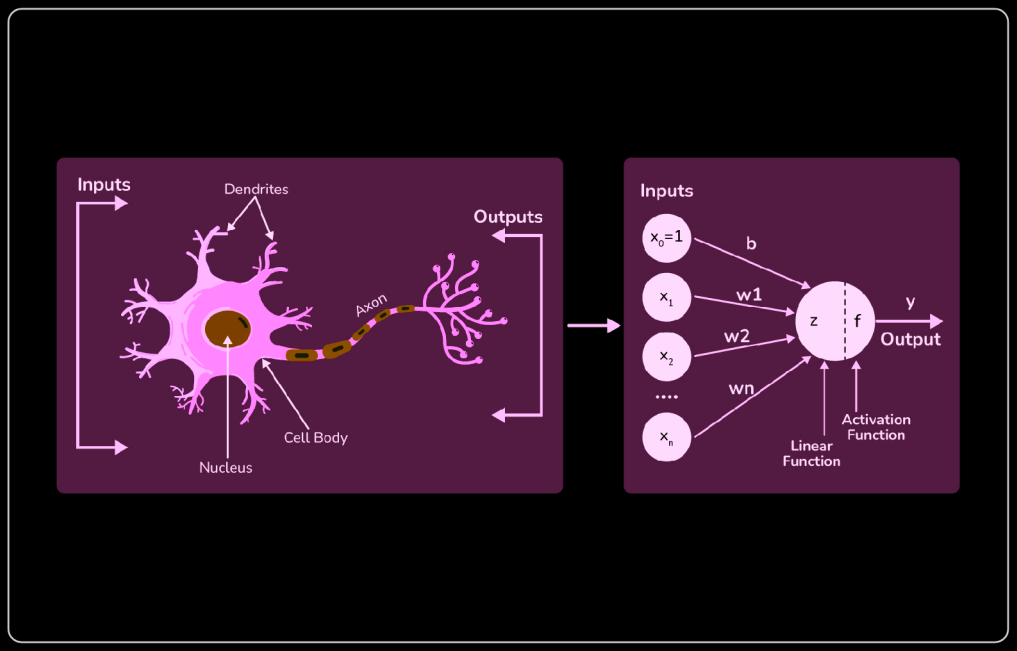


Figure — Left: a biological neuron (dendrites collect signals, the axon sends one out). Right: the artificial version — inputs $x_1 \ldots x_n$ times weights, plus bias b (drawn as a constant input $x_0 = 1$), summed into z, then passed through activation f (course diagram).

Biology is only inspiration — neurons are differentiable arithmetic (slopes computable for every number).

Glossary for this chapter

Term	Meaning
Parameters	All learned weights and biases
Layer	Neurons reading the same inputs; a **dense** (fully connected, linear) layer links every input to every neuron
Input / hidden / output layer	Raw features / any layer between (values never in the data) / the layer producing the prediction
Hyperparameters	Choices you make: depth, width, learning rate, batch size…
Epoch	One full pass over the training set
Batch / mini-batch	The examples used for one weight update

The perceptron (1958) and its limit

Frank Rosenblatt's **perceptron** is one neuron with a **step** activation (1 if $w \cdot x + b > 0$, else 0). When wrong, nudge toward the answer: $w \leftarrow w + \eta(y - \hat{y})x$, $b \leftarrow b + \eta(y - \hat{y})$ (η = learning rate).

One neuron draws a **straight line** (a **hyperplane** in higher dimensions) through feature space: fine for AND, impossible for **XOR** ("exactly one input is 1"), whose classes no single line separates — made famous by Minsky and Papert's 1969 book *Perceptrons*.

```
# code/perceptron.py (core)
for epoch in range(epochs):                        # w = zeros, b = 0, lr = 1.0
    for xi, yi in zip(X, y):
        pred = 1 if xi @ w + b > 0 else 0          # step activation
        update = lr * (yi - pred)                  # 0 if correct, +-1 if wrong
        w += update * xi; b += update
    # ... stop ("converged") after an epoch with no errors
```

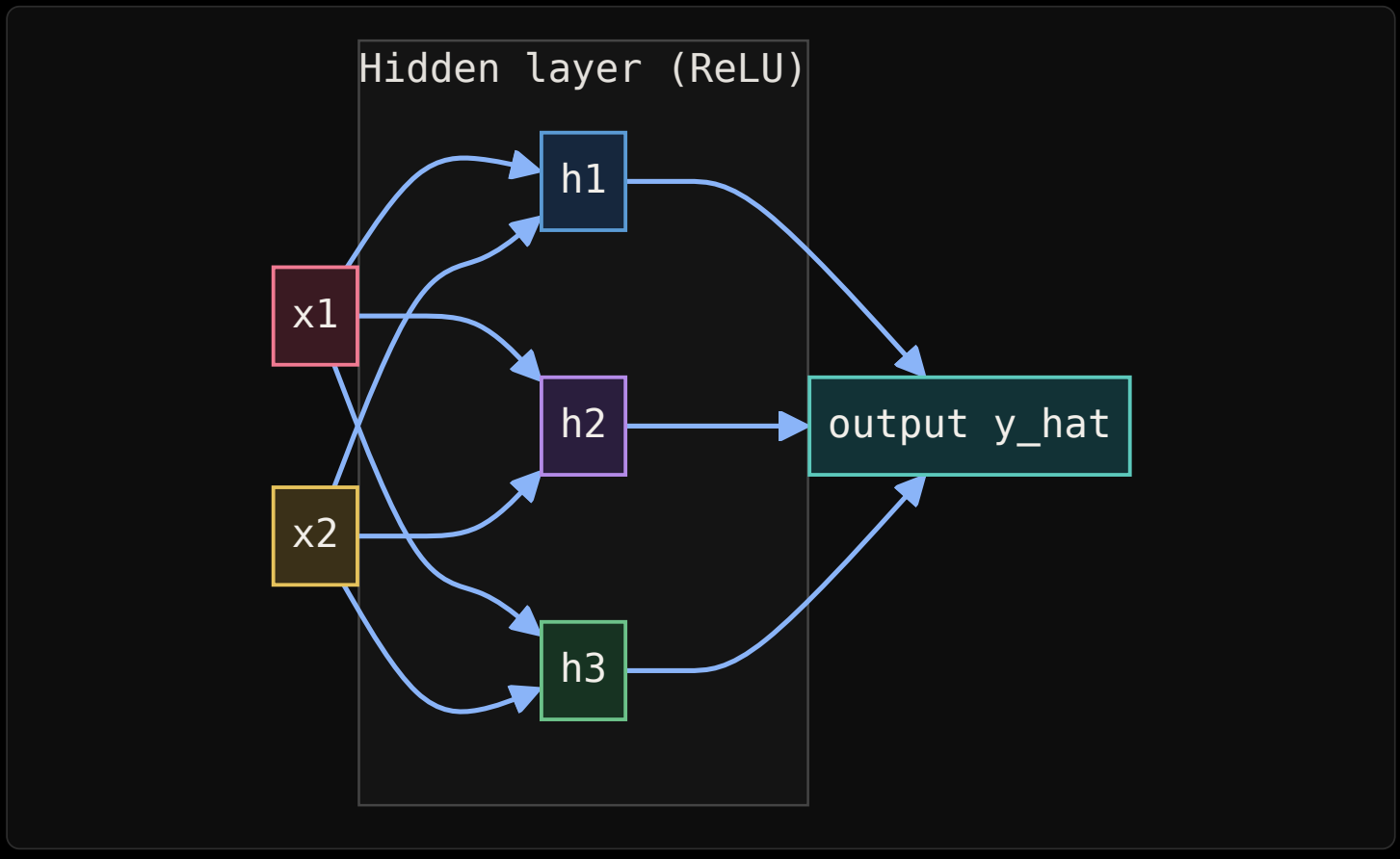
Output:

```
AND: converged=True after 6 epochs, w=[2. 1.], b=-2.0, preds=[0 0 0 1], target=[0 0 0 1]
XOR: converged=False after 20 epochs, w=[-1.  0.], b=1.0, preds=[1 1 0 0], target=[0 1 1 0]
```

The multi-layer perceptron (MLP)

Stack neurons: a **hidden layer** transforms inputs into new features for the output neuron. With two hidden neurons XOR is easy: one learns "x_1 OR x_2", another "x_1 AND x_2", the output "OR but not AND".

A network feeding forward with no loops is a **feed-forward network**; with all-dense layers it is a **multi-layer perceptron (MLP)** — the name stuck though modern MLPs use ReLU-like activations.



In matrix form a layer is **h = f(W x + b)** (one row of W per neuron); for a batch in rows of X, $H = f(X W^T + b)$. Dense-layer parameters = inputs × outputs + outputs; a 2 → 3 → 1 network has $(2 \cdot 3 + 3) + (3 \cdot 1 + 1) = 13$.

Why nonlinearity is essential

Without activations, $W_2(W_1 x + b_1) + b_2 = (W_2 W_1)x + (W_2 b_1 + b_2)$ — still one linear function. The **universal approximation theorem** (Cybenko 1989, Hornik 1991): one hidden layer with enough neurons can approximate any continuous function on a bounded region arbitrarily well — saying nothing about how many neurons or whether training finds them.

Why depth matters (hidden layers and "deep" networks)

Hidden layers build progressively more **abstract representations** (edges → textures and parts → objects). This **feature hierarchy** reuses lower-level pieces, so a deep net needs far fewer neurons than a wide shallow one. Depth costs computation, data and harder optimization (**gradients** — slopes telling each weight which way to move — must cross every layer; §10.6); ReLU, careful init, normalization, **residual connections** (adding a layer's input to its output; Chapter 11), GPUs and big datasets like ImageNet made deep nets trainable after ~2010.

Trade-offs

	Neural networks (MLP and beyond)	Gradient-boosted trees / linear models
Best data type	Images, audio, text, sequences — raw, high-dimensional, unstructured	Tabular rows and columns
Feature engineering	Learned automatically	Mostly manual
Data needed	Usually a lot	Works on small data
Compute	High (GPUs at real sizes)	Low
Interpretability	Low	Medium–high
Tuning effort	Many hyperparameters	Fewer

When not to use it

- Small tabular datasets: gradient-boosted trees (Chapter 7) usually win in seconds.
- Decisions that must be explained to a regulator when a simple model suffices, or no compute budget.

Common mistakes

- Forgetting activations between layers (collapses to linear).
- "More layers is always better" — deeper nets overfit and train harder without the tricks below.

Real-world examples

Google Photos search, Face ID, self-driving perception (CNNs — convolutional networks for images — and transformers — attention-based sequence networks; Chapters 11–12); voice-assistant speech recognition; YouTube and Meta recommenders running MLPs on learned **embeddings** (vectors representing each user or item; e.g. Meta's open-source DLRM).

In the interview

> **30-second answer:** "A neuron computes a weighted sum plus bias and applies a nonlinear activation. An MLP stacks layers; hidden layers learn intermediate features and depth composes simple features into complex ones. Without nonlinearities the stack collapses to one linear map. A single perceptron draws one hyperplane, so it can't learn XOR; one hidden layer can."

- *Parameters in a dense 256 → 128 layer?* 256·128 + 128 = 32,896.
- *What does universal approximation guarantee?* That a one-hidden-layer approximator exists — not that it's small or trainable.

Self-check

1. What does a bias do? → Shifts the threshold, so a neuron can fire with all-zero inputs (or stay quiet with large ones).

TL;DR

Neuron = weighted sum + bias + nonlinearity; MLP = layers of them. Hidden layers learn features, depth builds a hierarchy, nonlinearity makes stacking useful.

10.2 Activation functions

What it is

The **activation function** is the nonlinearity applied to each pre-activation; it governs gradient flow and, at the output, what values the network can produce.

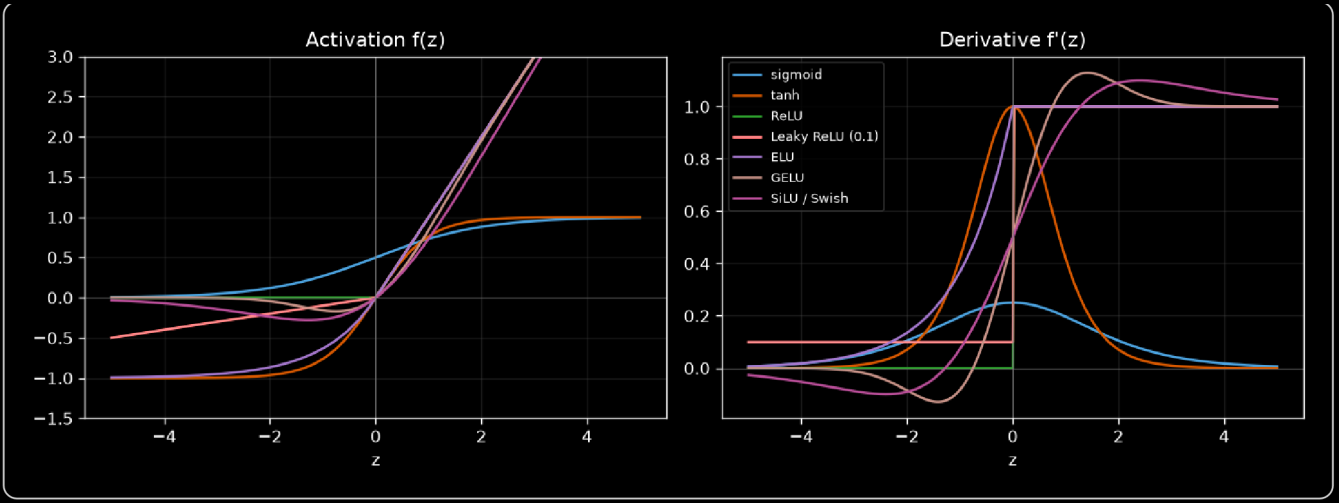


Figure — Common activations (left) and their derivatives (right), computed with PyTorch autograd (§10.4) by `code/activations.py`. The sigmoid's derivative never exceeds 0.25, and sigmoid and tanh flatten to 0 for large |z| ("saturation").

The table

Name	Formula	Output range	Derivative	Where used	Pros	Cons
Sigmoid σ	1 / (1 + e^−z)	(0, 1)	σ(z)(1 − σ(z)), max 0.25	Binary / multi-label outputs; LSTM gates	Reads as a probability	Saturates; not zero-centred
Tanh	(e^z − e^−z)/(e^z + e^−z)	(−1, 1)	1 − tanh²(z), max 1	RNN/LSTM hidden states	Zero-centred	Saturates for large |z|
ReLU	max(0, z)	[0, ∞)	1 if z > 0, else 0	Default hidden layer	Cheap, no saturation for z > 0	"Dying ReLU": stuck at z < 0, zero gradient forever
Leaky ReLU (PReLU if α is learned)	z if z > 0 else αz (α≈0.01)	(−∞, ∞)	1 or α	Hidden layers when ReLUs die	Never exactly zero gradient	Extra hyperparameter α
ELU	z if z > 0 else α(e^z − 1)	(−α, ∞)	1 or αe^z	Hidden layers	Smooth, mean nearer 0	exp is slower
GELU	z·Φ(z) (Φ = standard normal CDF)	≈(−0.17, ∞)	Smooth	Transformers (BERT, GPT)	Smooth ReLU-like	More compute
SiLU / Swish	z·σ(z)	≈(−0.28, ∞)	Smooth	EfficientNet; LLM feed-forward (SwiGLU)	Smooth	More compute
Softmax	softmax$_i$(z) = e^z$_i$ / Σ$_j$ e^z$_j$	each (0,1), sum = 1	—	Single-label multi-class output	Probability distribution	Output layer only
Identity (linear)	z	(−∞, ∞)	1	Regression output	Unbounded	—

Key terms: *RNNs/LSTMs* (recurrent networks; LSTMs have "gates" deciding what to remember) process sequences step by step (Chapter 12). *Transformers* are the attention networks behind *BERT*, *GPT* and *LLMs* (large language models); *EfficientNet* is an image-network family. *Zero-centred* = outputs spread around 0; *saturates* = curve goes flat, slope ≈ 0. Φ (standard normal *CDF*) = probability a bell-curve value is below z. *SwiGLU* = gated SiLU.

From `code/activations.py`:

```
sigmoid  f(-2,0,2)=[0.119 0.5   0.881]  f'=[0.105 0.25  0.105]
tanh     f(-2,0,2)=[-0.964  0.     0.964]  f'=[0.071 1.    0.071]
ReLU     f(-2,0,2)=[0. 0. 2.]  f'=[0. 0. 1.]
softmax([2,1,0.1]) = [0.659 0.242 0.099]
```

Softmax (worked in Chapter 5, section 5.9): z = (2, 1, 0.1) → (0.659, 0.242, 0.099). Adding a constant to all scores changes nothing, so libraries subtract max(z) first to avoid overflow.

How to choose

1. **Hidden layers: ReLU** (Leaky ReLU if many units die); **GELU/SiLU** in transformers; tanh/sigmoid only inside recurrent gates (Chapter 12).
2. **Output layer** by task:

Task	Output activation	Loss
Regression	None (identity)	MSE / MAE / Huber
Binary classification	Sigmoid (1 unit)	Binary cross-entropy
Multi-class, one label	Softmax (K units)	Categorical cross-entropy
Multi-label (several true)	Sigmoid per label	Binary cross-entropy per label

In PyTorch the output layer usually emits **raw logits** and the loss applies sigmoid/softmax internally (`BCEWithLogitsLoss`, `CrossEntropyLoss`) — more numerically stable.

Common mistakes

- `softmax` in the model plus `nn.CrossEntropyLoss` (which already applies log-softmax) — trains poorly.
- Sigmoid in every hidden layer of a deep net → vanishing gradients.
- ReLU on a regression output that must go negative.

Real-world examples

AlexNet (2012) popularised ReLU (~6× faster training than tanh on CIFAR-10); BERT and GPT-2/3 use GELU; EfficientNet SiLU; LLaMA SwiGLU in its feed-forward blocks.

In the interview

> **30-second answer:** "Activations add nonlinearity. ReLU is the hidden-layer default — cheap, no saturation for positive inputs — with dead units as its weakness (Leaky ReLU helps). Sigmoid and tanh saturate, so they live at outputs and in RNN gates. GELU/SiLU are the smooth transformer defaults. Output: identity for regression, sigmoid for binary/multi-label, softmax for multi-class."

- *ReLU's gradient at 0?* z is exactly 0 with ~zero probability; frameworks define it (PyTorch: 0).
- *Softmax vs sigmoid for 3 classes?* Softmax if exactly one is true; independent sigmoids if several can be.

TL;DR

ReLU (GELU in transformers) inside; identity / sigmoid / softmax at the output by task; sigmoid and tanh saturate and cause vanishing gradients in deep stacks.

10.3 The forward pass and loss functions

What it is

The **forward pass** runs a batch through the layers (h = f(W h_prev + b)) to a **prediction** ŷ or raw logits ("P(cat) = 0.92"). The **loss function** compares ŷ with y, returning one number averaged over the batch; training minimises the average training loss.

Loss functions

Loss	Formula (one example)	Task	Notes
MSE	$(\hat{y} - y)^2$	Regression	Punishes big errors; outlier-sensitive. `nn.MSELoss`
MAE / L1	$\lvert\hat{y} - y\rvert$	Regression	Outlier-robust; constant gradient size. `nn.L1Loss`
Huber / Smooth L1	$\frac{1}{2}(\hat{y}-y)^2$ if $\lvert\hat{y}-y\rvert \le \delta$ else $\delta(\lvert\hat{y}-y\rvert - \frac{1}{2}\delta)$	Regression	Quadratic near 0, linear far away. `nn.HuberLoss`
Binary cross-entropy (BCE)	$-[y \log p + (1-y) \log(1-p)]$	Binary / multi-label	$p = \sigma(\text{logit})$. `nn.BCEWithLogitsLoss` takes logits
Categorical cross-entropy	$-\log p_{\text{true}}$	Multi-class	$p = \text{softmax}(\text{logits})$. `nn.CrossEntropyLoss` takes logits + integer labels
Hinge	$\max(0, 1 - y \cdot s)$, $y \in \{-1, +1\}$	Max-margin classification	SVM loss; rare in deep learning

Worked cross-entropy. Probabilities (0.7, 0.2, 0.1), true class 0: $-\ln 0.7 = 0.357$; true class 2: $-\ln 0.1 = 2.303$ — confident wrong answers punished hardest. A uniform guess over K classes has loss $\ln K$ ($\ln 10 = 2.303$ for digits) — a sanity check for the first step.

Why not MSE for classification? Cross-entropy's gradient at the logits is **$p - y$** (§10.4), large when confidently wrong; MSE through a sigmoid is multiplied by σ', tiny when saturated — learning stalls when most needed. Cross-entropy is also the maximum-likelihood loss for a Bernoulli or categorical model.

Common mistakes

- Probabilities into `CrossEntropyLoss` / `BCEWithLogitsLoss` (they expect logits).
- Mixed target formats: `CrossEntropyLoss` takes integer indices (`long`, shape N) *or* class probabilities (`float`, N×K).
- Accuracy as a training loss — zero gradient almost everywhere.

Self-check

1. First-step loss, 4 classes, sensible init? → ~$\ln 4 = 1.386$.

TL;DR

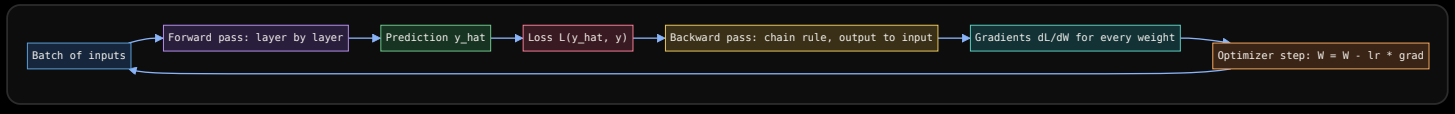
Forward pass = predictions layer by layer. MSE/MAE/Huber for regression, BCE for binary/multi-label, cross-entropy for multi-class — fed logits in PyTorch.

10.4 Backpropagation

What it is

Backpropagation computes the **gradient** of the loss for every weight ("nudge this up a bit — how does the loss change?") by applying the **chain rule** from the output backwards, reusing intermediate results, at only ~2–3× the cost of a forward pass regardless of parameter count. Popularised for neural nets by Rumelhart, Hinton and Williams (1986). Fundamentals: Chapter 2, section 2.4.

Backprop only computes gradients; **gradient descent** uses them: $\theta \leftarrow \theta - \eta \cdot \partial L/\partial \theta$ ($\partial L/\partial \theta$ = **partial derivative**, the slope w.r.t. θ alone; η = **learning rate**).



This forward → loss → backward → update loop over mini-batches and epochs is **how every neural network learns**.

The chain rule in one line

If $L = g(b)$ and $b = h(a)$, then $\partial L/\partial a = \partial L/\partial b \cdot \partial b/\partial a$. Backprop multiplies local derivatives from the loss back to each weight; each layer needs only the incoming gradient and its own local derivative.

Worked example with numbers

2 inputs → 2 hidden (ReLU) → 1 output (sigmoid), BCE loss. $x = (1, 2)$, $y = 1$; $W_1 = [[0.5, -0.2], [0.3, 0.4]]$ (row j = weights into hidden unit j), $b_1 = (0.1, -0.5)$; $w_2 = (0.7, -0.6)$, $b_2 = 0.2$; $\eta = 0.1$.

Forward pass

Step	Computation	Value
z_1 (hidden unit 1)	$0.5 \cdot 1 + (-0.2) \cdot 2 + 0.1$	0.2
z_1 (hidden unit 2)	$0.3 \cdot 1 + 0.4 \cdot 2 - 0.5$	0.6
$h = \text{ReLU}(z_1)$	Both positive, unchanged	(0.2, 0.6)
z_2	$0.7 \cdot 0.2 + (-0.6) \cdot 0.6 + 0.2$	−0.02
$\hat{y} = \sigma(z_2)$	1/(1 + e^0.02)	0.4950
$L = -\ln \hat{y}$	(y = 1)	0.7032

Backward pass (previous gradient × local derivative)

Gradient	Rule	Value
$\partial L/\partial z_2$	Sigmoid + BCE simplifies to $\hat{y} - y$	−0.5050
$\partial L/\partial w_2$	$\partial L/\partial z_2 \cdot h$	(−0.101, −0.303)
$\partial L/\partial b_2$	$\partial L/\partial z_2 \cdot 1$	−0.5050
$\partial L/\partial h$	$\partial L/\partial z_2 \cdot w_2$	(−0.3535, 0.303)
$\partial L/\partial z_1$	$\partial L/\partial h \cdot \text{ReLU}'(z_1)$, ReLU' = 1 (both $z_1 > 0$)	(−0.3535, 0.303)
$\partial L/\partial W_1$	$\partial L/\partial z_1$ (column) × x^T (row) — outer product	[[−0.3535, −0.707], [0.303, 0.606]]
$\partial L/\partial b_1$	$\partial L/\partial z_1$	(−0.3535, 0.303)

Why $\hat{y} - y$? $\partial L/\partial \hat{y} = -y/\hat{y} + (1-y)/(1-\hat{y})$ and $\sigma'(z) = \hat{y}(1 - \hat{y})$; their product cancels to $\hat{y} - y$. Softmax + cross-entropy likewise gives $\partial L/\partial \text{logits} = p - \text{one_hot}(y)$ (1 at the true class, 0 elsewhere).

Reading the gradients. $\partial L/\partial w_2 < 0$, so descent *increases* w_2, pushing $\hat{y}$ toward 1. Hidden unit 2 has a negative outgoing weight, so the network wants it *smaller*: its incoming weights get positive gradients and shrink.

Update ($\eta = 0.1$): $w_2 \leftarrow (0.7, -0.6) - 0.1 \cdot (-0.101, -0.303) = (0.7101, -0.5697)$. After updating everything, $\hat{y} = 0.5756$ and loss 0.5523 (from 0.7032).

The script checks the hand gradients against PyTorch's automatic differentiation:

```
# code/backprop_by_hand.py (backward part)
dz2 = y_hat - y                 # dL/dz2 for sigmoid + BCE
dw2 = dz2 * h;  db2 = dz2
dh  = dz2 * w2
dz1 = dh * (z1 > 0)             # ReLU'(z1)
dW1 = np.outer(dz1, x);  db1 = dz1
# ... then the same network in torch, loss.backward(), and np.allclose on every gradient
```

Output (last lines; the rest repeats the tables):

```
autograd agrees with hand-derived gradients: True
after one SGD step (lr=0.1): y_hat 0.4950 -> 0.5756, loss 0.7032 -> 0.5523
```

A 2-layer network from scratch in NumPy

`code/numpy_mlp.py` does this for a whole batch (rows = examples), tanh hidden units, sigmoid output, **full-batch** gradient descent (each update uses the whole training set). *fan_in* = inputs feeding a layer (*fan_out* = its outputs).

```python
class TwoLayerNet:
    """x -> tanh(x W1 + b1) -> sigmoid(h W2 + b2). Rows of X are examples."""
    # __init__: W1, W2 ~ N(0, 1/fan_in) (LeCun-style scale), biases 0
    def forward(self, X):
        self.X = X
        self.H = np.tanh(X @ self.W1 + self.b1)
        self.P = 1 / (1 + np.exp(-(self.H @ self.W2 + self.b2)))
        return self.P

    def backward(self, y):
        n = len(y)
        dZ2 = (self.P - y[:, None]) / n                 # BCE + sigmoid, averaged over batch
        dW2 = self.H.T @ dZ2;  db2 = dZ2.sum(0)
        dH = dZ2 @ self.W2.T
        dZ1 = dH * (1 - self.H ** 2)                     # tanh'(z) = 1 - tanh(z)^2
        dW1 = self.X.T @ dZ1;  db1 = dZ1.sum(0)
        return dW1, db1, dW2, db2

    def step(self, grads, lr):
        for p, g in zip((self.W1, self.b1, self.W2, self.b2), grads):
            p -= lr * g
```

`forward` stores X, H, P because backward needs them (why training uses more memory than **inference**, i.e. just predicting). A weight matrix's gradient is always (layer input)^T @ (gradient at the layer's output). The script also runs a **numerical gradient check** (nudge a weight by ±ε, measure the loss change) — the standard test for a hand-written backward pass.

Output:

```
XOR  loss 0.687 -> 0.0012, probs = [0.    0.999 0.999 0.002]
Moons loss 0.682 -> 0.0363, train acc 0.986, test acc 0.956
gradient check  numerical: [-0.02769  -0.015654 -0.024803]  analytical: [-0.02769  -0.015654 -0.024803]
```

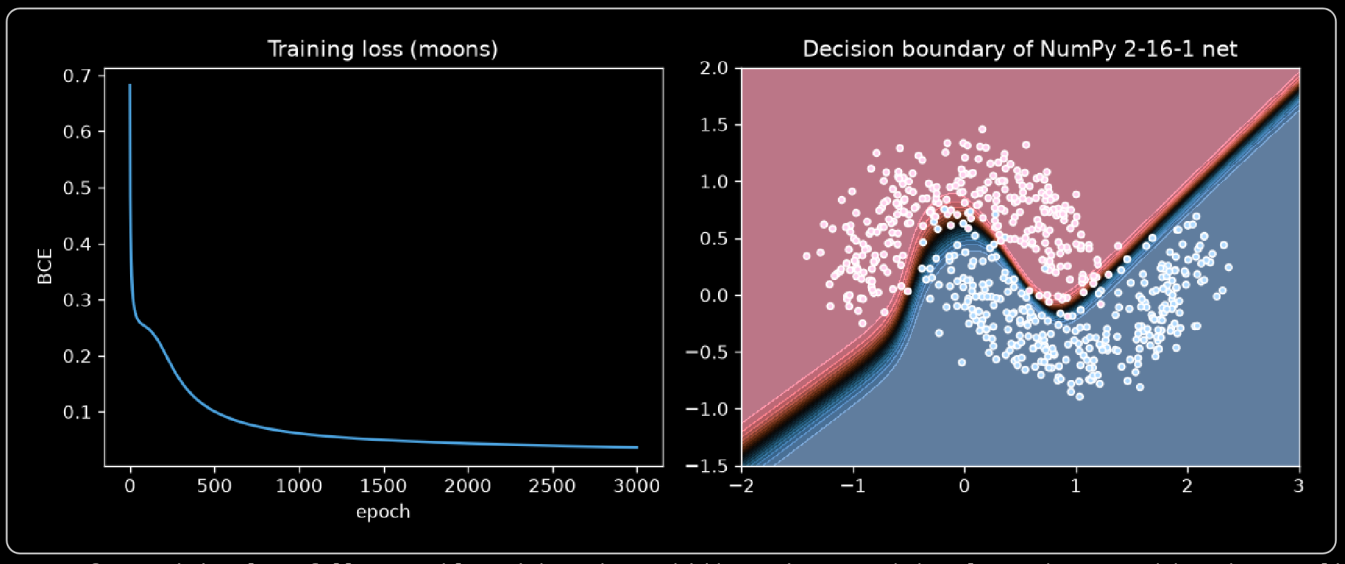


Figure — Left: training loss falls smoothly. Right: the 16-hidden-unit network has learned a curved boundary no linear model could draw.

Computational graphs and autograd

Frameworks record every **tensor** (multi-dimensional array) operation in a **computational graph**; `loss.backward()` walks it in reverse, accumulating into each parameter's `.grad`. This is **reverse-mode automatic differentiation** — exact, and cheap when one scalar output (the loss) depends on many inputs. Karpathy's *micrograd* builds such an engine in ~100 lines.

Common mistakes

- Not zeroing gradients between PyTorch steps (they **accumulate**); summing instead of averaging the batch loss (effective LR then scales with batch size); hand-written backward passes without a gradient check.

In the interview

30-second answer: "Backprop applies the chain rule from the loss backwards through the computational graph, getting $\partial L/\partial \theta$ for every parameter for a small constant multiple of a forward pass. Each layer multiplies the

incoming gradient by its local derivative. An optimizer then steps against the gradient. For sigmoid+BCE or softmax+cross-entropy, the gradient at the logits is prediction minus target."

- *Why reverse mode?* One scalar output, millions of inputs: one backward sweep gets every partial; forward mode needs one sweep per input.
- *What must the forward pass store?* Activations the local derivatives need — so training memory grows with batch size and depth (**gradient checkpointing** trades recompute for memory).

TL;DR

Forward computes the loss; backward applies the chain rule layer by layer; gradient descent subtracts LR × gradient. Autograd automates the backward pass.

10.5 Optimizers and learning-rate schedules

What it is

An **optimizer** turns gradients into updates. The **learning rate** is the single most important deep-learning hyperparameter: too big overshoots, too small crawls; schedules and adaptive optimizers move fast first and slowly near the target.

How the main optimizers work

g = mini-batch gradient, η = learning rate, θ = a parameter. (Plain GD and its batch/mini-batch/stochastic variants: Chapter 2, section 2.5.)

Optimizer	Update rule (per parameter)	Idea	PyTorch defaults
SGD (stochastic gradient descent)	$\theta \leftarrow \theta - \eta\, g$	Step on a mini-batch gradient estimate	`SGD(lr=…)` — always set lr
SGD + momentum	$v \leftarrow \mu\, v + g$; $\theta \leftarrow \theta - \eta\, v$	Running "velocity": consistent directions accelerate, zig-zags cancel	μ typically 0.9
Nesterov momentum	Gradient at the look-ahead point	Better-behaved momentum	`nesterov=True`
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta\, g/(\sqrt{s} + \varepsilon)$	Per-parameter step, shrinking for frequently updated params	Good for sparse features; step decays too much for deep nets
RMSProp	$s \leftarrow \rho\, s + (1-\rho)\, g^2$; $\theta \leftarrow \theta - \eta\, g/(\sqrt{s} + \varepsilon)$	AdaGrad with a **moving average** (recent values weighted more), so steps don't vanish	lr 0.01, ρ (`alpha`) 0.99
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $s \leftarrow \beta_2 s + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\, \hat{m}/(\sqrt{\hat{s}} + \varepsilon)$	Momentum + RMSProp + bias correction for the zero start	lr 1e-3, β = (0.9, 0.999), ε = 1e-8
AdamW	Adam plus $\theta \leftarrow \theta - \eta\, \lambda\, \theta$, separate from the gradient	"Decoupled" **weight decay** (shrink every weight toward 0 each step; §10.7) — the correct L2-style shrinkage with Adam	weight_decay 0.01

- **Momentum:** a heavy ball speeding along consistent slopes and damping zig-zags in narrow "ravines".
- **Adaptive methods** (RMSProp, Adam) divide each parameter's step by its recent gradient magnitude: steep, noisy directions take smaller steps, flat ones bigger; mini-batch noise can also help escape sharp minima.
- **Adam's first step** (g = 0.5, β_1 = 0.9, β_2 = 0.999): m = 0.05, s = 0.00025; $\hat{m}$ = 0.5, $\hat{s}$ = 0.25; step = η·0.5/√0.25 = η. Without correction: η·0.05/0.0158 ≈ 3.2η — distorted. The step is ≈η whatever the gradient's size: Adam's steps are scale-free.

`code/optimizers.py` runs four optimizers via `torch.optim` on (a) a 2-D ravine $f(x, y) = 0.05x^2 + 2y^2$ (40× steeper in y), from (−10, 2), and (b) an MLP on two moons, each near its largest well-behaved LR (SGD 0.45, momentum 0.05, RMSProp 0.3, Adam 0.5):

```
p = torch.tensor([-10.0, 2.0], requires_grad=True)
opt = torch.optim.SGD([p], lr=0.05, momentum=0.9)   # or RMSprop / Adam
for _ in range(60):
    opt.zero_grad(); ravine(p).backward(); opt.step()
```

Output:

```
(a) ravine f = 0.05 x^2 + 2 y^2, start (-10, 2), 60 steps
  SGD                final point [-0.631  0.   ]  loss 1.99e-02
  SGD+momentum(0.9)  final point [ 0.358 -0.073]  loss 1.71e-02
  RMSProp            final point [-0.  0.]  loss 3.77e-14
  Adam               final point [-0.215  0.045]  loss 6.42e-03
(b) MLP 2-32-32-1 on moons, 200 full-batch epochs
  SGD                loss after 20 epochs 0.639, after 200 0.265
  SGD+momentum(0.9)  loss after 20 epochs 0.310, after 200 0.083
  RMSProp            loss after 20 epochs 0.171, after 200 0.080
  Adam               loss after 20 epochs 0.274, after 200 0.075
```

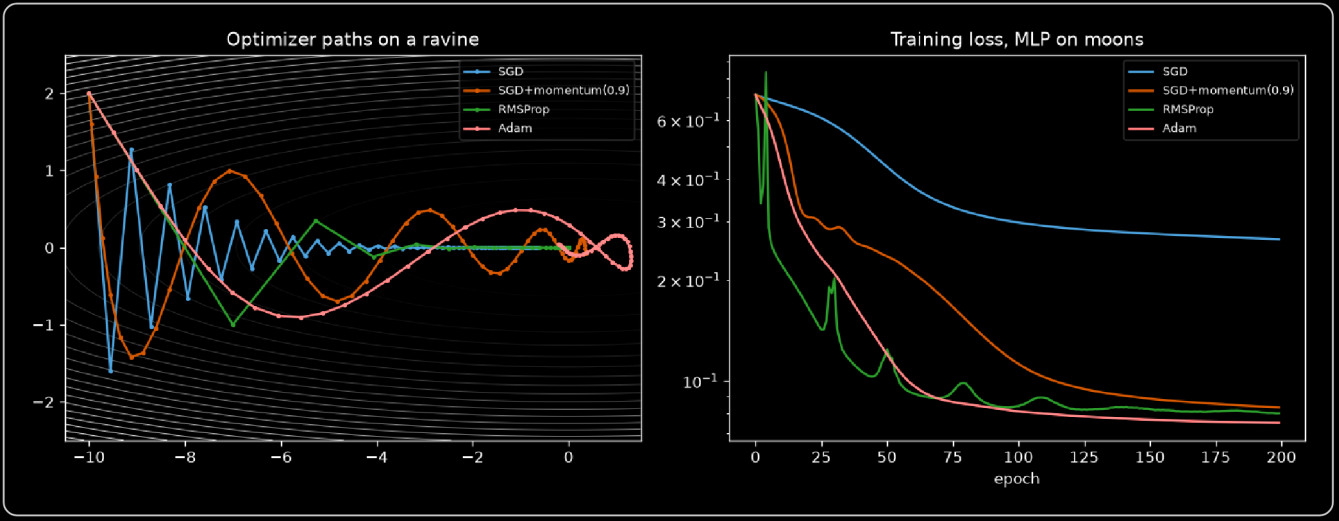


Figure — Left: SGD's LR is capped by the steep y-direction, so it crawls in x; momentum overshoots but travels fast; RMSProp and Adam rescale each coordinate and head almost straight in. Right: momentum and adaptive methods reach low loss far sooner.

RMSProp shines because its per-coordinate scaling exactly undoes this axis-aligned ravine — don't over-read toys. In practice **Adam/AdamW** is the default for transformers and most new models; **SGD with momentum** plus a schedule stays competitive (sometimes generalising slightly better) for CNN image classifiers.

Learning-rate schedules

A **schedule** changes η during training: big steps early, small late.

Schedule	Shape	PyTorch	Typical use
Constant	Flat	—	Baselines, small problems with Adam
Step decay	×γ every k epochs	`StepLR`, `MultiStepLR`	Classic ResNet (landmark image network, Chapter 11) recipe: ÷10 at epochs 30, 60, 90
Exponential decay	×γ every step/epoch	`ExponentialLR`	Simple smooth decay
Reduce on plateau	Drop when validation stalls	`ReduceLROnPlateau`	Unknown epoch count
Cosine annealing	Half cosine from η_max to ~0	`CosineAnnealingLR`	Modern default for vision and LLMs
Linear warmup + decay	Ramp from ~0, then cosine/linear decay	`LinearLR` + `SequentialLR`	Transformers (warmup avoids early Adam instability)
One-cycle	Warm up to max, anneal far below start	`OneCycleLR`	Fast training (Leslie Smith's "super-convergence")

```
# code/lr_schedules.py (warmup + cosine) - each step: opt.step() THEN sched.step()
sched = S.SequentialLR(opt, [S.LinearLR(opt, start_factor=0.01, total_iters=10),
                             S.CosineAnnealingLR(opt, T_max=90)], milestones=[10])
```

Output:

```
Constant                lr at steps 0/10/50/99: 0.1000 0.1000 0.1000 0.10000
StepLR (x0.5 every 25)  lr at steps 0/10/50/99: 0.1000 0.1000 0.0250 0.01250
ExponentialLR (0.97)    lr at steps 0/10/50/99: 0.1000 0.0737 0.0218 0.00490
CosineAnnealingLR       lr at steps 0/10/50/99: 0.1000 0.0976 0.0500 0.00002
Warmup 10 + cosine      lr at steps 0/10/50/99: 0.0010 0.1000 0.0587 0.00003
OneCycleLR              lr at steps 0/10/50/99: 0.0040 0.0295 0.0794 0.00000
```

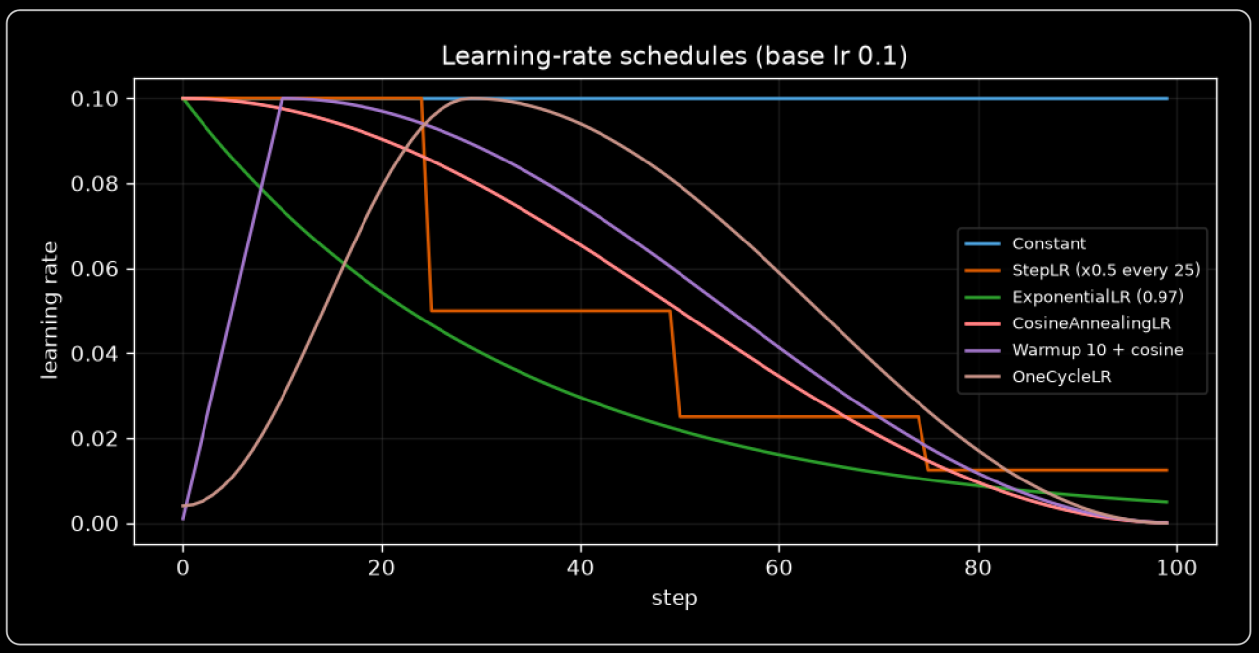


Figure — The six schedules over 100 steps.

Trade-offs

	SGD + momentum	Adam / AdamW
Tuning	LR and schedule matter a lot	Works near lr=1e-3 out of the box
Convergence speed	Slower early	Fast early
Generalization	Often excellent on CNNs	Excellent on transformers (AdamW)
Memory	1 extra buffer per parameter	2 extra buffers per parameter

Common mistakes

- `Adam(weight_decay=…)` expecting AdamW — there decay is added to the gradient (L2) and distorted by adaptive scaling; use `AdamW`.
- `scheduler.step()` before `optimizer.step()` (PyTorch warns; first LR skipped).
- Changing batch size without revisiting LR (SGD heuristic: scale LR roughly linearly with batch size, plus warmup).

Real-world examples

GPT-3 and LLaMA use Adam/AdamW with linear warmup + cosine decay; the original ResNet SGD, momentum 0.9, LR ÷10 on plateaus; fast.ai popularised one-cycle and the LR finder.

In the interview

30-second answer: "SGD steps against the mini-batch gradient; momentum accumulates a velocity so consistent directions speed up and oscillations cancel; RMSProp divides each step by a running RMS (root mean square, a typical size) of that parameter's gradients; Adam combines both with bias correction, and AdamW decouples weight decay. A schedule — typically warmup then cosine decay — gives big steps early and fine steps late."

- *Why warmup with Adam?* Early second-moment estimates are poor and gradients large; a small initial LR prevents destructive first updates.
- *Why bias correction?* m and s start at 0, biasing early averages toward 0; dividing by $1 - \beta^t$ fixes it.

TL;DR

Learning rate first. Adam/AdamW (lr ≈ 3e-4 to 1e-3) is the safe default; SGD+momentum with a schedule is strong for CNNs; warmup + cosine or one-cycle.

10.6 Initialization and vanishing/exploding gradients

The problem it solves

The gradient reaching layer 1 is a **product** of one factor per layer above. Factors below 1 shrink it exponentially with depth — **vanishing gradients** (early layers barely learn); above 1 grow it — **exploding gradients** (huge updates, NaN loss). Forward activations can likewise fade or blow up. Two sources:

1. **Activation derivatives:** sigmoid' ≤ 0.25, so ten sigmoid layers scale the gradient by at most $0.25^{10} \approx 9.5 \times 10^{-7}$.
2. **Weight scale:** badly scaled W shrinks or grows the signal's variance by a constant factor per layer.

How initialization fixes it

Keep the **variance** of activations (forward) and gradients (backward) roughly constant across layers. $z = \Sigma\, w_i x_i$ sums fan_in independent terms, so $\mathrm{Var}(z) = \text{fan_in} \cdot \mathrm{Var}(w) \cdot \mathrm{Var}(x)$; $\mathrm{Var}(w) = 1/\text{fan_in}$ keeps $\mathrm{Var}(z) = \mathrm{Var}(x)$ (fan_in = 256 → std 0.0625). Xavier also balances the backward pass (fan_out); He doubles the variance for ReLU:

Scheme	Weight variance	Designed for	PyTorch
Xavier / Glorot (2010)	2 / (fan_in + fan_out) (normal version)	tanh, sigmoid, linear	`nn.init.xavier_normal_` / `xavier_uniform_`
He / Kaiming (2015)	2 / fan_in	ReLU family (ReLU zeroes half the signal, so double the variance)	`nn.init.kaiming_normal_(w, nonlinearity="relu")`
LeCun	1 / fan_in	SELU (self-normalising activation), linear	`kaiming_normal_(w, nonlinearity="linear")`

Biases usually start at 0. **Never give all weights one value** (e.g. zeros): every neuron in a layer computes the same thing with the same gradient forever — **symmetry** never breaks. PyTorch's `nn.Linear` default (Kaiming-uniform with a = √5, i.e. U(−1/√fan_in, 1/√fan_in)) is fine for small nets; explicit He init is common for deep ReLU nets.

`code/init_and_gradients.py` pushes random data through a **20-layer, 256-wide MLP**, measuring activation spread and the gradient at layer 1:

```
tanh  + small N(0, 0.01)  activation std layer1=0.163 layer20=1.22e-16   |grad W1|=7.05e-18
tanh  + large N(0, 1)     activation std layer1=0.975 layer20=0.974   |grad W1|=8.04e+07
tanh  + Xavier            activation std layer1=0.63 layer20=0.16   |grad W1|=0.0255
...
ReLU  + large N(0, 1)     activation std layer1=14.6 layer20=8.73e+20   |grad W1|=inf
ReLU  + Xavier            activation std layer1=0.911 layer20=0.000722   |grad W1|=9.78e-06
ReLU  + He (Kaiming)      activation std layer1=1.29 layer20=0.739   |grad W1|=0.292
```

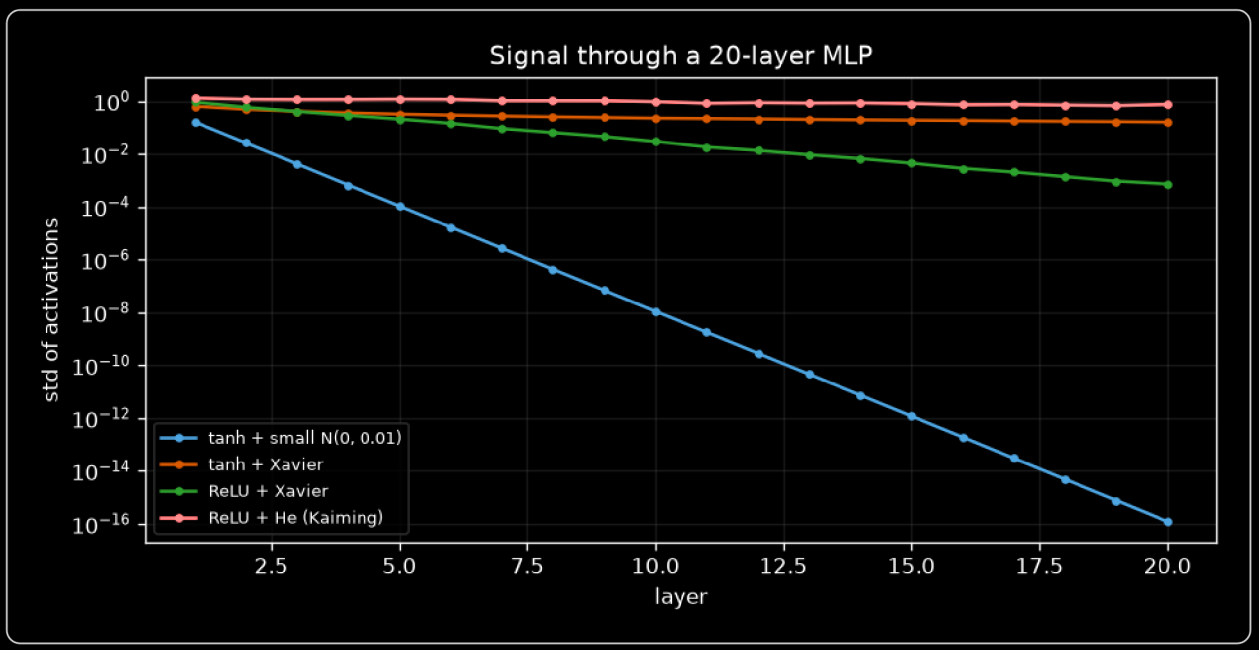


Figure — With too-small weights the signal dies within a few layers; ReLU with Xavier init shrinks by about half per layer; ReLU with He init keeps it steady.

Too-small weights → signal and gradient vanish (10^{-18}); too-large → tanh saturates and gradients explode, ReLU overflows to infinity; He for ReLU keeps everything near 1. (Xavier + tanh still decays because tanh squashes; PyTorch's tanh gain 5/3, `nn.init.calculate_gain("tanh")`, compensates.)

The full toolbox against vanishing/exploding gradients

Fix	How it helps
ReLU-family activations	Derivative exactly 1 for positive inputs
Good initialization (He/Xavier)	Variance ≈ constant across layers at the start
Normalization layers (BatchNorm, LayerNorm — §10.7)	Re-centre and re-scale activations every layer
Residual (skip) connections (Chapter 11)	Identity "highway" for gradients: y = x + F(x)
Gradient clipping	Caps the gradient **norm** (overall length) (`clip_grad_norm_(params, 1.0)`); standard for RNNs and transformers
Gated units (LSTM/GRU — Chapter 12)	Additive memory paths through time

Common mistakes

- All-zero hidden-weight init; very deep custom nets with default init, no normalization or residuals.
- Not monitoring per-layer gradient norms — they diagnose most "loss won't move" bugs.

In the interview

30-second answer: "Gradients are products of per-layer factors, so in deep nets they shrink or grow exponentially — caused by saturating activations (sigmoid' ≤ 0.25) and badly scaled weights. Fixes: ReLU, variance-preserving init (Xavier for tanh, He for ReLU), normalization, residual connections, and clipping for explosions."

- *Why He's factor 2?* ReLU zeroes about half the inputs, halving variance.
- *Why not zero init?* Symmetry — identical neurons get identical gradients.

Self-check

1. He variance for fan_in = 256? → 2/256 ≈ 0.0078 (std ≈ 0.088).

TL;DR

Multiplied factors → vanishing/exploding gradients. ReLU + He (tanh + Xavier), normalization, residuals, clipping.

10.7 Regularization: fighting overfitting

The problem it solves

A big network can **memorise** a small training set — **overfitting** (training loss falls, validation loss rises; bias–variance in Chapter 6, section 6.7). **Regularization** is anything improving generalisation, usually by limiting how freely the model fits.

`code/regularization.py` deliberately overfits a 3-hidden-layer, 256-unit MLP (~133k parameters) on only **80** noisy two-moons points (validated on 2,000), then tries each remedy (each epoch: `model.train()` — dropout on, BatchNorm on batch statistics — with optional input jitter; then `model.eval()` for validation; early stopping keeps the best weights). Output:

```
No regularization            epochs=400  train acc=1.000  val acc=0.831  val loss=2.296
Dropout p=0.5                epochs=400  train acc=0.975  val acc=0.840  val loss=1.290
Weight decay 1.0 (AdamW)     epochs=400  train acc=1.000  val acc=0.826  val loss=0.792
BatchNorm                    epochs=400  train acc=0.938  val acc=0.858  val loss=0.399
Input-noise augmentation     epochs=400  train acc=0.925  val acc=0.858  val loss=0.422
Early stopping (patience 30) epochs= 48  train acc=0.887  val acc=0.872  val loss=0.317
```

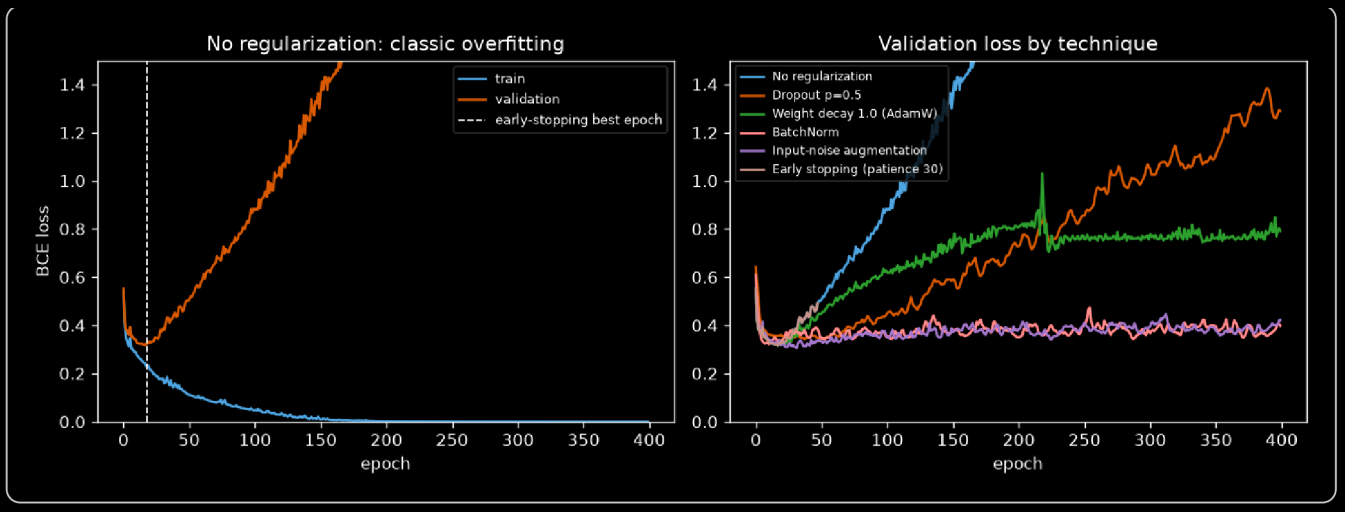


Figure — Left: unregularized, validation loss bottoms near epoch 18 then climbs while training loss → 0. Right: each technique flattens the validation curve.

Every method improves validation **loss**; accuracy gains are modest (early stopping, augmentation and BatchNorm help most). Real projects combine several.

Dropout

In training each unit's output is zeroed with probability p (0.5 for large dense layers, 0.1–0.3 elsewhere), independently per forward pass, so a different "thinned" network trains each step. Units can't form fragile **co-adaptations** and learn redundant robust features — roughly an ensemble of sub-networks. PyTorch's `nn.Dropout(p)` is **inverted dropout**: survivors are scaled by 1/(1 − p) in training so the expectation is unchanged, and `model.eval()` disables it. E.g. p = 0.5: (2, 4, 6, 8) → (0, 8, 12, 0).

Weight decay (L2 regularization)

Adds λ·Σw² to the loss — for SGD, shrinking each weight by (1 − 2ηλ) per step (the penalty's gradient is 2λw). Smaller weights give smoother functions less able to chase noise. With Adam use **AdamW**. Typical λ: 1e-4 to 1e-2 (SGD L2), 0.01–0.1 (AdamW); biases and normalization parameters often excluded.

Batch normalization

What it is. A layer (Ioffe & Szegedy, 2015) normalising each feature over the **mini-batch** to mean 0, variance 1, then applying learnable scale γ and shift β:

$\hat{x} = (x - \mu_{batch}) / \sqrt{(\sigma^2_{batch} + \varepsilon)}$, $y = \gamma \hat{x} + \beta$

Worked: batch values (2, 4, 6): μ = 4, σ² = 2.667, $\sqrt{(\sigma^2 + \varepsilon)} \approx 1.633$, $\hat{x}$ = (−1.225, 0, 1.225); γ = 2, β = 0.5 → y = (−1.95, 0.5, 2.95). γ, β let the network undo the normalization if useful.

Training uses batch statistics and updates **running averages** of μ, σ²; evaluation uses the running averages (so single-example inference works).

Benefits: faster, more stable training, **higher usable learning rates**, less init sensitivity, mild **regularizing** noise (each example normalised with a random batch's statistics). The paper credited reducing "internal covariate shift" (layer inputs changing distribution as earlier layers learn); Santurkar et al. (2018) argue it mainly smooths the loss landscape.

Limits: noisy with tiny batches, awkward for variable-length sequences. **LayerNorm** (normalise across one example's features) is standard in transformers and RNNs; **GroupNorm** suits small-batch vision.

Early stopping

Evaluate validation loss each epoch, keep the best weights, stop after `patience` epochs without improvement and restore them — free, and the best single method in the demo. The validation set now drives a decision, so report final numbers on a separate test set. (General treatment: Chapter 6, section 6.10.)

Data augmentation

Extra examples via label-preserving transformations: flips, crops, small rotations, colour jitter (images); time shifts, noise (audio); synonym swaps or **back-translation** (translate to another language and back) for text. Here, small Gaussian noise on the 2-D inputs; image augmentation in Chapter 11.

Trade-offs

Technique	Knob	Cost / watch out
Dropout	p	Slower convergence; needs `eval()`; less used with BatchNorm in CNNs
Weight decay	λ	Free; AdamW with Adam
BatchNorm	momentum, eps	Small batches; train/eval differences
Early stopping	patience	Needs a validation set; restore best weights
Data augmentation	strength	CPU time; must not change the label (a rotated "6" becomes "9")
Smaller model / more data	—	Often the most effective of all

Common mistakes

- Evaluating in `train()` mode; augmenting validation/test data; early-stopping on training loss.

Real-world examples

AlexNet (2012): dropout 0.5 in dense layers plus heavy augmentation. ResNets: BatchNorm after every **convolution** (the sliding-filter operation of image networks, Chapter 11). Transformers: LayerNorm, dropout, AdamW weight decay.

In the interview

> **30-second answer:** "Dropout randomly zeroes units so the net can't rely on co-adapted features; weight decay penalises large weights; BatchNorm normalises per mini-batch, allowing higher learning rates and adding mild noise; early stopping halts when validation loss stops improving; augmentation adds label-preserving variants. More data is the best regularizer."

- *Dropout at test time?* Nothing — inverted dropout already scaled in training.
- *BatchNorm at inference?* Running mean/variance from training, not batch statistics.

Self-check

1. Transformer normalization? → LayerNorm (or the cheaper RMSNorm).

TL;DR

Overfitting = train loss ↓, val loss ↑. Combine early stopping, weight decay (AdamW), dropout, normalization and augmentation; more data beats them all.

10.8 Practical training habits

Normalize the inputs

Wildly different scales (pixels 0–255 beside incomes in the hundreds of thousands) stretch the loss surface into a ravine. **Min-max scaling** to [0, 1] (or [−1, 1]) suits bounded data such as pixels (÷255); **standardization (z-score)** to mean 0, std 1 suits most tabular features, per-channel for images (ImageNet mean/std). Compute statistics on the **training set only** (scalers in detail: Chapter 4, section 4.5).

Mini-batches

One example per update is slow and noisy; the full dataset is memory-hungry with few updates; **mini-batches** compromise.

Batch size	Gradient estimate	Updates per epoch	Memory	Notes
Small (16–64)	Noisy	Many	Low	Noise often helps generalisation
Large (256–4096)	Smooth	Few	High	GPU-efficient; needs larger LR + warmup

Start at 32–256. Erratic training → larger batch or smaller LR; out of GPU memory → smaller batch or **gradient accumulation** (sum gradients over several small batches before one `step()`).

Class imbalance

With a rare class (fraud, disease, defects), always predicting the majority scores high accuracy and is useless (full treatment: Chapter 4, section 4.9).

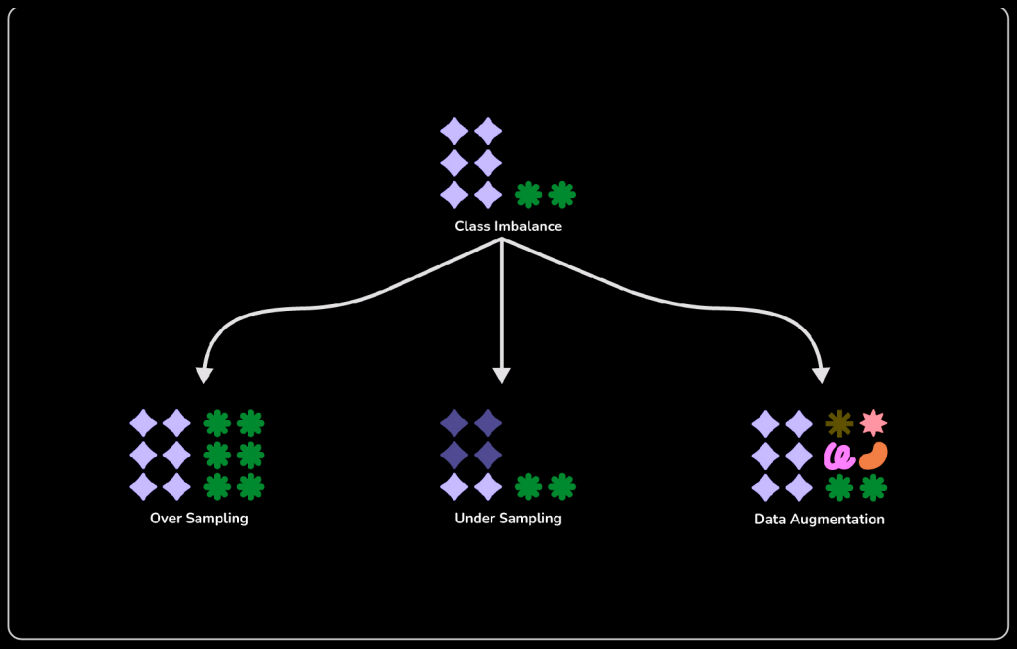


Figure — An imbalanced dataset (top) and three remedies: oversampling the minority class, undersampling the majority class, and augmentation to create new minority examples (course diagram).

Method	How	Risk
Oversampling	Repeat or synthesise minority examples (SMOTE interpolates between minority neighbours; augmentation for images); `WeightedRandomSampler`	Overfitting to repeats
Undersampling	Drop majority examples	Throws away information
Class weights	Minority mistakes cost more: `CrossEntropyLoss(weight=…)`, `BCEWithLogitsLoss(pos_weight=…)`	Probabilities no longer calibrated
Threshold tuning / better metrics	Precision, recall, F1, PR-AUC (Chapter 6); tune the threshold	—

`code/practical_tips.py` (95% / 5% synthetic data); class weight = n / (k · n_c) (n examples, k classes, n_c in class c): 2000/(2·112) ≈ 8.9 minority, 2000/(2·1888) ≈ 0.53 majority:

```
class counts in train: [1888, 112]  (minority = 5.6%)
plain cross-entropy          accuracy=0.957  minority recall=0.369
class-weighted (n/(k*n_c))   accuracy=0.939  minority recall=0.640
(always predicting the majority class would score accuracy 0.945)
```

Accuracy barely moves (near the useless baseline) but the weighted loss finds 64% of rare cases instead of 37%.

Reading loss curves

What you see	Likely cause	What to try
Loss flat from step 0, near ln(K)	LR far too small, or a bug (shuffled labels, missing `optimizer.step()`)	Overfit one batch; check the pipeline
Loss up / NaN / wild spikes	LR too high, exploding gradients, log(0)	Lower LR, clipping, logit-based losses
Training ↓, validation ↓ then ↑	Overfitting	Regularization, early stopping, more data
Both high and close	Underfitting	Bigger model, longer, higher LR, better features
Validation better than training	Dropout/augmentation only in training, or easier validation set	Usually fine — check the split
Noisy, jagged curve	Small batch or high LR	Larger batch, lower LR, smooth the plot

Overfit a small batch first

Train repeatedly on **one mini-batch**: a correct pipeline drives its loss near zero within a few hundred steps; if not, a bug (wrong labels or loss, no gradient flow) no tuning will fix. Also check the **first** loss ≈ chance (ln 10 = 2.303 for

10 classes).

```
1) Overfit one batch of 32 examples
  step   0  loss on the single batch = 2.3176
  step  50  loss on the single batch = 0.0008
  step 200  loss on the single batch = 0.0002
  expected start ~ ln(10) = 2.303 (uniform guess over 10 classes)
```

The learning-rate finder (LR range test)

Leslie Smith's trick, popularised by fast.ai: start tiny, **multiply the LR by a constant after every mini-batch** (e.g. 1e-5 → 10), record the smoothed loss — flat while too small, dropping once useful, exploding when too large. Pick a point on the steep descent, commonly **one-tenth of the LR at the loss minimum**.

In `code/practical_tips.py` the multiplier is (10 / 1e-5)^(1/149) over 150 batches, loss smoothed by an exponential moving average (0.9·old + 0.1·new), stopping once smoothed loss exceeds 4× its best:

```
2) LR range test (SGD+momentum, lr grows 1e-5 -> 10)
  lowest smoothed loss at lr=0.619; loss blew up by lr=2.27; rule of thumb: pick ~1/10 of the minimum -> 0.062
3) Training curves for three learning rates (SGD+momentum, 30 epochs)
  lr=0.0001 final train loss 2.252  val loss 2.250
  lr=0.05   final train loss 0.035  val loss 0.111
  lr=3      final train loss 2.443  val loss 2.444
```

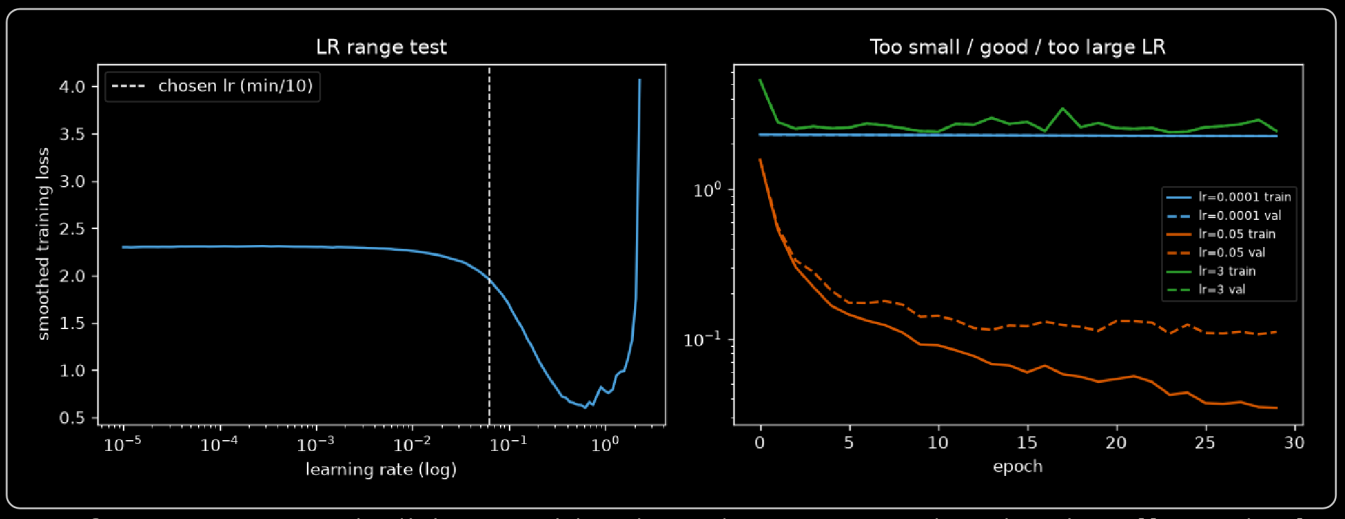


Figure — Left: LR range test on the digits MLP. Right: the LR the test suggests (≈0.05) trains well; 1e-4 barely moves in 30 epochs; 3.0 bounces around chance level.

A debugging checklist

Inspect examples and labels after the pipeline → chance-level first loss → overfit one batch → LR finder (or Adam 1e-3) → train with validation curves → then regularize and tune, against a simple baseline (logistic regression, gradient boosting). This is Karpathy's "A Recipe for Training Neural Networks": know the data, start simple, verify each piece, then scale.

In the interview

30-second answer: "Normalise inputs with training-set statistics, start from a known-good architecture and Adam, check the initial loss equals chance, overfit one batch to rule out bugs, pick the LR with a range test, then read train/validation curves for under- vs overfitting. For imbalance, use class weights or resampling and judge by recall/precision/PR-AUC, not accuracy."

Self-check

1. Initial loss 8.0 on 10 classes? → Far above ln 10 ≈ 2.3: bad init/scale, wrong labels or loss — investigate first.

TL;DR

Train-set normalisation; batches of 32–256; weights/resampling plus proper metrics for imbalance; chance-level first loss, overfit one batch, LR finder, read the curves.

10.9 PyTorch basics

What it is

PyTorch (created at Meta AI, now governed by the PyTorch Foundation) is the most widely used deep-learning framework in research and common in production:

Piece	What it does
`torch.Tensor`	N-dimensional array like NumPy's, but can live on a GPU and track gradients
autograd	Records operations on tensors with `requires_grad=True`; `.backward()` fills `.grad`
`nn.Module`	Base class for models/layers; registers parameters and sub-modules; you write `forward()`
`nn.Linear`, `nn.Conv2d`, `nn.ReLU`, `nn.Dropout` …	Ready-made layers (`torch.nn.functional` has stateless, parameter-free versions)
`torch.optim`	Optimizers (SGD, Adam, AdamW…) and `lr_scheduler`
`Dataset` / `DataLoader`	Index into data; batch, shuffle and optionally load in parallel worker processes

How it works — the training loop

Five lines per mini-batch: **forward → loss → zero_grad → backward → step.**

```
# code/pytorch_basics.py (core)
class WineNet(nn.Module):
    def __init__(self, n_in, n_hidden, n_classes):
        super().__init__()                          # registers sub-modules and parameters
        self.net = nn.Sequential(nn.Linear(n_in, n_hidden), nn.ReLU(),
                                 nn.Linear(n_hidden, n_classes))   # logits, no softmax
    def forward(self, x):
        return self.net(x)

mean, std = Xtr.mean(0), Xtr.std(0)                 # standardize with TRAIN statistics only
Xtr, Xte = (Xtr - mean) / std, (Xte - mean) / std
train_ds = TensorDataset(torch.tensor(Xtr, dtype=torch.float32), torch.tensor(ytr, dtype=torch.long))
train_dl = DataLoader(train_ds, batch_size=32, shuffle=True)

model = WineNet(13, 32, 3)
loss_fn = nn.CrossEntropyLoss()                     # = log-softmax + negative log-likelihood
opt = torch.optim.Adam(model.parameters(), lr=1e-2)

for epoch in range(1, 31):
    model.train()
    for xb, yb in train_dl:
        logits = model(xb)                          # 1. forward
        loss = loss_fn(logits, yb)                  # 2. loss
        opt.zero_grad()                             # 3. clear old gradients
        loss.backward()                             # 4. backprop
        opt.step()                                  # 5. update weights

    model.eval()
    with torch.no_grad(): ...                       # evaluate: no graph building

torch.save(model.state_dict(), "wine.pt")           # save weights; the script reloads them into a fresh WineNet
```

Output (tensor demos and model printout trimmed):

```
...
autograd dy/dx at x=2: 7.0
...
trainable parameters: 547
epoch  1  train loss 0.702 acc 0.944 | test loss 0.684 acc 0.907
epoch  5  train loss 0.092 acc 0.992 | test loss 0.083 acc 0.981
epoch 10  train loss 0.022 acc 1.000 | test loss 0.031 acc 0.981
epoch 30  train loss 0.003 acc 1.000 | test loss 0.018 acc 0.981
reloaded model test acc: 0.9814814814814815
...
```

Checks: $y = x^2 + 3x \rightarrow dy/dx$ at 2 = 7; parameters = 13·32 + 32 + 32·3 + 3 = 547. `model.train()` / `model.eval()` switch dropout and BatchNorm; `torch.no_grad()` saves memory and time; `state_dict()` (ordered dict of parameter and buffer tensors) is the recommended save/load format.

GPU: `device = "cuda" if torch.cuda.is_available() else "cpu"` (Apple Silicon: `"mps"`); move model and every batch with `.to(device)`. Mixed precision (`torch.autocast`: 16-bit numbers where safe) and `torch.compile(model)` (compiles to optimised code) speed up real workloads.

Custom Dataset: subclass `Dataset` with `__len__` and `__getitem__(i)` returning one (x, y); `DataLoader(ds, batch_size=64, shuffle=True, num_workers=4)` batches, shuffles and loads in parallel.

Common mistakes

- Missing `optimizer.zero_grad()`, `model.eval()` or `torch.no_grad()`; an unshuffled training loader.
- dtype mismatches: `CrossEntropyLoss` wants `long` labels; `BCEWithLogitsLoss` float targets shaped like the logits.
- `.item()`/`.numpy()` every step on a GPU (forces the CPU to wait for the GPU).

Real-world examples

PyTorch underlies Meta's production models, OpenAI's research code and most Hugging Face (hub of pre-trained models) models. PyTorch Lightning, Hugging Face `Trainer` and fastai hide the loop but run these five steps.

In the interview

> **30-second answer:** "Subclass nn.Module and write forward; wrap a Dataset in a DataLoader for batching and shuffling; each step is forward, loss, zero_grad, backward, optimizer.step. Switch model.train()/eval() for dropout and BatchNorm, use torch.no_grad() for inference, and save the state_dict."

- *Why zero_grad?* `.backward()` adds into `.grad`; without zeroing, gradients from several batches sum (done on purpose in gradient accumulation).
- *`nn.ReLU` vs `F.relu`?* Same maths; the module fits in `nn.Sequential`, the function is called in `forward`.

Self-check

1. Parameters of `nn.Linear(100, 10)`? → 1,010.

TL;DR

Tensors + autograd + nn.Module + DataLoader + optimizer. Loop: forward → loss → zero_grad → backward → step. train()/eval(), no_grad(), save state_dict.

Chapter TL;DR

- **Neuron** = weighted sum + bias + activation; **MLP** = stacked layers building a feature hierarchy; one hidden layer solves XOR.
- **Activations/losses:** ReLU/GELU inside; identity/sigmoid/softmax out; MSE, BCE, cross-entropy on logits.
- **Backprop** = chain rule to every weight; then $\theta \leftarrow \theta - \eta \nabla L$ with SGD/momentum/Adam(W) and a warmup + cosine schedule.
- **Deep-net stability:** ReLU, He/Xavier init, normalization, residuals, clipping. **Regularization:** dropout, weight decay, early stopping, augmentation, more data.
- **Practice:** chance-level first loss, overfit one batch, LR finder, read the curves; PyTorch's forward/loss/zero_grad/backward/step loop.

Next: Chapter 11 applies these ideas to images with convolutional networks; Chapter 12 to sequences with RNNs and attention.